

Chapter 5. Memory management

In this chapter, you'll learn how Windows implements virtual memory and how it manages the subset of virtual memory kept in physical memory. We'll also describe the internal structure and components that make up the memory manager, including key data structures and algorithms. Before examining these mechanisms, we'll review the basic services provided by the memory manager and key concepts such as reserved memory versus committed memory and shared memory.

Introduction to the memory manager

By default, the virtual size of a process on 32-bit Windows is 2 GB. If the image is marked specifically as large address space–aware, and the system is booted with a special option (described in the section “x86 address space layouts” later in this chapter), a 32-bit process can grow to be up to 3 GB on 32-bit Windows and to 4 GB on 64-bit Windows. The process virtual address space size on 64-bit Windows 8 and Server 2012 is 8192 GB (8 TB) and on 64 bit Windows 8.1 (and later) and Server 2012 R2 (and later), it is 128 TB.

As you saw in [Chapter 2](#), “[System architecture](#)”—specifically in [Table 2-2](#)—the maximum amount of physical memory currently supported by Windows ranges from 2 GB to 24 TB, depending on which version and edition of Windows you are running. Because the virtual address space might be larger or smaller than the physical memory on the machine, the memory manager has two primary tasks:

- Translating, or mapping, a process's virtual address space into physical memory so that when a thread running in the context of that process reads or writes to the virtual address space, the correct physical address is referenced. (The subset of a process's virtual address space that is physically resident is called the *working set*. Working sets are described in more detail in the section “[Working sets](#)” later in this chapter.)
- Paging some of the contents of memory to disk when it becomes overcommitted—that is, when running threads try to use more physical memory than is currently available—and bringing the contents back into physical memory when needed.

In addition to providing virtual memory management, the memory manager provides a core set of services on which the various Windows environment subsystems are built. These services include memory-mapped files (internally called *section objects*), copy-on-write memory, and support for applications using large, sparse address spaces. The memory manager also provides a way for a process to allocate and use larger amounts of physical memory than can be mapped into the process virtual address space at one time—for example, on 32-bit systems with more than 3 GB of physical memory. This is explained in the section “[Address Windowing Extensions](#)” later in this chapter.



Note

There is a Control Panel applet (System) that provides control over the size, number, and locations of paging files. Its nomenclature suggests that virtual memory is the same thing as the paging file. This is not the case. The paging file is only one aspect of virtual memory. In fact, even if you run with no page file at all, Windows will still be using virtual memory. This distinction is explained in more detail later in this chapter.

Memory manager components

The memory manager is part of the Windows executive and therefore exists in the file Ntoskrnl.exe. It's the largest component in the executive, hinting at its importance and complexity. No parts of the memory manager exist in the HAL. The memory manager consists of the following components:

- A set of executive system services for allocating, deallocating, and managing virtual memory, most of which are exposed through the Windows API or kernel-mode device driver interfaces
- A translation-not-valid and access fault trap handler for resolving hardware-detected memory-management exceptions and making virtual pages resident on behalf of a process
- Six key top-level routines, each running in one of six different kernel-mode threads in the System process:
 - **The balance set manager (KeBalanceSetManager, priority 17)** This calls an inner routine, the working set manager (MmWorkingSetManager), once per second as well as when free memory falls below a certain threshold. The working set manager drives the overall memory-management policies, such as working set trimming, aging, and modified page writing.
 - **The process/stack swapper (KeSwapProcessOrStack, priority 23)** This performs both process and kernel thread stack inswapping and outswapping. The balance set manager and the thread-scheduling code in the kernel awaken this thread when an inswap or outswap operation needs to take place.
 - **The modified page writer (MiModifiedPageWriter, priority 18)** This writes dirty pages on the modified list back to the appropriate paging files. This thread is awakened when the size of the modified list needs to be reduced.
 - **The mapped page writer (MiMappedPageWriter, priority 18)** This writes dirty pages in mapped files to disk or remote storage. It is awakened when the size of the modified list needs to be reduced or if pages for mapped files have been on the modified list for more than 5 minutes. This second modified page writer thread is necessary because it can generate page faults that result in requests for free pages. If there were no free pages and only one modified page writer thread, the system could deadlock waiting for free pages.
 - **The segment dereference thread (MiDereferenceSegmentThread, priority 19)** This is responsible for cache reduction as well as for page file growth and shrinkage. For example, if there is no virtual address space for paged pool growth, this thread trims the page cache so that the paged pool used to anchor it can be freed for reuse.
 - **The zero page thread (MiZeroPageThread, priority 0)** This zeroes out pages on the free list so that a cache of zero pages is available to satisfy future demand-zero page faults. In some cases, memory zeroing is done by a faster function called MiZeroInParallel. See the note in the "[Page list dynamics](#)" section later in this chapter.

Each of these components is covered in more detail later in the chapter except for the segment dereference thread, which is covered in Chapter 14, "Cache manager," in Part 2.

Large and small pages

Memory management is done in distinct chunks called *pages*. This is because the hardware memory management unit translates virtual to physical addresses at the granularity of a page. Hence, a page is the smallest unit of protection at the hardware level. (The various page-protection options are described in the section “[Protecting memory](#)” later in this chapter.) The processors on which Windows runs support two page sizes: small and large. The actual sizes vary based on the processor architecture, and they are listed in [Table 5-1](#).

Architecture	Small Page Size	Large Page Size	Small Pages per Large Page
x86 (PAE)	4 KB	2 MB	512
x64	4 KB	2 MB	512
ARM	4 KB	4 MB	1024

TABLE 5-1 Page sizes



Note

Some processors support configurable page sizes, but Windows does not use this feature.

The primary advantage of large pages is speed of address translation for references to data within the large page. This advantage exists because the first reference to any byte within a large page will cause the hardware’s translation look-aside buffer (TLB) (described in the section “[Address translation](#)” later in this chapter), to have in its cache the information necessary to translate references to any other byte within the large page. If small pages are used, more TLB entries are needed for the same range of virtual addresses, thus increasing the recycling of entries as new virtual addresses require translation. This, in turn, means having to go back to the page table structures when references are made to virtual addresses outside the scope of a small page whose translation has been cached. The TLB is a very small cache; thus, large pages make better use of this limited resource.

To take advantage of large pages on systems with more than 2 GB of RAM, Windows maps with large pages the core operating system images (Ntoskrnl.exe and Hal.dll) as well as core operating system data (such as the initial part of non-paged pool and the data structures that describe the state of each physical memory page). Windows also automatically maps I/O space requests (calls by device drivers to MmMapIoSpace) with large pages if the request is of a satisfactorily large page length and alignment. In addition, Windows allows applications to map their images, private memory, and page file-backed sections with large pages (see the MEM_LARGE_PAGES flag on the VirtualAlloc, VirtualAllocEx, and VirtualAllocExNuma functions). You can also specify other device drivers to be mapped with large pages by adding a multistring registry value LargePageDrivers to the key HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management and specifying the names of the drivers as separately null-terminated strings.

Attempts to allocate large pages may fail after the operating system has been running for an extended period because the physical memory for each large page must occupy a significant number (refer to [Table 5-1](#)) of physically contiguous small pages. This extent of physical pages must furthermore begin on a large page boundary. For example, physical pages 0–511 could be used as a large page on an x64 system, as could physical pages 512–1,023, but pages 10–521 could not. Free physical memory does become fragmented as the system runs. This is not a problem for allocations using small pages but can cause large page allocations to fail.

The memory is also always non-pageable because the page file system does not support large pages. Because the memory is non-pageable, the caller is required to have the `SeLockMemoryPrivilege` to be able to allocate using large pages. Also, the allocated memory is not considered part of the process working set (described in the section “[Working sets](#)” later in this chapter); nor are large page allocations subject to job-wide limits on virtual memory usage.

On Windows 10 version 1607 x64 and Server 2016 systems, large pages may also be mapped with huge pages, which are 1 GB in size. This is done automatically if the allocation size requested is larger than 1 GB, but it does not have to be a multiple of 1 GB. For example, an allocation of 1040 MB would result in using one huge page (1024 MB) plus 8 “normal” large pages (16 MB divided by 2 MB).

There is an unfortunate side effect of large pages. Each page (whether huge, large, or small) must be mapped with a single protection that applies to the entire page. This is because hardware memory protection is on a per-page basis. If a large page contains, for example, both read-only code and read/write data, the page must be marked as read/write, meaning that the code will be writable. As a result, device drivers or other kernel-mode code could, either maliciously or due to a bug, modify what is supposed to be read-only operating system or driver code without causing a memory access violation. If small pages are used to map the operating system’s kernel-mode code, the read-only portions of `Ntoskrnl.exe` and `Hal.dll` can be mapped as read-only pages. Using small pages does reduce efficiency of address translation, but if a device driver (or other kernel-mode code) attempts to modify a read-only part of the operating system, the system will crash immediately with the exception information pointing at the offending instruction in the driver. If the write were allowed to occur, the system would likely crash later (in a harder-to-diagnose way) when some other component tried to use the corrupted data.

If you suspect you are experiencing kernel code corruptions, enable Driver Verifier (described in [Chapter 6](#), “[I/O system](#)”), which will disable the use of large pages.



Note

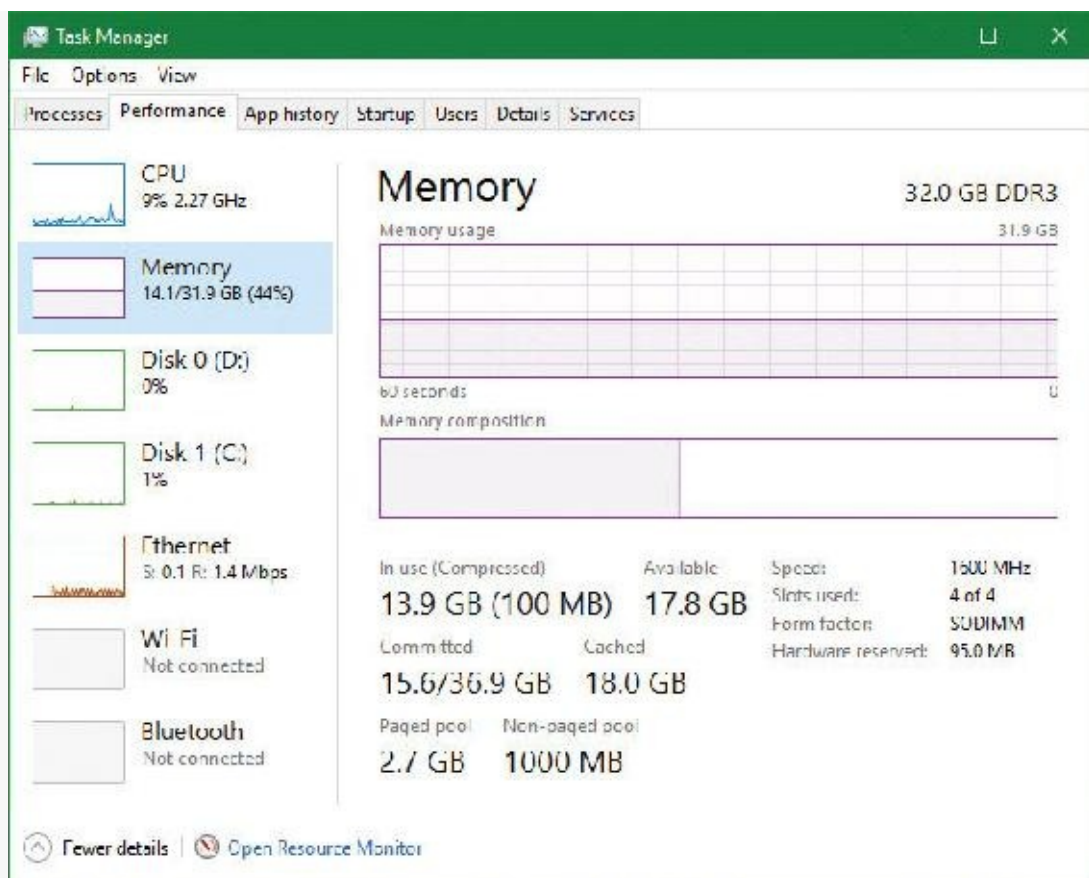
The term *page* used in this and later chapters refers to a small page unless otherwise indicated or apparent by context.

Examining memory usage

The Memory and Process performance-counter categories provide access to most of the details about system and process memory utilization. Throughout this chapter, we’ll include references to specific performance counters that contain information related to the component being described. We’ve included relevant examples and experiments throughout the chapter. One word of caution, however: Different utilities use varying and sometimes inconsistent or confusing names when displaying memory information. The following experiment illustrates this point. (We’ll explain the terms used in this example in subsequent sections.)

EXPERIMENT: Viewing system memory information

The Performance tab in the Windows Task Manager, shown in the following screenshot from a Windows 10 version 1607 system (click the **Memory** tab on the left in the **Performance** tab), displays basic system memory information. This information is a subset of the detailed memory information available through performance counters. It includes data on both physical and virtual memory usage. The table that follows shows the meaning of the memory-related values.

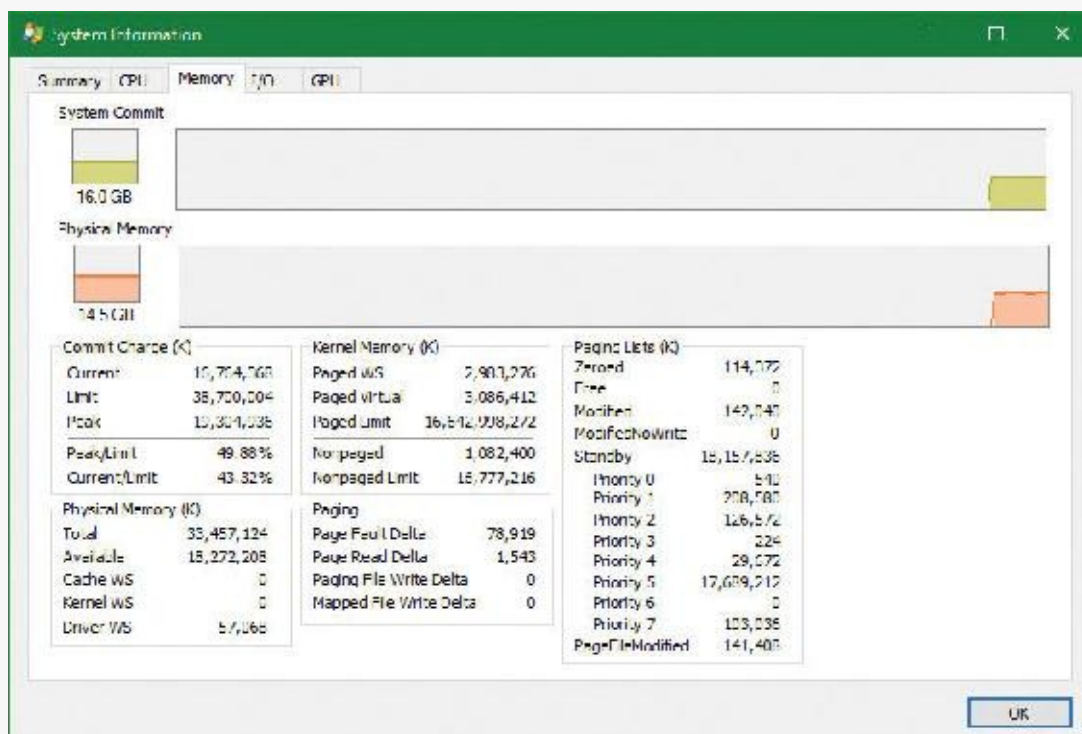


Task Manager Value	Definition
Memory usage histogram	The chart's line height reflects physical memory in use by Windows (not available as a performance counter). The area above the line is equal to the Available value in the bottom section. The total height of the graph is equal to the total value shown at the top right of the graph (31.9 GB in this example). This represents the total RAM usable by the operating system, and does not include BIOS shadow pages, device memory, and so on.
Memory composition	This details the relation between memory that is actively used, standby, modified, and free-zero (all described later in this chapter).
Total physical memory (top right of graph)	This shows the physical memory usable by Windows.
In Use (Compressed)	This is the physical memory currently being used. The amount of compressed physical memory is in parentheses. Hovering over the value shows the amount of memory saved by the compression. (Memory compression is discussed in the section "Memory compression" later in this chapter.)
Cached	This is the sum of the following performance counters in the Memory category: Cache Bytes, Modified Page List Bytes, Standby Cache Core Bytes, Standby Cache Normal Priority Bytes, and Standby Cache Reserve Bytes.
Available	This is the amount of memory that is immediately available for use by the operating system, processes, and drivers. It is equal to the combined size of the standby, free, and zero page lists.
Free	This shows the free and zero-page list bytes. To see this information, hover over the right-most part of the Memory Composition bar (assuming you have enough free memory to hover over it).
Committed	The two numbers shown here are equal to the values in the Committed Bytes and Commit Limit performance counters, respectively.
Paged Pool	This is the total size of the paged pool, including both free and allocated regions.
Non-Paged Pool	This is the total size of the non-paged pool, including both free and allocated regions.

To see the specific usage of the paged and non-paged pool, use the Poolmon utility, described later in this chapter in the “[Monitoring pool usage](#)” section.

The Process Explorer tool from Sysinternals can show considerably more data about physical and virtual memory. On its main screen, click the **View** menu, choose **System Information**, and click the **Memory** tab. Here is an example of a display from a 64-bit Windows 10 system. (We will explain most

of these counters in the relevant sections later in this chapter.)



Two other Sysinternals tools show extended memory information:

- **VMMMap** This shows the usage of virtual memory within a process to a fine level of detail.
- **RAMMap** This shows detailed physical memory usage.

These tools are featured in experiments found later in this chapter.

Finally, the `!vm` command in the kernel debugger shows the basic memory-management information available through the memory-related performance counters. This command can be useful if you're looking at a crash dump or hung system. Here's an example of its output from a 64-bit Windows 10 system with 32 GB of RAM:

[Click here to view code image](#)

```
lkd> !vm
Page File: \??\C:\pagefile.sys
  Current:    1048576 Kb   Free Space:    1034696 Kb
  Minimum:    1048576 Kb   Maximum:      4194304 Kb
Page File: \??\C:\swapfile.sys
  Current:      16384 Kb   Free Space:      16376 Kb
  Minimum:      16384 Kb   Maximum:    24908388 Kb
No Name for Paging File
  Current:    58622948 Kb   Free Space:    57828340 Kb
  Minimum:    58622948 Kb   Maximum:    58622948 Kb

Physical Memory:      8364281 ( 33457124 Kb)
Available Pages:      4627325 ( 18509300 Kb)
ResAvail Pages:       7215930 ( 28863720 Kb)
Locked IO Pages:      0 ( 0 Kb)
Free System PTEs:     4295013448 (17180053792 Kb)
Modified Pages:        68167 ( 272668 Kb)
Modified PF Pages:     68158 ( 272632 Kb)
```

```

Modified No Write Pages:          0 (          0 Kb)
NonPagedPool Usage:              495 (        1980 Kb)
NonPagedPoolNx Usage:           269858 (    1079432 Kb)
NonPagedPool Max:               4294967296 (17179869184 Kb)
PagedPool 0 Usage:              371703 (    1486812 Kb)
PagedPool 1 Usage:              99970 (      399880 Kb)
PagedPool 2 Usage:             100021 (      400084 Kb)
PagedPool 3 Usage:              99916 (      399664 Kb)
PagedPool 4 Usage:              99983 (      399932 Kb)
PagedPool Usage:                771593 (    3086372 Kb)
PagedPool Maximum:             4160749568 (16642998272 Kb)
Session Commit:                 12210 (      48840 Kb)
Shared Commit:                  344197 (    1376788 Kb)
Special Pool:                   0 (          0 Kb)
Shared Process:                 19244 (      76976 Kb)
Pages For MDLs:                 419675 (    1678700 Kb)
Pages For AWE:                   0 (          0 Kb)
NonPagedPool Commit:            270387 (    1081548 Kb)
PagedPool Commit:              771593 (    3086372 Kb)
Driver Commit:                  24984 (      99936 Kb)
Boot Commit:                    100044 (    400176 Kb)
System PageTables:              5948 (      23792 Kb)
VAD/PageTable Bitmaps:          18202 (      72808 Kb)
ProcessLockedFilePages:         299 (       1196 Kb)
Pagefile Hash Pages:            33 (        132 Kb)
Sum System Commit:              1986816 (    7947264 Kb)
Total Private:                  2126069 (    8504276 Kb)
Misc/Transient Commit:          18422 (      73688 Kb)
Committed pages:                4131307 (   16525228 Kb)
Commit limit:                   9675001 (   38700004 Kb)
...

```

The values not in parentheses are in small pages (4 KB). We will describe many of the details of the output of this command throughout this chapter.

Internal synchronization

Like all other components of the Windows executive, the memory manager is fully reentrant and supports simultaneous execution on multiprocessor systems. That is, it allows two threads to acquire resources in such a way that they don't corrupt each other's data. To accomplish the goal of being fully reentrant, the memory manager uses several different internal synchronization mechanisms, such as spinlocks and interlocked instructions, to control access to its own internal data structures. (Synchronization objects are discussed in Chapter 8, "System mechanisms", in Part 2.)

Some of the system-wide resources to which the memory manager must synchronize access include:

- Dynamically allocated portions of the system virtual address space
- System working sets

- Kernel memory pools
- The list of loaded drivers
- The list of paging files
- Physical memory lists
- Image base randomization address space layout randomization (ASLR) structures
- Each individual entry in the page frame number (PFN) database

Per-process memory-management data structures that require synchronization include the following:

- **Working set lock** This is held while changes are made to the working set list.
- **Address space lock** This is held whenever the address space is being changed.

Both these locks are implemented using pushlocks. These are described in Chapter 8 in Part 2.

Services provided by the memory manager

The memory manager provides a set of system services to allocate and free virtual memory, share memory between processes, map files into memory, flush virtual pages to disk, retrieve information about a range of virtual pages, change the protection of virtual pages, and lock the virtual pages into memory.

Like other Windows executive services, memory-management services allow their caller to supply a process handle indicating the particular process whose virtual memory is to be manipulated. The caller can thus manipulate either its own memory or (with proper permissions) the memory of another process. For example, if a process creates a child process, by default it has the right to manipulate the child process's virtual memory. Thereafter, the parent process can allocate, deallocate, read, and write memory on behalf of the child process by calling virtual memory services and passing a handle to the child process as an argument. This feature is used by subsystems to manage the memory of their client processes. It is also essential for implementing debuggers because debuggers must be able to read and write to the memory of the process being debugged.

Most of these services are exposed through the Windows API. As shown in [Figure 5-1](#), the Windows API has four groups of functions for managing memory in applications:

- **Virtual API** This is the lowest-level API for general memory allocations and deallocations. It always works on page granularity. It is also the most powerful, supporting the full capabilities of the memory manager. Functions include `VirtualAlloc`, `VirtualFree`, `VirtualProtect`, `VirtualLock`, and others.
- **Heap API** This provides functions for small allocations (typically less than a page). It uses the Virtual API internally, but adds management on top of it. Heap manager functions include `HeapAlloc`, `HeapFree`, `HeapCreate`, `HeapReAlloc` and others. The heap manager is discussed in the section “[Heap manager](#)” later in this chapter.
- **Local/Global APIs** These are leftovers from 16-bit Windows and are now implemented using the Heap API.
- **Memory-mapped files** These functions allow mapping files as memory and/or sharing memory between cooperating processes. Memory-mapped file functions include `CreateFileMapping`, `OpenFileMapping`, `MapViewOfFile`, and others.

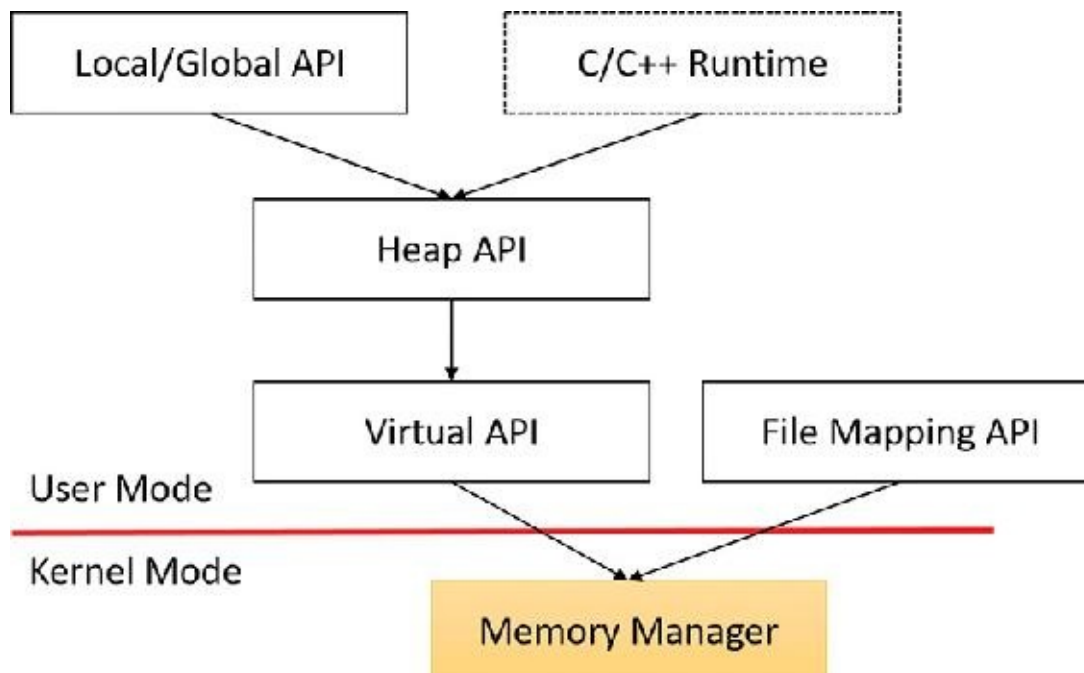


FIGURE 5-1 Memory API groups in user mode.

The dotted box shows a typical C/C++ runtime implementation of memory management (functions such as `malloc`, `free`, `realloc`, C++ operator `new` and `delete`) using the Heap API. The box is dotted because this implementation is compiler-dependent and certainly not mandatory (although quite common). The C runtime equivalents that are implemented in `Ntdll.dll` use the Heap API.

The memory manager also provides several services to other kernel-mode components inside the executive as well as to device drivers. These include allocating and deallocating physical memory and locking pages in physical memory for direct memory access (DMA) transfers. These functions begin with the prefix `Mm`. In addition, although not strictly part of the memory manager, some executive support routines that begin with `Ex` are used to allocate and deallocate from the system heaps (paged and non-paged pool) as well as to manipulate look-aside lists. We'll touch on these topics later in this chapter in the section "[Kernel-mode heaps \(system memory pools\)](#)."

Page states and memory allocations

Pages in a process virtual address space are either free, reserved, committed, or shareable. Committed and shareable pages are pages that, when accessed, ultimately translate to valid pages in physical memory. Committed pages are also referred to as private pages. This is because committed pages cannot be shared with other processes, whereas shareable pages can be (but might be in use by only one process).

Private pages are allocated through the Windows `VirtualAlloc`, `VirtualAllocEx`, and `Virtual-AllocExNuma` functions, which lead eventually to the executive in the function `NtAllocateVirtualMemory` inside the memory manager. These functions are capable of committing memory as well as reserving memory. Reserving memory means setting aside a range of contiguous virtual addresses for possible future use (such as an array) while consuming negligible system resources, and then committing portions of the reserved space as needed as the application runs. Or, if the size requirements are known in advance, a process can reserve and commit in the same function call. In either case, the resulting committed pages can then be accessed by any thread in the process. Attempting to access free or reserved memory results in an access violation exception because the page isn't mapped to any storage that can resolve the reference.

If committed (private) pages have never been accessed before, they are created at the time of first access as zero-initialized pages (or *demand zero*). Private committed pages may later be automatically written to the paging file by the operating system if required by demand for physical memory. Private refers to the fact that these pages are normally inaccessible to any other process.



Note

Some functions, such as `ReadProcessMemory` and `WriteProcessMemory`, appear to permit cross-process memory access, but these are implemented by running kernel-mode code in the context of the target process. (This is referred to as *attaching to the process*.) They also require that the security descriptor of the target process grant the accessor the `PROCESS_VM_READ` or `PROCESS_VM_WRITE` right, respectively, or that the accessor holds the `SeDebugPrivilege`, which is by default granted only to members of the administrators group.

Shared pages are usually mapped to a view of a section. This in turn is part or all of a file, but may instead represent a portion of page file space. All shared pages can potentially be shared with other processes. Sections are exposed in the Windows API as file-mapping objects.

When a shared page is first accessed by any process, it will be read in from the associated mapped file unless the section is associated with the paging file, in which case it is created as a zero-initialized page. Later, if it is still resident in physical memory, the second and subsequent processes accessing it can simply use the same page contents that are already in memory. Shared pages might also have been prefetched by the system.

Two upcoming sections of this chapter, “[Shared memory and mapped files](#)” and “[Section objects](#),” go into much more detail about shared pages. Pages are written to disk through a mechanism called *modified page writing*. This occurs as pages are moved from a process’s working set to a system-wide list called the *modified page list*. From there, they are written to disk or remote storage. (Working sets and the modified list are explained later in this chapter.) Mapped file pages can also be written back to their original files on disk with an explicit call to `FlushViewOfFile` or by the mapped page writer as memory demands dictate.

You can decommit private pages and/or release address space with the `VirtualFree` or `VirtualFreeEx` function. The difference between decommitment and release is similar to the difference between reservation and commitment. Decommited memory is still reserved, but released memory has been freed; it is neither committed nor reserved.

Using the two-step process of reserving and then committing virtual memory defers committing pages—and, thereby, defers adding to the system commit charge described in the next section—until needed, but keeps the convenience of virtual contiguity. Reserving memory is a relatively inexpensive operation because it consumes very little actual memory. All that needs to be updated or constructed is the relatively small internal data structures that represent the state of the process address space. We’ll explain these data structures, called *page tables* and *Virtual Address Descriptors* (VADs), later in this chapter.

One extremely common use for reserving a large space and committing portions of it as needed is the user-mode stack for each thread. When a thread is created, a stack is created by reserving a contiguous portion of the process address space. (The default size is 1 MB but you can override this size with the `CreateThread` and `CreateRemoteThread(Ex)` function calls or change it on an executable image basis by using the `/STACK` linker flag.) By default, the initial page in the stack is committed and

the next page is marked as a guard page (which isn't committed) that traps references beyond the end of the committed portion of the stack and expands it.

EXPERIMENT: Reserved versus committed pages

You can use the `TestLimit` Sysinternals utility to allocate large amounts of reserved or private committed virtual memory. You can then observe the difference via Process Explorer. Follow these steps:

1. Open two command prompt windows.
2. Invoke `TestLimit` in one of the command prompt windows to create a large amount of reserved memory:

[Click here to view code image](#)

```
C:\temp>testlimit -r 1 -c 800
```

```
Testlimit v5.24 - test Windows limits  
Copyright (C) 2012-2015 Mark Russinovich  
Sysinternals - www.sysinternals.com
```

```
Process ID: 18468
```

```
Reserving private bytes 1 MB at a time ...  
Leaked 800 MB of reserved memory (800 MB total leaked).  
Lasterror: 0  
The operation completed successfully.
```

3. In the other command prompt window, create a similar amount of committed memory:

[Click here to view code image](#)

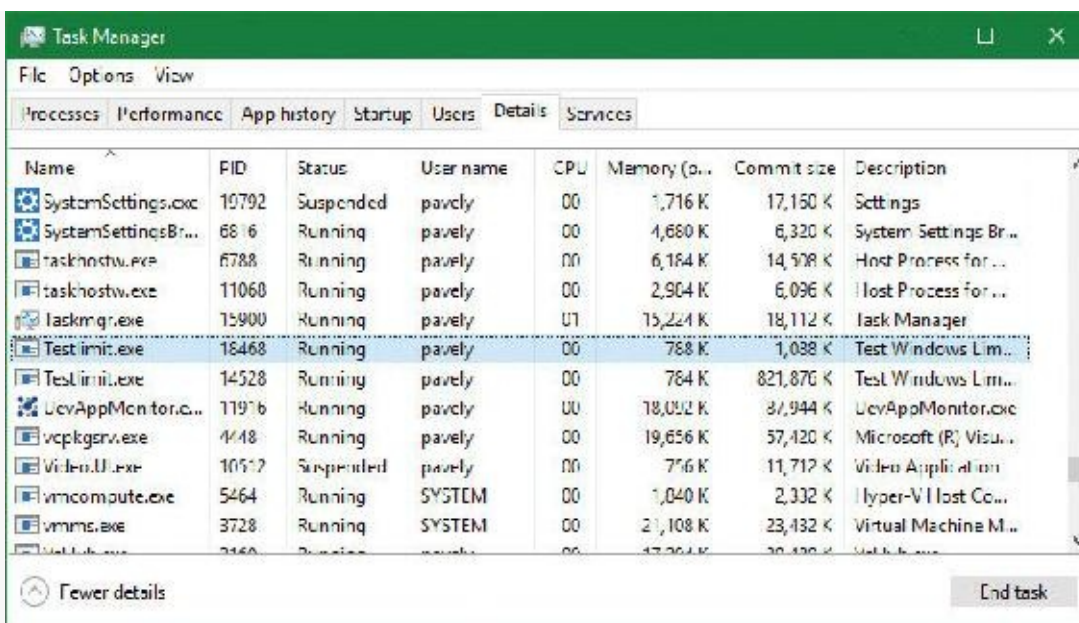
```
C:\temp>testlimit -m 1 -c 800
```

```
Testlimit v5.24 - test Windows limits  
Copyright (C) 2012-2015 Mark Russinovich  
Sysinternals - www.sysinternals.com
```

```
Process ID: 14528
```

```
Leaking private bytes 1 KB at a time ...  
Leaked 800 MB of private memory (800 MB total leaked).  
Lasterror: 0  
The operation completed successfully.
```

4. Run Task Manager, click the **Details** tab, and add a **Commit Size** column.
5. Find the two instances of `TestLimit.exe` in the list. They should look something like the following:



6. Notice that Task Manager shows the committed size but it has no counters that reveal the reserved memory in the other TestLimit process.

7. Open Process Explorer.

8. Click the **Process Memory** tab and enable the **Private Bytes** and **Virtual Size** columns.

9. Find the two TestLimit.exe processes in the main display:

Process	CPU	Private Bytes	Virtual Size	Working Set	PID	Description	Company Name
System Idle Process	0.00	0 K	0 K	0 K	0		
System	0.15	128 K	8,768 K	4,848 K	4		
System Idle Process	0.00	0 K	0 K	0 K	0		
System Settings.exe	0.00	17,160 K	21,478,226,124 K	41,268 K	10792	Settings	Microsoft Corporation
System Settings Broker.exe	0.00	6,320 K	21,478,226,124 K	25,632 K	6816	System Settings Broker	Microsoft Corporation
taskhostw.exe	0.00	14,508 K	21,478,226,124 K	21,560 K	6788	Host Process for Windows T...	Microsoft Corporation
taskhostv.exe	0.00	6,096 K	21,478,226,124 K	19,100 K	11060	Host Process for Windows T...	Microsoft Corporation
taskmgr.exe	0.08	18,032 K	21,478,226,124 K	41,552 K	15900	Task Manager	Microsoft Corporation
TestLimit.exe	0.00	1,000 K	892,752 K	5,552 K	18468	Test Windows Limits	Sysinternals - www.sysinter...
TestLimit.exe	0.00	821,876 K	892,752 K	5,516 K	14528	Test Windows Limits	Sysinternals - www.sysinter...
DevAppMonitor.exe	0.02	81,944 K	843,086 K	24,340 K	11916		Microsoft Corporation
vcplgsvr.exe	0.00	57,420 K	11,712,960 K	53,320 K	4448	Microsoft (R) Visual C++ Res...	Microsoft Corporation
VideoUI.exe	0.00	11,712 K	248,168 K	21,888 K	10512	Video Application	Microsoft Corporation
vmcompute.exe	0.00	2,332 K	21,478,226,124 K	9,844 K	5464	Hyper-V Host Compute Serv...	Microsoft Corporation
vmtoolsd.exe	0.00	23,432 K	21,478,226,124 K	38,864 K	3728	Virtual Machine Monitor	Microsoft Corporation
WslLib.exe	0.00	30,304 K	242,560 K	49,424 K	3160	WslLib.exe	Microsoft Corporation
WavesAudio.exe	0.00	4,420 K	81,672 K	13,376 K	10280	Waves MaxxAudio Service...	Waves Audio Ltd.
Windows Defender	0.00	12,404 K	21,478,226,124 K	31,400 K	5792	Windows Defender	Microsoft Corporation

10. Notice that the virtual sizes of the two processes are identical, but only one shows a Private Bytes value that is comparable to the Virtual Size value. The large difference in the other TestLimit process (process ID 18468) is due to the reserved memory. You could make the same comparison in Performance Monitor by looking at the Virtual Bytes and Private Bytes counters in the Process category.

Commit charge and commit limit

On the Performance tab in Task Manager, on the Memory page, there is a Committed label with two numbers underneath it. The memory manager keeps track of private committed memory usage on a global basis, termed *commitment* or *commit charge*. This is the first of the two numbers, which represents the total of all committed virtual memory in the system.

There is a system-wide limit, called the *system commit limit* or simply the *commit limit*, on the amount of committed virtual memory that can exist at any one time. This limit corresponds to the current total size of all paging files plus the amount of RAM that is usable by the operating system. This is the second of the two numbers displayed under the Committed label. The memory manager can increase the commit limit automatically by expanding one or more of the paging files if they are not already at their configured maximum size.

Commit charge and the system commit limit are explained in more detail in the section “[Commit charge and the system commit limit](#)” later in this chapter.

Locking memory

In general, it’s better to let the memory manager decide which pages remain in physical memory. However, there might be special circumstances when it might be necessary for an application or device driver to lock pages in physical memory. Pages can be locked in memory in two ways:

- Windows applications can call the `VirtualLock` function to lock pages in their process working set. Pages locked using this mechanism remain in memory until explicitly unlocked or until the process that locked them terminates. The number of pages a process can lock can’t exceed its minimum working set size minus eight pages. If a process needs to lock more pages, it can increase its working set minimum with the `SetProcessWorkingSetSizeEx` function, discussed later in this chapter in the section “[Working set management](#).”
- Device drivers can call the `MmProbeAndLockPages`, `MmLockPagableCodeSection`, `MmLockPagable-DataSection`, or `MmLockPagableSectionByHandle` kernel-mode functions. Pages locked using this mechanism remain in memory until explicitly unlocked. The last three of these APIs enforce no quota on the number of pages that can be locked in memory because the resident available page charge is obtained when the driver first loads. This ensures that it can never cause a system crash due to overlocking. For the first API, quota charges must be obtained or the API will return a failure status.

Allocation granularity

Windows aligns each region of reserved process address space to begin on an integral boundary defined by the value of the system allocation granularity, which can be retrieved from the Windows `GetSystemInfo` or `GetNativeSystemInfo` functions. This value is 64 KB, a granularity that is used by the memory manager to efficiently allocate metadata (for example, VADs, bitmaps, and so on) to support various process operations. In addition, if support were added for future processors with larger page sizes (for example, up to 64 KB) or virtually indexed caches that require system-wide physical-to-virtual page alignment, the risk of requiring changes to applications that made assumptions about allocation alignment would be reduced.



Note

Windows kernel-mode code isn't subject to the same restrictions. It can reserve memory on a single-page granularity (although this is not exposed to device drivers for the reasons detailed earlier). This level of granularity is primarily used to pack TEB allocations more densely. Because this mechanism is internal only, this code can easily be changed if a future platform requires different values. Also, for the purposes of supporting 16-bit and MS-DOS applications on x86 systems only, the memory manager provides the `MEM_DOS_LIM` flag to the `MapViewOfFileEx` API, which is used to force the use of single-page granularity.

Finally, when a region of address space is reserved, Windows ensures that the size and base of the region is a multiple of the system page size, whatever that might be. For example, because x86 systems use 4 KB pages, if you tried to reserve a region of memory 18 KB in size, the actual amount reserved on an x86 system would be 20 KB. If you specified a base address of 3 KB for an 18 KB region, the actual amount reserved would be 24 KB. Note that the VAD for the allocation would then also be rounded to 64 KB alignment/length, thus making the remainder of it inaccessible.

Shared memory and mapped files

As is true with most modern operating systems, Windows provides a mechanism to share memory among processes and the operating system. *Shared memory* can be defined as memory that is visible to more than one process or that is present in more than one process virtual address space. For example, if two processes use the same DLL, it would make sense to load the referenced code pages for that DLL into physical memory only once and share those pages between all processes that map the DLL, as illustrated in [Figure 5-2](#).

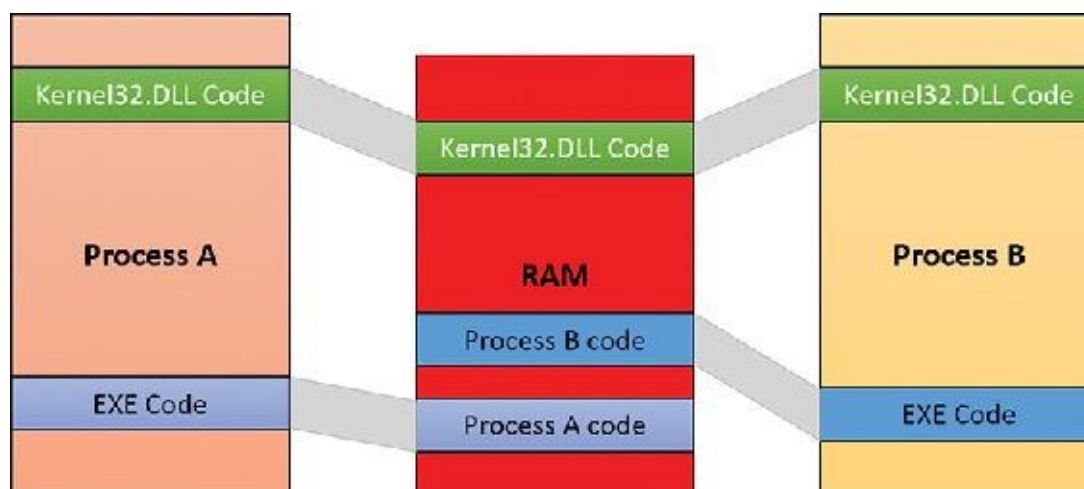


FIGURE 5-2 Sharing memory between processes.

Each process would still maintain its private memory areas to store private data but the DLL code and unmodified data pages could be shared without harm. As we'll explain later, this kind of sharing happens automatically because the code pages in executable images—EXE and DLL files, and several other types like screen savers (SCR), which are essentially DLLs under other names—are mapped as execute-only and writable pages are mapped as copy-on-write. (See the "[Copy-on-write](#)" section later in this chapter for more information.)

[Figure 5-2](#) shows two processes, based on different images, that share a DLL mapped just once to physical memory. The images (EXE) code itself is not shared in this case because the two processes run

different images. The EXE code would be shared between processes that run the same image, such as two or more processes running Notepad.exe.

The underlying primitives in the memory manager used to implement shared memory are called *section objects*, which are exposed as file-mapping objects in the Windows API. The internal structure and implementation of section objects are described later in this chapter in the section “[Section objects](#).”

This fundamental primitive in the memory manager is used to map virtual addresses whether in main memory, in the page file, or in some other file that an application wants to access as if it were in memory. A section can be opened by one process or by many. In other words, section objects don’t necessarily equate to shared memory.

A section object can be connected to an open file on disk (called a *mapped file*) or to committed memory (to provide shared memory). Sections mapped to committed memory are called *page-file-backed sections* because the pages are written to the paging file (as opposed to a mapped file) if demands on physical memory require it. (Because Windows can run with no paging file, page-file-backed sections might in fact be “backed” only by physical memory.) As with any other empty page that is made visible to user mode (such as private committed pages), shared committed pages are always zero-filled when they are first accessed to ensure that no sensitive data is ever leaked.

To create a section object, call the Windows `CreateFileMapping`, `CreateFileMappingFromApp`, or `CreateFileMappingNuma(Ex)` function, specifying a previously opened file handle to map it to (or `INVALID_HANDLE_VALUE` for a page-file-backed section) and optionally a name and security descriptor. If the section has a name, other processes can open it with `OpenFileMapping` or the `CreateFileMapping*` functions. Or you can grant access to section objects through either handle inheritance (by specifying that the handle be inheritable when opening or creating the handle) or handle duplication (by using `DuplicateHandle`). Device drivers can also manipulate section objects with the `ZwOpenSection`, `ZwMapViewOfSection`, and `ZwUnmapViewOfSection` functions.

A section object can refer to files that are much larger than can fit in the address space of a process. (If the paging file backs a section object, sufficient space must exist in the paging file and/or RAM to contain it.) To access a very large section object, a process can map only the portion of the section object that it requires (called a *view* of the section) by calling the `MapViewOfFile(Ex)`, `MapViewOfFileFromApp`, or `MapViewOfFileExNuma` function and then specifying the range to map. Mapping views permits processes to conserve address space because only the views of the section object needed at the time must be mapped into memory.

Windows applications can use mapped files to conveniently perform I/O to files by simply making them appear as data in memory within their address space. User applications aren’t the only consumers of section objects; the image loader uses section objects to map executable images, DLLs, and device drivers into memory, and the cache manager uses them to access data in cached files. (For information on how the cache manager integrates with the memory manager, see Chapter 14 in Part 2.) The implementation of shared memory sections, both in terms of address translation and the internal data structures, is explained in the section “[Section objects](#)” later in this chapter.

You can list the memory-mapped files in a process by using Process Explorer. To do so, configure the lower pane to show the DLL view. (Open the **View** menu, select **Lower Pane View**, and choose **DLLs**.) Note that this is more than just a list of DLLs—it represents all memory-mapped files in the process address space. Some of these are DLLs, one is the image file (EXE) being run, and additional entries might represent memory-mapped data files.

[illegible]

Protecting memory

- All system-wide data structures and memory pools used by kernel-mode system components can be accessed only while in kernel mode. User-mode threads can't access these pages. If they attempt to do so, the hardware generates a fault, which the memory manager reports to the thread as an access violation.
- Each process has a separate, private address space, protected from access by any thread belonging to another process. Even shared memory is not really an exception to this because each process accesses the shared regions using addresses that are part of its own virtual address space. The only exception is if another process has virtual memory read or write access to the process object (or holds `SeDebugPrivilege`) and thus can use the `ReadProcessMemory` or `WriteProcessMemory` function. Each time a thread references an address, the virtual memory hardware, in concert with the memory manager, intervenes and translates the virtual address into a physical one. By controlling how virtual addresses are translated, Windows can ensure that threads running in one process don't inappropriately access a page belonging to another process.
- In addition to the implicit protection offered by virtual-to-physical address translation, all processors supported by Windows provide some form of hardware-controlled memory protection

such as read/write, read-only, and so on. (The exact details of such protection vary according to the processor.) For example, code pages in the address space of a process are marked read-only and are thus protected from modification by user threads. [Table 5-2](#) lists the memory-protection options defined in the Windows API. (See the documentation for the `VirtualProtect`, `VirtualProtectEx`, `VirtualQuery`, and `VirtualQueryEx` functions.)

Attribute	Description
<code>PAGE_NOACCESS</code>	Any attempt to read from, write to, or execute code in this region causes an access violation.
<code>PAGE_READONLY</code>	Any attempt to write to (and on processors with no execute support, execute code in) memory causes an access violation, but reads are permitted.
<code>PAGE_READWRITE</code>	The page is readable and writable but not executable.
<code>PAGE_EXECUTE</code>	Any attempt to write to code in memory in this region causes an access violation, but execution (and read operations on all existing processors) is permitted.
<code>PAGE_EXECUTE_READ</code> [*]	Any attempt to write to memory in this region causes an access violation, but executes and reads are permitted.
<code>PAGE_EXECUTE_READWRITE</code> [*]	The page is readable, writable, and executable. Any attempted access will succeed.
<code>PAGE_WRITECOPY</code>	Any attempt to write to memory in this region causes the system to give the process a private copy of the page. On processors with no-execute support, attempts to execute code in memory in this region cause an access violation.
<code>PAGE_EXECUTE_WRITECOPY</code>	Any attempt to write to memory in this region causes the system to give the process a private copy of the page. Reading and executing code in this region is permitted. (No copy is made in this case.)
<code>PAGE_GUARD</code>	Any attempt to read from or write to a guard page raises an <code>EXCEPTION_GUARD_PAGE</code> exception and turns off the guard page status. Guard pages thus act as a one-shot alarm. Note that this flag can be specified with any of the page protections listed in this table except <code>PAGE_NOACCESS</code> .
<code>PAGE_NOCACHE</code>	This uses physical memory that is not cached. This is not recommended for general usage. It is useful for device drivers—for example, mapping a video frame buffer with no caching.
<code>PAGE_WRITECOMBINE</code>	This enables write-combined memory accesses. When enabled, the processor does not cache memory writes (possibly causing significantly more memory traffic than if memory writes were cached), but it does try to aggregate write requests to optimize performance. For example, if multiple writes are made to the same address, only the most recent write might occur. Separate writes to adjacent addresses may be similarly collapsed into a single large write. This is not typically used for general applications, but it is useful for device drivers—for example, mapping a video frame buffer as write combined.
<code>PAGE_TARGETS_INVALID</code> and <code>PAGE_TARGETS_NO_UPDATE</code> (Windows 10 and Windows Server 2016)	These values control behavior of Control Flow Guard (CFG) for executable code in these pages. Both constants have the same value but are used in different calls, essentially acting as a toggle. <code>PAGE_TARGETS_INVALID</code> indicates indirect calls should fail CFG and crash the process. <code>PAGE_TARGETS_NO_UPDATE</code> allows <code>VirtualProtect</code> calls that change the page range to allow execution to not update CFG state. See Chapter 7, “Security,” for more information on CFG.

^{*}No execute protection is supported on processors that have the necessary hardware support (for example, all x64 processors) but not in older x86 processors. If unsupported, “execute” translates to “read.”

TABLE 5-2 Memory-protection options defined in the Windows API

- Shared memory section objects have standard Windows access control lists (ACLs) that are checked when processes attempt to open them, thus limiting access of shared memory to those processes with the proper rights. Access control also comes into play when a thread creates a section to contain a mapped file. To create the section, the thread must have at least read access to the underlying file object or the operation will fail.

Once a thread has successfully opened a handle to a section, its actions are still subject to the memory manager and the hardware-based page protections described earlier. A thread can change the page-level protection on virtual pages in a section if the change doesn’t violate the permissions in the ACL for that section object. For example, the memory manager allows a thread to change the pages of a read-only section to have copy-on-write access but not to have read/write access. The copy-on-write access is permitted because it has no effect on other processes sharing the data.

Data Execution Prevention

Data Execution Prevention (DEP), or no-execute (NX) page protection, causes an attempt to transfer control to an instruction in a page marked as “no execute” to generate an access fault. This can prevent certain types of malware from exploiting bugs in the system through the execution of code placed in a data page such as the stack. DEP can also catch poorly written programs that don’t correctly set permissions on pages from which they intend to execute code. If an attempt is made in kernel mode to execute code in a page marked as “no execute,” the system will crash with the bug check code `ATTEMPTED_EXECUTE_OF_NOEXECUTE_MEMORY (0xFC)`. (See Chapter 15, “Crash dump analysis,” in Part 2 for an explanation of these codes.) If this occurs in user mode, a `STATUS_ACCESS_VIOLATION (0xC0000005)` exception is delivered to the thread attempting the illegal reference. If a process allocates memory that needs to be executable, it must explicitly mark such pages by specifying the `PAGE_EXECUTE`, `PAGE_EXECUTE_READ`, `PAGE_EXECUTE_READWRITE`, or `PAGE_EXECUTE_WRITECOPY` flags on the page-granularity memory-allocation functions.

On 32-bit x86 systems that support DEP, bit 63 in the page table entry (PTE) is used to mark a page as non-executable. Therefore, the DEP feature is available only when the processor is running in Physical Address Extension (PAE) mode, without which page table entries are only 32 bits wide. (See the section “x86 virtual address translation” later in this chapter.) Thus, support for hardware DEP on 32-bit systems requires loading the PAE kernel (`%SystemRoot%\System32\Ntkrnlpa.exe`), which currently is the only supported kernel on x86 systems.

On ARM systems, DEP is set to `AlwaysOn`.

On 64-bit versions of Windows, execution protection is always applied to all 64-bit processes and device drivers and can be disabled only by setting the `nx BCD` option to `AlwaysOff`. Execution protection for 32-bit programs depends on system configuration settings, described shortly. On 64-bit Windows, execution protection is applied to thread stacks (both user and kernel mode), user-mode pages not specifically marked as executable, the kernel paged pool, and the kernel session pool. For a description of kernel memory pools, see the section “[Kernel-mode heaps \(system memory pools\)](#).” However, on 32-bit Windows, execution protection is applied only to thread stacks and user-mode pages, not to the paged pool and session pool.

The application of execution protection for 32-bit processes depends on the value of the `BCD nx` option. To change the settings, open the **Data Execution Prevention** tab in the Performance Options dialog box (see [Figure 5-3](#)). (To open this dialog box, right-click **Computer**, select **Properties**, click **Advanced System Settings**, and choose **Performance Settings**.) When you configure no-execute protection in the Performance Options dialog box, the `BCD nx` option is set to the appropriate value. [Table 5-3](#) lists the variations of the values and how they correspond to the Data Execution Prevention tab. The registry lists 32-bit applications that are excluded from execution protection under the `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Layers` key, with the value name being the full path of the executable and the data set to `DisableNXShowUI`.

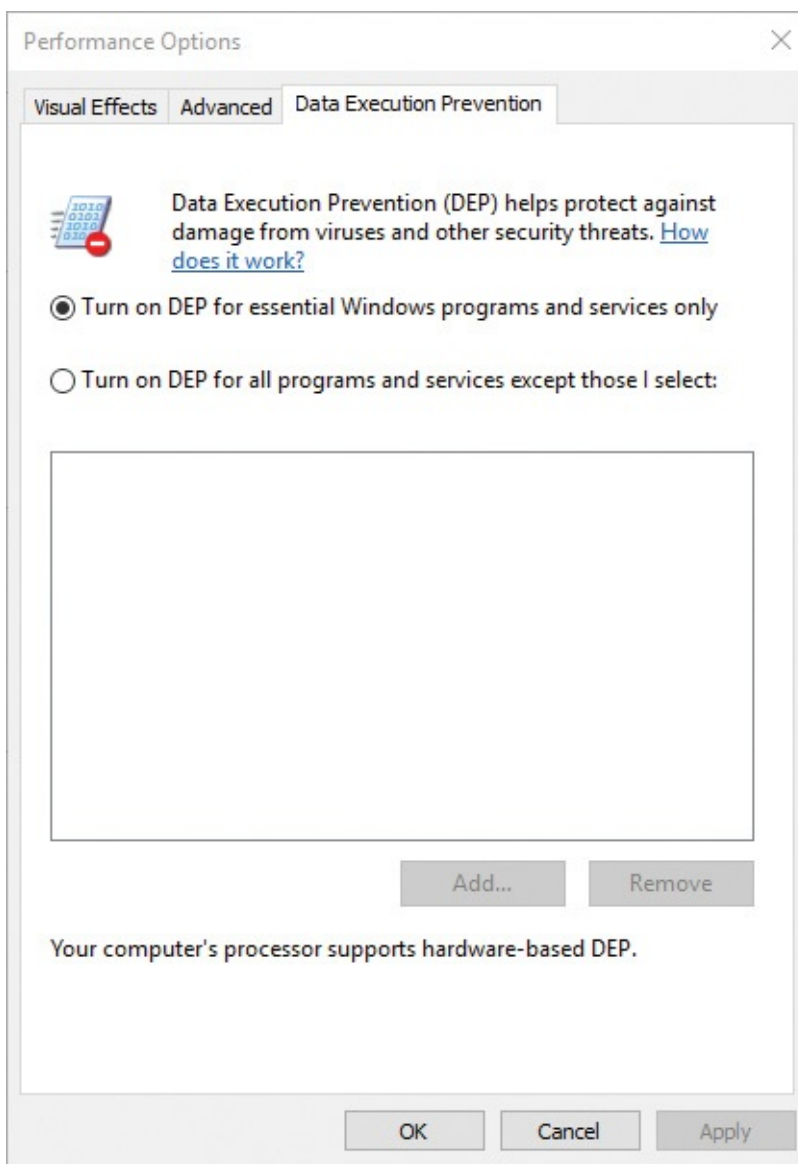


FIGURE 5-3 Data Execution Prevention tab settings.

BCD nx Value	Option on Data Execution Prevention Tab	Explanation
OptIn	Turn on DEP for Essential Windows Programs and Services Only	This enables DEP for core Windows system images. It enables 32-bit processes to dynamically configure DEP for their lifetime.
OptOut	Turn on DEP for All Programs and Services Except Those I Select	This enables DEP for all executables except those specified. It enables 32-bit processes to dynamically configure DEP for their lifetime. It also enables system compatibility fixes for DEP.
AlwaysOn	There is no dialog box option for this setting	This enables DEP for all components with no ability to exclude certain applications. It disables dynamic configuration for 32-bit processes and disables system compatibility fixes.
AlwaysOff	There is no dialog box option for this setting	This disables DEP (not recommended). It also disables dynamic configuration for 32-bit processes.

TABLE 5-3 BCD nx values

On Windows client versions (both 64-bit and 32-bit), execution protection for 32-bit processes is configured by default to apply only to core Windows operating system executables. That is, the nx BCD option is set to **OptIn**. This is to avoid breaking 32-bit applications that might rely on being able to execute code in pages not specifically marked as executable, such as self-extracting or packed applications. On Windows server systems, execution protection for 32-bit applications is configured by default to apply to all 32-bit programs. That is, the nx BCD option is set to **OptOut**.

Even if you force DEP to be enabled, there are still other methods through which applications can

disable DEP for their own images. For example, regardless of which execution-protection options are enabled, the image loader will verify the signature of the executable against known copy-protection mechanisms (such as SafeDisc and SecuROM) and disable execution protection to provide compatibility with older copy-protected software such as computer games. (See [Chapter 3](#) for more information about the image loader.)

EXPERIMENT: Looking at DEP protection on processes

Process Explorer can show you the current DEP status for all the processes on your system, including whether the process is opted in or benefiting from permanent protection. To look at the DEP status for processes, right-click any column in the process tree, choose **Select Columns**, and then select **DEP Status** on the **Process Image** tab. There are three possible values:

- **DEP (permanent)** This means the process has enabled DEP because it is a “necessary Windows program or service.”
 - **DEP** This means the process opted in to DEP. This may be due to a system-wide policy to opt in to all 32-bit processes, because of an API call such as `SetProcessDEPPolicy`, or because the `/NXCOMPAT` linker flag was set when the image was built.
 - **Nothing** If the column displays no information for this process, DEP is disabled because of either a system-wide policy or an explicit API call or shim.
-

Additionally, to provide compatibility with older versions of the Active Template Library (ATL) framework (version 7.1 or earlier), the Windows kernel provides an ATL thunk emulation environment. This environment detects ATL thunk code sequences that have caused the DEP exception and emulates the expected operation. Application developers can request that ATL thunk emulation not be applied by using the latest Microsoft C++ compiler and specifying the `/NXCOMPAT` flag (which sets the `IMAGE_DLLCHARACTERISTICS_NX_COMPAT` flag in the PE header), which tells the system that the executable fully supports DEP. Note that ATL thunk emulation is permanently disabled if the `AlwaysOn` value is set.

Finally, if the system is in `OptIn` or `OptOut` mode and executing a 32-bit process, the `SetProcessDEPPolicy` function allows a process to dynamically disable DEP or to permanently enable it. When it is enabled through this API, DEP cannot be disabled programmatically for the lifetime of the process. This function can also be used to dynamically disable ATL thunk emulation if the image wasn’t compiled with the `/NXCOMPAT` flag. On 64-bit processes or systems booted with `AlwaysOff` or `AlwaysOn`, the function always returns a failure. The `GetProcessDEPPolicy` function returns the 32-bit per-process DEP policy (it fails on 64-bit systems, where the policy is always the same—enabled), while `GetSystemDEPPolicy` can be used to return a value corresponding to the policies in [Table 5-3](#).

Copy-on-write

Copy-on-write page protection is an optimization the memory manager uses to conserve physical memory. When a process maps a copy-on-write view of a section object that contains read/write pages, the memory manager delays the copying of pages until the page is written to instead of making a process private copy at the time the view is mapped. For example, in [Figure 5-4](#), two processes are sharing three pages, each marked copy-on-write, but neither of the two processes has attempted to modify any data on the pages.

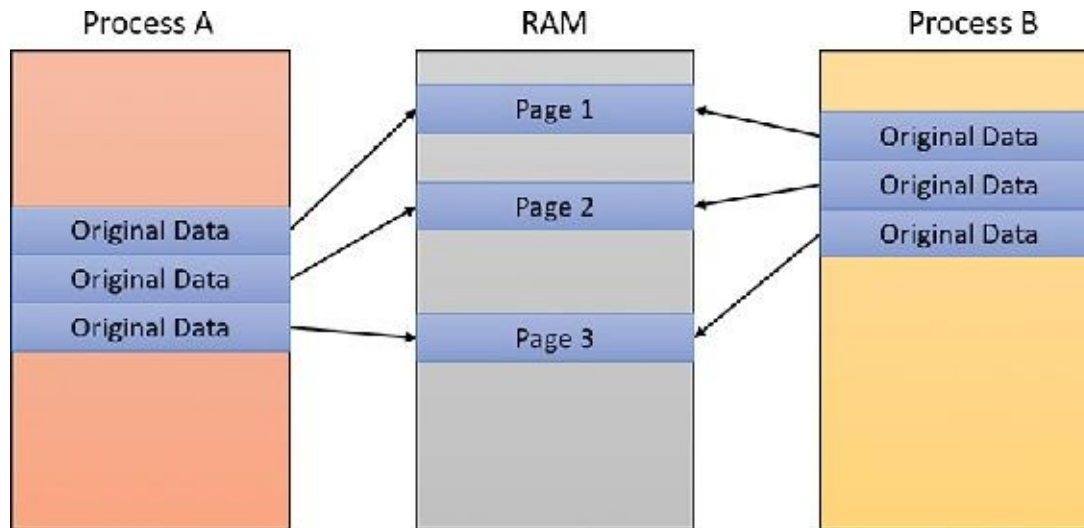


FIGURE 5-4 The “before” of copy-on-write.

If a thread in either process writes to a page, a memory-management fault is generated. The memory manager sees that the write is to a copy-on-write page, so instead of reporting the fault as an access violation, it does the following:

1. It allocates a new read/write page in physical memory.
2. It copies the contents of the original page to the new page.
3. It updates the corresponding page-mapping information (explained later in this chapter) in this process to point to the new location.
4. It dismisses the exception, causing the instruction that generated the fault to be re-executed.

This time, the write operation succeeds. However, as shown in [Figure 5-5](#), the newly copied page is now private to the process that did the writing and isn’t visible to the other process still sharing the copy-on-write page. Each new process that writes to that same shared page will also get its own private copy.

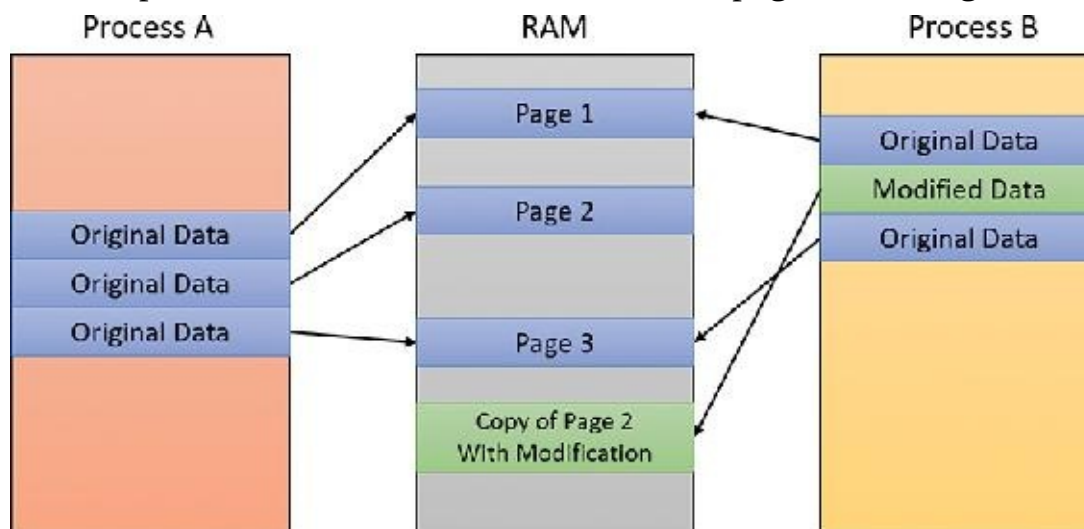


FIGURE 5-5 The “after” of copy-on-write.

One application of copy-on-write is to implement breakpoint support in debuggers. For example, by default, code pages start out as execute-only. If a programmer sets a breakpoint while debugging a program, however, the debugger must add a breakpoint instruction to the code. It does this by first changing the protection on the page to `PAGE_EXECUTE_READWRITE` and then changing the instruction stream. Because the code page is part of a mapped section, the memory manager creates a private copy for the process with the breakpoint set, while other processes continue using the unmodified code page.

Copy-on-write is one example of an evaluation technique called *lazy evaluation* that the memory manager uses as often as possible. Lazy-evaluation algorithms avoid performing an expensive operation until absolutely required. If the operation is never required, no time is wasted on it.

To examine the rate of copy-on-write faults, see the Write Copies/Sec performance counter in the Memory category of the Performance Monitor tool.

Address Windowing Extensions

Although the 32-bit version of Windows can support up to 64 GB of physical memory (refer to [Table 2-2](#)), each 32-bit user process has only a 2 GB virtual address space by default. (You can configure this to up to 3 GB when using the `increaseuserva` BCD option, described in the upcoming section “[Virtual address space layouts](#).”) An application that needs to make more than 2 GB (or 3 GB) of data easily available in a single process could do so via file mapping, remapping a part of its address space into various portions of a large file. However, significant paging would be involved upon each remap.

For higher performance (and more fine-grained control), Windows provides a set of functions called *Address Windowing Extensions (AWE)*. These functions allow a process to allocate more physical memory than can be represented in its virtual address space. It then can access the physical memory by mapping a portion of its virtual address space into selected portions of the physical memory at various times.

You allocate and use memory via the AWE functions in three steps:

1. You allocate the physical memory to be used. The application uses the Windows functions `AllocateUserPhysicalPages` or `AllocateUserPhysicalPagesNuma`. (These require the `SeLockMemoryPrivilege`.)
2. You create one or more regions of virtual address space to act as windows to map views of the physical memory. The application uses the Win32 `VirtualAlloc`, `VirtualAllocEx`, or `Virtual-AllocExNuma` function with the `MEM_PHYSICAL` flag.
3. Steps 1 and 2 are, generally speaking, initialization steps. To actually use the memory, the application uses `MapUserPhysicalPages` or `MapUserPhysicalPagesScatter` to map a portion of the physical region allocated in step 1 into one of the virtual regions, or windows, allocated in step 2.

[Figure 5-6](#) shows an example. The application has created a 256 MB window in its address space and has allocated 4 GB of physical memory. It can then use `MapUserPhysicalPages` or `MapUserPhysical-PageScatter` to access any portion of the physical memory by mapping the desired portion of memory into the 256 MB window. The size of the application’s virtual address space window determines the amount of physical memory the application can access with any given mapping. To access another portion of the allocated RAM, the application can simply remap the area.

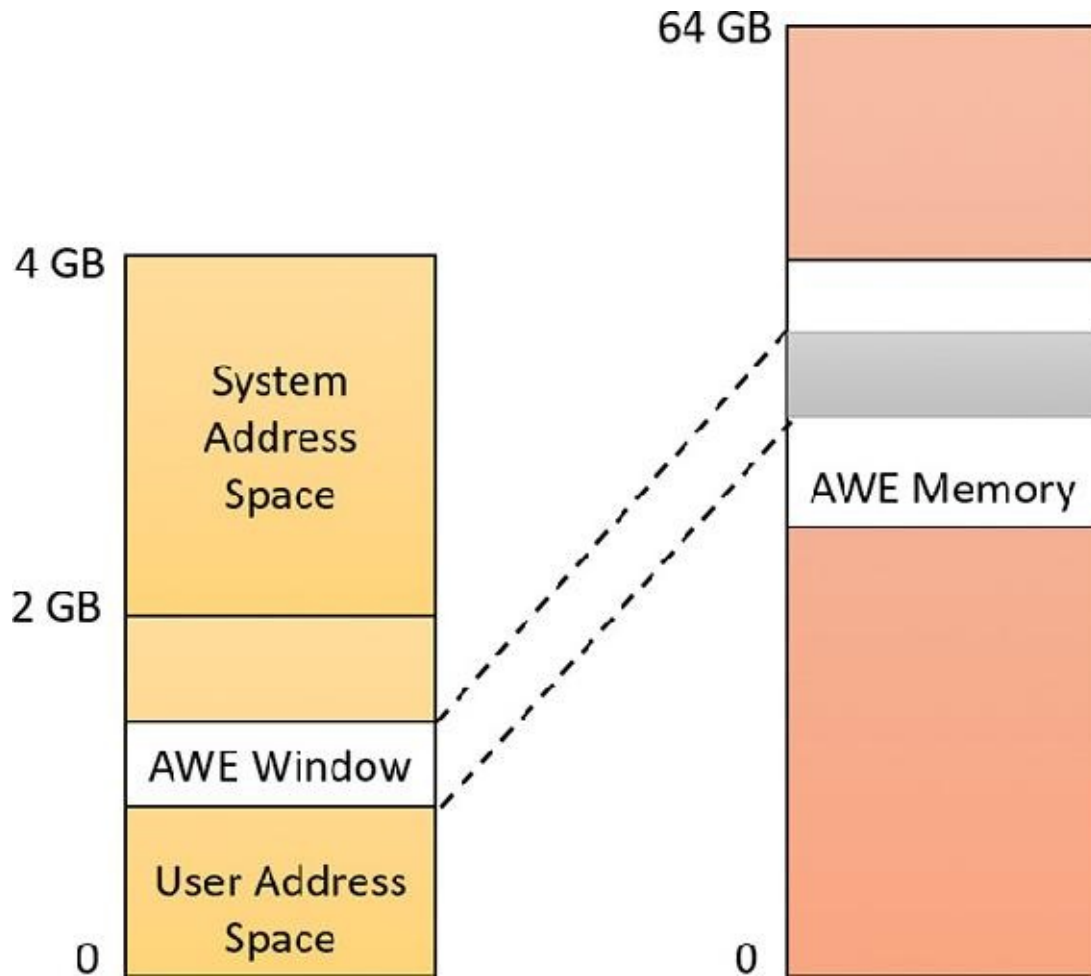


FIGURE 5-6 Using AWE to map physical memory.

The AWE functions exist on all editions of Windows and are usable regardless of how much physical memory a system has. However, AWE is most useful on 32-bit systems with more than 2 GB of physical memory because it provides a way for a 32-bit process to access more RAM than its virtual address space would otherwise allow. Another use is for security purposes. Because AWE memory is never paged out, the data in AWE memory can never have a copy in the paging file that someone could examine by rebooting into an alternate operating system. (`VirtualLock` provides the same guarantee for pages in general.)

Finally, there are some restrictions on memory allocated and mapped by the AWE functions:

- Pages can't be shared between processes.
- The same physical page can't be mapped to more than one virtual address.
- Page protection is limited to read/write, read-only, and no access.

AWE is less useful on 64 bit Windows systems because these systems support 128 TB of virtual address space per process, while allowing a maximum of only 24 TB of RAM (on Windows Server 2016 systems). Therefore, AWE is not necessary to allow an application to use more RAM than it has virtual address space; the amount of RAM on the system will always be smaller than the process virtual address space. AWE remains useful, however, for setting up non-pageable regions of a process address space. It provides finer granularity than the file-mapping APIs. (The system page size is 4 KB rather than 64 KB.)

For a description of the page table data structures used to map memory on systems with more than 4 GB of physical memory, see the section “x86 virtual address translation.”

Kernel-mode heaps (system memory pools)

At system initialization, the memory manager creates two dynamically sized memory pools, or heaps, that most kernel-mode components use to allocate system memory:

- **Non-paged pool** This consists of ranges of system virtual addresses that are guaranteed to reside in physical memory at all times. Thus, they can be accessed at any time without incurring a page fault—meaning they can be accessed from any IRQL. One of the reasons a non-paged pool is required is because page faults can’t be satisfied at DPC/dispatch level or above. Therefore, any code and data that might execute or be accessed at or above DPC/dispatch level must be in non-pageable memory.
- **Paged pool** This is a region of virtual memory in system space that can be paged into and out of the system. Device drivers that don’t need to access the memory from DPC/dispatch level or above can use paged pool. It is accessible from any process context.

Both memory pools are in the system part of the address space and are mapped in the virtual address space of every process. The executive provides routines to allocate and deallocate from these pools. For information on these routines, see the functions that start with `ExAllocatePool`, `ExAllocatePoolWithTag`, and `ExFreePool` in the Windows Development Kit (WDK) documentation.

Systems start with four paged pools, which are combined to make the overall system paged pool, and two non-paged pools. More are created—as many as 64—depending on the number of NUMA nodes on the system. Having more than one paged pool reduces the frequency of system code blocking on simultaneous calls to pool routines. Additionally, the different pools created are mapped across different virtual address ranges that correspond to different NUMA nodes on the system. The different data structures, such as the large page look-aside lists, to describe pool allocations are also mapped across different NUMA nodes.

In addition to the paged and non-paged pools, there are a few other pools with special attributes or uses. For example, there is a pool region in session space that is used for data that is common to all processes in the session. Allocations from another pool, called *special pool*, are surrounded by pages marked as “no access” to help isolate problems in code that accesses memory before or after the region of pool it allocated.

Pool sizes

A non-paged pool starts at an initial size based on the amount of physical memory on the system and then grows as needed. For a non-paged pool, the initial size is 3 percent of system RAM. If this is less than 40 MB, the system will instead use 40 MB as long as 10 percent of RAM results in more than 40 MB. Otherwise, 10 percent of RAM is chosen as a minimum. Windows dynamically chooses the maximum size of the pools and allows a given pool to grow from its initial size to the maximums shown in [Table 5-4](#).

Pool Type	Maximum on 32-Bit Systems	Maximum on 64 bit Systems (Windows 8, Server 2012)	Maximum on 64-Bit Systems (Windows 8.1, 10, Server 2012 R2, 2016)
Non-paged	75 percent of physical memory or 2 GB, whichever is smaller	75 percent of physical memory or 128 GB, whichever is smaller	16 TB
Paged	2 GB	384 GB	15.5 TB

TABLE 5-4 Maximum pool sizes

Four of these computed sizes are stored in kernel variables in Windows 8.x and Server 2012/R2. Three of these are exposed as performance counters and one is computed only as a performance counter value. Windows 10 and Server 2016 moved the global variables into fields in a global memory management

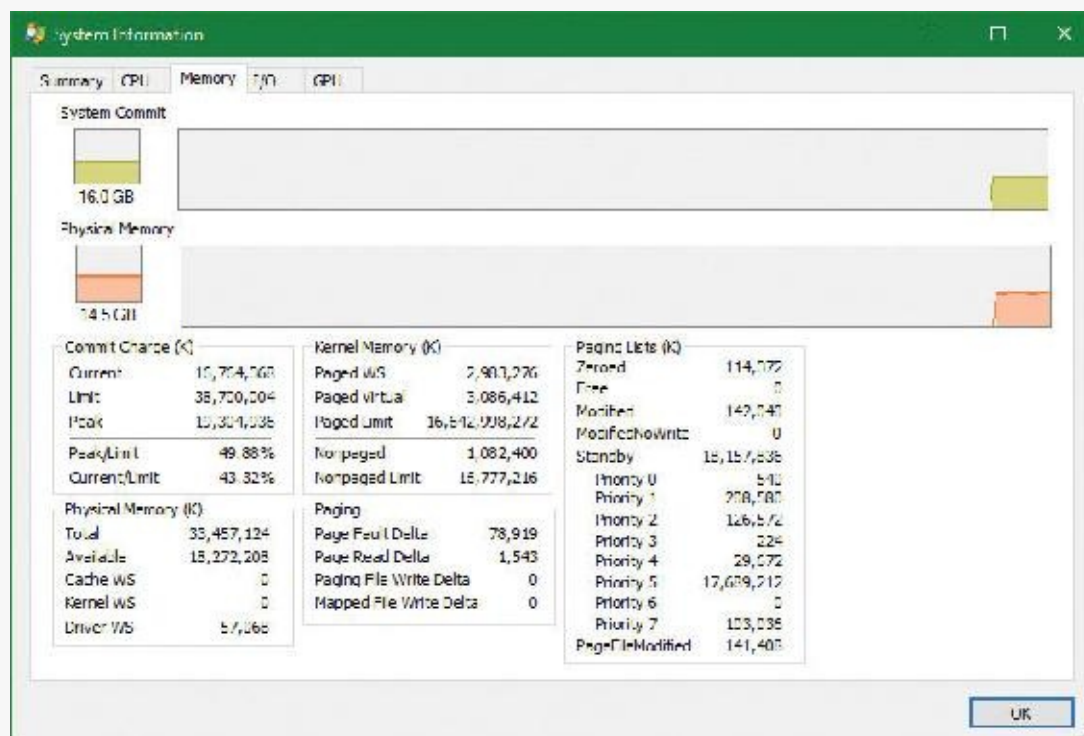
structure (MI_SYSTEM_INFORMATION) named MiState. Within this lies a variable named Vs (of type _MI_VISIBLE_STATE) where this information resides. The global variable MiVisibleState also points to that VS member. These variables and counters are listed in [Table 5-5](#).

Kernel Variable	Performance Counter	Description
MmSizeOfNonPagedPoolInBytes	Memory: Pool non-paged bytes	This is the size of the initial non-paged pool. It can be reduced or enlarged automatically by the system if memory demands dictate. The kernel variable will not show these changes, but the performance counter will.
MmMaximumNonPagedPoolInBytes (Windows 8.x and Server 2012/R2)	Not available	This is the maximum size of a non-paged pool.
MiVisibleState->MaximumNonPagePoolInBytes (Windows 10 and Server 2016)	Not available	This is the maximum size of a non-paged pool.
Not available	Memory: Pool paged bytes	This is the current total virtual size of paged pool.
WorkingSetSize (number of pages) in the MmPagedPoolWs struct (type MMSUPPORT) (Windows 8.x and Server 2012/R2)	Memory: Pool paged resident bytes	This is the current physical (resident) size of paged pool.
MmSizeOfPagedPoolInBytes (Windows 8.x and Server 2012/R2)	Not available	This is the maximum (virtual) size of a paged pool.
MiState.Vs.SizeOfPagedPoolInBytes (Windows 10 and Server 2016)	Not available	This is the maximum (virtual) size of a paged pool.

TABLE 5-5 System pool size variables and performance counters

EXPERIMENT: Determining the maximum pool sizes

You can obtain the pool maximums by using either Process Explorer or live kernel debugging (explained in [Chapter 1](#)). To view pool maximums with Process Explorer, select the **View** menu, choose **System Information**, and then click the **Memory** tab. The pool limits are displayed in the Kernel Memory section, as shown here:



Note

For Process Explorer to retrieve this information, it must have access to the symbols for the kernel running on your system. For a description of how to configure Process Explorer to use symbols, see the experiment “[Viewing process details with Process Explorer](#)” in [Chapter 1](#).

To view the same information by using the kernel debugger, you can use the `!vm` command as was shown previously in this chapter.

Monitoring pool usage

The Memory performance counter object has separate counters for the non-paged pool and paged pool (both virtual and physical). In addition, the Poolmon utility (in the WDK Tools directory) allows you to monitor the detailed usage of non-paged and paged pool. When you run Poolmon, you should see a display like the one shown in [Figure 5-7](#).

FIGURE 5-7 Poolmon output.

Any highlighted lines you might see represent changes to the display. (You can disable the highlighting feature by typing / while running Poolmon; type / again to re-enable highlighting.) Type ? while Poolmon is running to bring up its help screen. You can configure which pools you want to monitor (paged, non-paged, or both) and the sort order. For example, by pressing the **P** key until only non-paged allocations are shown, and then the **D** key to sort by the Diff (differences) column, you can find out what kind of structures are most numerous in non-paged pool. Also, the command-line options are shown, which allow you to monitor specific tags (or every tag but one tag). For example, the command `poolmon -iCM` will monitor only CM tags (allocations from the configuration manager, which manages the registry). The columns have the meanings shown in [Table 5-6](#).

Column	Explanation
Tag	This is a four-byte tag given to the pool allocation.
Type	This is the pool type (paged or non-paged).
Allocs	This is a count of all allocations. The number in parentheses shows the difference in the Allocs column since the last update.
Frees	This is the count of all frees. The number in parentheses shows the difference in the Frees column since the last update.
Diff	This is the count of allocations minus frees.
Bytes	This is the total bytes consumed by this tag. The number in parentheses shows the difference in the Bytes column since the last update.
Per Alloc	This is the size in bytes of a single instance of this tag.

TABLE 5-6 Poolmon columns

For a description of the meaning of the pool tags used by Windows, see the Pooltag.txt file in the Triage subdirectory where the Debugging tools for Windows are located. Because third-party device-driver pool tags are not listed in this file, you can use the -C switch on the 32-bit version of Poolmon that comes with the WDK to generate a local pool tag file (Localtag.txt). This file will contain pool tags used by drivers found on your system, including third-party drivers. (Note that if a device-driver binary has been deleted after it was loaded, its pool tags will not be recognized.)

Alternatively, you can search the device drivers on your system for a pool tag by using the Strings.exe tool from Sysinternals. For example, the following command displays drivers that contain the string "abcd":

[Click here to view code image](#)

strings %SYSTEMROOT%\system32\drivers*.sys | findstr /i "abcd"

Device drivers do not necessarily have to be located in %SystemRoot%\System32\Drivers. They can be in any folder. To list the full path of all loaded drivers, follow these steps:

- 1. Open the **Start** menu and type **Msiinfo32** (System Information should appear).
- 2. Run System Information.
- 3. Select **Software Environment**.
- 4. Choose **System Drivers**. If a device driver has been loaded and then deleted from the system, it will not be listed here.

An alternative way to view pool usage by device driver is to enable the pool-tracking feature of Driver Verifier, explained in [Chapter 6](#). While this makes the mapping from pool tag to device driver unnecessary, it does require a reboot (to enable Driver Verifier on the desired drivers). After rebooting with pool tracking enabled, you can either run the graphical Driver Verifier Manager (%SystemRoot%\System32\Verifier.exe) or use the `Verifier /Log` command to send the pool-usage information to a file.

Finally, you can view pool usage with the kernel debugger `!poolused` command. The `!poolused 2` command shows non-paged pool usage sorted by pool tag using the most amount of pool. The `!poolused 4` command lists paged-pool usage, again sorted by pool tag using the most amount of pool. The following example shows the partial output from these two commands:

[Click here to view code image](#)

```
lkd> !poolused 2
.....
Sorting by NonPaged Pool Consumed

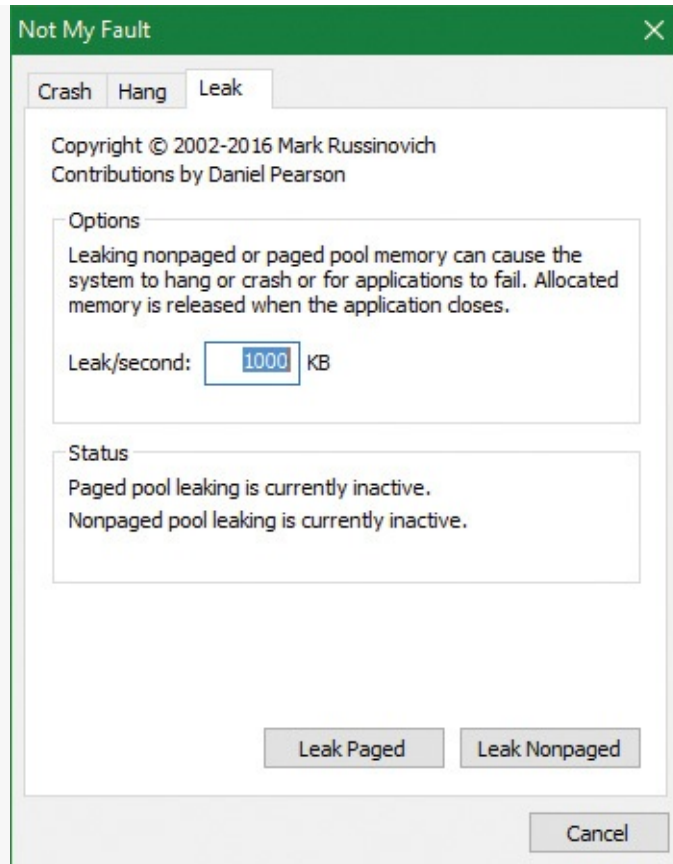
Tag           Allocs      NonPaged      Allocs      Paged      Used
              Used
File          626381      260524032      0           0           0      File
objects
Ntfx          733204      227105872      0           0           0      General
Allocation , Binary:
MmCa          513713      148086336      0           0           0      ntfs.sys
areas for mapped
files ,
Binary: nt!mm
FMSl          732490      140638080      0           0           0      STREAM_LIST
structure ,
Binary:
fltmgr.sys
CcSc          104420      56804480       0           0           0      Cache
Manager Shared Cache Map
, Binary:
nt!cc
SQSF          283749      45409984       0           0           0      UNKNOWN
```

pooltag 'SQSF', please						update
pooltag.txt						
FMfz	382318	42819616	0	0		FILE_LIST_C
structure ,						Binary:
fltmgr.sys						
FMsc	36130	32950560	0	0		SECTION_COM
structure ,						Binary:
fltmgr.sys						
EtwB	517	31297568	107	105119744		Etw Buffer
, Binary: nt!etw						
DFmF	382318	30585440	382318	91756320		UNKNOWN
pooltag 'DFmF', please						update
pooltag.txt						
DFmE	382318	18351264	0	0		UNKNOWN
pooltag 'DFmE', please						update
pooltag.txt						
FSfc	382318	18351264	0	0		Unrecognize
File System Run						Time
allocations (update						pooltag.w)
, Binary: nt!fsrtl						
smNp	4295	17592320	0	0		ReadyBoost
store node pool						allocations
, Binary: nt!store						or
rdyboost.sys						
Thre	5780	12837376	0	0		Thread
objects , Binary: nt!ps						
Pool	8	12834368	0	0		Pool
tables, etc.						

EXPERIMENT: Troubleshooting a pool leak

In this experiment, you will fix a real paged pool leak on your system so that you can use the techniques described in the previous section to track down the leak. The leak will be generated by the Notmyfault tool from Sysinternals. Follow these steps:

1. Run Notmyfault.exe for your OS bitness (for example, the 64 bit on a 64-bit system).
2. Notmyfault.exe loads the Myfault.sys device driver and presents a Not My Fault dialog box with the Crash tab selected. Click the **Leak** tab. It should look something like this:



3. Ensure that the **Leak/Second** setting is set to 1000 KB.
4. Click the **Leak Paged** button. This causes Notmyfault to begin sending requests to the Myfault device driver to allocate paged pool. Notmyfault will continue sending requests until you click the **Stop Paged** button. Paged pool is not normally released even when you close a program that has caused it to occur (by interacting with a buggy device driver). The pool is permanently leaked until you reboot the system. However, to make testing easier, the Myfault device driver detects that the process was closed and frees its allocations.
5. While the pool is leaking, open Task Manager, click the **Performance** tab, and select the **Memory** label. Notice the **Paged Pool** value climbing. You can also check this with Process Explorer's System Information display (select the **View** menu, choose **System Information**, and click the **Memory** tab).
6. To determine which pool tag is leaking, run Poolmon and press the **B** key to sort by the number of bytes.
7. Press **P** twice so that Poolmon shows only paged pool. Notice the Leak pool tag climbing to the top of the list. (Poolmon shows changes to pool allocations by highlighting the lines that change.)
8. Click the **Stop Paged** button so that you don't exhaust paged pool on your system.
9. Using the technique described in the previous section, run Strings (from Sysinternals) to look for driver binaries that contain the Leak pool tag. This should display a match on the file Myfault.sys, thus confirming it as the driver using the Leak pool tag.

[Click here to view code image](#)

```
Strings %SystemRoot%\system32\drivers\*.sys | findstr Leak
```

Look-aside lists

Windows provides a fast memory-allocation mechanism called *look-aside lists*. The basic difference between pools and look-aside lists is that while general pool allocations can vary in size, a look-aside list contains only fixed-sized blocks. Although the general pools are more flexible in terms of what they can supply, look-aside lists are faster because they don't use any spinlocks.

Executive components and device drivers can create look-aside lists that match the size of frequently allocated data structures by using the `ExInitializeNPagedLookasideList` (for non-paged allocations) and `ExInitializePagedLookasideList` (for paged allocation) functions, as documented in the WDK. To minimize the overhead of multiprocessor synchronization, several executive subsystems such as the I/O manager, cache manager, and object manager create separate look-aside lists for each processor for their frequently accessed data structures. The executive also creates a general per-processor paged and non-paged look-aside list for small allocations (256 bytes or less).

If a look-aside list is empty (as it is when it's first created), the system must allocate from the paged or non-paged pool. But if it contains a freed block, the allocation can be satisfied very quickly. (The list grows as blocks are returned to it.) The pool-allocation routines automatically tune the number of freed buffers that look-aside lists store according to how often a device driver or executive subsystem allocates from the list. The more frequent the allocations, the more blocks are stored on a list. Look-aside lists are automatically reduced in size if they aren't being allocated from. (This check happens once per second when the balance set manager system thread wakes up and calls the `ExAdjustLook-asideDepth` function.)

EXPERIMENT: Viewing the system look-aside lists

You can display the contents and sizes of the various system look-aside lists with the kernel debugger `!lookaside` command. The following excerpt is from the output of this command:

[Click here to view code image](#)

```
lkd> !lookaside
```

```
Lookaside "nt!CcTwilightLookasideList" @
0xffffffff800c6f54300 Tag(hex): 0x6b576343 "CcWk"
    Type                =          0200 NonPagedPoolNx
    Current Depth        =           0 Max Depth      =           4
    Size                 =          128 Max Alloc      =          512
    AllocateMisses       =       728323 FreeMisses    =       728271
    TotalAllocates       =    1030842 TotalFrees      =    1030766
    Hit Rate             =          29% Hit Rate      =          29%
```

```
Lookaside "nt!IopSmallIrpLookasideList" @
0xffffffff800c6f54500 Tag(hex): 0x73707249 "Irps"
    Type                =          0200 NonPagedPoolNx
    Current Depth        =           0 Max Depth      =           4
    Size                 =          280 Max Alloc      =         1120
    AllocateMisses       =       44683 FreeMisses    =       43576
    TotalAllocates       =     232027 TotalFrees      =     230903
    Hit Rate             =          80% Hit Rate      =          81%
```

```
Lookaside "nt!IopLargeIrpLookasideList" @
0xffffffff800c6f54600 Tag(hex): 0x6c707249 "Irpl"
    Type                =          0200 NonPagedPoolNx
    Current Depth        =           0 Max Depth      =           4
    Size                 =         1216 Max Alloc      =         4864
    AllocateMisses       =     143708 FreeMisses    =     142551
    TotalAllocates       =     317297 TotalFrees      =     316131
    Hit Rate             =          54% Hit Rate      =          54%
```

...

```
Total NonPaged currently allocated for above lists =           0
Total NonPaged potential for above lists           =        13232
Total Paged currently allocated for above lists     =           0
Total Paged potential for above lists               =         4176
```

Heap manager

Most applications allocate smaller blocks than the 64-KB minimum allocation granularity possible using page-granularity functions such as `VirtualAlloc`. Allocating such a large area for relatively small allocations is not optimal from a memory usage and performance standpoint. To address this, Windows provides a component called the *heap manager*, which manages allocations inside larger memory areas reserved using the page-granularity memory-allocation functions. The allocation granularity in the heap manager is relatively small: 8 bytes on 32-bit systems, and 16 bytes on 64-bit systems. The heap manager has been designed to optimize memory usage and performance in the case of these smaller allocations.

The heap manager exists in two places: `Ntdll.dll` and `Ntoskrnl.exe`. The subsystem APIs (such as the Windows heap APIs) call the functions in `Ntdll.dll`, and various executive components and device drivers call the functions in `Ntoskrnl.exe`. Its native interfaces (prefixed with `Rtl`) are available only for use in internal Windows components or kernel-mode device drivers. The documented Windows API interfaces to the heap (prefixed with `Heap`) are forwarders to the native functions in `Ntdll.dll`. In addition, legacy APIs (prefixed with either `Local` or `Global`) are provided to support older Windows applications. These also internally call the heap manager, using some of its specialized interfaces to support legacy behavior. The most common Windows heap functions are:

- **HeapCreate or HeapDestroy** These create or delete, respectively, a heap. The initial reserved and committed size can be specified at creation.
- **HeapAlloc** This allocates a heap block. It is forwarded to **RtlAllocateHeap** in `Ntdll.dll`.
- **HeapFree** This frees a block previously allocated with `HeapAlloc`.
- **HeapReAlloc** This changes the size of an existing allocation, growing or shrinking an existing block. It is forwarded to `RtlReAllocateHeap` in `Ntdll.dll`.
- **HeapLock and HeapUnlock** These control mutual exclusion to heap operations.
- **HeapWalk** This enumerates the entries and regions in a heap.

Process heaps

Each process has at least one heap: the default process heap. The default heap is created at process startup and is never deleted during the process's lifetime. It defaults to 1 MB in size, but you can make it bigger by specifying a starting size in the image file by using the `/HEAP` linker flag. This size is just the initial reserve, however. It will expand automatically as needed. You can also specify the initial committed size in the image file.

The default heap can be explicitly used by a program or implicitly used by some Windows internal functions. An application can query the default process heap by making a call to the Windows `GetProcessHeap` function. Processes can also create additional private heaps with the `HeapCreate` function. When a process no longer needs a private heap, it can recover the virtual address space by calling `HeapDestroy`. An array with all heaps is maintained in each process, and a thread can query them with the Windows `GetProcessHeaps` function.

A Universal Windows Platform (UWP) app process includes at least three heaps:

- The default process heap just described.
- A shared heap used to pass large arguments to the process's session `Csrss.exe` instance. This is created by the `CsrClientConnectToServer` `Ntdll.dll` function, which executes early in the process initialization done by `Ntdll.dll`. The heap handle is available in the global variable `CsrPortHeap` (in `Ntdll.dll`).

- A heap created by the Microsoft C runtime library. Its handle is stored in the global variable `_crthheap` (in the `msvcrt` module). This heap is the one used internally by the C/C++ memory-allocation functions such as `malloc`, `free`, `operator new/delete`, and so on.

A heap can manage allocations either in large memory regions reserved from the memory manager via `VirtualAlloc` or from memory-mapped file objects mapped in the process address space. The latter approach is rarely used in practice (and is not exposed by the Windows API), but it's suitable for scenarios where the content of the blocks needs to be shared between two processes or between a kernel-mode and a user-mode component. The Win32 GUI subsystem driver (Win32k.sys) uses such a heap for sharing GDI and USER objects with user mode. If a heap is built on top of a memory-mapped file region, certain constraints apply with respect to the component that can call heap functions:

- The internal heap structures use pointers, and therefore do not allow remapping to different addresses in other processes.
- The synchronization across multiple processes or between a kernel component and a user process is not supported by the heap functions.
- In the case of a shared heap between user mode and kernel mode, the user-mode mapping should be read-only to prevent user-mode code from corrupting the heap's internal structures, which would result in a system crash. The kernel-mode driver is also responsible for not putting any sensitive data in a shared heap to avoid leaking it to user mode.

Heap types

Until Windows 10 and Server 2016, there was just one heap type, which we'll call the *NT heap*. The NT heap is augmented by an optional front-end layer, which if used, consists of the *low-fragmentation heap (LFH)*.

Windows 10 introduced a new heap type called *segment heap*. The two heap types include common elements but are structured and implemented differently. By default, the segment heap is used by all UWP apps and some system processes, while the NT heap is used by all other processes. This can be changed in the registry as described in the section "[The segment heap](#)" later in this chapter.

The NT heap

As shown in [Figure 5-8](#), the NT heap in user mode is structured in two layers: a front-end layer and the heap back end (sometimes called the heap core). The back end handles the basic functionality and includes the management of blocks inside segments, the management of the segments, policies for extending the heap, committing and decommitting memory, and management of large blocks.

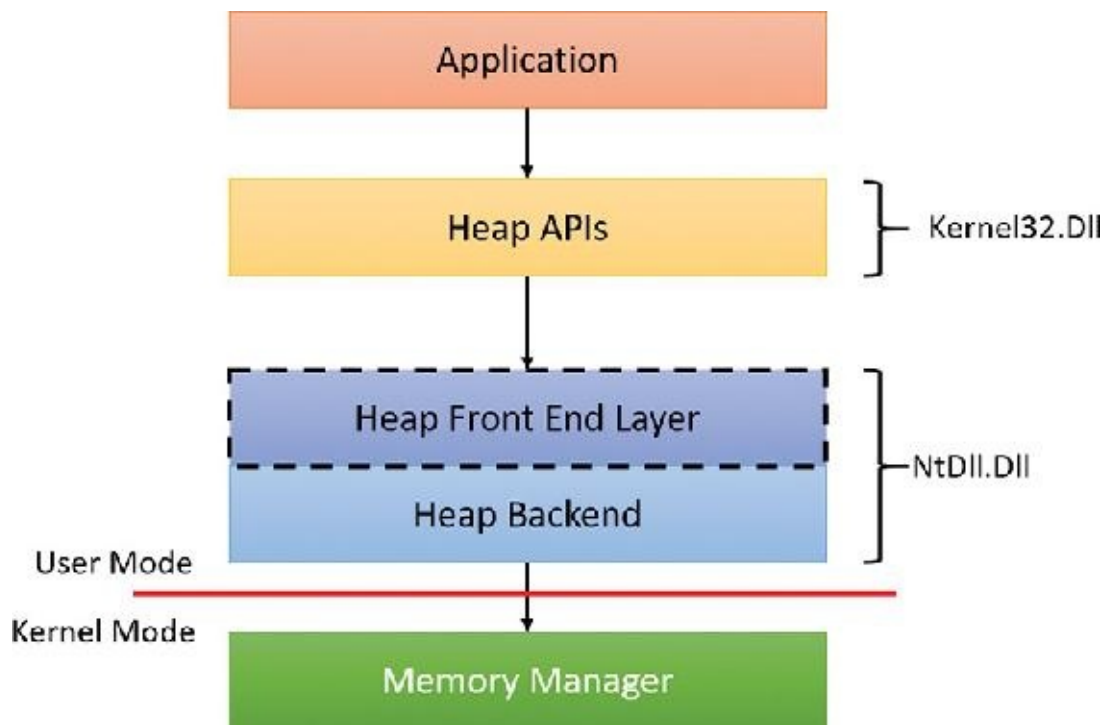


FIGURE 5-8 NT heap layers in user mode.

For user-mode heaps only, a front-end heap layer can exist on top of the core functionality. Windows supports one optional front end layer, the LFH, described in the upcoming section “[The low-fragmentation heap.](#)”

Heap synchronization

The heap manager supports concurrent access from multiple threads by default. However, if a process is single threaded or uses an external mechanism for synchronization, it can tell the heap manager to avoid the overhead of synchronization by specifying the `HEAP_NO_SERIALIZE` flag either at heap creation or on a per-allocation basis. If heap synchronization is enabled, there is one lock per heap that protects all internal heap structures.

A process can also lock the entire heap and prevent other threads from performing heap operations for operations that would require consistent states across multiple heap calls. For instance, enumerating the heap blocks in a heap with the Windows function `HeapWalk` requires locking the heap if multiple threads can perform heap operations simultaneously. Locking and unlocking a heap can be done with the `HeapLock` and `HeapUnlock` functions, respectively.

The low-fragmentation heap

Many applications running in Windows have relatively small heap memory usage—usually less than 1 MB. For this class of applications, the heap manager’s best-fit policy helps keep a low memory footprint for each process. However, this strategy does not scale for large processes and multiprocessor machines. In these cases, memory available for heap usage might be reduced due to heap fragmentation. Performance can suffer in scenarios where only certain sizes are often used concurrently from different threads scheduled to run on different processors. This happens because several processors need to modify the same memory location (for example, the head of the look-aside list for that particular size) at the same time, thus causing significant contention for the corresponding cache line.

The LFH avoids fragmentation by managing allocated blocks in predetermined different block-size ranges called *buckets*. When a process allocates memory from the heap, the LFH chooses the bucket that

maps to the smallest block large enough to hold the required size. (The smallest block is 8 bytes.) The first bucket is used for allocations between 1 and 8 bytes, the second for allocations between 9 and 16 bytes, and so on, until the 32nd bucket, which is used for allocations between 249 and 256 bytes, followed by the 33rd bucket, which is used for allocations between 257 and 272 bytes, and so on. Finally, the 128th bucket, which is the last, is used for allocations between 15,873 and 16,384 bytes. (This is known as a *binary buddy* system.) If the allocation is larger than 16,384 bytes, the LFH simply forwards it to the underlying heap back end. [Table 5-7](#) summarizes the different buckets, their granularity, and the range of sizes they map to.

Buckets	Granularity	Range
1–32	8	1–256
33–48	16	257–512
49–64	32	513–1,024
65–80	64	1,025–2,048
81–96	128	2,049–4,096
97–112	256	4,097–8,192
113–128	512	8,193–16,384

TABLE 5-7 LFH buckets

The LFH addresses these issues by using the core heap manager and look-aside lists. The Windows heap manager implements an automatic tuning algorithm that can enable the LFH by default under certain conditions, such as lock contention or the presence of popular size allocations that have shown better performance with the LFH enabled. For large heaps, a significant percentage of allocations is frequently grouped in a relatively small number of buckets of certain sizes. The allocation strategy used by LFH is to optimize the usage for these patterns by efficiently handling same-size blocks.

To address scalability, the LFH expands the frequently accessed internal structures to a number of slots that is two times larger than the current number of processors on the machine. The assignment of threads to these slots is done by an LFH component called the *affinity manager*. Initially, the LFH starts using the first slot for heap allocations; however, if a contention is detected when accessing some internal data, the LFH switches the current thread to use a different slot. Further contentions will spread threads on more slots. These slots are controlled for each size bucket to improve locality and minimize the overall memory consumption.

Even if the LFH is enabled as a front-end heap, the less frequent allocation sizes may continue to use the core heap functions to allocate memory, while the most popular allocation classes will be performed from the LFH. Once the LFH is enabled for a specific heap, it cannot be disabled. The `HeapSetInformation` API with the `HeapCompatibilityInformation` class that was able to remove the LFH layer in Windows 7 and earlier versions of Windows is now ignored.

The segment heap

[Figure 5-9](#) shows the architecture of the segment heap, introduced in Windows 10.

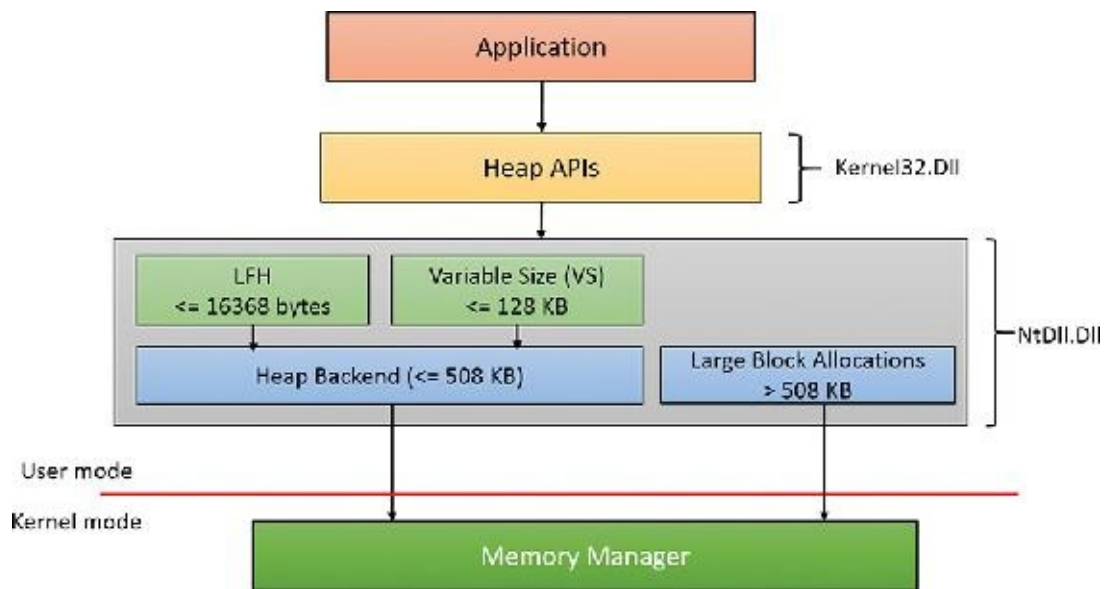


FIGURE 5-9 Segment heap.

The actual layer that manages an allocation depends on the allocation size as follows:

- For small sizes (less than or equal to 16,368 bytes), the LFH allocator is used, but only if the size is determined to be a common one. This is a similar logic to the LFH front layer of the NT heap. If the LFH has not kicked in yet, the variable size (VS) allocator will be used instead.
- For sizes less than or equal to 128 KB (and not serviced by the LFH), the VS allocator is used. Both VS and LFH allocators use the back end to create the required heap sub-segments as necessary.
- Allocations larger than 128 KB and less than or equal to 508 KB are serviced directly by the heap back end.
- Allocations larger than 508 KB are serviced by calling the memory manager directly (**VirtualAlloc**) since these are so large that using the default 64 KB allocation granularity (and rounding to the nearest page size) is deemed good enough.

Here is a quick comparison of the two heap implementations:

- In some scenarios, the segment heap may be somewhat slower than the NT heap. However, it's likely that future Windows versions would make it on par with the NT heap.
- The segment heap has a lower memory footprint for its metadata, making it better suited for low-memory devices such as phones.
- The segment heap's metadata is separated from the actual data, while the NT heap's metadata is interspersed with the data itself. This makes the segment heap more secure, as it's more difficult to get to the metadata of an allocation given just a block address.
- The segment heap can be used only for a growable heap. It cannot be used with a user-supplied memory mapped file. If such a segment heap creation is attempted, an NT heap is created instead.
- Both heaps support LFH-type allocations, but their internal implementation is completely different. The segment heap has a more efficient implementation in terms of memory consumption and performance.

As mentioned, UWP apps use segment heaps by default. This is mainly because of their lower memory footprint, which is suitable for low-memory devices. It's also used with certain system processes based on executable name: `csrss.exe`, `lsass.exe`, `runtimebroker.exe`, `services.exe`, `smss.exe`, and `svchost.exe`.

The segment heap is not the default heap for desktop apps because there are some compatibility

concerns that may affect existing applications. It's likely that in future versions, however, it will become the default. To enable or disable the segment heap for a specific executable, you can set an Image File Execution Options value named `FrontEndHeapDebugOptions` (DWORD):

- Bit 2 (4) to disable segment heap
- Bit 3 (8) to enable segment heap

You can also globally enable or disable the segment heap by adding a value named `Enabled` (DWORD) to the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Segment Heap` registry key. A zero value disables the segment heap and a non-zero value enables it.

EXPERIMENT: Viewing basic heap information

In this experiment, we'll examine some heaps of a UWP process.

1. Using Windows 10, run the Windows calculator. (Click the **Start** button and type **Calculator** to find it.)
2. The calculator in Windows 10 has been turned into a UWP app (`Calculator.Exe`). Run `WinDbg` and attach to the calculator process.
3. Once attached, `WinDbg` breaks into the process. Issue the `!heap` command to get a quick summary of heaps in the process:

[Click here to view code image](#)

```
0:033> !heap
Heap Address      NT/Segment Heap
2531eb90000       Segment Heap
2531e980000       NT Heap
2531eb10000       Segment Heap
25320a40000       Segment Heap
253215a0000       Segment Heap
253214f0000       Segment Heap
2531eb70000       Segment Heap
25326920000       Segment Heap
253215d0000       NT Heap
```

4. Notice the various heaps with their handle and type (segment or NT). The first heap is the default process heap. Because it's growable and not using any preexisting memory block, it's created as a segment heap. The second heap is used with a user-defined memory block (described earlier in the "[Process heaps](#)" section). Because this feature is currently unsupported by the segment heap, it's created as an NT heap.
5. An NT heap is managed by the `NtDll!_HEAP` structure. Let's view this structure for the second heap:

[Click here to view code image](#)

```
0:033> dt ntdll!_heap 2531e980000
+0x000 Segment      : _HEAP_SEGMENT
+0x000 Entry        : _HEAP_ENTRY
+0x010 SegmentSignature : 0xffeefee
+0x014 SegmentFlags  : 1
```

```

+0x018 SegmentListEntry : _LIST_ENTRY [ 0x00000253'1e980120 -
0x00000253'1e980120 ]
+0x028 Heap              : 0x00000253'1e980000 _HEAP
+0x030 BaseAddress       : 0x00000253'1e980000 Void
+0x038 NumberOfPages    : 0x10
+0x040 FirstEntry        : 0x00000253'1e980720 _HEAP_ENTRY
+0x048 LastValidEntry    : 0x00000253'1e990000 _HEAP_ENTRY
+0x050 NumberOfUnCommittedPages : 0xf
+0x054 NumberOfUnCommittedRanges : 1
+0x058 SegmentAllocatorBackTraceIndex : 0
+0x05a Reserved          : 0
+0x060 UCRSegmentList    : _LIST_ENTRY [ 0x00000253'1e980fe0 -
0x00000253'1e980fe0 ]
+0x070 Flags             : 0x8000
+0x074 ForceFlags        : 0
+0x078 CompatibilityFlags : 0
+0x07c EncodeFlagMask    : 0x100000
+0x080 Encoding          : _HEAP_ENTRY
+0x090 Interceptor       : 0
+0x094 VirtualMemoryThreshold : 0xff00
+0x098 Signature         : 0xeeffeeff
+0x0a0 SegmentReserve    : 0x100000
+0x0a8 SegmentCommit     : 0x2000
+0x0b0 DeCommitFreeBlockThreshold : 0x100
+0x0b8 DeCommitTotalFreeThreshold : 0x1000
+0x0c0 TotalFreeSize     : 0x8a
+0x0c8 MaximumAllocationSize : 0x00007fff'ffdeffff
+0x0d0 ProcessHeapsListIndex : 2
...
+0x178 FrontEndHeap      : (null)
+0x180 FrontHeapLockCount : 0
+0x182 FrontEndHeapType   : 0 ''
+0x183 RequestedFrontEndHeapType : 0 ''
+0x188 FrontEndHeapUsageData : (null)
+0x190 FrontEndHeapMaximumIndex : 0
+0x192 FrontEndHeapStatusBitmap : [129] ""
+0x218 Counters           : _HEAP_COUNTERS
+0x290 TuningParameters   : _HEAP_TUNING_PARAMETERS

```

6. Notice the **FrontEndHeap** field. This field indicates whether a front-end layer exists. In the preceding output, it's `null`, meaning there is no front-end layer. A non-`null` value indicates an LFH front-end layer (since it's the only one defined).

7. A segment heap is defined with the `NtDll!_SEGMENT_HEAP` structure. Here's the default process heap:

[Click here to view code image](#)

```
0:033> dt ntdll!_segment_heap 2531eb90000
```

```

+0x000 TotalReservedPages : 0x815
+0x008 TotalCommittedPages : 0x6ac
+0x010 Signature : 0xddeeddee
+0x014 GlobalFlags : 0
+0x018 FreeCommittedPages : 0
+0x020 Interceptor : 0
+0x024 ProcessHeapListIndex : 1
+0x026 GlobalLockCount : 0
+0x028 GlobalLockOwner : 0
+0x030 LargeMetadataLock : _RTL_SRWLOCK
+0x038 LargeAllocMetadata : _RTL_RB_TREE
+0x048 LargeReservedPages : 0
+0x050 LargeCommittedPages : 0
+0x058 SegmentAllocatorLock : _RTL_SRWLOCK
+0x060 SegmentListHead : _LIST_ENTRY [ 0x00000253'1ec00000 -
0x00000253'28a00000 ]
+0x070 SegmentCount : 8
+0x078 FreePageRanges : _RTL_RB_TREE
+0x088 StackTraceInitVar : _RTL_RUN_ONCE
+0x090 ContextExtendLock : _RTL_SRWLOCK
+0x098 AllocatedBase : 0x00000253'1eb93200 ""
+0x0a0 UncommittedBase : 0x00000253'1eb94000 "--- memory
read error at
address 0x00000253'1eb94000 ---"
+0x0a8 ReservedLimit : 0x00000253'1eba5000 "--- memory
read error at
address 0x00000253'1eba5000 ---"
+0x0b0 VsContext : _HEAP_VS_CONTEXT
+0x120 LfhContext : _HEAP_LFH_CONTEXT

```

8. Notice the **Signature** field. It's used to distinguish between the two types of heaps.
9. Notice the **SegmentSignature** field of the **_HEAP** structure. It is in the same offset (0x10). This is how functions such as **RtlAllocateHeap** know which implementation to turn to based on the heap handle (address) alone.
10. Notice the last two fields in the **_SEGMENT_HEAP**. These contain the VS and LFH allocator information.
11. To get more information on each heap, issue the **!heap -s** command:

[Click here to view code image](#)

```
0:033> !heap -s
```

Reserved Heap		Committed	Global	Process Heap	Total	Total
Address	Signature	Flags	List	Bytes Index	Bytes (K)	(K)

2531eb90000	ddeeddee	0	1	8276	6832
2531eb10000	ddeeddee	0	3	1108	868
25320a40000	ddeeddee	0	4	1108	16
253215a0000	ddeeddee	0	5	1108	20
253214f0000	ddeeddee	0	6	3156	816
2531eb70000	ddeeddee	0	7	1108	24
25326920000	ddeeddee	0	8	1108	32

NT HEAP STATS BELOW

LFH Key : 0xd7b666e8f56a4b98

Termination on corruption : ENABLED

Affinity manager status:

- Virtual affinity limit 8
- Current entries in use 0
- Statistics: Swaps=0, Resets=0, Allocs=0

Lock	Heap	Flags	Reserv	Commit	Virt	Free	List	UO
Fast			(k)	(k)	(k)	(k)		
length	blocks							
cont. heap								

000002531e980000								
00008000	64	4	64	2	1	1	0	
0								
00000253215d0000								
00000001	16	16	16	10	1	1	0	
N/A								

12. Notice the first part of the output. It shows extended information on segment heaps (if any).
The second part shows extended information on NT heaps in the process.

The !heap debugger command provides a multitude of options for viewing, investigating, and searching heaps. See the “Debugger Tools for Windows” documentation for more information.

Heap security features

As the heap manager has evolved, it has taken an increased role in early detection of heap usage errors and in mitigating effects of potential heap-based exploits. These measures exist to lessen the security effect of potential vulnerabilities in applications. Both the NT-heap and the segment-heap implementations have multiple mechanisms that reduce the likelihood of memory exploitation.

The metadata used by the heaps for internal management is packed with a high degree of randomization to make it difficult for an attempted exploit to patch the internal structures to prevent crashes or conceal the attack attempt. These blocks are also subject to an integrity-check mechanism on the header to detect simple corruptions such as buffer overruns. Finally, the heap uses a small degree of randomization of the base address or handle. By using the `HeapSetInformation` API with the `Heap-EnableTerminationOnCorruption` class, processes can opt in for an automatic termination in case of detected inconsistencies to avoid executing unknown code.

As an effect of block metadata randomization, using the debugger to simply dump a block header as an area of memory is not that useful. For example, the size of the block and whether it is busy are not easy to spot from a regular dump. The same applies to LFH blocks. They have a different type of metadata stored in the header, also partially randomized. To dump these details, the `!heap -i` command in the debugger does all the work to retrieve the metadata fields from a block, also flagging checksum or free-list inconsistencies if they exist. The command works for both LFH and regular heap blocks. The total size of the blocks, the user-requested size, the segment owning the block, and the header partial checksum are available in the output, as shown in the following sample. Because the randomization algorithm uses the heap granularity, the `!heap -i` command should be used only in the proper context of the heap containing the block. In the example, the heap handle is `0x001a0000`. If the current heap context were different, the decoding of the header would be incorrect. To set the proper context, the same `!heap -i` command with the heap handle as an argument must be executed first.

[Click here to view code image](#)

```
0:004> !heap -i 000001f72a5e0000
Heap context set to the heap 0x000001f72a5e0000

0:004> !heap -i 000001f72a5eb180
Detailed information for block entry 000001f72a5eb180
Assumed heap          : 0x000001f72a5e0000 (Use !heap -i NewHeapHandle
to change)
Header content        : 0x2FB544DC 0x1000021F (decoded : 0x7F01007E
0x10000048)
Owning segment        : 0x000001f72a5e0000 (offset 0)
Block flags           : 0x1 (busy )
Total block size      : 0x7e units (0x7e0 bytes)
Requested size        : 0x7d0 bytes (unused 0x10 bytes)
Previous block size   : 0x48 units (0x480 bytes)
Block CRC             : OK - 0x7f
Previous block        : 0x000001f72a5ead00
Next block            : 0x000001f72a5eb960
```

Segment heap-specific security features

The segment heap implementation uses many security mechanisms to make it harder to corrupt memory or to allow code injection by an attacker. Here are a few of them:

- **Fail fast on linked list node corruption** The segment heap uses linked lists to track segments and sub-segments. As with the NT heap, checks are added in the list node insertion and removal to prevent arbitrary memory writes due to corrupted list nodes. If a corrupted node is detected, the process is terminated via a call to `RtlFailFast`.
- **Fail fast on red-black (RB) tree node corruption** The segment heap uses RB trees to track free back-end and VS allocations. Node insertion and deletion functions validate the nodes involved or, if corrupted, invoke the fail-fast mechanism.
- **Function pointer decoding** Some aspects of the segment heap allow for callbacks (in `VsContext` and `LfhContext` structures, part of the `_SEGMENT_HEAP` structure). An attacker can override these callbacks to point to his or her own code. However, the function pointers are encoded by using a XOR function with an internal random heap key and the context address, both of which cannot be guessed in advance.
- **Guard pages** When LFH and VS sub-segments and large blocks are allocated, a guard page is added at the end. This helps to detect overflows and corruption of adjacent data. See the section “[Stacks](#)” later in this chapter for more information on guard pages.

Heap debugging features

The heap manager includes several features to help detect bugs by using the following heap settings:

- **Enable tail checking** The end of each block carries a signature that is checked when the block is released. If a buffer overrun destroys the signature entirely or partially, the heap will report this error.
- **Enable free checking** A free block is filled with a pattern that is checked at various points when the heap manager needs to access the block, such as at removal from the free list to satisfy an allocate request. If the process continues to write to the block after freeing it, the heap manager will detect changes in the pattern and the error will be reported.
- **Parameter checking** This function consists of extensive checking of the parameters passed to the heap functions.
- **Heap validation** The entire heap is validated at each heap call.
- **Heap tagging and stack traces support** This function supports the specification of tags for allocation and/or captures user-mode stack traces for the heap calls to help narrow the possible causes of a heap error.

The first three options are enabled by default if the loader detects that a process is started under the control of a debugger. (A debugger can override this behavior and turn off these features.) You can specify the heap debugging features for an executable image by setting various debugging flags in the image header using the `Gflags` tool. (See the next experiment and the section “Windows global flags” in Chapter 8 in Part 2.) Alternatively, you can enable heap debugging options using the `!heap` command in the standard Windows debuggers. (See the debugger help for more information.)

Enabling heap-debugging options affects all heaps in the process. Also, if any of the heap-debugging options are enabled, the LFH will be disabled automatically and the core heap will be used (with the required debugging options enabled). The LFH is also not used for heaps that are not expandable

(because of the extra overhead added to the existing heap structures) or for heaps that do not allow serialization.

Pageheap

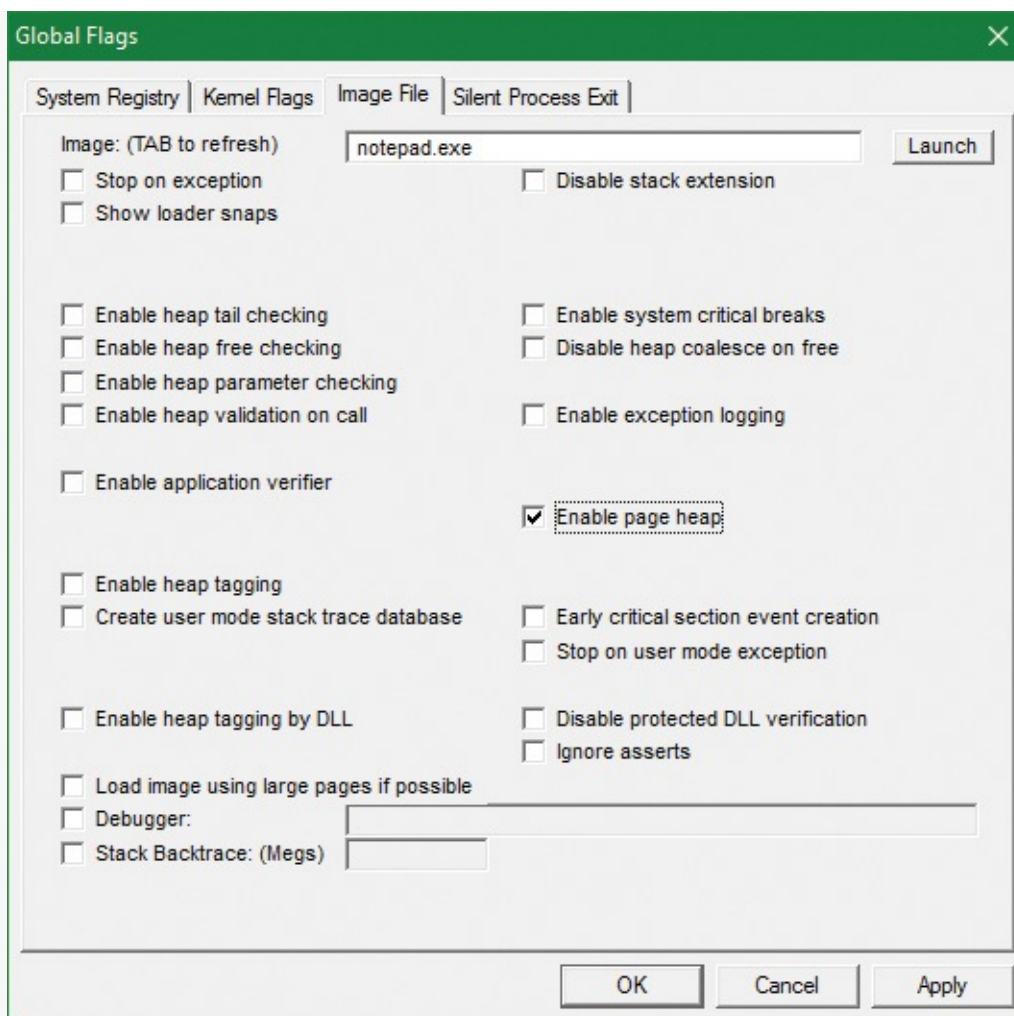
Because the tail and free checking options described in the preceding sections might discover corruptions that occurred well before the problem was detected, an additional heap debugging capability, called *pageheap*, is provided. Pageheap directs all or part of the heap calls to a different heap manager. You can enable pageheap using the Gflags tool (part of the Debugging Tools for Windows). When enabled, the heap manager places allocations at the end of pages and reserves the page that immediately follows. Because reserved pages are not accessible, any buffer overruns that occur will cause an access violation, making it easier to detect the offending code. Optionally, pageheap allows for the placement of blocks at the beginning of the pages, with the preceding page reserved, to detect buffer underrun problems (a rare occurrence). Pageheap also can protect freed pages against any access to detect references to heap blocks after they have been freed.

Note that using the pageheap can cause you to run out of address space (in 32-bit processes) because of the significant overhead added for small allocations. Also, performance can suffer due to the increase of references to demand zero pages, loss of locality, and additional overhead caused by frequent calls to validate heap structures. A process can reduce the impact by specifying that the pageheap be used only for blocks of certain sizes, address ranges, and/or originating DLLs.

EXPERIMENT: Using pageheap

In this experiment, you'll turn on pageheap for Notepad.exe and see its effects.

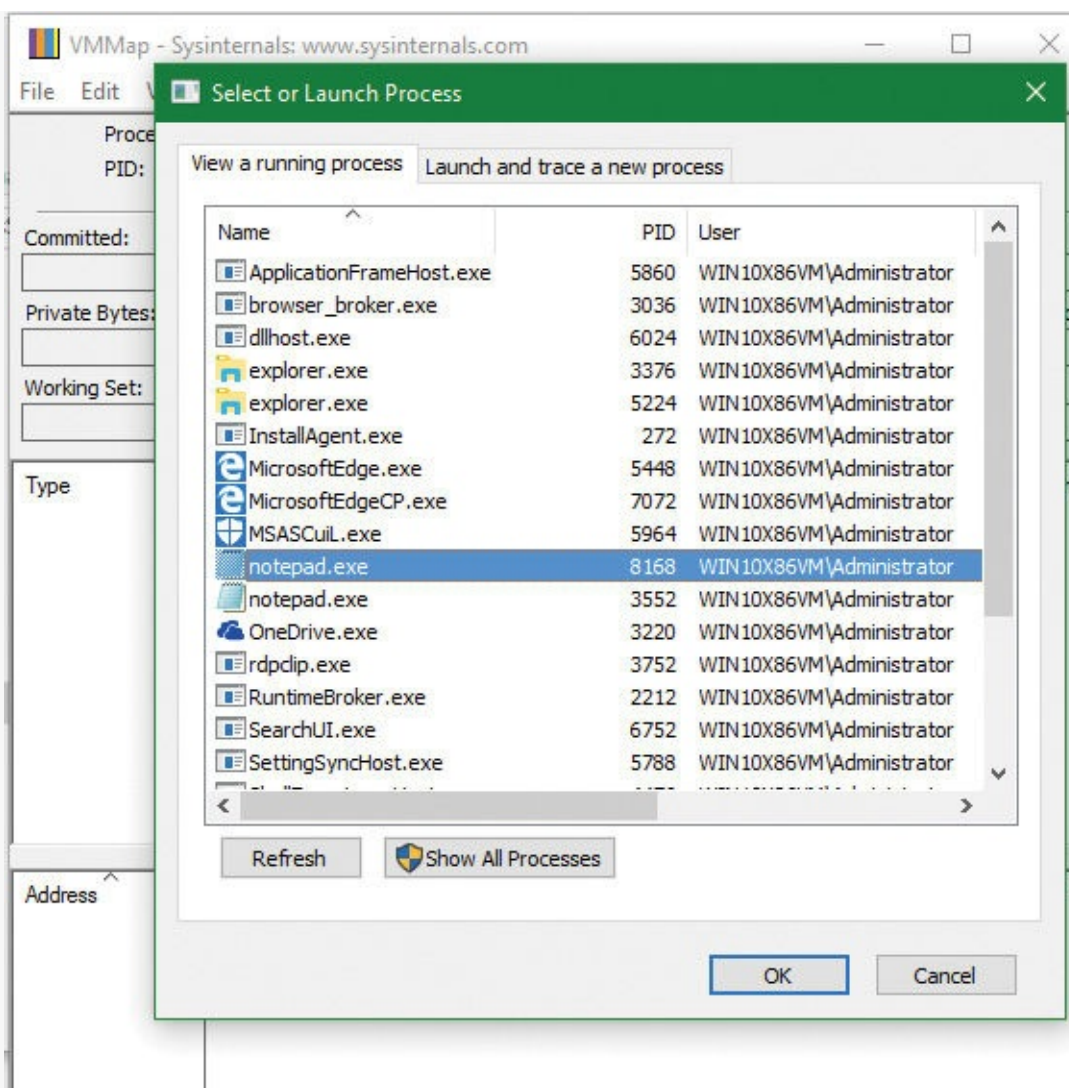
1. Run Notepad.exe.
2. Open Task Manager, click to the **Details** tab, and add the **Commit Size** column to the display.
3. Notice the commit size of the notepad instance you just launched.
4. Run Gflags.exe, located in the folder where Debugging Tools for Windows is installed (requires elevation).
5. Click the **Image File** tab.
6. In the **Image** text box, type **notepad.exe**. Then press the **Tab** key. The various check boxes should be selected.
7. Select the **Enable Page Heap** check box. The dialog box should look like this:



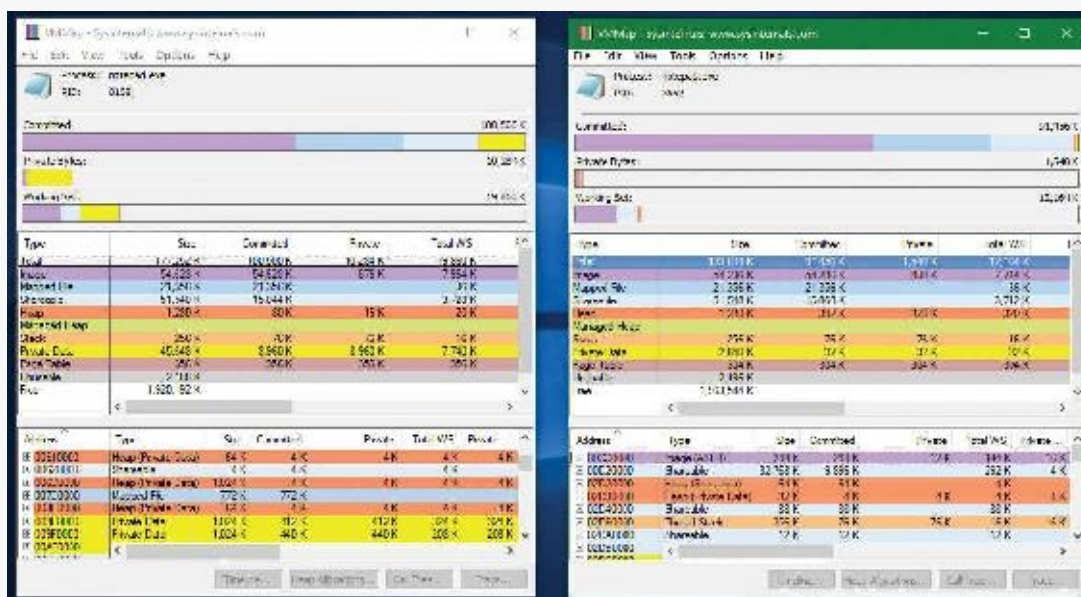
8. Click **Apply**.
9. Run another instance of Notepad. (Don't close the first one.)
10. In Task Manager, compare the commit size of both notepad instances. Notice that the second instance has a much larger commit size even though both are empty notepad processes. This is due to the extra allocations that pageheap provides. Here's a screenshot from 32-bit Windows 10:

Name	PID	User name	Commit size	CPU	Memory (p. ^
MsMpEng.exe	2460	SYSTEM	95,416 K	00	49,720 I
NisSrv.exe	4116	LOCAL SERVI...	9,056 K	00	3,024 I
notepad.exe	8168	Administrator	10,248 K	00	8,452 I
notepad.exe	3552	Administrator	1,532 K	00	1,020 I
OneDrive.exe	3220	Administrator	8,620 K	00	5,088 I

11. To get a better sense of the extra memory allocated, use the VMMap Sysinternals tool. While the notepad processes are still running, open VMMap.exe and select the notepad instance that is using pageheap:



12. Open another instance of VMMAP and select the other notepad instance. Place the windows side by side to see both:



13. Notice that the difference in the commit size is clearly visible in the Private Data (yellow) part.

14. Click the **Private Data** line in the middle display on both VMMAP instances to see its parts in the bottom display (sorted by size in the screenshot):

that are not actually part of the heap, or not part of the heap identified by the heap handle argument to `HeapFree`, are simply ignored. In addition, no blocks are actually freed once `exit` or `RtlExitUserProcess` has been called.

The FTH server continues to monitor the failure rate of the application after the mitigations have been installed. If the failure rate does not improve, the mitigations are removed.

You can observe the activity of the fault-tolerant heap in the Event Viewer. Follow these steps:

1. Open a Run prompt and type **eventvwr.msc**.
2. In the left pane, choose **Event Viewer**, select **Applications and Services Logs**, choose **Microsoft**, select **Windows**, and click **Fault-Tolerant-Heap**.
3. Click the **Operational** log.
4. The FTH may be disabled completely in the registry. in the `HKLM\Software\Microsoft\FTH` key, set the `Enabled` value to 0.

That same key contains the various FTH settings, such as the delay mentioned earlier and an exclusion list of executables (which includes by default system processes such as `smss.exe`, `csrss.exe`, `wininit.exe`, `services.exe`, `winlogon.exe` and `taskhost.exe`). A rule list is also included (`RuleList` value), which lists the modules and exception type (and some flags) to watch for in order for FTH to kick in. By default, a single rule is listed, indicating heap problems in `Ntdll.dll` of type `STATUS_ACCESS_VIOLATION` (`0xc0000005`).

The FTH does not normally operate on services, and it is disabled on Windows server systems for performance reasons. A system administrator can manually apply the shim to an application or service executable by using the Application Compatibility Toolkit.

Virtual address space layouts

This section describes the components in the user and system address space, followed by the specific layouts on 32-bit (x86 and ARM) and 64-bit (x64) systems. This information will help you to understand the limits on process and system virtual memory on these platforms.

Three main types of data are mapped into the virtual address space in Windows:

- **Per-process private code and data** As explained in [Chapter 1](#), each process has a private address space that cannot be accessed by other processes. That is, a virtual address is always evaluated in the context of the current process and cannot refer to an address defined by any other process. Threads within the process can therefore never access virtual addresses outside this private address space. Even shared memory is not an exception to this rule, because shared memory regions are mapped into each participating process, and so are accessed by each process using per-process addresses. Similarly, the cross-process memory functions (`Read-ProcessMemory` and `WriteProcessMemory`) operate by running kernel-mode code in the context of the target process. The process virtual address space, called *page tables*, is described in the “[Address translation](#)” section. Each process has its own set of page tables. They are stored in kernel-mode-only accessible pages so that user-mode threads in a process cannot modify their own address space layout.
- **Session-wide code and data** Session space contains information that is common to each session. (For a description of sessions, see [Chapter 2](#).) A session consists of the processes and other system objects such as the window station, desktops, and windows that represent a single user’s logon session. Each session has a session-specific paged pool area used by the kernel-mode portion of the

Windows subsystem (Win32k.sys) to allocate session-private GUI data structures. In addition, each session has its own copy of the Windows subsystem process (Csrss.exe) and logon process (Winlogon.exe). The Session Manager process (Smss.exe) is responsible for creating new sessions, which includes loading a session-private copy of Win32k.sys, creating the session-private object manager namespace (see Chapter 8 in Part 2 for more details on the object manager), and creating the session-specific instances of the Csrss.exe and Winlogon.exe processes. To virtualize sessions, all session-wide data structures are mapped into a region of system space called *session space*. When a process is created, this range of addresses is mapped to the pages associated with the session that the process belongs to.

- **System-wide code and data** System space contains global operating system code and data structures visible by kernel-mode code regardless of which process is currently executing. System space consists of the following components:
 - **System code** This contains the OS image, HAL, and device drivers used to boot the system.
 - **Nonpaged pool** This is the non-pageable system memory heap.
 - **Paged pool** This is the pageable system memory heap.
 - **System cache** This is virtual address space used to map files open in the system cache. (See Chapter 11, “Startup and shutdown,” in Part 2 for detailed information.)
 - **System page table entries (PTEs)** This is the pool of system PTEs used to map system pages such as I/O space, kernel stacks, and memory descriptor lists. You can see how many system PTEs are available by using Performance Monitor to examine the value of the Memory: Free System Page Table Entries counter.
 - **System working set lists** These are the working set list data structures that describe the three system working sets: system cache, paged pool, and system PTEs.
 - **System mapped views** This is used to map Win32k.sys, the loadable kernel-mode part of the Windows subsystem, as well as kernel-mode graphics drivers it uses. (See [Chapter 2](#) for more information on Win32k.sys.)
 - **Hyperspace** This is a special region used to map the process working set list and other per-process data that doesn’t need to be accessible in arbitrary process context. Hyperspace is also used to temporarily map physical pages into the system space. One example of this is invalidating page table entries in page tables of processes other than the current one, such as when a page is removed from the standby list.
 - **Crash dump information** This is reserved to record information about the state of a system crash.
 - **HAL usage** This is system memory reserved for HAL-specific structures.

Now that we’ve described the basic components of the virtual address space in Windows, let’s examine the specific layout on the x86, ARM, and x64 platforms.

x86 address space layouts

By default, each user process on 32-bit versions of Windows has a 2 GB private address space. (The operating system takes the remaining 2 GB.) However, for x86, the system can be configured with the `increaseuserva` BCD boot option to permit user address spaces up to 3 GB. Two possible address space layouts are shown in [Figure 5-10](#).

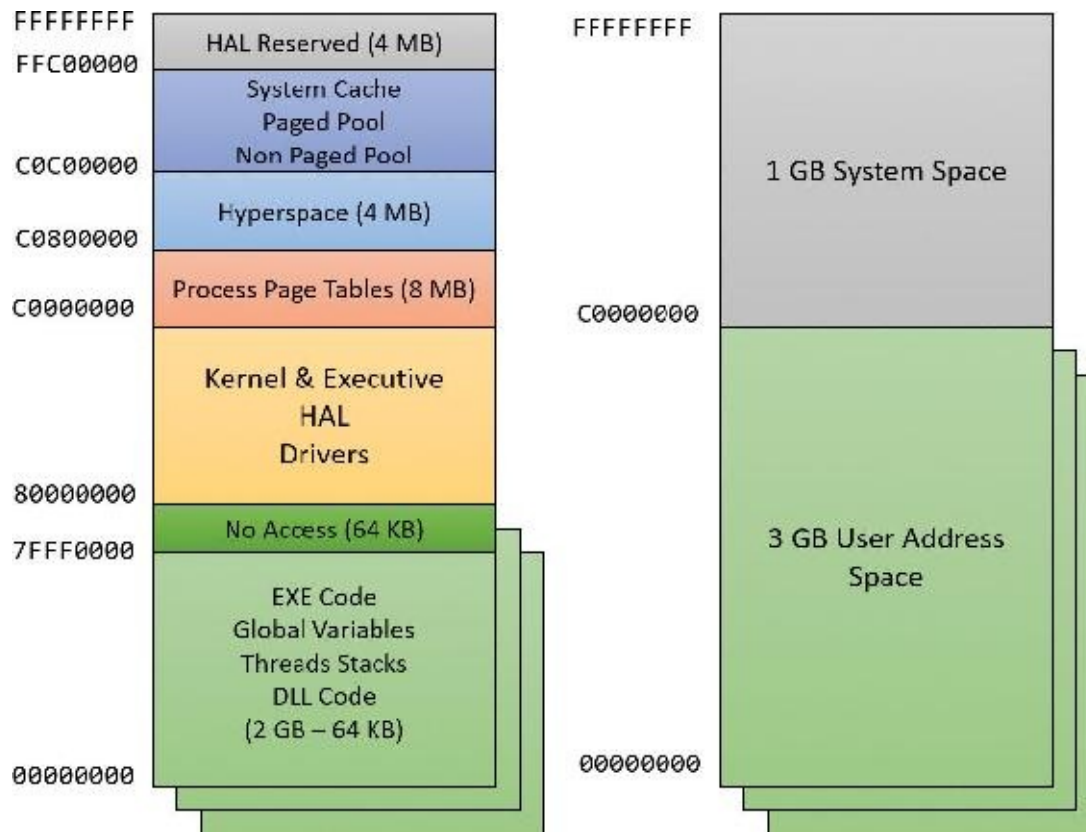


FIGURE 5-10 x86 virtual address space layouts (2 GB on the left, 3 GB on the right).

The ability of a 32-bit process to grow beyond 2 GB was added to accommodate the need for 32-bit applications to keep more data in memory than could be done with a 2 GB address space. Of course, 64-bit systems provide a much larger address space.

For a process to grow beyond 2 GB of address space, the image file must have the `IMAGE_FILE_LARGE_ADDRESS_AWARE` flag set in the image header (in addition to the global `increaseuserva` setting). Otherwise, Windows reserves the additional address space for that process so that the application won't see virtual addresses greater than `0x7FFFFFFF`. Access to the additional virtual memory is opt-in because some applications assume they'll be given at most 2 GB of the address space. Because the high bit of a pointer referencing an address below 2 GB is always zero (31 bits are needed to reference a 2 GB address space), these applications would use the high bit in their pointers as a flag for their own data—clearing it, of course, before referencing the data. If they ran with a 3 GB address space, they would inadvertently truncate pointers that have values greater than 2 GB, causing program errors, including possible data corruption. You set this flag by specifying the `/LARGEADDRESSAWARE` linker flag when building the executable. Alternatively, use the Property page in Visual Studio (choose **Linker**, select **System**, and click **Enable Large Addresses**). You can add the flag to an executable image even without building (no source code required) by using a tool such as `Editbin.exe` (part of the Windows SDK tools), assuming the file is not signed. This flag has no effect when running the application on a system with a 2 GB user address space.

Several system images are marked as large address space aware so that they can take advantage of systems running with large process address spaces. These include the following:

- **Lsass.exe** The Local Security Authority Subsystem
- **Inetinfo.exe** Internet Information Server
- **Chkdsk.exe** The Check Disk utility
- **Sms.exe** The Session Manager

EXPERIMENT: Checking whether an application is large address aware

You can use the Dumpbin utility from the Visual Studio Tools (and older versions of the Windows SDK) to check other executables to see if they support large address spaces. Use the `/headers` flag to display the results. Here's a sample output of Dumpbin on the Session Manager:

[Click here to view code image](#)

```
dumpbin /headers c:\windows\system32\smss.exe
Microsoft (R) COFF/PE Dumper Version 14.00.24213.1
Copyright (C) Microsoft Corporation. All rights reserved.
Dump of file c:\windows\system32\smss.exe
PE signature found
File Type: EXECUTABLE IMAGE
FILE HEADER VALUES
      14C machine (x86)
        5 number of sections
57898F8A time date stamp Sat Jul 16 04:36:10 2016
        0 file pointer to symbol table
        0 number of symbols
      E0 size of optional header
    122 characteristics
          Executable
          Application can handle large (>2GB) addresses
          32 bit word machine
```

Finally, memory allocations using `VirtualAlloc`, `VirtualAllocEx`, and `VirtualAllocExNuma` start with low virtual addresses and grow higher by default. Unless a process allocates a lot of memory or has a very fragmented virtual address space, it will never get back very high virtual addresses. Therefore, for testing purposes, you can force memory allocations to start from high addresses by using the `MEM_TOP_DOWN` flag to the `VirtualAlloc*` functions or by adding a `DWORD` registry value named `AllocationPreference` to the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key and setting its value to `0x100000`.

The following output shows runs of the `TestLimit` utility (shown in previous experiments) leaking memory on a 32-bit Windows machine booted without the `increaseuserva` option:

[Click here to view code image](#)

```
Testlimit.exe -r
```

```
Testlimit v5.24 - test Windows limits
Copyright (C) 2012-2015 Mark Russinovich
Sysinternals - www.sysinternals.com
```

```
Process ID: 5500
```

Reserving private bytes (MB)...
Leaked 1978 MB of reserved memory (1940 MB total leaked). Lasterror:
8

The process managed to reserve close to the 2 GB limit (but not quite). The process address space has the EXE code and various DLLs mapped, so naturally it's not possible in a normal process to reserve the entire address space.

On that same system, you can switch to a 3 GB address space by running the following command from an administrative command window:

[Click here to view code image](#)

```
C:\WINDOWS\system32>bcdedit /set increaseuserva 3072
```

The operation completed successfully.

Notice that the command allows you to specify any number (in MB) between 2,048 (the 2 GB default) to 3,072 (the 3 GB maximum). After you restart the system so that the setting can take effect, running TestLimit again produces the following:

[Click here to view code image](#)

```
Testlimit.exe -r
```

```
Testlimit v5.24 - test Windows limits  
Copyright (C) 2012-2015 Mark Russinovich  
Sysinternals - www.sysinternals.com
```

```
Process ID: 2308
```

Reserving private bytes (MB)...
Leaked 2999 MB of reserved memory (2999 MB total leaked). Lasterror:
8

TestLimit was able to leak close to 3 GB, as expected. This is only possible because TestLimit was linked with /LARGEADDRESSAWARE. Had it not been, the results would have been essentially the same as on the system booted without `increaseuserva`.



Note

To revert a system to the normal 2 GB address space per process, run the `bcdedit /deletevalue increaseuserva` command.

x86 system address space layout

The 32-bit versions of Windows implement a dynamic system address space layout by using a virtual address allocator. (We'll describe this functionality later in this section.) There are still a few specifically reserved areas, as shown in [Figure 5-10](#). However, many kernel-mode structures use dynamic address space allocation. These structures are therefore not necessarily virtually contiguous with themselves. Each can easily exist in several disjointed pieces in various areas of system address space. The uses of system address space that are allocated in this way include the following:

- Non-paged pool
- Paged pool
- Special pool
- System PTEs
- System mapped views
- File system cache
- PFN database
- Session space

x86 session space

For systems with multiple sessions (which is almost always the case, as session 0 is used by system processes and services, while session 1 is used for the first logged on user), the code and data unique to each session are mapped into system address space but shared by the processes in that session. [Figure 5-11](#) shows the general layout of session space. The sizes of the components of session space, just like the rest of kernel system address space, are dynamically configured and resized by the memory manager on demand.



FIGURE 5-11 x86 session space layout (not proportional).

EXPERIMENT: Viewing sessions

You can display which processes are members of which sessions by examining the session ID. You can do this using Task Manager, Process Explorer, or the kernel debugger. Using the kernel debugger, you can list the active sessions with the `!session` command as follows:

```
lkd> !session
Sessions on machine: 3
Valid Sessions: 0 1 2
Current Session 2
```

You can then set the active session using the `!session -s` command and display the address of the session data structures and the processes in that session with the `!sprocess` command:

[Click here to view code image](#)

```
lkd> !session -s 1
Sessions on machine: 3
Implicit process is now d4921040
Using session 1
```

```
lkd> !sprocess
Dumping Session 1
```

```
_MM_SESSION_SPACE d9306000
_MMSESSION        d9306c80
PROCESS d4921040   SessionId: 1   Cid: 01d8   Peb:
00668000   ParentCid: 0138
           DirBase: 179c5080   ObjectTable: 00000000   HandleCount:    0.
           Image: smss.exe

PROCESS d186c180   SessionId: 1   Cid: 01ec   Peb:
00401000   ParentCid: 01d8
           DirBase: 179c5040   ObjectTable: d58d48c0   HandleCount: <Data Not
Accessible>
           Image: csrss.exe

PROCESS d49acc40   SessionId: 1   Cid: 022c   Peb:
03119000   ParentCid: 01d8
           DirBase: 179c50c0   ObjectTable: d232e5c0   HandleCount: <Data Not
Accessible>
           Image: winlogon.exe

PROCESS dc0918c0   SessionId: 1   Cid: 0374   Peb:
003c4000   ParentCid: 022c
           DirBase: 179c5160   ObjectTable: dc28f6c0   HandleCount: <Data Not
Accessible>
           Image: LogonUI.exe

PROCESS dc08e900   SessionId: 1   Cid: 037c   Peb:
```

00d8b000 ParentCid: 022c
DirBase: 179c5180 ObjectTable: dc249640 HandleCount: <Data Not Accessible>
Image: dwm.exe

To view the details of the session, dump the MM_SESSION_SPACE structure using the dt command, as follows:

[Click here to view code image](#)

```
lkd> dt nt!_mm_session_space d9306000
+0x000 ReferenceCount      : 0n4
+0x004 u                   : <unnamed-tag>
+0x008 SessionId          : 1
+0x00c ProcessReferenceToSession : 0n6
+0x010 ProcessList        : _LIST_ENTRY [ 0xd4921128 - 0xdc08e9e8 ]
+0x018 SessionPageDirectoryIndex : 0x1617f
+0x01c NonPagablePages    : 0x28
+0x020 CommittedPages     : 0x290
+0x024 PagedPoolStart     : 0xc0000000 Void
+0x028 PagedPoolEnd       : 0xffbfffffff Void
+0x02c SessionObject      : 0xd49222b0 Void
+0x030 SessionObjectHandle : 0x800003ac Void
+0x034 SessionPoolAllocationFailures : [4] 0
+0x044 ImageTree          : _RTL_AVL_TREE
+0x048 LocaleId           : 0x409
+0x04c AttachCount        : 0
+0x050 AttachGate         : _KGATE
+0x060 WsListEntry        : _LIST_ENTRY [ 0xcdcde060 - 0xd6307060 ]
+0x080 Lookaside          : [24] _GENERAL_LOOKASIDE
+0xc80 Session            : _MMSESSION
...

```

EXPERIMENT: Viewing session space utilization

You can view session space memory utilization with the `!vm 4` command in the kernel debugger. For example, the following output was taken from a 32-bit Windows client system with a remote desktop connection, resulting in three sessions—the default two sessions plus the remote session. (The addresses are for the `MM_SESSION_SPACE` objects shown earlier.)

[Click here to view code image](#)

```
lkd> !vm 4
...
Terminal Server Memory Usage By Session:

Session ID 0 @ d6307000:
Paged Pool Usage:      2012 Kb
NonPaged Usage:       108 Kb
Commit Usage:         2292 Kb

Session ID 1 @ d9306000:
Paged Pool Usage:      2288 Kb
NonPaged Usage:       160 Kb
Commit Usage:         2624 Kb

Session ID 2 @ cdcde000:
Paged Pool Usage:      7740 Kb
NonPaged Usage:       208 Kb
Commit Usage:         8144 Kb

Session Summary
Paged Pool Usage:      12040 Kb
NonPaged Usage:       476 Kb
Commit Usage:         13060 Kb
```

System page table entries

System page table entries (PTEs) are used to dynamically map system pages such as I/O space, kernel stacks, and the mapping for memory descriptor lists (MDLs, discussed to some extent in [Chapter 6](#)). System PTEs aren't an infinite resource. On 32-bit Windows, the number of available system PTEs is such that the system can theoretically describe 2 GB of contiguous system virtual address space. On Windows 10 64 bit and Server 2016, system PTEs can describe up to 16 TB of contiguous virtual address space.

EXPERIMENT: Viewing system PTE information

You can see how many system PTEs are available by examining the value of the Memory: Free System Page Table Entries counter in Performance Monitor or by using the `!sysptes` or `!vm` command in the debugger. You can also dump the `_MI_SYSTEM_PTE_TYPE` structure as part of the memory state (`MiState`) variable (or the `MiSystemPteInfo` global variable on Windows 8.x/2012/R2). This will also show you how many PTE allocation failures occurred on the system. A high count indicates a problem and possibly a system PTE leak.

[Click here to view code image](#)

```
kd> !sysptes
System PTE Information
  Total System Ptes 216560
    starting PTE: c0400000
  free blocks: 969    total free: 16334    largest free block: 264

kd> ? MiState
Evaluate expression: -2128443008 = 81228980

kd> dt nt!_MI_SYSTEM_INFORMATION SystemPtes
+0x3040 SystemPtes : _MI_SYSTEM_PTE_STATE

kd> dt nt!_mi_system_pte_state SystemViewPteInfo 81228980+3040
+0x10c SystemViewPteInfo : _MI_SYSTEM_PTE_TYPE

kd> dt nt!_mi_system_pte_type 81228980+3040+10c
+0x000 Bitmap           : _RTL_BITMAP
+0x008 BasePte           : 0xc0400000 _MMPTE
+0x00c Flags             : 0xe
+0x010 VaType            : c ( MiVaDriverImages )
+0x014 FailureCount      : 0x8122bae4 -> 0
+0x018 PteFailures       : 0
+0x01c SpinLock          : 0
+0x01c GlobalPushLock    : (null)
+0x020 Vm                : 0x8122c008 _MMSUPPORT_INSTANCE
+0x024 TotalSystemPtes   : 0x120
+0x028 Hint              : 0x2576
+0x02c LowestBitEverAllocated : 0xc80
+0x030 CachedPtes        : (null)
+0x034 TotalFreeSystemPtes : 0x73
```

If you are seeing lots of system PTE failures, you can enable system PTE tracking by creating a new DWORD value in the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key called `TrackPtes` and setting its value to 1. You can then use `!sysptes 4` to show a list of allocators.

ARM address space layout

As shown in [Figure 5-12](#), the ARM address space layout is nearly identical to the x86 address space. The memory manager treats ARM-based systems exactly as x86 systems in terms of pure memory management. The differences are at the address translation layer, described in the section “[Address translation](#)” later in this chapter.

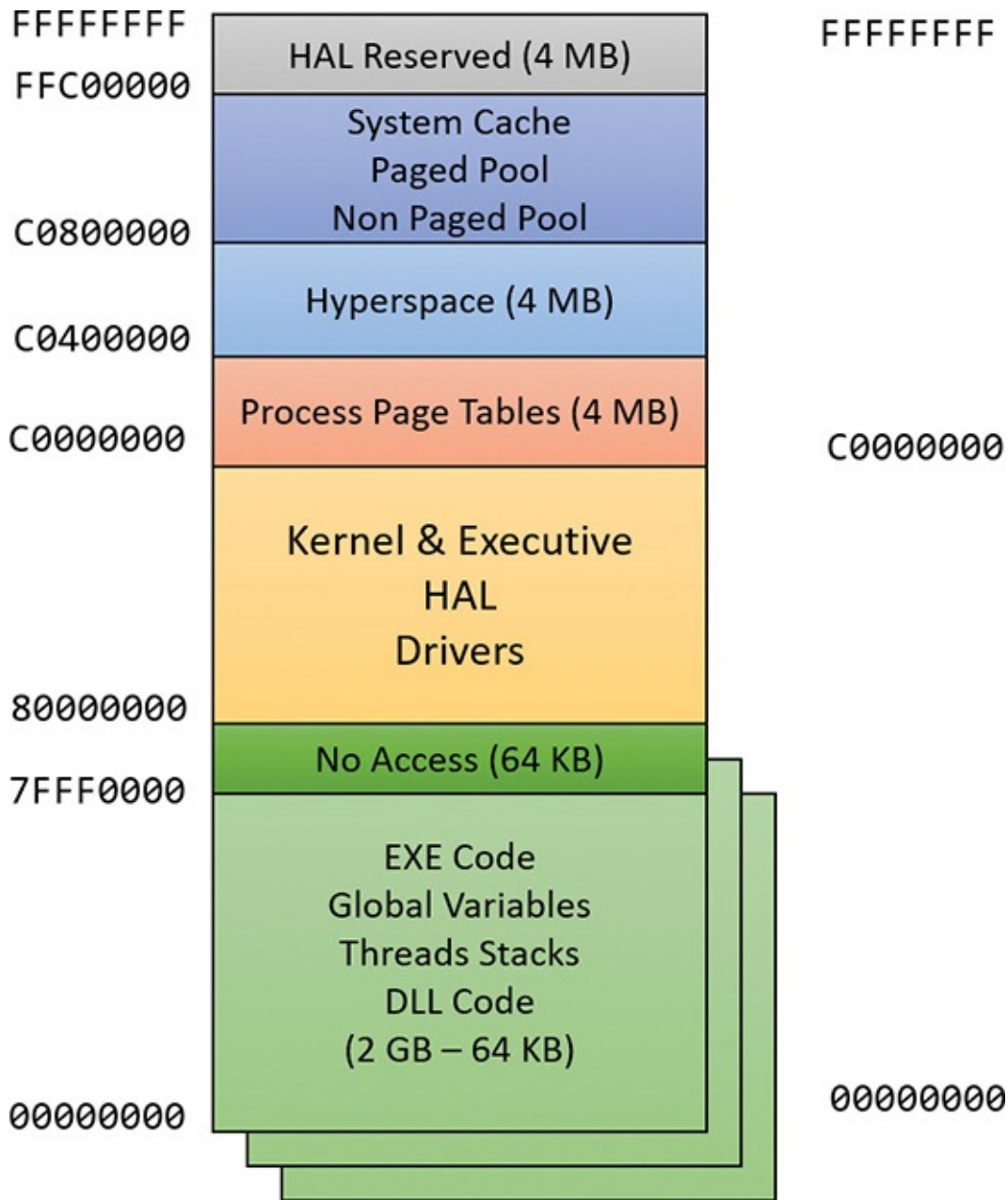


FIGURE 5-12 ARM virtual address space layout.

64-bit address space layout

The theoretical 64-bit virtual address space is 16 exabytes (EB), or 18,446,744,073,709,551,616 bytes. Current processor limitations allow for 48 address lines only, limiting the possible address space to 256 TB (2 to the 48th power). The address space is divided in half, where the lower 128 TB are available as private user processes and the upper 128 TB are system space. System space is divided into several different-sized regions (Windows 10 and Server 2016), as shown in [Figure 5-13](#). Clearly, 64 bits provides a tremendous leap in terms of address space sizes as opposed to 32 bit. The actual starts of various kernel sections are not necessarily those shown, as ASLR is in effect in kernel space in the latest versions of Windows.

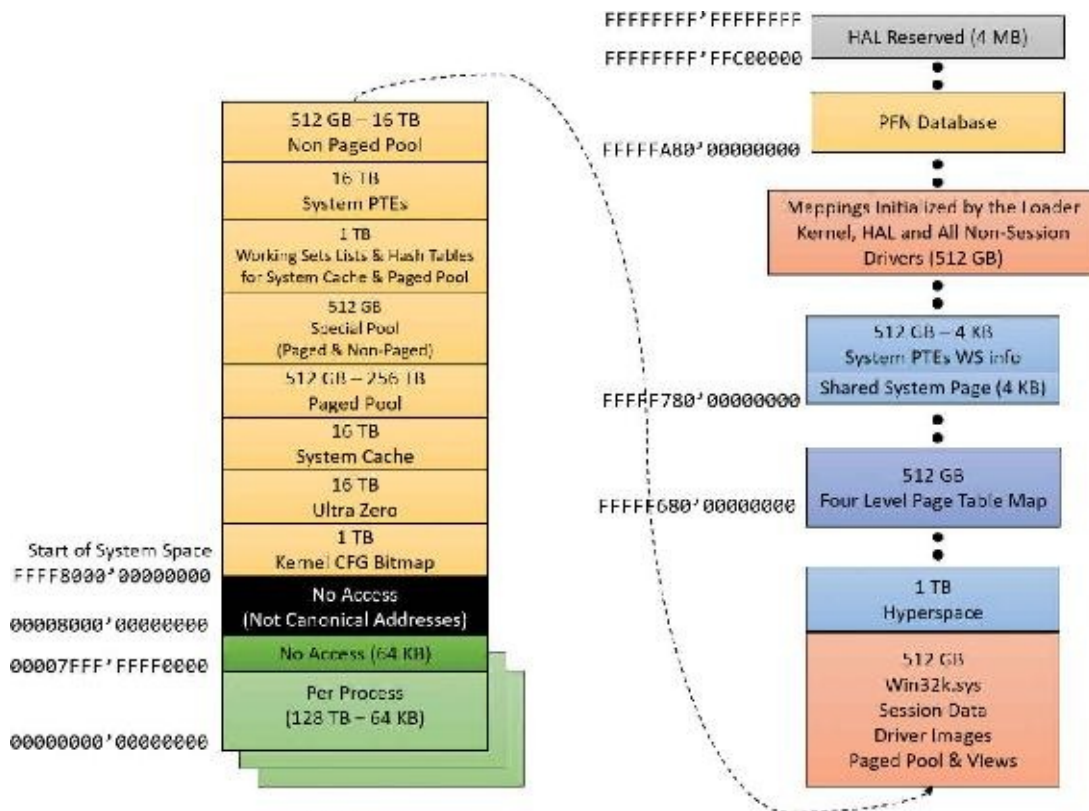


FIGURE 5-13 x64 address space layout.



Note

Windows 8 and Server 2012 are limited to 16 TB of address space. This is because of Windows implementation limitations, described in Chapter 10 of the sixth edition of *Windows Internals Part 2*. Of these, 8 TB is per process and the other 8 TB is used for system space.

Thirty-two-bit images that are large address space aware receive an extra benefit while running on 64-bit Windows (under Wow64). Such an image will actually receive all 4 GB of user address space available. After all, if the image can support 3 GB pointers, 4 GB pointers should not be any different, because unlike the switch from 2 GB to 3 GB, there are no additional bits involved. The following output shows TestLimit running as a 32-bit application, reserving address space on a 64-bit Windows machine.

[Click here to view code image](#)

```
C:\Tools\Sysinternals>Testlimit.exe -r
```

```
Testlimit v5.24 - test Windows limits
Copyright (C) 2012-2015 Mark Russinovich
Sysinternals - www.sysinternals.com
```

```
Process ID: 264
```

```
Reserving private bytes (MB)...
```

```
Leaked 4008 MB of reserved memory (4008 MB total leaked). Lasterror:
8
```

```
Not enough storage is available to process this command.
```

These results depend on TestLimit having been linked with the /LARGEADDRESSAWARE option. Had it not been, the results would have been about 2 GB for each. Sixty-four-bit applications linked without /LARGEADDRESSAWARE are constrained to the first 2 GB of the process virtual address space, just like 32-bit applications. (This flag is set by default in Visual Studio for 64-bit builds.)

x64 virtual addressing limitations

As discussed, 64 bits of virtual address space allow for a possible maximum of 16 EB of virtual memory—a notable improvement over the 4 GB offered by 32-bit addressing. Obviously, neither today’s computers nor tomorrow’s are even close to requiring support for that much memory.

Accordingly, to simplify chip architecture and avoid unnecessary overhead—particularly in address translation (described later)—AMD’s and Intel’s current x64 processors implement only 256 TB of virtual address space. That is, only the low-order 48 bits of a 64-bit virtual address are implemented. However, virtual addresses are still 64 bits wide, occupying 8 bytes in registers or when stored in memory. The high-order 16 bits (bits 48 through 63) must be set to the same value as the highest-order implemented bit (bit 47), in a manner similar to sign extension in two’s complement arithmetic. An address that conforms to this rule is said to be a *canonical address*.

Under these rules, the bottom half of the address space starts at 0x0000000000000000 as expected, but ends at 0x00007FFFFFFFFFFFFF. The top half of the address space starts at 0xFFFF800000000000 and ends at 0xFFFFFFFFFFFFFFFF. Each canonical portion is 128 TB. As newer processors implement more of the address bits, the lower half of memory will expand upward toward 0x7FFFFFFFFFFFFFFFFF, while the upper half will expand downward toward 0x8000000000000000.

Dynamic system virtual address space management

Thirty-two-bit versions of Windows manage the system address space through an internal kernel virtual allocator mechanism, described in this section. Currently, 64-bit versions of Windows have no need to use the allocator for virtual address space management (and thus bypass the cost) because each region is statically defined (refer to [Figure 5-13](#)).

When the system initializes, the `MiInitializeDynamicVa` function sets up the basic dynamic ranges and sets the available virtual address to all available kernel space. It then initializes the address space ranges for boot loader images, process space (hyperspace), and the HAL through the `MiInitialize-SystemVaRange` function, which is used to set hard-coded address ranges (on 32-bit systems only). Later, when non-paged pool is initialized, this function is used again to reserve the virtual address ranges for it. Finally, whenever a driver loads, the address range is relabeled to a driver-image range instead of a boot-loaded range.

After this point, the rest of the system virtual address space can be dynamically requested and released through `MiObtainSystemVa` (and its analogous `MiObtainSessionVa`) and `MiReturnSystemVa`. Operations such as expanding the system cache, the system PTEs, non-paged pool, paged pool, and/or special pool; mapping memory with large pages; creating the PFN database; and creating a new session all result in dynamic virtual address allocations for a specific range.

Each time the kernel virtual address space allocator obtains virtual memory ranges for use by a certain type of virtual address, it updates the `MiSystemVaType` array, which contains the virtual address type for the newly allocated range. The values that can appear in `MiSystemVaType` are shown in [Table 5-8](#).

Region	Description	Limitable
<code>MiVaUnused</code> (0)	Unused	N/A
<code>MiVaSessionSpace</code> (1)	Addresses for session space	Yes
<code>MiVaProcessSpace</code> (2)	Addresses for process address space	No
<code>MiVaBootLoaded</code> (3)	Addresses for images loaded by the boot loader	No
<code>MiVaPfnDatabase</code> (4)	Addresses for the PFN database	No
<code>MiVaNonPagedPool</code> (5)	Addresses for the non-paged pool	Yes
<code>MiVaPagedPool</code> (6)	Addresses for the paged pool	Yes
<code>MiVaSpecialPoolPaged</code> (7)	Addresses for the special pool (paged)	No
<code>MiVaSystemCache</code> (8)	Addresses for the system cache	No
<code>MiVaSystemPtes</code> (9)	Addresses for system PTEs	Yes
<code>MiVaHal</code> (10)	Addresses for the HAL	No
<code>MiVaSessionGlobalSpace</code> (11)	Addresses for session global space	No
<code>MiVaDriverImages</code> (12)	Addresses for loaded driver images	No
<code>MiVaSpecialPoolNonPaged</code> (13)	Addresses for the special pool (non-paged)	Yes
<code>MiVaSystemPtesLarge</code> (14)	Addresses for large page PTEs	Yes

TABLE 5-8 System virtual address types

Although the ability to dynamically reserve virtual address space on demand allows better management of virtual memory, it would be useless without the ability to free this memory. As such, when the paged pool or system cache can be shrunk, or when special pool and large page mappings are freed, the associated virtual address is freed. Another case is when the boot registry is released. This allows dynamic management of memory depending on each component’s use. Additionally, components can reclaim

memory through `MiReclaimSystemVa`, which requests virtual addresses associated with the system cache to be flushed out (through the dereference segment thread) if available virtual address space has dropped below 128 MB. Reclaiming can also be satisfied if initial non-paged pool has been freed.

In addition to better proportioning and better management of virtual addresses dedicated to different kernel memory consumers, the dynamic virtual address allocator also has advantages when it comes to memory footprint reduction. Instead of having to manually pre-allocate static page table entries and page tables, paging-related structures are allocated on demand. On both 32-bit and 64-bit systems, this reduces boot-time memory usage because unused addresses won't have their page tables allocated. It also means that on 64-bit systems, the large address space regions that are reserved don't need to have their page tables mapped in memory. This allows them to have arbitrarily large limits, especially on systems that have little physical RAM to back the resulting paging structures.

EXPERIMENT: Querying system virtual address usage (Windows 10 and Server 2016)

You can look at the current and peak usage of each system virtual address type by using the kernel debugger. The global variable `MiVisibleState` (of type `MI_VISIBLE_STATE`) provides information available in the public symbols. (The example is on x86 Windows 10.)

1. To get a sense of the data provided by `MiVisibleState`, dump the structure with values:

[Click here to view code image](#)

```
lkd> dt nt!_mi_visible_state poi(nt!MiVisibleState)
+0x000 SpecialPool      : _MI_SPECIAL_POOL
+0x048 SessionWsList    : _LIST_ENTRY [ 0x91364060 -
0x9a172060 ]
+0x050 SessionIdBitmap  : 0x8220c3a0 _RTL_BITMAP
+0x054 PagedPoolInfo    : _MM_PAGED_POOL_INFO
+0x070 MaximumNonPagedPoolInPages : 0x80000
+0x074 SizeOfPagedPoolInPages : 0x7fc00
+0x078 SystemPteInfo    : _MI_SYSTEM_PTE_TYPE
+0x0b0 NonPagedPoolCommit : 0x3272
+0x0b4 BootCommit       : 0x186d
+0x0b8 MdlPagesAllocated : 0x105
+0x0bc SystemPageTableCommit : 0x1e1
+0x0c0 SpecialPagesInUse : 0
+0x0c4 WsOverheadPages  : 0x775
+0x0c8 VadBitmapPages   : 0x30
+0x0cc ProcessCommit    : 0xb40
+0x0d0 SharedCommit     : 0x712a
+0x0d4 DriverCommit     : 0n7276
+0x100 SystemWs         : [3] _MMSUPPORT_FULL
+0x2c0 SystemCacheShared : _MMSUPPORT_SHARED
+0x2e4 MapCacheFailures : 0
+0x2e8 PagefileHashPages : 0x30
+0x2ec PteHeader        : _SYSPTES_HEADER
+0x378 SessionSpecialPool : 0x95201f48 _MI_SPECIAL_POOL
+0x37c SystemVaTypeCount : [15] 0
+0x3b8 SystemVaType     : [1024] ""
```

```
+0x7b8 SystemVaTypeCountFailures : [15] 0
+0x7f4 SystemVaTypeCountLimit : [15] 0
+0x830 SystemVaTypeCountPeak : [15] 0
+0x86c SystemAvailableVa : 0x38800000
```

2. Notice the last arrays with 15 elements each, corresponding to the system virtual address types from [Table 5-8](#). Here are the `SystemVaTypeCount` and `SystemVaTypeCountPeak` arrays:

[Click here to view code image](#)

```
lkd> dt nt!_mi_visible_state poi(nt!mivisiblestate) -a
SystemVaTypeCount
```

```
+0x37c SystemVaTypeCount :
```

```
[00] 0
[01] 0x1c
[02] 0xb
[03] 0x15
[04] 0xf
[05] 0x1b
[06] 0x46
[07] 0
[08] 0x125
[09] 0x38
[10] 2
[11] 0xb
[12] 0x19
[13] 0
[14] 0xd
```

```
lkd> dt nt!_mi_visible_state poi(nt!mivisiblestate) -a
SystemVaTypeCountPeak
```

```
+0x830 SystemVaTypeCountPeak :
```

```
[00] 0
[01] 0x1f
[02] 0
[03] 0x1f
[04] 0xf
[05] 0x1d
[06] 0x51
[07] 0
[08] 0x1e6
[09] 0x55
[10] 0
[11] 0xb
[12] 0x5d
[13] 0
[14] 0xe
```

EXPERIMENT: Querying system virtual address usage (Windows 8.x and Server 2012/R2)

You can look at the current and peak usage of each system virtual address type by using the kernel debugger. For each system virtual address type described in [Table 5-8](#), the `MiSystemVaTypeCount`, `MiSystemVaTypeCountFailures`, and `MiSystemVaTypeCountPeak` global arrays in the kernel contain the sizes, count failures, and peak sizes for each type. The size is in multiples of a PDE mapping (see the “[Address translation](#)” section later in this chapter), which is effectively the size of a large page (2 MB on x86). Here’s how you can dump the usage for the system, followed by the peak usage. You can use a similar technique for the failure counts. (The example is from a 32-bit Windows 8.1 system.)

[Click here to view code image](#)

```
lkd> dd /c 1 MiSystemVaTypeCount L f
81c16640  00000000
81c16644  0000001e
81c16648  0000000b
81c1664c  00000018
81c16650  0000000f
81c16654  00000017
81c16658  0000005f
81c1665c  00000000
81c16660  000000c7
81c16664  00000021
81c16668  00000002
81c1666c  00000008
81c16670  0000001c
81c16674  00000000
81c16678  0000000b
lkd> dd /c 1 MiSystemVaTypeCountPeak L f
81c16b60  00000000
81c16b64  00000021
81c16b68  00000000
81c16b6c  00000022
81c16b70  0000000f
81c16b74  0000001e
81c16b78  0000007e
81c16b7c  00000000
81c16b80  000000e3
81c16b84  00000027
81c16b88  00000000
81c16b8c  00000008
81c16b90  00000059
81c16b94  00000000
81c16b98  0000000b
```

Theoretically, the different virtual address ranges assigned to components can grow arbitrarily in size if enough system virtual address space is available. In practice, on 32-bit systems, the kernel allocator

implements the ability to set limits on each virtual address type for the purposes of both reliability and stability. (On 64-bit systems, kernel address space exhaustion is currently not a concern.) Although no limits are imposed by default, system administrators can use the registry to modify these limits for the virtual address types that are currently marked as limitable (see [Table 5-8](#)).

If the current request during the `MiObtainSystemVa` call exceeds the available limit, a failure is marked (see the previous experiment) and a reclaim operation is requested regardless of available memory. This should help alleviate memory load and might allow the virtual address allocation to work during the next attempt. Recall, however, that reclaiming affects only system cache and non-paged pool.

EXPERIMENT: Setting system virtual address limits

The `MiSystemVaTypeCountLimit` array contains limitations for system virtual address usage that can be set for each type. Currently, the memory manager allows only certain virtual address types to be limited, and it provides the ability to use an undocumented system call to set limits for the system dynamically during run time. (These limits can also be set through the registry, as described at <http://msdn.microsoft.com/en-us/library/bb870880.aspx>.) These limits can be set for those types marked in [Table 5-8](#).

You can use the `MemLimit` utility (found in this book's downloadable resources) on 32-bit systems to query and set the different limits for these types and to see the current and peak virtual address space usage. Here's how you can query the current limits with the `-q` flag:

[Click here to view code image](#)

```
C:\Tools>MemLimit.exe -q
```

```
MemLimit v1.01 - Query and set hard limits on system VA space
consumption
```

```
Copyright (C) 2008-2016 by Alex Ionescu
www.alex-ionescu.com
```

```
System Va Consumption:
```

Type	Current	Peak	Limit
Non Paged Pool	45056 KB	55296 KB	0 KB
Paged Pool	151552 KB	165888 KB	0 KB
System Cache	446464 KB	479232 KB	0 KB
System PTEs	90112 KB	135168 KB	0 KB
Session Space	63488 KB	73728 KB	0 KB

As an experiment, use the following command to set a limit of 100 MB for paged pool:

```
memlimit.exe -p 100M
```

Now use the Sysinternals `TestLimit` tool to create as many handles as possible. Normally, with enough paged pool, the number should be around 16 million. But with the limit to 100 MB, it's less:

[Click here to view code image](#)

```
C:\Tools\Sysinternals>Testlimit.exe -h
```

```
Testlimit v5.24 - test Windows limits
```

Process ID: 4780

Creating handles...

Created **10727844** handles. Lasterror: 1450

See Chapter 8 in Part 2 for more information about objects, handles, and page-pool consumption.

System virtual address space quotas

The system virtual address space limits described in the previous section allow for the limiting of system-wide virtual address space usage of certain kernel components. However, they work only on 32-bit systems when applied to the system as a whole. To address more specific quota requirements that system administrators might have, the memory manager collaborates with the process manager to enforce either system-wide or user-specific quotas for each process.

You can configure the `PagedPoolQuota`, `NonPagedPoolQuota`, `PagingFileQuota`, and `WorkingSetPagesQuota` values in the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key to specify how much memory of each type a given process can use. This information is read at initialization, and the default system quota block is generated and then assigned to all system processes. (User processes get a copy of the default system quota block unless per-user quotas have been configured, as explained next.)

To enable per-user quotas, you can create subkeys under the `HKLM\SYSTEM\CurrentControlSet\Session Manager\Quota System` registry key, each one representing a given user SID. The values mentioned previously can then be created under this specific SID subkey, enforcing the limits only for the processes created by that user. [Table 5-9](#) shows how to configure these values (which can be done at run time or not) and which privileges are required.

Value Name	Description	Value Type	Dynamic	Privilege
<code>PagedPoolQuota</code>	This is the maximum size of paged pool that can be allocated by this process.	Size in MB	Only for processes running with the system token	<code>SeIncreaseQuotaPrivilege</code>
<code>NonPagedPoolQuota</code>	This is the maximum size of non-paged pool that can be allocated by this process.	Size in MB	Only for processes running with the system token	<code>SeIncreaseQuotaPrivilege</code>
<code>PagingFileQuota</code>	This is the maximum number of pages that a process can have backed by the page file.	Pages	Only for processes running with the system token	<code>SeIncreaseQuotaPrivilege</code>
<code>WorkingSetPagesQuota</code>	This is the maximum number of pages that a process can have in its working set (in physical memory).	Pages	Yes	<code>SeIncreaseBasePriorityPrivilege</code> unless operation is a purge request

TABLE 5-9 Process quota types

User address space layout

Just as address space in the kernel is dynamic, the user address space is also built dynamically. The addresses of the thread stacks, process heaps, and loaded images (such as DLLs and an application's executable) are dynamically computed (if the application and its images support it) through the ASLR mechanism.

At the operating system level, user address space is divided into a few well-defined regions of memory, as shown in [Figure 5-14](#). The executable and DLLs themselves are present as memory-mapped image files, followed by the heap(s) of the process and the stack(s) of its thread(s). Apart from these regions (and some reserved system structures such as the TEBs and PEB), all other memory allocations are run-time dependent and generated. ASLR is involved with the location of all these run-time-dependent regions and, combined with DEP, provides a mechanism for making remote exploitation of a system through memory manipulation harder to achieve. Because Windows code and data are placed at dynamic locations, an attacker cannot typically hard-code a meaningful offset into either a program or a system-supplied DLL.

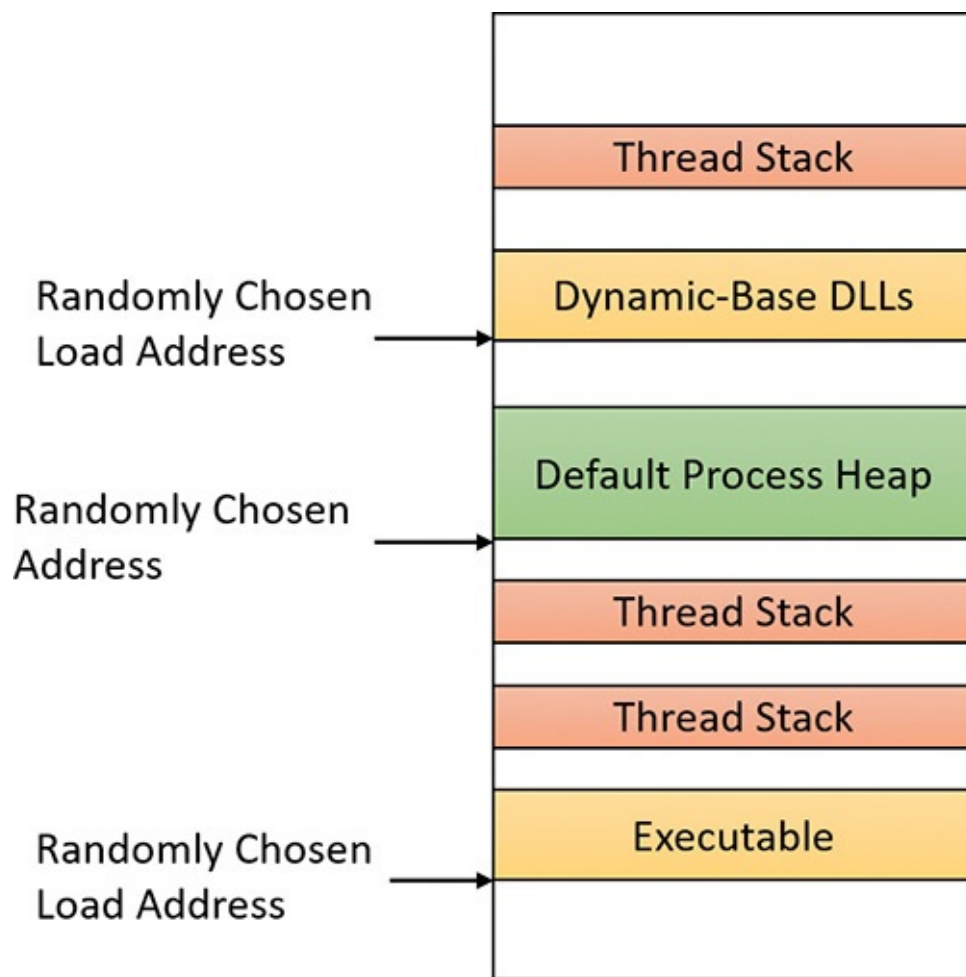


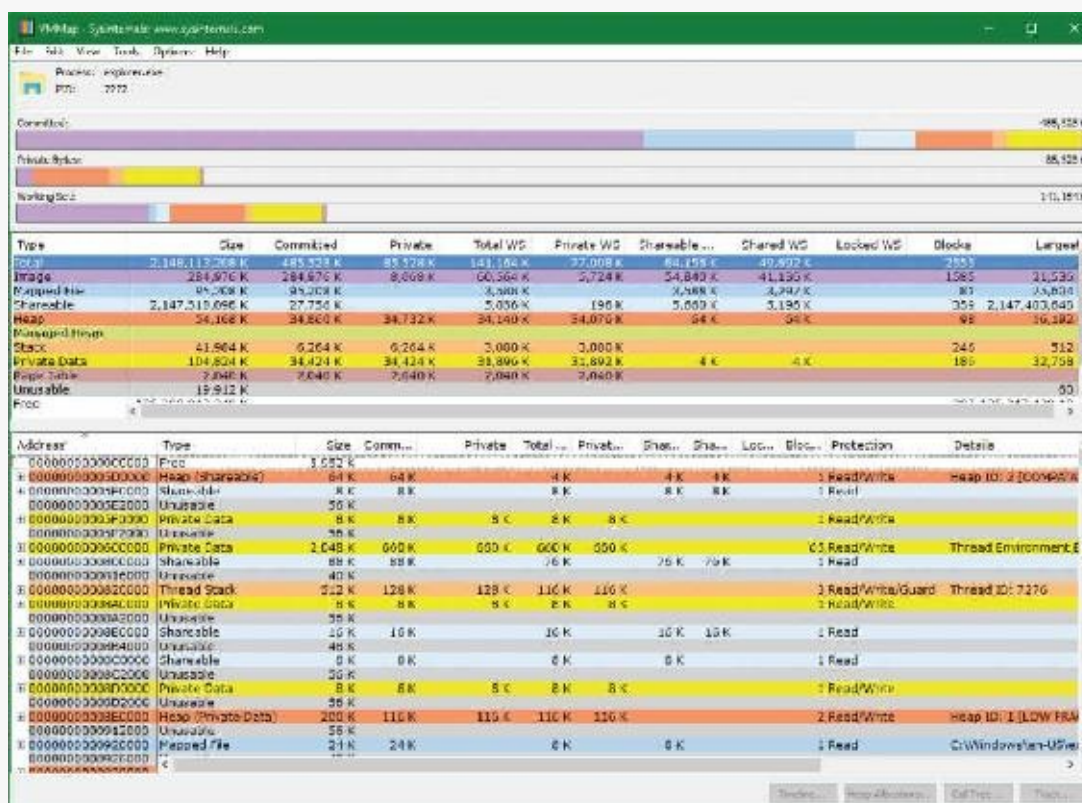
FIGURE 5-14 User address space layout with ASLR enabled.

EXPERIMENT: Analyzing user virtual address space

The VMMap utility from Sysinternals can show you a detailed view of the virtual memory being used by any process on your machine. This information is divided into categories for each type of allocation, summarized as follows:

- **Image** This displays memory allocations used to map the executable and its dependencies (such as dynamic libraries) and any other memory-mapped image (portable executable format) files.
- **Mapped File** This displays memory allocations for memory mapped data files.
- **Shareable** This displays memory allocations marked as shareable, typically including shared memory (but not memory-mapped files, which are listed under either Image or Mapped File).
- **Heap** This displays memory allocated for the heap(s) that this process owns.
- **Managed Heap** This displays memory allocated by the .NET CLR (managed objects). It would show nothing for a process that does not use .NET.
- **Stack** This displays memory allocated for the stack of each thread in this process.
- **Private Data** This displays memory allocations marked as private other than the stack and heap, such as internal data structures.

The following shows a typical view of Explorer (64 bit) as seen through VMMap:



Depending on the type of memory allocation, VMMap can show additional information such as file names (for mapped files), heap IDs and types (for heap allocations), and thread IDs (for stack allocations). Furthermore, each allocation's cost is shown both in committed memory and working set memory. The size and protection of each allocation is also displayed.

ASLR begins at the image level, with the executable for the process and its dependent DLLs. Any image file that has specified ASLR support in its PE header (IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE), typically specified by using the

/DYNAMICBASE linker flag in Microsoft Visual Studio, and contains a relocation section will be processed by ASLR. When such an image is found, the system selects an image offset valid globally for the current boot. This offset is selected from a bucket of 256 values, all of which are 64 KB aligned.

Image randomization

For executables, the load offset is calculated by computing a delta value each time an executable is loaded. This value is a pseudo-random 8-bit number from 0x10000 to 0xFE0000, calculated by taking the current processor's time stamp counter (TSC), shifting it by four places, and then performing a division modulo 254 and adding 1. This number is then multiplied by the allocation granularity of 64 KB discussed earlier. By adding 1, the memory manager ensures that the value can never be 0, so executables will never load at the address in the PE header if ASLR is being used. This delta is then added to the executable's preferred load address, creating one of 256 possible locations within 16 MB of the image address in the PE header.

For DLLs, computing the load offset begins with a per-boot, system-wide value called the *image bias*. This is computed by `MiInitializeRelocations` and stored in the global memory state structure (`MI_SYSTEM_INFORMATION`) in the `MiState.Sections.ImageBias` fields (`MiImageBias` global variable in Windows 8.x/2012/R2). This value corresponds to the TSC of the current CPU when this function was called during the boot cycle, shifted and masked into an 8-bit value. This provides 256 possible values on 32 bit systems; similar computations are done for 64-bit systems with more possible values as the address space is vast. Unlike executables, this value is computed only once per boot and shared across the system to allow DLLs to remain shared in physical memory and relocated only once. If DLLs were remapped at different locations inside different processes, the code could not be shared. The loader would have to fix up address references differently for each process, thus turning what had been shareable read-only code into process-private data. Each process using a given DLL would have to have its own private copy of the DLL in physical memory.

Once the offset is computed, the memory manager initializes a bitmap called `ImageBitMap` (`MiImageBitMap` global variable in Windows 8.x/2012/R2), which is part of the `MI_SECTION_STATE` structure. This bitmap is used to represent ranges from 0x50000000 to 0x78000000 for 32-bit systems (see the numbers for 64-bit systems below), and each bit represents one unit of allocation (64 KB, as mentioned). Whenever the memory manager loads a DLL, the appropriate bit is set to mark its location in the system. When the same DLL is loaded again, the memory manager shares its section object with the already relocated information.

As each DLL is loaded, the system scans the bitmap from top to bottom for free bits. The `ImageBias` value computed earlier is used as a start index from the top to randomize the load across different boots as suggested. Because the bitmap will be entirely empty when the first DLL (which is always `Ntdll.dll`) is loaded, its load address can easily be calculated. (Sixty-four-bit systems have their own bias.)

- **32 bit** $0x78000000 - (\text{ImageBias} + \text{NtdllSizein64KBChunks}) * 0x10000$
- **64 bit** $0x7FFFFFFFF0000 - (\text{ImageBias64High} + \text{NtdllSizein64KBChunks}) * 0x10000$

Each subsequent DLL will then load in a 64 KB chunk below. Because of this, if the address of `Ntdll.dll` is known, the addresses of other DLLs could easily be computed. To mitigate this possibility, the order in which known DLLs are mapped by the Session Manager during initialization is also randomized when `Smss.exe` loads.

Finally, if no free space is available in the bitmap (which would mean that most of the region defined for ASLR is in use), the DLL relocation code defaults back to the executable case, loading the DLL at a 64 KB chunk within 16 MB of its preferred base address.

EXPERIMENT: Calculating the load address of Ntdll.dll

With what you learned in the previous section, you can calculate the load address of Ntdll.dll with the kernel variable information. The following calculation is done on a Windows 10 x86 system:

1. Start local kernel debugging.
2. Find the ImageBias value:

[Click here to view code image](#)

```
lkd> ? nt!mystate
Evaluate expression: -2113373760 = 820879c0
lkd> dt nt!_mi_system_information sections.imagebias 820879c0
      +0x500 Sections          :
      +0x0dc ImageBias         : 0x6e
```

3. Open Explorer and find the size of Ntdll.dll in the System32 directory. On this system, it's 1547 KB = 0x182c00, so the size in 64 KB chunks is 0x19 (always rounding up). The result is $0x78000000 - (0x6E + 0x19) * 0x10000 = 0x77790000$.
4. Open Process Explorer, find any process, and look at the load address (in the Base or Image Base columns) of Ntdll.dll. You should see the same value.
5. Try to do the same for a 64-bit system.

Stack randomization

The next step in ASLR is to randomize the location of the initial thread's stack and, subsequently, of each new thread. This randomization is enabled unless the `StackRandomizationDisabled` flag was enabled for the process, and consists of first selecting one of 32 possible stack locations separated by either 64 KB or 256 KB. You select this base address by finding the first appropriate free memory region and then choosing the x th available region, where x is once again generated based on the current processor's TSC shifted and masked into a 5-bit value. (This allows for 32 possible locations.)

When this base address has been selected, a new TSC-derived value is calculated—this one 9 bits long. The value is then multiplied by 4 to maintain alignment, which means it can be as large as 2,048 bytes (half a page). It is added to the base address to obtain the final stack base.

Heap randomization

ASLR randomizes the location of the initial process heap and subsequent heaps when created in user mode. The `RtlCreateHeap` function uses another pseudo-random TSC-derived value to determine the base address of the heap. This value, 5 bits this time, is multiplied by 64 KB to generate the final base address, starting at 0, giving a possible range of 0x00000000 to 0x001F0000 for the initial heap. Additionally, the range before the heap base address is manually deallocated to force an access violation if an attack is doing a brute-force sweep of the entire possible heap address range.

ASLR in kernel address space

ASLR is also active in kernel address space. There are 64 possible load addresses for 32-bit drivers and 256 for 64-bit drivers. Relocating user-space images requires a significant amount of work area in kernel space, but if kernel space is tight, ASLR can use the user-mode address space of the System process for this work area. On Windows 10 (version 1607) and Server 2016, ASLR is implemented for most system memory regions, such as paged and non-paged pools, system cache, page tables, and the PFN database (initialized by `MiAssignTopLevelRanges`).

Controlling security mitigations

As you've seen, ASLR and many other security mitigations in Windows are optional because of their potential compatibility effects: ASLR applies only to images with the `IMAGE_DLL_CHARACTERISTICS_DYNAMIC_BASE` bit in their image headers, hardware no-execute (DEP) can be controlled by a combination of boot options and linker options, and so on. To allow both enterprise customers and individual users more visibility and control of these features, Microsoft publishes the Enhanced Mitigation Experience Toolkit (EMET). EMET offers centralized control of the mitigations built into Windows and adds several more mitigations not yet part of the Windows product. Additionally, EMET provides notification capabilities through the event log to let administrators know when certain software has experienced access faults because mitigations have been applied. Finally, EMET enables manual opt-out for certain applications that might exhibit compatibility issues in certain environments, even though they were opted in by the developer.



Note

EMET is in version 5.51 at the time of this writing. Its end of support has been extended to the end of July 2018. However, some of its features are integrated in current Windows versions.

EXPERIMENT: Looking at ASLR protection on processes

You can use Process Explorer from Sysinternals to look over your processes (and, just as important, the DLLs they load) to see if they support ASLR. Even if just one DLL loaded by a process does not support ASLR, it can make the process much more vulnerable to attacks.

To look at the ASLR status for processes, follow these steps:

1. Right-click any column in the process tree and choose **Select Columns**.
2. Select **ASLR Enabled** on the Process Image and DLL tabs.
3. Notice that all in-box Windows programs and services are running with ASLR enabled, but third-party applications may or may not run with ASLR.

In the example, we have highlighted the Notepad.exe process. In this case, its load address is 0x7FF7D76B0000. If you were to close all instances of Notepad and then start another, you would find it at a different load address. If you shut down and reboot the system and then try the experiment again, you will find that the ASLR-enabled DLLs are at different load addresses after each boot.

[illegible]

Address translation

Now that you've seen how Windows structures the virtual address space, let's look at how it maps these address spaces to real physical pages. User applications and system code reference virtual addresses. This section starts with a detailed description of 32-bit x86 address translation in PAE mode (the only mode supported in recent versions of Windows) and continues with a description of the differences on the ARM and x64 platforms. The next section describes what happens when such a translation doesn't resolve to a physical memory address (page fault) and explains how Windows manages physical memory via working sets and the page frame database.

x86 virtual address translation

The original x86 kernel supported no more than 4 GB of physical memory, based on the CPU hardware available at the time. The Intel x86 Pentium Pro processor introduced a memory-mapping mode called *Physical Address Extension (PAE)*. With the proper chipset, the PAE mode allows 32-bit operating systems access to up to 64 GB of physical memory on current Intel x86 processors (up from 4 GB without PAE) and up to 1,024 GB of physical memory when running on x64 processors in legacy mode (although Windows currently limits this to 64 GB due to the size of the PFN database required to describe so much memory). Since then, Windows has maintained two separate x86 kernels—one that did not support PAE and one that did. Starting with Windows Vista, an x86 Windows installation always installs the PAE kernel even if the system's physical memory is not higher than 4 GB. This allows Microsoft to maintain a single x86 kernel, as the benefits of the non-PAE kernel in terms of performance and memory footprint became negligible (and is required for hardware no-execute support). Thus, we'll describe only x86 PAE address translation. Interested readers can read the relevant section in the sixth edition of this book for the non-PAE case.

Using data structures that the memory manager creates and maintains called *page tables*, the CPU translates virtual addresses into physical addresses. Each page of virtual address space is associated with a system-space structure called a *page table entry (PTE)*, which contains the physical address to which the virtual one is mapped. For example, [Figure 5-15](#) shows how three consecutive virtual pages might be mapped to three physically discontinuous pages on an x86 system. There may not even be any PTEs for regions that have been marked as reserved or committed but never accessed, because the page table itself might be allocated only when the first page fault occurs. (The dashed line connecting the virtual pages to the PTEs in [Figure 5-15](#) represents the indirect relationship between virtual pages and physical memory.)

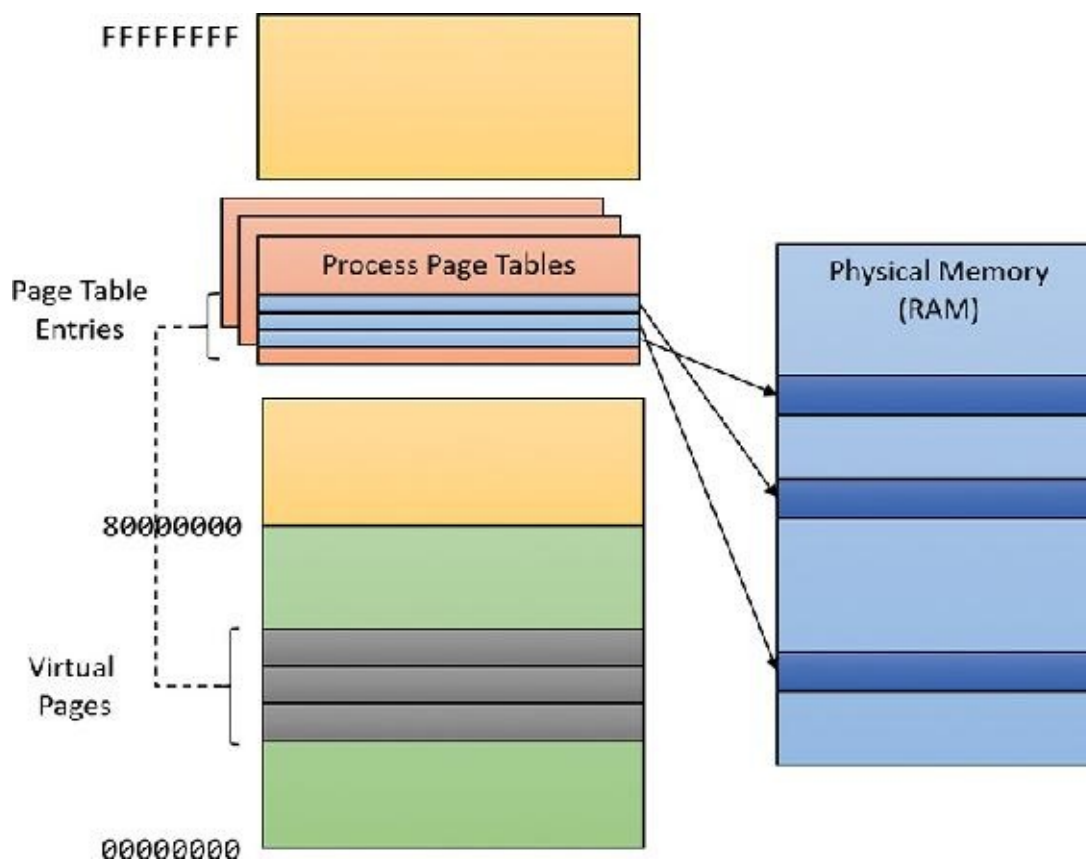


FIGURE 5-15 Mapping virtual addresses to physical memory (x86).



Note

Even kernel-mode code (such as device drivers) cannot reference physical memory addresses directly, but it may do so indirectly by first creating virtual addresses mapped to them. For more information, see the memory descriptor list (MDL) support routines described in the WDK documentation.

The actual translation process and the layout of the page tables and page directories (described shortly), are determined by the CPU. The operating system must follow suit and build the structures correctly in memory for the whole concept to work. [Figure 5-16](#) depicts a general diagram of x86 translation. The general concept, however, is the same for other architectures.

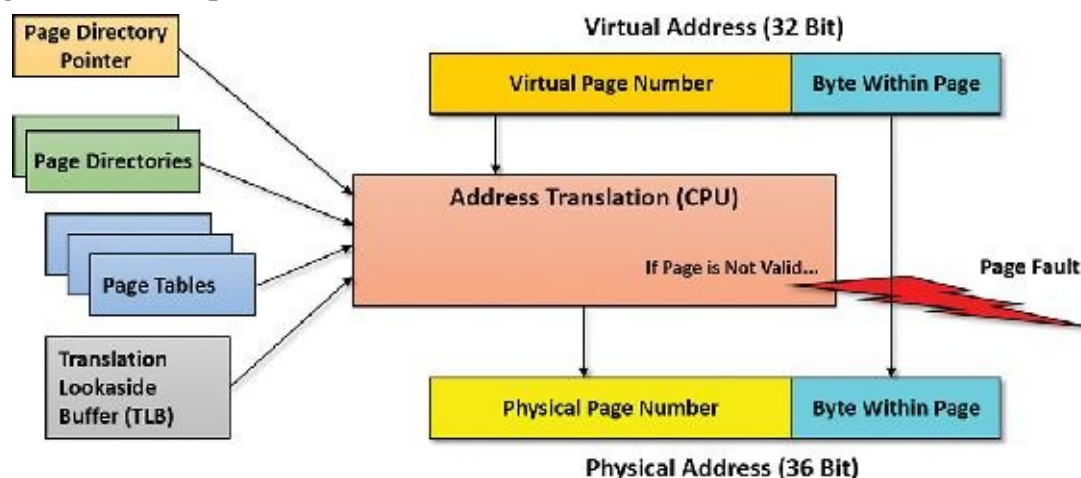


FIGURE 5-16 Virtual address translation overview.

As shown in [Figure 5-16](#), the input to the translation system consists of a 32-bit virtual address (since this is the addressable range with 32 bit) and a bunch of memory-related structures (page tables, page directories, a single page directory pointer table [PDPT], and translation lookaside buffers, all described shortly). The output should be a 36-bit physical address in RAM where the actual byte is located. The number 36 comes from the way the page tables are structured and, as mentioned, dictated by the processor. When mapping small pages (the common case, shown in [Figure 5-16](#)), the least significant 12 bits from the virtual address are copied directly to the resulting physical address. 12 bits is exactly 4 KB—the size of a small page.

If the address cannot be translated successfully (for example, the page may not be in physical memory but resides in a page file), the CPU throws an exception known as a *page fault* that indicates to the OS that the page cannot be located. Because the CPU has no idea where to find the page (page file, mapped file, or something else), it relies on the OS to get the page from wherever it's located (if possible), fix the page tables to point to it, and request that the CPU tries translation again. (Page faults are described in the section “[Page files](#)” later in this chapter.)

[Figure 5-17](#) depicts the entire process of translating x86 virtual to physical addresses.

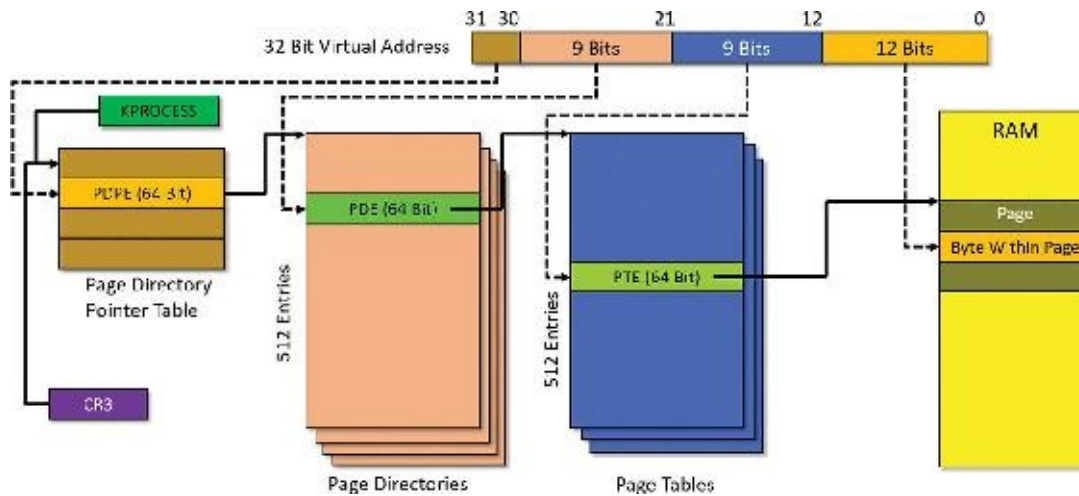


FIGURE 5-17 x86 virtual address translation.

The 32-bit virtual address to be translated is logically segregated into four parts. As you've seen, the lower 12 bits are used as-is to select a specific byte within a page. The translation process starts with a single PDPT per process, which resides in physical memory at all times. (Otherwise, how would the system locate it?) Its physical address is stored in the KPROCESS structure for each process. The special x86 register CR3 stores this value for the currently executing process (that is, one of its threads made the access to the virtual address). This means that when a context switch occurs on a CPU, if the new thread is running under a different process than the old thread, then the CR3 register must be loaded with the new process's page directory pointer address from its KPROCESS structure. The PDPT must be aligned on a 32-byte boundary and must furthermore reside in the first 4 GB of RAM (because CR3 on x86 is still a 32-bit register).

Given the layout in [Figure 5-17](#), the sequence of translating a virtual address to a physical one goes as follows:

1. The two most significant bits of the virtual address (bits 30 and 31) provide an index into the PDPT. This table has four entries. The entry selected—the page directory pointer entry (PDPE)—points to the physical address of a page directory.
2. A page directory contains 512 entries, one of which is selected by bits 21 to 29 (9 bits) from the virtual address. The selected page directory entry (PDE) points to the physical address of a page table.
3. A page table also contains 512 entries, one of which is selected by bits 13 to 28 (9 bits) from the virtual address. The selected page table entry (PTE) points to the physical address of the start of the page.
4. The virtual address offset (lower 12 bits) is added to the PTE pointed-to address to give the final physical address requested by the caller.

Every entry value in the various tables is also called a page frame number (PFN) because it points to a page-aligned address. Each entry is 64 bits wide—so the size of a page directory or page table is no larger than a 4 KB page—but only 24 bits are strictly necessary to describe a 64 GB physical range (combined with the 12 bits of the offset for an address range with a total of 36 bits). This means there are more bits than needed for the actual PFN values.

One of the extra bits in particular is paramount to the whole mechanism: the valid bit. This bit indicates whether the PFN data is indeed valid and therefore whether the CPU should execute the procedure just outlined. If the bit is clear, however, it indicates a page fault. The CPU raises an exception and expects the OS to handle the page fault in some meaningful way. For example, if the page in question was

previously written to disk, then the memory manager should read it back to a free page in RAM, fix the PTE, and tell the CPU to try again.

Because Windows provides a private address space for each process, each process has its own PDPT, page directories, and page tables to map that process’s private address space. However, the page directories and page tables that describe system space are shared among all processes (and session space is shared only among processes in a session). To avoid having multiple page tables describing the same virtual memory, the page directory entries that describe system space are initialized to point to the existing system page tables when a process is created. If the process is part of a session, session space page tables are also shared by pointing the session space page directory entries to the existing session page tables.

Page tables and page table entries

Each PDE points to a page table. A page table is a simple array of PTEs. So, too, is a PDPT. Every page table has 512 entries and each PTE maps a single page (4 KB). This means a page table can map a 2 MB address space (512 x 4 KB). Similarly, a page directory has 512 entries, each pointing to a page table. This means a page directory can map 512 x 2 MB or 1 GB of address space. This makes sense because there are four PDPEs, which together can map the entire 32-bit 4 GB address space.

For large pages, the PDE points with 11 bits to the start of a large page in physical memory, where the byte offset is taken for the low 21 bits of the original virtual address. This means such a PDE mapping a large page does not point to any page table.

The layout of a page directory and page table is essentially the same. You can use the kernel debugger !pte command to examine PTEs. (See the upcoming experiment “[Translating addresses](#).”) We’ll discuss valid PTEs here and invalid PTEs in the section “[Page fault handling](#).” Valid PTEs have two main fields: the PFN of the physical page containing the data or of the physical address of a page in memory, and some flags that describe the state and protection of the page. (See [Figure 5-18](#).)

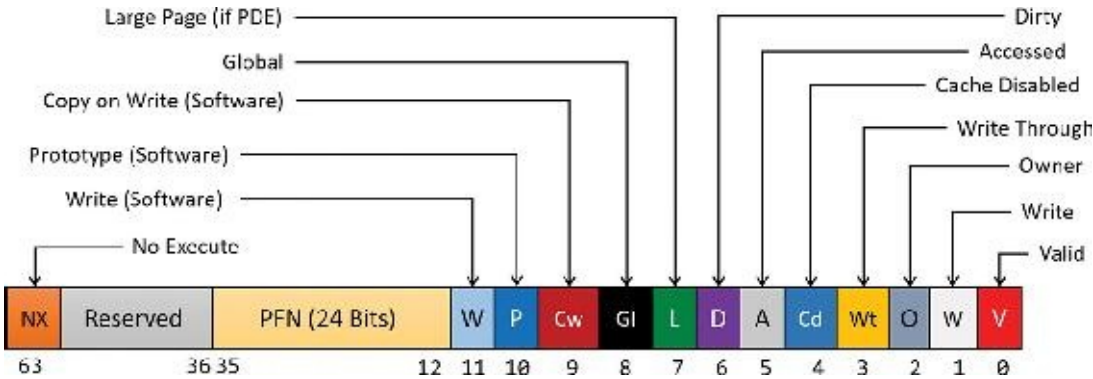


FIGURE 5-18 Valid x86 hardware PTEs.

The bits labeled “Software” and “Reserved” in [Figure 5-18](#) are ignored by the memory management unit (MMU) inside the CPU regardless of whether the PTE is valid. These bits are stored and interpreted by the memory manager. [Table 5-10](#) briefly describes the hardware-defined bits in a valid PTE.

Name of Bit	Meaning
Accessed	The page has been accessed.
Cache disabled	This disables CPU caching for that page.
Copy-on-write	The page is using copy-on-write (described earlier).
Dirty	The page has been written to.
Global	Translation applies to all processes. For example, a translation buffer flush won't affect this PTE.
Large page	This indicates that the PDE maps a 2 MB page. (Refer to the section "Large and small pages" earlier in this chapter.)
No execute	This indicates that code cannot execute in the page. (It can be used for data only.)
Owner	This indicates whether user-mode code can access the page or whether the page is limited to kernel-mode access.
Prototype	The PTE is a prototype PTE, which is used as a template to describe shared memory associated with section objects.
Valid	This indicates whether the translation maps to a page in physical memory.
Write through	This marks the page as write-through or, if the processor supports the page attribute table, write-combined. This is typically used to map video frame buffer memory.
Write	This indicates to the MMU whether the page is writable.

TABLE 5-10 PTE status and protection bits

On x86 systems, a hardware PTE contains two bits that can be changed by the MMU: the dirty bit and the accessed bit. The MMU sets the accessed bit whenever the page is read or written (provided it is not already set). The MMU sets the dirty bit whenever a write operation occurs to the page. The operating system is responsible for clearing these bits at the appropriate times. They are never cleared by the MMU.

The x86 MMU uses a write bit to provide page protection. When this bit is clear, the page is read-only. When it is set, the page is read/write. If a thread attempts to write to a page with the write bit clear, a memory-management exception occurs. In addition, the memory manager's access fault handler (described later in the section "[Page fault handling](#)") must determine whether the thread can be allowed to write to the page (for example, if the page was really marked copy-on-write) or whether an access violation should be generated.

Hardware versus software write bits in page table entries

The additional write bit implemented in software (refer to [Table 5-10](#)) is used to force an update of the dirty bit to be synchronized with updates to Windows memory management data. In a simple implementation, the memory manager would set the hardware write bit (bit 1) for any writable page. A write to any such page will cause the MMU to set the dirty bit in the PTE. Later, the dirty bit will tell the memory manager that the contents of that physical page must be written to backing store before the physical page can be used for something else.

In practice, on multiprocessor systems, this can lead to race conditions that are expensive to resolve. At any time, the MMUs of the various processors can set the dirty bit of any PTE that has its hardware write bit set. The memory manager must, at various times, update the process working set list to reflect the state of the dirty bit in a PTE. The memory manager uses a pushlock to synchronize access to the working set list. But on a multiprocessor system, even while one processor is holding the lock, the dirty bit might be changed by MMUs of other CPUs. This raises the possibility of missing an update to a dirty bit.

To avoid this, the Windows memory manager initializes both read-only and writable pages with the hardware write bit (bit 1) of their PTEs set to 0 and records the true writable state of the page in the software write bit (bit 11). On the first write access to such a page, the processor will raise a memory-

management exception because the hardware write bit is clear, just as it would be for a true read-only page. In this case, though, the memory manager learns that the page actually *is* writable (via the software write bit), acquires the working set pushlock, sets the dirty bit and the hardware write bit in the PTE, updates the working set list to note that the page has been changed, releases the working set pushlock, and dismisses the exception. The hardware write operation then proceeds as usual, but the setting of the dirty bit is made to happen with the working set list pushlock held.

On subsequent writes to the page, no exceptions occur because the hardware write bit is set. The MMU will redundantly set the dirty bit, but this is benign because the “written-to” state of the page is already recorded in the working set list. Forcing the first write to a page to go through this exception handling may seem to be excessive overhead. However, it happens only once per writable page as long as the page remains valid. Furthermore, the first access to almost any page already goes through memory-management exception handling because pages are usually initialized in the invalid state (the PTE bit 0 is clear). If the first access to a page is also the first write access to the page, the dirty bit handling just described will occur within the handling of the first-access page fault, so the additional overhead is small. Finally, on both uniprocessor and multiprocessor systems, this implementation allows flushing of the translation look-aside buffer (described in the next section) without holding a lock for each page being flushed.

Translation look-aside buffer

As you’ve learned, each hardware address translation requires three lookups:

- One to find the right entry in the PDPT
- One to find the right entry in the page directory (which provides the location of the page table)
- One to find the right entry in the page table

Because doing three additional memory lookups for every reference to a virtual address would quadruple the required bandwidth to memory, resulting in poor performance, all CPUs cache address translations so that repeated accesses of the same addresses don’t have to be repeatedly translated. This cache is an array of associative memory called the *translation lookaside buffer (TLB)*. Associative memory is a vector whose cells can be read simultaneously and compared to a target value. In the case of the TLB, the vector contains the virtual-to-physical page mappings of the most recently used pages, as shown in [Figure 5-19](#), and the type of page protection, size, attributes, and so on applied to each page. Each entry in the TLB is like a cache entry whose tag holds portions of the virtual address and whose data portion holds a physical page number, protection field, valid bit, and usually a dirty bit indicating the condition of the page to which the cached PTE corresponds. If a PTE’s global bit is set (as is done by Windows for system space pages that are visible to all processes), the TLB entry isn’t invalidated on process context switches.

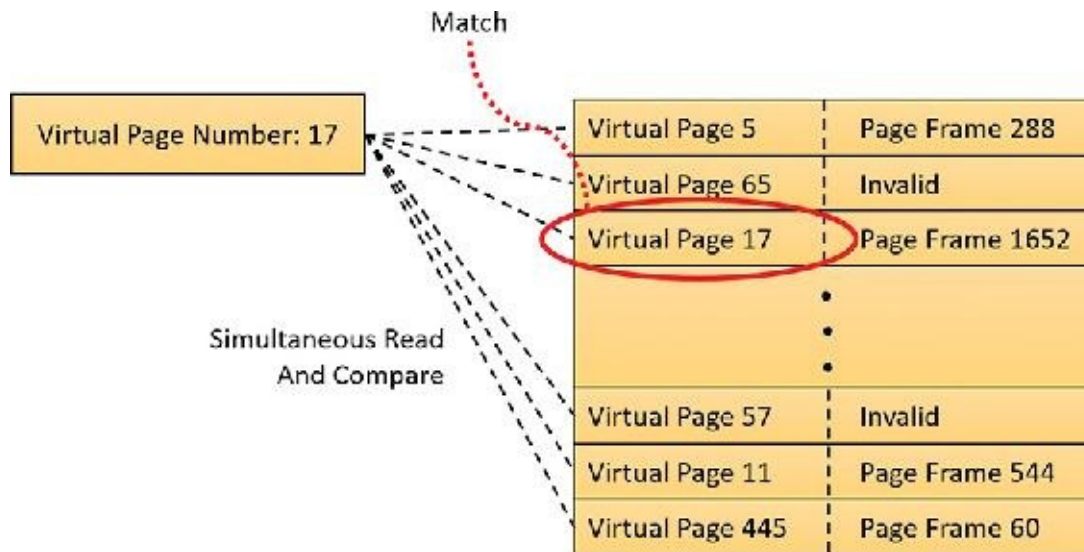


FIGURE 5-19 Accessing the TLB.

Frequently used virtual addresses are likely to have entries in the TLB, which provides extremely fast virtual-to-physical address translation and, therefore, fast memory access. If a virtual address isn't in the TLB, it might still be in memory, but multiple memory accesses are needed to find it, which makes the access time slightly slower. If a virtual page has been paged out of memory or if the memory manager changes the PTE, the memory manager is required to explicitly invalidate the TLB entry. If a process accesses it again, a page fault occurs, and the memory manager brings the page back into memory (if needed) and re-creates its PTE (which then results in an entry for it in the TLB).

EXPERIMENT: Translating addresses

To clarify how address translation works, this experiment shows an example of translating a virtual address on an x86 PAE system using the available tools in the kernel debugger to examine the PDPT, page directories, page tables, and PTEs. In this example, you'll work with a process that has virtual address 0x3166004, currently mapped to a valid physical address. In later examples, you'll see how to follow address translation for invalid addresses with the kernel debugger.

First convert 0x3166004 to binary and break it into the three fields used to translate an address. In binary, 0x3166004 is 11.0001.0110.0110.0000.0000.0100. Breaking it into the component fields yields the following:

31 30 29	21 20	12 11	0
00	00.0011.000	1.0110.0110	0000.0000.0100
Page Directory Pointer Index (0)	Page Directory Index (24)	Page Table Index (0x166 or 358)	Byte Offset (4)

To start the translation process, the CPU needs the physical address of the process's PDPT. This is found in the CR3 register while a thread in that process is running. You can display this address by looking at the DirBase field in the output of the !process command, as shown here:

[Click here to view code image](#)

```
lkd> !process -1 0
PROCESS 99aa3040 SessionId: 2 Cid: 1690 Peb:
03159000 ParentCid: 0920
```

DirBase: 01024800 ObjectTable: b3b386c0 HandleCount: <Data Not Accessible>

Image: windbg.exe

The DirBase field shows that the PDPT is at physical address 0x1024800. As shown in the preceding illustration, the PDPT index field in the sample virtual address is 0. Therefore, the PDPT entry that contains the physical address of the relevant page directory is the first entry in the PDPT, at physical address 0x1024800.

The kernel debugger !pte command displays the PDE and PTE that describe a virtual address, as shown here:

[Click here to view code image](#)

```
lkd> !pte 3166004
                               VA 03166004
PDE at C06000C0               PTE at C0018B30
contains 0000000056238867     contains 800000005DE61867
pfn 56238      - - -DA--UWEV  pfn 5de61      - - -DA--UW-V
```

The debugger does not show the PDPT, but it is easy to display given its physical address:

[Click here to view code image](#)

```
lkd> !dq 01024800 L4
# 1024800 00000000'53c88801 00000000'53c89801
# 1024810 00000000'53c8a801 00000000'53c8d801
```

Here we have used the debugger extension command !dq. This is similar to the dq command (display as quadwords—64 bit values) but lets us examine memory by physical rather than virtual address. Because we know the PDPT is only four entries long, we added the L 4 length argument to keep the output uncluttered.

As illustrated, the PDPT index (the two most significant bits) from the sample virtual address equal 0, so the PDPT entry you want is the first displayed quadword. PDPT entries have a format similar to PDEs and PTEs, so you can see that this one contains a PFN of 0x53c88 (always page-aligned), for a physical address of 0x53c88000. That's the physical address of the page directory.

The !pte output shows the PDE address 0xC06000C0 as a virtual address, not physical. On x86 systems, the first process page directory starts at virtual address 0xC0600000. In this case, the PDE address is 0xC0—that is, 8 bytes (the size of an entry) times 24 added to the page directory start address. Therefore, the page directory index field of the sample virtual address is 24. That means you're looking at the 25th PDE in the page directory.

The PDE provides the PFN of the needed page table. In this example, the PFN is 0x56238, so the page table starts at physical address 0x56238000. To this the MMU will add the page table index field (0x166) from the virtual address multiplied by 8 (the size of a PTE in bytes). The resulting physical address of the PTE is 0x56238B30.

The debugger shows that this PTE is at *virtual* address 0xC0018B30. Notice that the byte offset portion (0xB30) is the same as that from the physical address, as is always the case in address translation. Because the memory manager maps page tables starting at 0xC0000000, adding 0xB30 to 0xC0018000 (the 0x18 is entry 24 as you've seen) yields the virtual address shown in the kernel debugger output: 0xC0018B30. The debugger shows that the PFN field of the PTE is 0x5DE61.

Finally, you can consider the byte offset from the original address. As described, the MMU concatenates the byte offset to the PFN from the PTE, giving a physical address of 0x5DE61004. This is the physical address that corresponds to the original virtual address of 0x3166004...at the moment.

The flags bits from the PTE are interpreted to the right of the PFN number. For example, the PTE that describes the page being referenced has flags of - - -DA- -UW-V. Here, A stands for accessed (the page has been read), U for user-mode accessible (as opposed to kernel-mode accessible only), W for writable page (rather than just readable), and V for valid (the PTE represents a valid page in physical memory).

To confirm the calculation of the physical address, look at the memory in question via both its virtual and its physical addresses. First, using the debugger's `dd` (display dwords) command on the virtual address, you see the following:

[Click here to view code image](#)

```
lkd> dd 3166004 L 10
03166004  00000034 00000006 00003020 0000004e
03166014  00000000 00020020 0000a000 00000014
```

And with the `!dd` command on the physical address just computed, you see the same contents:

[Click here to view code image](#)

```
lkd> !dd 5DE61004 L 10
#5DE61004 00000034 00000006 00003020 0000004e
#5DE61014 00000000 00020020 0000a000 00000014
```

You could similarly compare the displays from the virtual and physical addresses of the PTE and PDE.

x64 virtual address translation

Address translation on x64 is similar to x86, but with a fourth level added. Each process has a top-level extended page directory called the *page map level 4 table* that contains the physical locations of 512 third-level structures, called *page directory pointers*. The page parent directory is analogous to the x86 PAE PDPT, but there are 512 of them instead of just one, and each page parent directory is an entire page containing 512 entries instead of just four. Like the PDPT, the page parent directory's entries contain the physical locations of second-level page directories, each of which in turn contains 512 entries providing the locations of the individual page tables. Finally, the page tables, each of which contains 512 page table entries, contain the physical locations of the pages in memory. All the "physical locations" in the preceding description are stored in these structures as PFNs.

Current implementations of the x64 architecture limit virtual addresses to 48 bits. The components that make up this 48-bit virtual address and the connection between the components for translation purposes are shown in [Figure 5-20](#), and the format of an x64 hardware PTE is shown in [Figure 5-21](#).

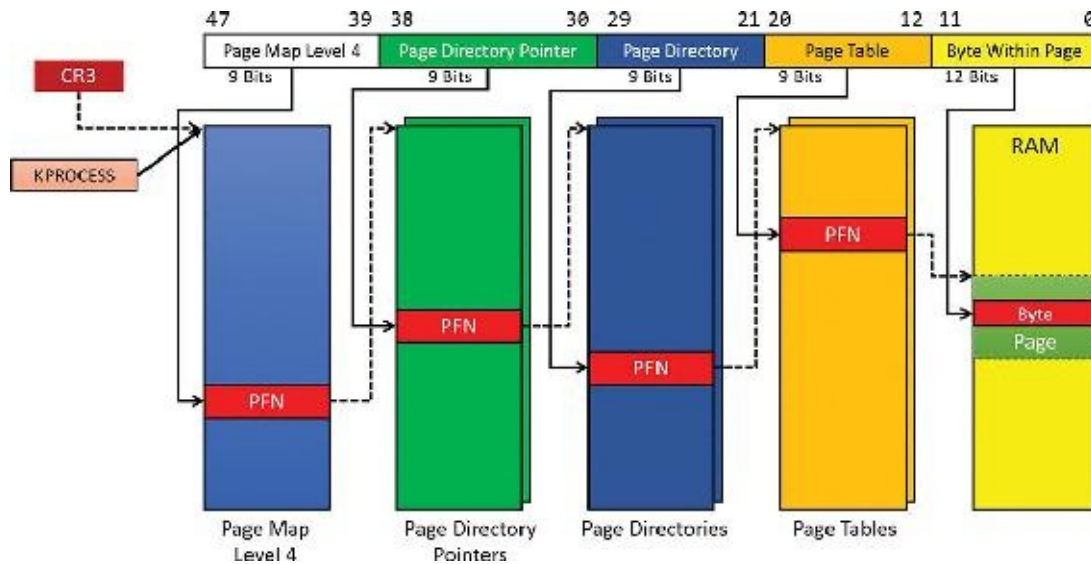


FIGURE 5-20 x64 address translation.

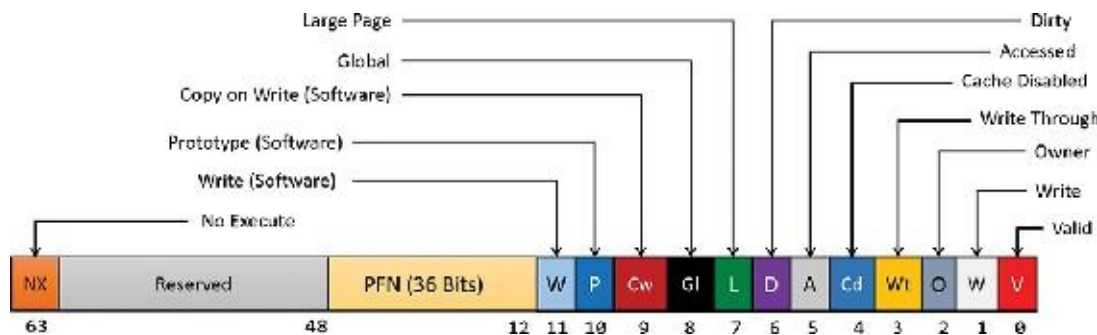


FIGURE 5-21 x64 hardware PTE.

ARM virtual address translation

Virtual address translation on ARM 32-bit processors uses a single page directory with 1,024 entries, each 32 bits in size. The translation structures are shown in [Figure 5-22](#).

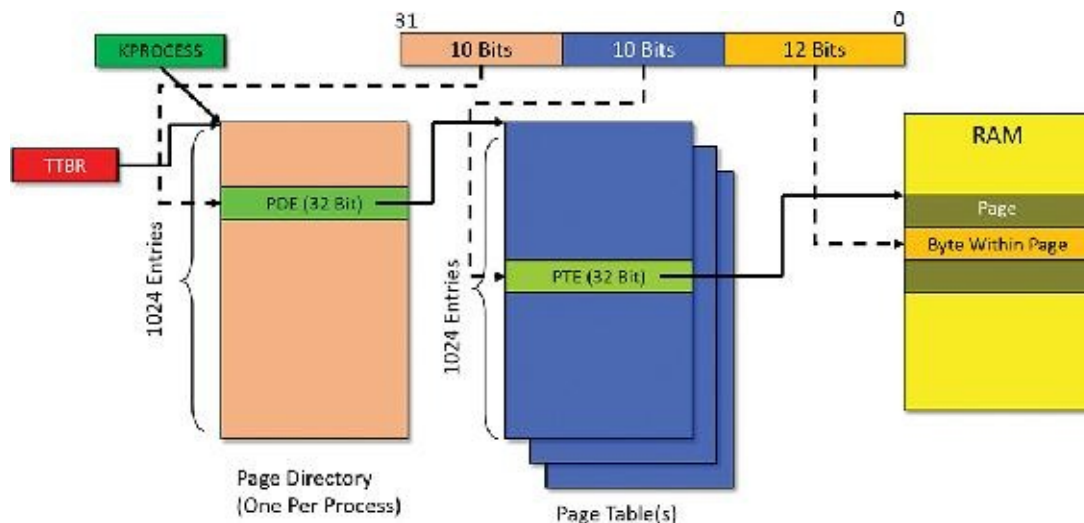


FIGURE 5-22 ARM virtual address translation structures.

Every process has a single page directory, with its physical address stored in the TTBR register (similar to the CR3 x86/x64 register). The 10 most significant bits of the virtual address select a PDE that may point to one of 1,024 page tables. A specific PTE is selected by the next 10 bits of the virtual address. Each valid PTE points to the start of a page in physical memory, where the offset is given by the

lower 12 bits of the address (just as in the x86 and x64 cases). The scheme in [Figure 5-22](#) suggests that the addressable physical memory is 4 GB because each PTE is smaller (32 bits) than the x86/x64 case (64 bits), and indeed only 20 bits are used for the PFN. ARM processors do support a PAE mode (similar to x86), but Windows does not use this functionality. Future Windows versions may support the ARM 64-bit architecture, which will alleviate the physical address limitations as well as dramatically increase the virtual address space for processes and the system.

Curiously, the layout of valid PTE, PDE, and large page PDE are not the same. [Figure 5-23](#) shows the layout of a valid PTE for ARMv7, currently used by Windows. For more information, consult the official ARM documentation.

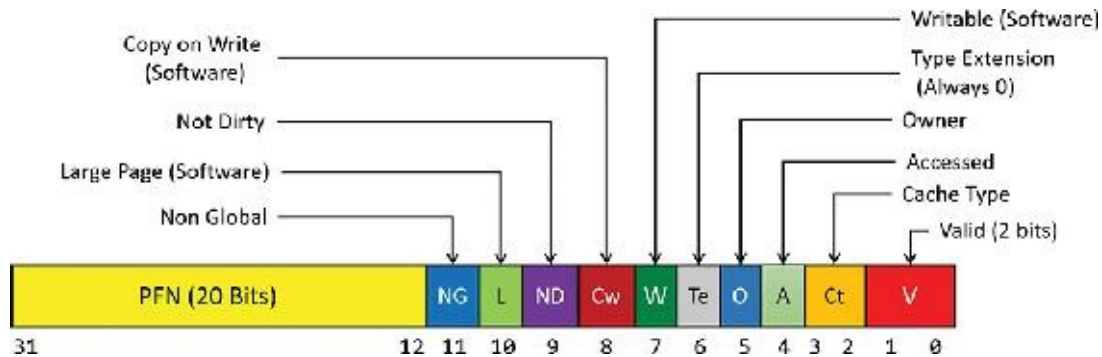


FIGURE 5-23 ARM valid PTE layout.

Page fault handling

Earlier, you saw how address translations are resolved when the PTE is valid. When the PTE valid bit is clear, this indicates that the desired page is for some reason not currently accessible to the process. This section describes the types of invalid PTEs and how references to them are resolved.



Note

Only the 32-bit x86 PTE formats are detailed in this section. PTEs for 64-bit and ARM systems contain similar information, but their detailed layout is not presented.

A reference to an invalid page is called a *page fault*. The kernel trap handler (see the section “Trap dispatching” in Chapter 8 in Part 2) dispatches this kind of fault to the memory manager fault handler function, `MmAccessFault`, to resolve. This routine runs in the context of the thread that incurred the fault and is responsible for attempting to resolve the fault (if possible) or raise an appropriate exception. These faults can be caused by a variety of conditions, as listed in [Table 5-11](#).

Reason for Fault	Result
Corrupt PTE/PDE	Bug-check (crash) the system with code 0x1A (MEMORY_MANAGEMENT).
Accessing a page that isn't resident in memory but is on disk in a page file or a mapped file	Allocate a physical page and read the desired page from disk and into the relevant working set.
Accessing a page that is on the standby or modified list	Transition the page to the relevant process, session, or system working set.
Accessing a page that isn't committed (for example, reserved address space or address space that isn't allocated)	Access violation exception.
Accessing a page from user mode that can be accessed only in kernel mode	Access violation exception.
Writing to a page that is read-only	Access violation exception.
Accessing a demand-zero page	Add a zero-filled page to the relevant working set.
Writing to a guard page	Guard-page violation (if there is a reference to a user-mode stack, perform automatic stack expansion).
Writing to a copy-on-write page	Make a process-private (or session-private) copy of the page and use it to replace the original in the process, session, or system working set.
Writing to a page that is valid but hasn't been written to the current backing store copy	Set the dirty bit in the PTE.
Executing code in a page that is marked as no execute	Access violation exception.
PTE permissions don't match enclave permissions (see the section "Memory enclaves" later in this chapter and the Windows SDK documentation for the CreateEnclave function)	User mode: access violation exception. Kernel mode: bug-check with code 0x50 (PAGE_FAULT_IN_NONPAGED_AREA).

TABLE 5-11 Reasons for access faults

The following section describes the four basic kinds of invalid PTEs that are processed by the access fault handler. Following that is an explanation of a special case of invalid PTEs, called *prototype PTEs*, which are used to implement shareable pages.

Invalid PTEs

If the valid bit of a PTE encountered during address translation is zero, the PTE represents an invalid page—one that will raise a memory-management exception, or page fault, upon reference. The MMU ignores the remaining bits of the PTE, so the operating system can use these bits to store information about the page that will assist in resolving the page fault.

The following list details the four kinds of invalid PTEs and their structure. These are often referred to as *software PTEs* because they are interpreted by the memory manager rather than the MMU. Some of the flags are the same as those for a hardware PTE, as described in [Table 5-10](#), and some of the bit fields have either the same or similar meanings to corresponding fields in the hardware PTE.

- **Page file** The desired page resides within a paging file. As illustrated in [Figure 5-24](#), 4 bits in the PTE indicate in which of 16 possible page files the page resides, and 32 bits provide the page number within the file. The pager initiates an in-page operation to bring the page into memory and make it valid. The page file offset is always non-zero and never all ones (that is, the very first and last pages in the page file are not used for paging) to allow for other formats, described next.

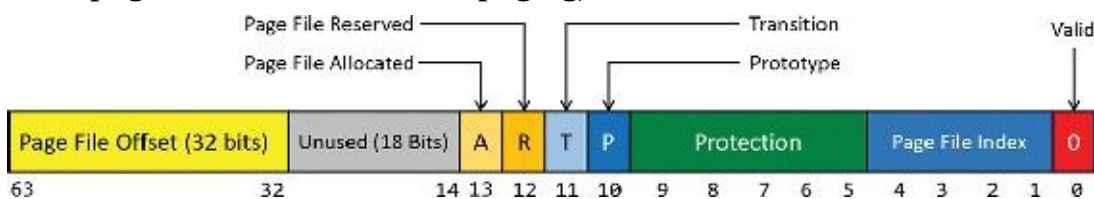


FIGURE 5-24 A PTE representing a page in a page file.

- **Demand zero** This PTE format is the same as the page file PTE shown in the previous entry but the

page file offset is zero. The desired page must be satisfied with a page of zeroes. The pager looks at the zero page list. If the list is empty, the pager takes a page from the free list and zeroes it. If the free list is also empty, it takes a page from one of the standby lists and zeroes it.

- **Virtual Address Descriptor** This PTE format is the same as the page file PTE shown previously but in this case the page file offset field is all one. This indicates a page whose definition and backing store, if any, can be found in the process's Virtual Address Descriptor (VAD) tree. This format is used for pages that are backed by sections in mapped files. The pager finds the VAD that defines the virtual address range encompassing the virtual page and initiates an in-page operation from the mapped file referenced by the VAD. (VADs are described in more detail in the section "[Virtual address descriptors](#)" later in this chapter.)
- **Transition** The transition bit is one. The desired page is in memory on either the standby, modified, or modified-no-write list or not on any list. The pager will remove the page from the list (if it is on one) and add it to the process working set. This is known as a *soft page fault* because no I/O is involved.
- **Unknown** The PTE is zero or the page table doesn't yet exist. (The PDE that would provide the physical address of the page table contains zero.) In both cases, the memory manager must examine the VADs to determine whether this virtual address has been committed. If so, page tables are built to represent the newly committed address space. If not—that is, if the page is reserved or hasn't been defined at all—the page fault is reported as an access violation exception.

Prototype PTEs

If a page can be shared between two processes, the memory manager uses a software structure called *prototype page table entries* (*prototype PTEs*) to map these potentially shared pages. For page-file-backed sections, an array of prototype PTEs is created when a section object is first created. For mapped files, portions of the array are created on demand as each view is mapped. These prototype PTEs are part of the segment structure (described in the section "[Section objects](#)" later in this chapter).

When a process first references a page mapped to a view of a section object (recall that VADs are created only when the view is mapped), the memory manager uses the information in the prototype PTE to fill in the real PTE used for address translation in the process page table. When a shared page is made valid, both the process PTE and the prototype PTE point to the physical page containing the data. To track the number of process PTEs that reference a valid shared page, a counter in its PFN database entry is incremented. Thus, the memory manager can determine when a shared page is no longer referenced by any page table and thus can be made invalid and moved to a transition list or written out to disk.

When a shareable page is invalidated, the PTE in the process page table is filled in with a special PTE that points to the prototype PTE that describes the page, as shown in [Figure 5-25](#). Thus, when the page is accessed, the memory manager can locate the prototype PTE using the information encoded in this PTE, which in turn describes the page being referenced.

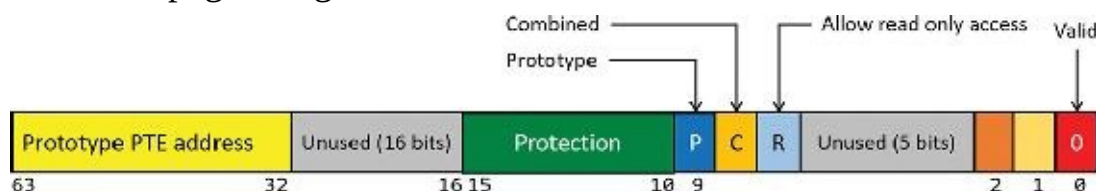


FIGURE 5-25 Structure of an invalid PTE that points to the prototype PTE.

A shared page can be in one of six different states, as described by the prototype PTE:

- **Active/valid** The page is in physical memory because of another process that accessed it.
- **Transition** The desired page is in memory on the standby or modified list (or not on any list).
- **Modified-no-write** The desired page is in memory and on the modified-no-write list. (Refer to [Table 5-11](#).)
- **Demand zero** The desired page should be satisfied with a page of zeroes.
- **Page file** The desired page resides within a page file.
- **Mapped file** The desired page resides within a mapped file.

Although the format of these prototype PTEs is the same as that of the real PTEs described earlier, the prototype PTEs aren't used for address translation. They are a layer between the page table and the PFN database and never appear directly in page tables.

By having all the accessors of a potentially shared page point to a prototype PTE to resolve faults, the memory manager can manage shared pages without needing to update the page tables of each process sharing the page. For example, a shared code or data page might be paged out to disk at some point. When the memory manager retrieves the page from disk, it needs only to update the prototype PTE to point to the page's new physical location. The PTEs in each of the processes sharing the page remain the same, with the valid bit clear and still pointing to the prototype PTE. Later, as processes reference the page, the real PTE will get updated.

[Figure 5-26](#) illustrates two virtual pages in a mapped view. One is valid and the other is invalid. As shown, the first page is valid and is pointed to by the process PTE and the prototype PTE. The second page is in the paging file—the prototype PTE contains its exact location. The process PTE (and any other processes with that page mapped) points to this prototype PTE.

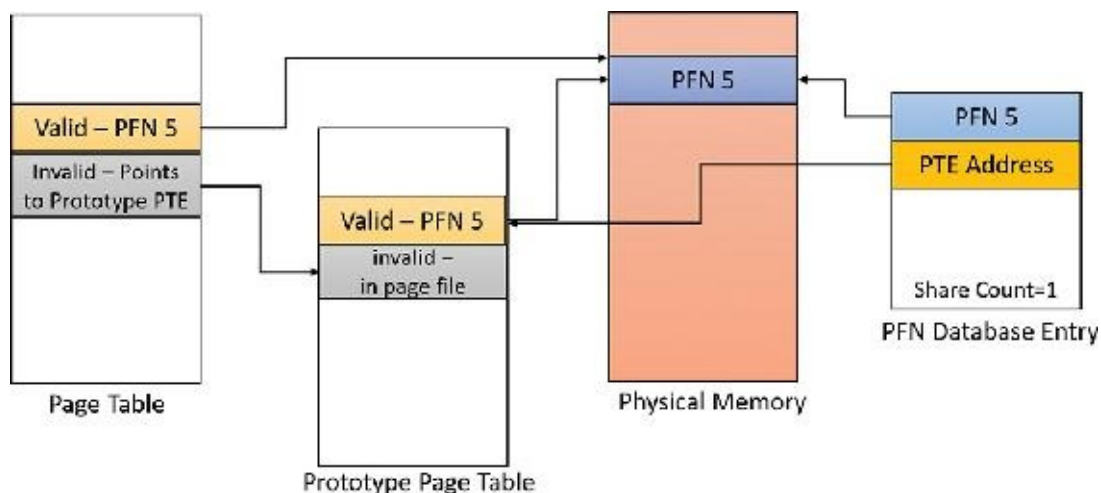


FIGURE 5-26 Prototype page table entries.

In-paging I/O

In-paging I/O occurs when a read operation must be issued to a file (paging or mapped) to satisfy a page fault. Also, because page tables are themselves pageable, the processing of a page fault can incur additional I/O if necessary when the system is loading the page table page that contains the PTE or the prototype PTE that describes the original page being referenced.

The in-page I/O operation is synchronous—that is, the thread waits on an event until the I/O completes—and isn't interruptible by asynchronous procedure call (APC) delivery. The pager uses a special modifier in the I/O request function to indicate paging I/O. Upon completion of paging I/O, the I/O system triggers an event, which wakes up the pager and allows it to continue in-page processing.

While the paging I/O operation is in progress, the faulting thread doesn't own any critical memory management synchronization objects. Other threads within the process can issue virtual memory functions and handle page faults while the paging I/O takes place. But there are a few interesting conditions that the pager must recognize when the I/O completes are exposed:

- Another thread in the same process or a different process could have faulted the same page (called a *collided page fault* and described in the next section).
- The page could have been deleted and remapped from the virtual address space.
- The protection on the page could have changed.
- The fault could have been for a prototype PTE, and the page that maps the prototype PTE could be out of the working set.

The pager handles these conditions by saving enough state on the thread's kernel stack before the paging I/O request such that when the request is complete, it can detect these conditions and, if necessary, dismiss the page fault without making the page valid. When and if the faulting instruction is reissued, the pager is again invoked and the PTE is reevaluated in its new state.

Collided page faults

The case when another thread in the same process or a different process faults a page that is currently being in-paged is known as a *collided page fault*. The pager detects and handles collided page faults optimally because they are common occurrences in multithreaded systems. If another thread or process faults the same page, the pager detects the collided page fault, noticing that the page is in transition and that a read is in progress. (This information is in the PFN database entry.) In this case, the pager may issue a wait operation on the event specified in the PFN database entry. Alternatively, it can choose to issue a parallel I/O to protect the file systems from deadlocks. (The first I/O to complete "wins," and the others are discarded.) This event was initialized by the thread that first issued the I/O needed to resolve the fault.

When the I/O operation completes, all threads waiting on the event have their wait satisfied. The first thread to acquire the PFN database lock is responsible for performing the in-page completion operations. These operations consist of checking I/O status to ensure that the I/O operation completed successfully, clearing the read-in-progress bit in the PFN database, and updating the PTE.

When subsequent threads acquire the PFN database lock to complete the collided page fault, the pager recognizes that the initial updating has been performed because the read-in-progress bit is clear and checks the in-page error flag in the PFN database element to ensure that the in-page I/O completed successfully. If the in-page error flag is set, the PTE isn't updated, and an in-page error exception is raised in the faulting thread.

Clustered page faults

The memory manager prefetches large clusters of pages to satisfy page faults and populate the system cache. The prefetch operations read data directly into the system's page cache instead of into a working set in virtual memory. Therefore, the prefetched data does not consume virtual address space, and the size of the fetch operation is not limited to the amount of virtual address space that is available. Also, no expensive TLB-flushing inter-processor interrupt (IPI) is needed if the page will be repurposed. The prefetched pages are put on the standby list and marked as in transition in the PTE. If a prefetched page is subsequently referenced, the memory manager adds it to the working set. However, if it is never referenced, no system resources are required to release it. If any pages in the prefetched cluster are already in memory, the memory manager does not read them again. Instead, it uses a dummy page to represent them so that an efficient single large I/O can still be issued, as shown in [Figure 5-27](#).

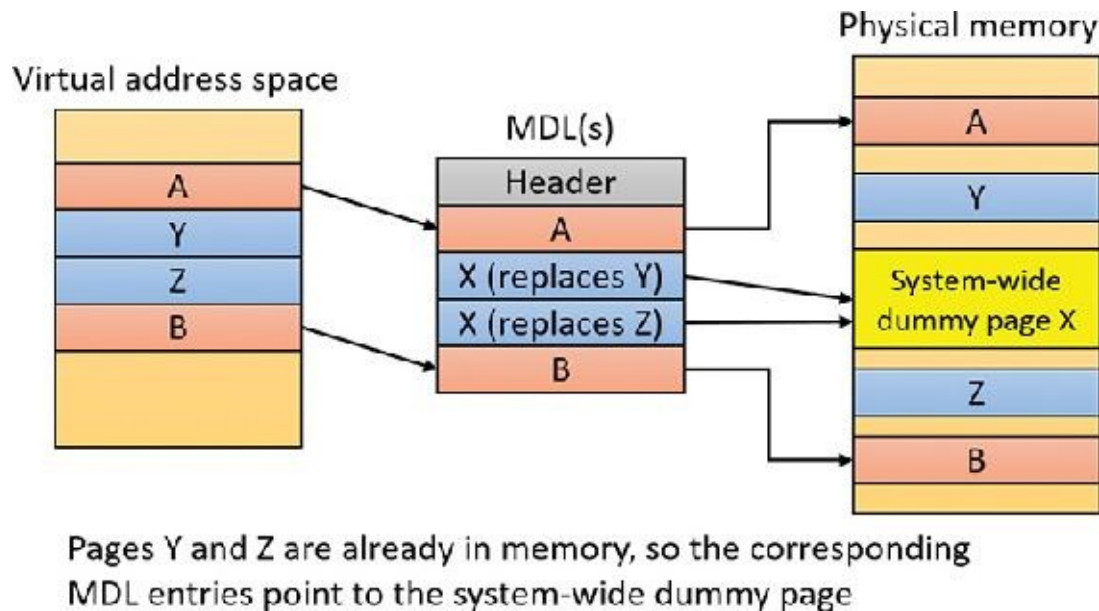


FIGURE 5-27 Usage of dummy page during virtual-address-to-physical-address mapping in an MDL.

In the figure, the file offsets and virtual addresses that correspond to pages A, Y, Z, and B are logically contiguous, although the physical pages themselves are not necessarily contiguous. Pages A and B are nonresident, so the memory manager must read them. Pages Y and Z are already resident in memory, so it is not necessary to read them. (In fact, they might already have been modified since they were last read in from their backing store, in which case it would be a serious error to overwrite their contents.) However, reading pages A and B in a single operation is more efficient than performing one read for page A and a second read for page B. Therefore, the memory manager issues a single read request that comprises all four pages (A, Y, Z, and B) from the backing store. Such a read request includes as many pages as it makes sense to read, based on the amount of available memory, the current system usage, and so on.

When the memory manager builds the MDL that describes the request, it supplies valid pointers to pages A and B. However, the entries for pages Y and Z point to a single system-wide dummy page X. The memory manager can fill the dummy page X with the potentially stale data from the backing store because it does not make X visible. However, if a component accesses the Y and Z offsets in the MDL, it sees the dummy page X instead of Y and Z.

The memory manager can represent any number of discarded pages as a single dummy page, and that page can be embedded multiple times in the same MDL or even in multiple concurrent MDLs that are being used for different drivers. Consequently, the contents of the locations that represent the discarded pages can change at any time. (See [Chapter 6](#) for more on MDLs.)

Page files

Page files store modified pages that are still in use by some process but have had to be written to disk because they were unmapped or memory pressure resulted in a trim. Page file space is reserved when the pages are initially committed, but the actual optimally clustered page file locations cannot be chosen until pages are written out to disk.

When the system boots, the Session Manager process (Smss.exe) reads the list of page files to open by examining the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PagingFiles registry value. This multistring registry value contains the name, minimum size, and maximum size of each paging file. Windows supports up to 16 paging files on x86 and x64 and up to 2 page files on ARM. On x86 and x64 systems, each page file can be up to 16 TB in size, while the maximum is 4 GB on ARM systems. Once open, the page files can't be deleted while the system is running because the System process maintains an open handle to each page file.

Because the page file contains parts of process and kernel virtual memory, for security reasons, the system can be configured to clear the page file at system shutdown. To enable this, set the `ClearPageFileAtShutdown` registry value in the HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management key to 1. Otherwise, after shutdown, the page file will contain whatever data happened to have been paged out while the system was up. This data could then be accessed by someone who gained physical access to the machine.

If the minimum and maximum paging file sizes are both zero (or not specified), this indicates a system-managed paging file. Windows 7 and Server 2008 R2 used a simple scheme based on RAM size alone as follows:

- **Minimum size** Set to the amount of RAM or 1 GB, whichever is larger
- **Maximum size** Set to 3 * RAM or 4 GB, whichever is larger

These settings are not ideal. For example, today's laptops and desktop machines can easily have 32 GB or 64 GB of RAM, and server machines can have hundreds of gigabytes of RAM. Setting the initial page file size to the size of RAM may result in a considerable loss of disk space, especially if disk sizes are relatively small and based on solid-state device (SSD). Furthermore, the amount of RAM in a system is not necessarily indicative of the typical memory workload on that system.

The current implementation uses a more elaborate scheme to derive a "good" minimum page file size based not only on RAM size, but also on page file history usage and other factors. As part of page-file creation and initialization, Smss.exe calculates page file minimum sizes based on four factors, stored in global variables:

- **RAM (SmpDesiredPfSizeBasedOnRAM)** This is the recommended page file size based on RAM.
- **Crash dump (SmpDesiredPfSizeForCrashDump)** This is the recommended page file size needed to be able to store a crash dump.
- **History (SmpDesiredPfSizeBasedOnHistory)** This is the recommended page file size based on usage history. Smss.exe uses a timer that triggers once an hour and records the page file usage.
- **Apps (SmpDesiredPfSizeForApps)** This is the recommended page file for Windows apps.
- These values are computed as shown in [Table 5-12](#).

Recommendation Base	Recommended Page File Size
RAM	If RAM ≤ 1 GB, then size = 1 GB. If RAM > 1 GB, then add 1/8 GB for every extra gigabyte of RAM, up to a maximum of 32 GB.
Crash dump	If a dedicated dump file is configured, then no page file is required for storing a dump file, and the size = 0. (You can configure this for a dedicated dump file by adding the value <code>DedicatedDumpFile</code> in the <code>HKLM\System\CurrentControlSet\Control\CrashControl</code> key.) If the dump type configured is set to Automatic (the default), then: If RAM < 4 GB, then size = RAM / 6. Otherwise, size = 2/3 GB + 1/8 GB for each extra gigabyte above 4 GB, capped to 32 GB. If there was a recent crash for which the page file was not large enough, then recommended size is increased to RAM size or 32 GB, whichever is smaller. If a full dump is configured, returned size = RAM size plus additional information size present in a dump file. If a kernel dump is configured, then size = RAM.
History	If enough samples have been logged, returns the 90 th percentile as the recommended size. Otherwise, returns the size based on RAM (above).
Apps	If it's a server, return zero. The recommended size is based on a factor that the Process Lifecycle Manager (PLM) uses to determine when to terminate an app. Current factor is 2.3 * RAM, which was considered with RAM = 1 GB (rough minimum for mobile devices). The recommended size (based on the mentioned factor) is around 2.5 GB. If this is more than RAM, RAM is subtracted. Otherwise, zero is returned.

TABLE 5-12 Base calculation for page file size recommendation

The maximum page file size for a system-managed size is set at three times the size of RAM or 4 GB, whichever is larger. The minimum (initial) page file size is determined as follows:

- If it's the first system-managed page file, then the base size is set based on page file history (refer to [Table 5-12](#)). Otherwise, the base size is based on RAM.
- If it's the first system-managed page file:
 - If the base size is smaller than the computed page file size for apps (`SmpDesiredPfSizeForApps`), then set the new base as the size computed for apps (refer to [Table 5-12](#)).
 - If the (new) base size is smaller than the computed size for crash dumps (`SmpDesiredPfSizeForCrashDump`), then set the new base to be the size computed for crash dumps.

EXPERIMENT: Viewing page files

To view the list of page files, look in the registry at the `PagingFiles` value in the `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management` key. This entry contains the paging file configuration settings modified through the Advanced System Settings dialog box. To access these settings, follow these steps:

1. Open Control Panel.
2. Click **System and Security** and then click **System**. This opens the System Properties dialog box, which you can also access by right-clicking on **Computer** in Explorer and selecting **Properties**.
3. Click **Advanced System Settings**.
4. In the Performance area, click **Settings**. This opens the Performance Options dialog box.
5. Click the **Advanced** tab.
6. In the Virtual Memory area, click **Change**.

EXPERIMENT: Viewing page file recommended sizes

To view the actual variables calculated in [Table 5-12](#), follow these steps (this experiment was done using an x86 Windows 10 system):

1. Start local kernel debugging.
2. Locate Smss.exe processes:

[Click here to view code image](#)

```
lkd> !process 0 0 smss.exe
PROCESS 8e54bc40 SessionId: none Cid: 0130 Peb:
02bab000 ParentCid:
0004
DirBase: bffe0020 ObjectTable: 8a767640 HandleCount: <Data
Not
Accessible>
Image: smss.exe

PROCESS 9985bc40 SessionId: 1 Cid: 01d4 Peb:
02f9c000 ParentCid: 0130
DirBase: bffe0080 ObjectTable: 00000000 HandleCount: 0.
Image: smss.exe

PROCESS a122dc40 SessionId: 2 Cid: 02a8 Peb:
02fcd000 ParentCid: 0130
DirBase: bffe0320 ObjectTable: 00000000 HandleCount: 0.
Image: smss.exe
```

3. Locate the first one (with a session ID of none), which is the master Smss.exe. (Refer to [Chapter 2](#) for more details.)
4. Switch the debugger context to that process:

[Click here to view code image](#)

```
lkd> .process /r /p 8e54bc40
Implicit process is now 8e54bc40
Loading User Symbols
..
```

5. Show the four variables described in the previous section. (Each one is 64 bits in size.)

[Click here to view code image](#)

```
lkd> dq smss!SmpDesiredPfSizeBasedOnRAM L1
00974cd0 00000000'4fff1a00
lkd> dq smss!SmpDesiredPfSizeBasedOnHistory L1
00974cd8 00000000'05a24700
lkd> dq smss!SmpDesiredPfSizeForCrashDump L1
00974cc8 00000000'1ffecd55
lkd> dq smss!SmpDesiredPfSizeForApps L1
00974ce0 00000000'00000000
```

6. Because there is a single volume (C:\) on this machine, a single page file would be created.

Assuming it wasn't specifically configured, it would be system managed. You can look at the actual file size of C:\PageFile.Sys on disk or use the !vm debugger command:

[Click here to view code image](#)

```
lkd> !vm 1
Page File: \??\C:\pagefile.sys
  Current:      524288 Kb   Free Space:      524280 Kb
  Minimum:      524288 Kb Maximum:      8324476 Kb
Page File: \??\C:\swapfile.sys
  Current:      262144 Kb   Free Space:      262136 Kb
  Minimum:      262144 Kb   Maximum:      4717900 Kb
No Name for Paging File
  Current:      11469744 Kb   Free Space:     11443108 Kb
  Minimum:      11469744 Kb   Maximum:      11469744 Kb
...
```

Notice the minimum size of C:\PageFile.sys (524288 KB). (We'll discuss the other page file entries in the next section). According to the variables, SmpDesiredPfSizeForCrashDump is the largest, so must be the determining factor (0x1FFECD55 = 524211 KB), which is very close to the listed value. (Page file sizes round up to multiple of 64 MB.)

To add a new page file, Control Panel uses the internal NtCreatePagingFile system service defined in Ntdll.dll. (This requires the SeCreatePagefilePrivilege.) Page files are always created as non-compressed files, even if the directory they are in is compressed. Their names are *PageFile.Sys* (except some special ones described in the next section). They are created in the root of partitions with the Hidden file attribute so they are not immediately visible. To keep new page files from being deleted, a handle is duplicated into the System process so that even after the creating process closes the handle to the new page file, a handle is nevertheless always open to it.

The swap file

In the UWP apps world, when an app goes to the background—for example, it is minimized—the threads in that process are suspended so that the process does not consume any CPU. The private physical memory used by the process can potentially be reused for other processes. If memory pressure is high, the private working set (physical memory used by the process) may be swapped out to disk to allow the physical memory to be used for other processes.

Windows 8 added another page file called a *swap file*. It is essentially the same as a normal page file but is used exclusively for UWP apps. It's created on client SKUs only if at least one normal page file was created (the normal case). Its name is *SwapFile.sys* and it resides in the system root partition—for example, C:\SwapFile.Sys.

After the normal page files are created, the HKLM\System\CurrentControlSet\Control\Session Manager\Memory Management registry key is consulted. If a DWORD value named SwapFileControl exists and its value is zero, swap file creation is aborted. If a value named SwapFile exists, it's read as a string with the same format as a normal page file, with a filename, an initial size, and a maximum size. The difference is that a value of zero for the sizes is interpreted as no swap file creation. These two registry values do not exist by default, which results in the creation of a SwapFile.sys file on the system root partition with a minimum size of 16 MB on fast (and small) disks (for example, SSD) or 256 MB on slow (or large SSD) disks. The maximum size of the swap file is set to

1.5 * RAM or 10 percent of the system root partition size, whichever is smaller. See [Chapter 7](#) in this book and Chapter 8, “System mechanisms,” and Chapter 9, “Management mechanisms,” in Part 2 for more on UWP apps.



Note

The swap file is not counted against the maximum page files supported.

The virtual page file

The `!vm` debugger command hints at another page file called “No Name for Paging File.” This is a virtual page file. As the name suggests, it has no actual file, but is used indirectly as the backing store for memory compression (described later in this chapter in the section “[Memory compression](#)”). It is large, but its size is set arbitrarily as not to run out of free space. The invalid PTEs for pages that have been compressed point to this virtual page file and allow the memory compression store to get to the compressed data when needed by interpreting the bits in the invalid PTE leading to the correct store, region, and index.

EXPERIMENT: Viewing swap file and virtual page file information

The `!vm` debugger command shows the information on all page files, including the swap file and the virtual page file:

[Click here to view code image](#)

```
lkd> !vm 1
Page File: \??\C:\pagefile.sys
  Current:      524288 Kb   Free Space:      524280 Kb
  Minimum:      524288 Kb   Maximum:        8324476 Kb
Page File: \??\C:\swapfile.sys
  Current:      262144 Kb   Free Space:      262136 Kb
  Minimum:      262144 Kb   Maximum:      4717900 Kb
No Name for Paging File
  Current:    11469744 Kb   Free Space:    11443108 Kb
  Minimum:    11469744 Kb   Maximum:      11469744 Kb
```

On this system, the swap file minimum size is 256 MB, as the system is a Windows 10 virtual machine. (The VHD behind the disk is considered a slow disk.) The maximum size of the swap file is about 4.5 GB, as the RAM on the system is 3 GB and disk partition size is 64 GB (the minimum of 4.5 GB and 6.4 GB).

Commit charge and the system commit limit

We are now in a position to more thoroughly discuss the concepts of commit charge and the system commit limit.

Whenever virtual address space is created—for example, by a `VirtualAlloc` (for committed memory) or `MapViewOfFile` call—the system must ensure that there is room to store it, either in RAM or in backing store, before successfully completing the create request. For mapped memory (other than sections mapped to the page file), the file associated with the mapping object referenced by the `MapViewOfFile` call provides the required backing store. All other virtual allocations rely on system-

managed shared resources for storage: RAM and the paging file(s). The purpose of the system commit limit and commit charge is to track all uses of these resources to ensure they are never overcommitted—that is, that there is never more virtual address space defined than there is space to store its contents, either in RAM or in backing store (on disk).



Note

This section makes frequent references to paging files. It is possible (though not generally recommended) to run Windows without any paging files. Essentially, this means that when RAM is exhausted, there is no room to grow and memory allocations fail, generating a blue screen. You can consider every reference to paging files here to be qualified by “if one or more paging files exist.”

Conceptually, the system commit limit represents the total committed virtual address space that can be created in addition to virtual allocations that are associated with their own backing store—that is, in addition to sections mapped to files. Its numeric value is simply the amount of RAM available to Windows plus the current sizes of any page files. If a page file is expanded or new page files are created, the commit limit increases accordingly. If no page files exist, the system commit limit is simply the total amount of RAM available to Windows.

Commit charge is the system-wide total of all committed memory allocations that must be kept in either RAM or in a paging file. From the name, it should be apparent that one contributor to commit charge is process-private committed virtual address space. However, there are many other contributors, some of them not so obvious.

Windows also maintains a per-process counter called the *process page file quota*. Many of the allocations that contribute to commit charge also contribute to the process page file quota. This represents each process’s private contribution to the system commit charge. Note, however, that this does not represent current page file usage. It represents the potential or maximum page file usage, should all these allocations have to be stored there.

The following types of memory allocations contribute to the system commit charge and, in many cases, to the process page file quota. (Some of these will be described in detail in later sections of this chapter.)

- **Private committed memory** This is memory allocated with the `VirtualAlloc` call with the `MEM_COMMIT` option. This is the most common type of contributor to the commit charge. These allocations are also charged to the process page file quota.
- **Page-file-backed mapped memory** This is memory allocated with a `MapViewOfFile` call that references a section object, which in turn is not associated with a file. The system uses a portion of the page file as the backing store instead. These allocations are not charged to the process page file quota.
- **Copy-on-write regions of mapped memory (even if it is associated with ordinary mapped files)** The mapped file provides backing store for its own unmodified content. However, should a page in the copy-on-write region be modified, it can no longer use the original mapped file for backing store. It must be kept in RAM or in a paging file. These allocations are not charged to the process page file quota.
- **Non-paged and paged pool and other allocations in system space that are not backed by explicitly associated files** Even the currently free regions of the system memory pools contribute to commit charge. The non-pageable regions are counted in the commit charge even though they will never be written to the page file because they permanently reduce the amount of RAM available for

private pageable data. These allocations are not charged to the process page file quota.

- **Kernel stacks** Threads' stacks when executing in kernel mode.
- **Page tables** Most of these are themselves pageable, and they are not backed by mapped files. However, even if they are not pageable, they occupy RAM. Therefore, the space required for them contributes to commit charge.
- **Space for page tables that are not yet actually allocated** As you'll see, where large areas of virtual space have been defined but not yet referenced (for example, private committed virtual space), the system need not actually create page tables to describe it. However, the space for these as-yet-nonexistent page tables is charged to commit charge to ensure that the page tables can be created when they are needed.
- **Allocations of physical memory made via the Address Windowing Extension (AWE) APIs** As discussed previously, consume physical memory directly.

For many of these items, the commit charge may represent the potential use of storage rather than its actual use. For example, a page of private committed memory does not actually occupy either a physical page of RAM or the equivalent page file space until it's been referenced at least once. Until then, it is a *demand-zero page* (described later). But commit charge accounts for such pages when the virtual space is first created. This ensures that when the page is later referenced, actual physical storage space will be available for it.

A region of a file mapped as copy-on-write has a similar requirement. Until the process writes to the region, all pages in it are backed by the mapped file. However, the process may write to any of the pages in the region at any time. When that happens, those pages are thereafter treated as private to the process. Their backing store is, thereafter, the page file. Charging the system commit for them when the region is first created ensures that there will be private storage for them later, if and when the write accesses occur.

A particularly interesting case occurs when reserving private memory and later committing it. When the reserved region is created with `VirtualAlloc`, system commit charge is not charged for the actual virtual region. On Windows 8 and Server 2012 and earlier versions, it is charged for any new page table pages that will be required to describe the region, even though these might not yet exist or even be eventually needed. Starting with Windows 8.1 and Server 2012 R2, page table hierarchies for reserved regions are not charged immediately; this means that huge reserved memory regions can be allocated without exhausting page tables. This becomes important in some security features, such as Control Flow Guard (CFG, see [Chapter 7](#) for more details). If the region or a part of it is later committed, system commit is charged to account for the size of the region (and page tables), as is the process page file quota.

To put it another way, when the system successfully completes, for example, a `VirtualAlloc` commit or `MapViewOfFile` call, it makes a commitment that the necessary storage will be available when needed, even if it wasn't needed at that moment. Thus, a later memory reference to the allocated region can never fail for lack of storage space. (Of course, it could still fail for other reasons, such as page protection, the region being deallocated, and so on.) The commit charge mechanism allows the system to keep this commitment.

The commit charge appears in the Performance Monitor counters as Memory: Committed Bytes. It is also the first of the two numbers displayed on the Task Manager's Performance tab with the legend Commit (the second being the commit limit), and it is displayed by Process Explorer's System Information Memory tab as Commit Charge – Current.

The process page file quota appears in the performance counters as Process: Page File Bytes. The same data appears in the Process: Private Bytes performance counter. (Neither term exactly describes the

true meaning of the counter.)

If the commit charge ever reaches the commit limit, the memory manager will attempt to increase the commit limit by expanding one or more page files. If that is not possible, subsequent attempts to allocate virtual memory that uses commit charge will fail until some existing committed memory is freed. The performance counters listed in [Table 5-13](#) allow you to examine private committed memory usage on a system-wide, per-process, or per-page-file, basis.

Performance Counter	Description
Memory: Committed Bytes	This is the number of bytes of virtual (not reserved) memory that has been committed. This number doesn't necessarily represent page file usage because it includes private committed pages in physical memory that have never been paged out. Rather, it represents the charged amount that must be backed by page file space and/or RAM.
Memory: Commit Limit	This is the number of bytes of virtual memory that can be committed without having to extend the paging files. If the paging files can be extended, this limit is soft.
Process: Page File Quota	This is the process's contribution to Memory: Committed Bytes.
Process: Private Bytes	This is the same as Process: Page File Quota.
Process: Working Set – Private	This is the subset of Process: Page File Quota that is currently in RAM and can be referenced without a page fault. It is also a subset of Process: Working Set.
Process: Working Set	This is the subset of Process: Virtual Bytes that is currently in RAM and can be referenced without a page fault.
Process: Virtual Bytes	This is the total virtual memory allocation of the process, including mapped regions, private committed regions, and private reserved regions.
Paging File: % Usage	This is the percentage of the page file space that is currently in use.
Paging File: % Usage Peak	This is the highest observed value of Paging File: % Usage.

TABLE 5-13 Committed memory and page file performance counters

Commit charge and page file size

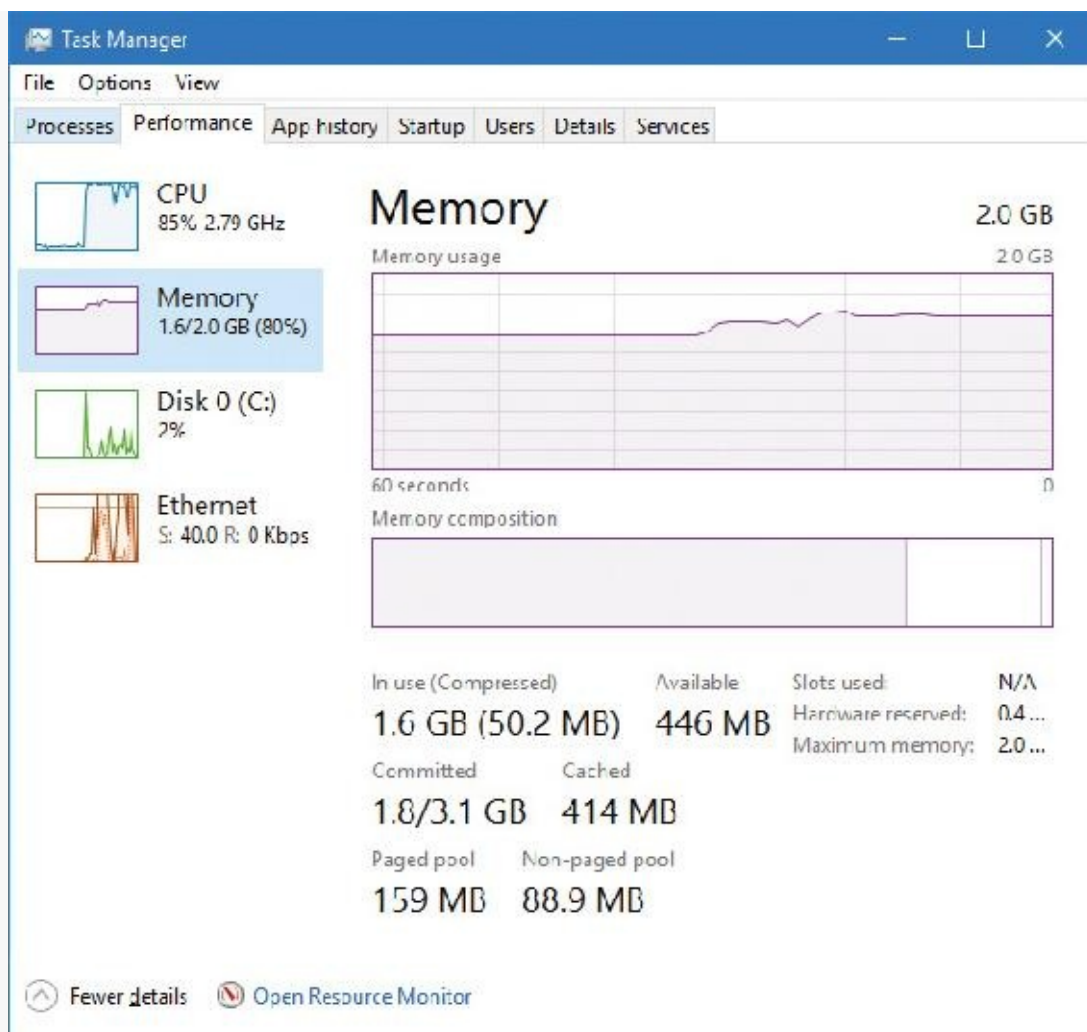
The counters in [Table 5-13](#) can assist you in choosing a custom page file size. The default policy based on the amount of RAM works acceptably for most machines, but depending on the workload, it can result in a page file that's unnecessarily large or not large enough.

To determine how much page-file space your system really needs based on the mix of applications that have run since the system booted, examine the peak commit charge in the Memory tab of Process Explorer's System Information display. This number represents the peak amount of page file space since the system booted that would have been needed if the system had to page out the majority of private committed virtual memory (which rarely happens).

If the page file on your system is too big, the system will not use it any more or less. In other words, increasing the size of the page file does not change system performance. It simply means the system can have more committed virtual memory. If the page file is too small for the mix of applications you are running, you might get a "system running low on virtual memory" error message. In this case, check to see whether a process has a memory leak by examining the process private bytes count. If no process appears to have a leak, check the system paged pool size. If a device driver is leaking paged pool, this might also explain the error. Refer to the "[Troubleshooting a pool leak](#)" experiment in the "[Kernel-mode heaps \(system memory pools\)](#)" section for information on troubleshooting a pool leak.

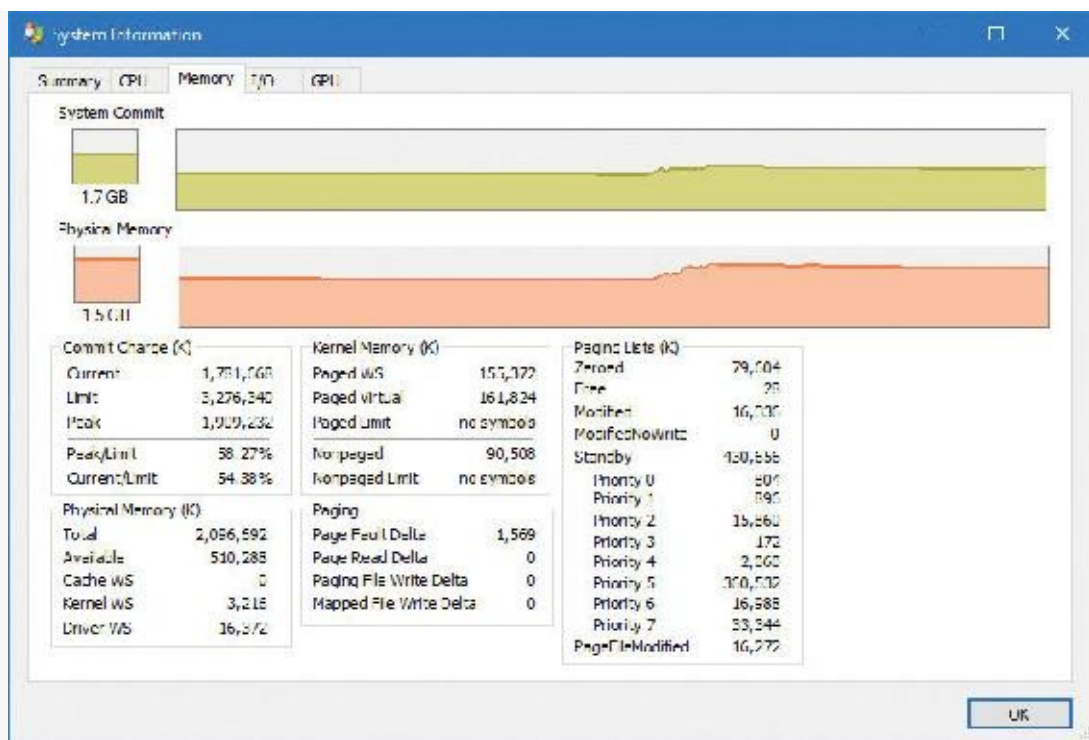
EXPERIMENT: Viewing page file usage with Task Manager

You can view committed memory usage with Task Manager. To do so, click its **Performance** tab. You'll see the following counters related to page files:



The system commit total is displayed under the **Committed** label as two numbers. The first number represents *potential* page file usage, not actual page file usage. It is how much page file space would be used if all the private committed virtual memory in the system had to be paged out all at once. The second number displayed is the *commit limit*, which is the maximum virtual memory usage that the system can support before running out of virtual memory. (This includes virtual memory backed in physical memory as well as by the paging files.) The commit limit is essentially the size of RAM plus the current size of the paging files. It therefore does not account for possible page file expansion.

Process Explorer's System Information display shows an additional piece of information about system commit usage—namely, the percentage of the peak as compared to the limit and the current usage as compared to the limit:



Stacks

Whenever a thread runs, it must have access to a temporary storage location in which to store function parameters, local variables, and the return address after a function call. This part of memory is called a *stack*. On Windows, the memory manager provides two stacks for each thread: the user stack and the kernel stack, as well as per-processor stacks called DPC stacks. [Chapter 2](#) briefly discussed how system calls cause the thread to switch from a user stack to its kernel stack. Now we'll look at some extra services the memory manager provides to efficiently use stack space.

User stacks

When a thread is created, the memory manager automatically reserves a predetermined amount of virtual memory, which by default is 1 MB. This amount can be configured in the call to the `CreateThread` or `CreateRemoteThread(Ex)` function or when compiling the application by using the `/STACK:reserve` switch in the Microsoft C/C++ compiler, which stores the information in the image header. Although 1 MB is reserved, only the first page of the stack will be committed (unless the PE header of the image specifies otherwise), along with a guard page. When a thread's stack grows large enough to touch the guard page, an exception occurs, causing an attempt to allocate another guard. Through this mechanism, a user stack doesn't immediately consume all 1 MB of committed memory but instead grows with demand. (However, it will never shrink back.)

EXPERIMENT: Creating the maximum number of threads

With only 2 GB of user address space available to each 32-bit process, the relatively large memory that is reserved for each thread's stack allows for an easy calculation of the maximum number of threads that a process can support: a little less than 2,048, for a total of nearly 2 GB of memory (unless the `increaseuserva` BCD option is used and the image is large address space aware). By forcing each new thread to use the smallest possible stack reservation size, 64 KB, the limit can grow to about 30,000 threads. You can test this for yourself by using the `TestLimit` utility from Sysinternals. Here is some sample output:

[Click here to view code image](#)

```
C:\Tools\Sysinternals>Testlimit.exe -t -n 64
```

```
Testlimit v5.24 - test Windows limits  
Copyright (C) 2012-2015 Mark Russinovich  
Sysinternals - www.sysinternals.com
```

```
Process ID: 17260
```

```
Creating threads with 64 KB stacks...  
Created 29900 threads. Lasterror: 8
```

If you attempt this experiment on a 64-bit Windows installation (with 128 TB of user address space available), you would expect to see potentially hundreds of thousands of threads created (assuming sufficient memory was available). Interestingly, however, `TestLimit` actually creates fewer threads than on a 32-bit machine. This has to do with the fact that `Testlimit.exe` is a 32-bit application and thus runs under the Wow64 environment. (See Chapter 8 in Part 2 for more information on Wow64.) Each thread will therefore have not only its 32-bit Wow64 stack but also its 64-bit stack, thus consuming more than twice the memory, while keeping only 2 GB of address space. To properly test the thread-creation limit on 64-bit Windows, use the `Testlimit64.exe` binary instead.

You will need to terminate `TestLimit` with Process Explorer or Task Manager. You cannot use Ctrl+C to break the application because this operation itself creates a new thread, which will not be possible once memory is exhausted.

Kernel stacks

Although user stack sizes are typically 1 MB, the amount of memory dedicated to the kernel stack is significantly smaller: 12 KB on 32-bit systems and 16 KB on 64-bit systems, followed by another guard page, for a total of 16 or 20 KB of virtual address space. Code running in the kernel is expected to have less recursion than user code, as well as contain more efficient variable use and keep stack buffer sizes low. Because kernel stacks live in system address space (which is shared by all processes), their memory usage has a bigger impact of the system.

Although kernel code is usually not recursive, interactions between graphics system calls handled by `Win32k.sys` and its subsequent callbacks into user mode can cause recursive re-entries in the kernel on the same kernel stack. As such, Windows provides a mechanism for dynamically expanding and shrinking the kernel stack from its initial size. As each additional graphics call is performed from the same thread, another 16 KB kernel stack is allocated. (This happens anywhere in system address space. The memory

manager provides the ability to jump stacks when nearing the guard page.) Whenever each call returns to the caller (unwinding), the memory manager frees the additional kernel stack that had been allocated, as shown in [Figure 5-28](#). This mechanism allows reliable support for recursive system calls as well as efficient use of system address space. It is provided for use by driver developers when performing recursive callouts through the `KeExpandKernelStackAndCallout (Ex)` APIs, as necessary.

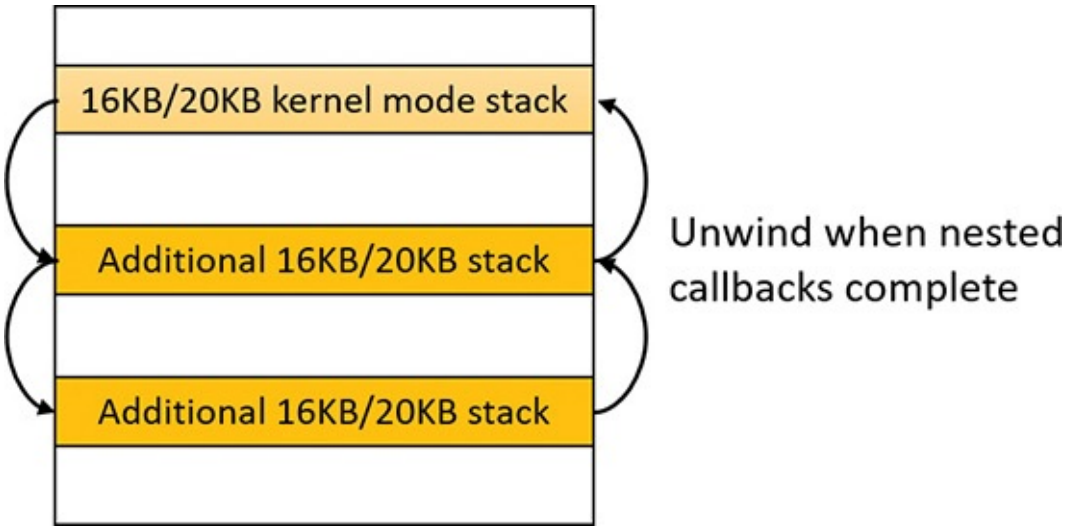


FIGURE 5-28 Kernel stack jumping.

EXPERIMENT: Viewing kernel-stack usage

You can use the RamMap tool from Sysinternals to display the physical memory currently being occupied by kernel stacks. Here’s a screenshot from the **Use Counts** tab:

Driver Locked	3,419,852 K	3,419,852 K	
Kernel Stack	47,496 K	41,380 K	6,116 K
Unused	10,773,740 K	77,504 K	

To view kernel-stack usage, try the following:

- 1. Repeat the previous TestLimit experiment, but don’t terminate TestLimit yet.
- 2. Switch to RamMap.
- 3. Open the **File** menu and select **Refresh** (or press **F5**). You should see a much higher kernel stack size:

Driver Locked	3,419,852 K	3,419,852 K	
Kernel Stack	346,076 K	339,916 K	6,160 K
Unused	10,318,232 K	77,680 K	

Running TestLimit a few more times (without closing previous instances) would easily exhaust physical memory on a 32-bit system, and this limitation results in one of the primary limits on system-wide 32-bit thread count.

DPC stack

Windows keeps a per-processor DPC stack available for use by the system whenever DPCs are executing. This approach isolates the DPC code from the current thread's kernel stack. (This is unrelated to the DPC's actual operation because DPCs run in arbitrary thread context. See [Chapter 6](#) for more on DPCs.) The DPC stack is also configured as the initial stack for handling the `sysenter` (x86), `svc` (ARM), or `syscall` (x64) instruction during a system call. The CPU is responsible for switching the stack when these instructions are executed, based on one of the model-specific registers (MSRs on x86/x64). However, Windows does not want to reprogram the MSR for every context switch because that is an expensive operation. Windows therefore configures the per-processor DPC stack pointer in the MSR.

Virtual address descriptors

The memory manager uses a demand-paging algorithm to know when to load pages into memory, waiting until a thread references an address and incurs a page fault before retrieving the page from disk. Like copy-on-write, demand paging is a form of *lazy evaluation*—waiting to perform a task until it is required.

The memory manager uses lazy evaluation not only to bring pages into memory but also to construct the page tables required to describe new pages. For example, when a thread commits a large region of virtual memory with `VirtualAlloc`, the memory manager could immediately construct the page tables required to access the entire range of allocated memory. But what if some of that range is never accessed? Creating page tables for the entire range would be a wasted effort. Instead, the memory manager waits to create a page table until a thread incurs a page fault. It then creates a page table for that page. This method significantly improves performance for processes that reserve and/or commit a lot of memory but access it sparsely.

The virtual address space that would be occupied by such as-yet-nonexistent page tables is charged to the process page file quota and to the system commit charge. This ensures that space will be available for them should they actually be created. With the lazy-evaluation algorithm, allocating even large blocks of memory is a fast operation. When a thread allocates memory, the memory manager must respond with a range of addresses for the thread to use. To do this, the memory manager maintains another set of data structures to keep track of which virtual addresses have been reserved in the process's address space and which have not. These data structures are known as *Virtual Address Descriptors* (VADs). VADs are allocated in non-paged pool.

Process VADs

For each process, the memory manager maintains a set of VADs that describes the status of the process's address space. VADs are organized into a self-balancing AVL tree (named after its inventors, Adelson-Velsky and Landis, where the heights of the two child subtrees of any node differ by at most 1; this makes the insertion, lookup, and deletion very fast). On average, this results in the fewest comparisons when searching for a VAD corresponding with a virtual address. There is one VAD for each virtually contiguous range of not-free virtual addresses that all have the same characteristics (reserved versus committed versus mapped, memory access protection, and so on). [Figure 5-29](#) shows a diagram of a VAD tree.

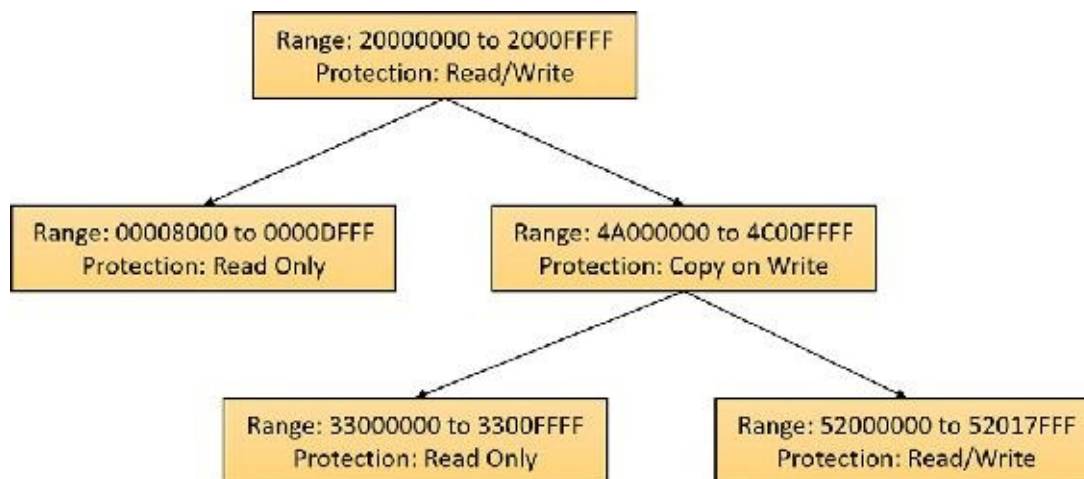


FIGURE 5-29 VADs.

When a process reserves address space or maps a view of a section, the memory manager creates a VAD to store any information supplied by the allocation request, such as the range of addresses being reserved, whether the range will be shared or private, whether a child process can inherit the contents of the range, and the page protection applied to pages in the range.

When a thread first accesses an address, the memory manager must create a PTE for the page containing the address. To do so, it finds the VAD whose address range contains the accessed address and uses the information it finds to fill in the PTE. If the address falls outside the range covered by the VAD or in a range of addresses that are reserved but not committed, the memory manager knows that the thread didn't allocate the memory before attempting to use it and therefore generates an access violation.

EXPERIMENT: Viewing VADs

You can use the kernel debugger's `!vad` command to view the VADs for a given process. First find the address of the root of the VAD tree with the `!process` command. Then specify that address to the `!vad` command, as shown in the following example of the VAD tree for a process running Explorer.exe:

[Click here to view code image](#)

```
lkd> !process 0 1 explorer.exe
PROCESS fffffc8069382e080
    SessionId: 1  Cid: 43e0    Peb: 00bc5000  ParentCid: 0338
    DirBase: 554ab7000  ObjectTable: fffffda8f62811d80  HandleCount:
823.
    Image: explorer.exe
    VadRoot fffffc806912337f0 Vads 505 Clone 0 Private 5088. Modified
2146. Locked 0.
...
lkd> !vad fffffc8068ae1e470
VAD          Level      Start          End Commit
fffffc80689bc52b0  9          640          64f      0
Mapped      READWRITE
Pagefile section, shared commit 0x10
fffffc80689be6900  8          650          651      0 Mapped      READONLY
Pagefile section, shared commit 0x2
fffffc80689bc4290  9          660          675      0 Mapped      READONLY
Pagefile section, shared commit 0x16
fffffc8068ae1f320  7          680          6ff      32
Private     READWRITE
fffffc80689b290b0  9          700          701      2
Private     READWRITE
fffffc80688da04f0  8          710          711      2
Private     READWRITE
fffffc80682795760  6          720          723      0 Mapped      READONLY
Pagefile section, shared commit 0x4
fffffc80688d85670 10          730          731      0 Mapped      READONLY
Pagefile section, shared commit 0x2
fffffc80689bdd9e0  9          740          741      2
Private     READWRITE
fffffc80688da57b0  8          750          755      0 Mapped      READONLY
\Windows\en-US\explorer.exe.mui
...
Total VADs: 574, average level: 8, maximum depth: 10
Total private commit: 0x3420 pages (53376 KB)
Total shared commit:  0x478 pages (4576 KB)
```

Rotate VADs

A video card driver must typically copy data from the user-mode graphics application to various other system memory, including the video card memory and the AGP port's memory, both of which have different caching attributes as well as addresses. To quickly allow these different views of memory to be mapped into a process, and to support the different cache attributes, the memory manager implements rotate VADs, which allow video drivers to transfer data directly by using the GPU and to rotate unneeded memory in and out of the process view pages on demand. [Figure 5-30](#) shows an example of how the same virtual address can rotate between video RAM and virtual memory.

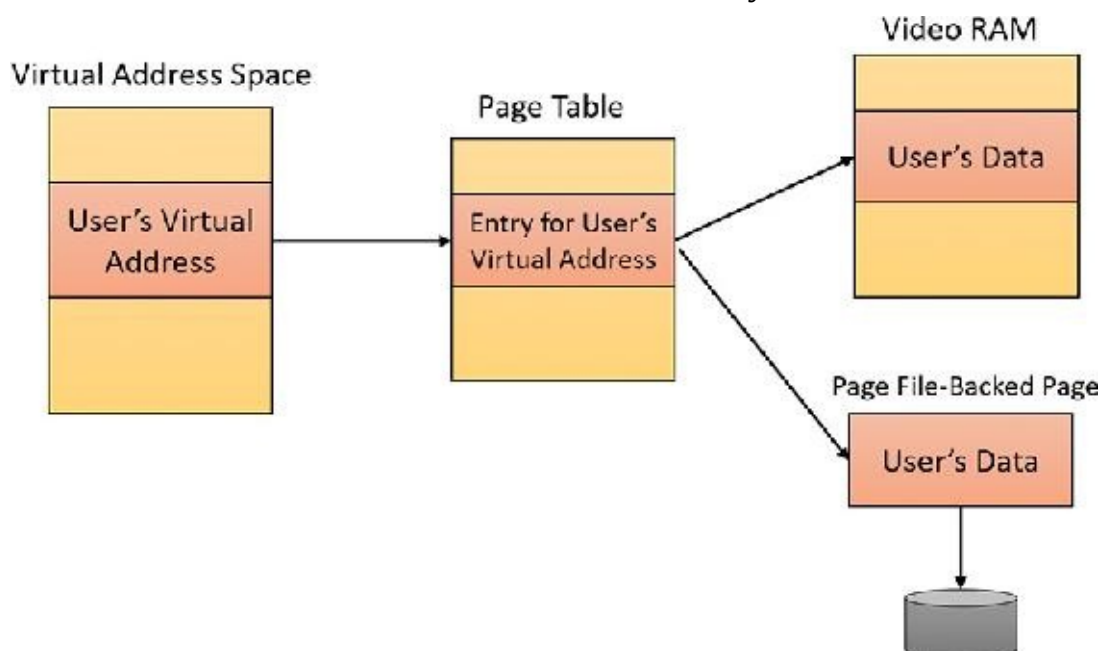


FIGURE 5-30 Rotate VADs.

NUMA

Each new release of Windows provides new enhancements to the memory manager to make better use of non-uniform memory architecture (NUMA) machines, such as large server systems as well as Intel i7 and AMD Opteron SMP workstations. The NUMA support in the memory manager adds intelligent knowledge of node information such as location, topology, and access costs to allow applications and drivers to take advantage of NUMA capabilities, while abstracting the underlying hardware details.

When the memory manager is initializing, it calls the `MiComputeNumaCosts` function to perform various page and cache operations on different nodes. It then computes the time it took for those operations to complete. Based on this information, it builds a node graph of access costs (the distance between a node and any other node on the system). When the system requires pages for a given operation, it consults the graph to choose the most optimal node (that is, the closest). If no memory is available on that node, it chooses the next closest node, and so on.

Although the memory manager ensures that, whenever possible, memory allocations come from the ideal processor's node (the *ideal node*) of the thread making the allocation, it also provides functions that allow applications to choose their own node, such as the `VirtualAllocExNuma`, `CreateFileMappingNuma`, `MapViewOfFileExNuma`, and `AllocateUserPhysicalPagesNuma` APIs.

The ideal node isn't used only when applications allocate memory but also during kernel operation and page faults. For example, when a thread running on a non-ideal processor takes a page fault, the memory

manager won't use the current node. Instead, it will allocate memory from the thread's ideal node. Although this might result in slower access time while the thread is still running on this CPU, overall memory access will be optimized as the thread migrates back to its ideal node. In any case, if the ideal node is out of resources, the closest node to the ideal node is chosen and not a random other node. Just like user-mode applications, however, drivers can specify their own node when using APIs such as `MmAllocatePagesForMdlEx` or `MmAllocateContiguousMemorySpecifyCacheNode`.

Various memory manager pools and data structures are also optimized to take advantage of NUMA nodes. The memory manager tries to evenly use physical memory from all the nodes on the system to hold the non-paged pool. When a non-paged pool allocation is made, the memory manager uses the ideal node as an index to choose a virtual memory address range inside non-paged pool that corresponds to physical memory belonging to this node. In addition, per-NUMA node pool free lists are created to efficiently leverage these types of memory configurations. Apart from non-paged pool, the system cache and system PTEs are also similarly allocated across all nodes, as well as the memory manager's look-aside lists.

Finally, when the system needs to zero pages, it does so in parallel across different NUMA nodes by creating threads with NUMA affinities that correspond to the nodes in which the physical memory is located. The logical prefetcher and SuperFetch (described in the section "Proactive memory management [SuperFetch]") also use the ideal node of the target process when prefetching, while soft page faults cause pages to migrate to the ideal node of the faulting thread.

Section objects

As noted earlier in this chapter in the "[Shared memory and mapped files](#)" section, the *section object*, which the Windows subsystem calls a *file mapping object*, represents a block of memory that two or more processes can share. A section object can be mapped to the paging file or to another file on disk.

The executive uses sections to load executable images into memory, and the cache manager uses them to access data in a cached file. (See Chapter 14 in Part 2 for more information on how the cache manager uses section objects.) You can also use section objects to map a file into a process address space. The file can then be accessed as a large array by mapping different views of the section object and reading or writing to memory rather than to the file—an activity called *mapped file I/O*. When the program accesses an invalid page (one not in physical memory), a page fault occurs and the memory manager automatically brings the page into memory from the mapped file or page file. If the application modifies the page, the memory manager writes the changes back to the file during its normal paging operations. (Alternatively, the application can flush a view explicitly by using the Windows `FlushViewOfFile` function.)

Like other objects, section objects are allocated and deallocated by the object manager. The object manager creates and initializes an object header, which it uses to manage the objects; the memory manager defines the body of the section object. (See Chapter 8 in Part 2 for more on the object manager). The memory manager also implements services that user-mode threads can call to retrieve and change the attributes stored in the body of section objects. The structure of a section object is shown in [Figure 5-31](#). [Table 5-14](#) summarizes the unique attributes stored in section objects.

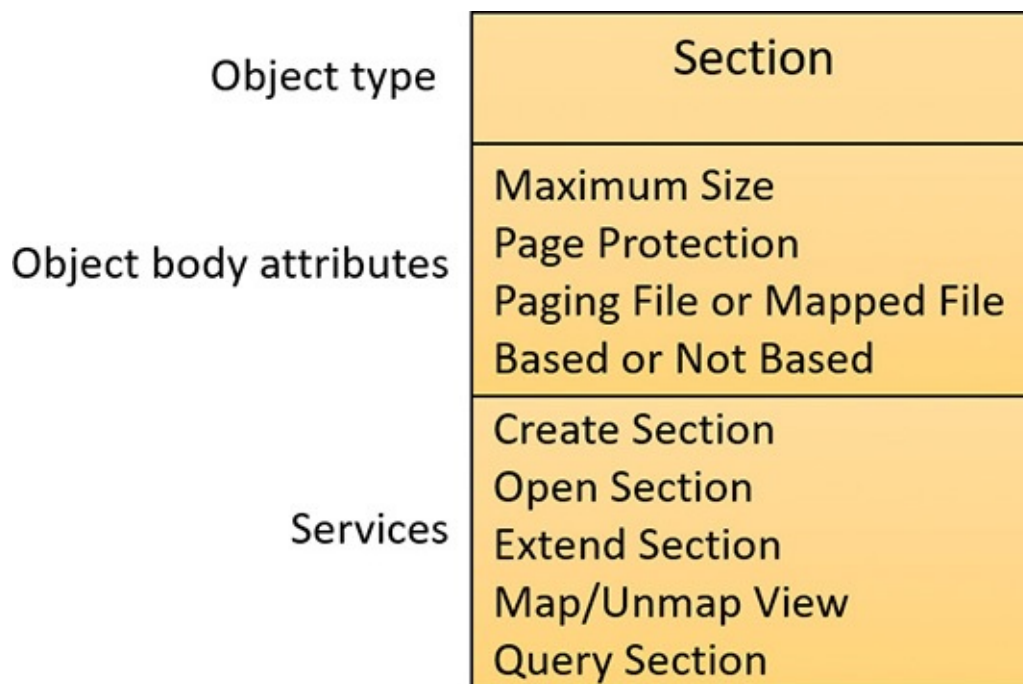


FIGURE 5-31 A section object.

Attribute	Purpose
Maximum size	This is the largest size to which the section can grow in bytes. If mapping a file, this is the maximum size of the file.
Page protection	This is page-based memory protection assigned to all pages in the section when it is created.
Paging file or mapped file	This indicates whether the section is created empty (backed by the paging file—as explained earlier, page-file-backed sections use page-file resources only when the pages need to be written out to disk) or loaded with a file (backed by the mapped file).
Based or not based	This indicates whether a section is a based section, which must appear at the same virtual address for all processes sharing it, or a non-based section, which can appear at different virtual addresses for different processes.

TABLE 5-14 Section object body attributes

EXPERIMENT: Viewing section objects

You can use Process Explorer from Sysinternals to see files mapped by a process. Follow these steps:

1. Open the **View** menu, choose **Lower Pane View**, and select **DLLs**.
2. Open the **View** menu, choose **Select Columns**, choose **DLL**, and enable the **Mapping Type** column.
3. Notice the files marked as Data in the Mapping column. These are mapped files rather than DLLs and other files the image loader loads as modules. Section objects that are backed by a page file are indicated in the Name column as <Pagefile Backed>. Otherwise, the file name is shown.

types of file access as well as to providing caching for files. The section object pointers structure points to one or two control areas. One control area is used to map the file when it is accessed as a data file and the other is used to map the file when it is run as an executable image. A control area in turn points to subsection structures that describe the mapping information for each section of the file (read-only, read/write, copy-on-write, and so on). The control area also points to a segment structure allocated in paged pool, which in turn points to the prototype PTEs used to map to the actual pages mapped by the section object. As described earlier in this chapter, process page tables point to these prototype PTEs, which in turn map the pages being referenced.

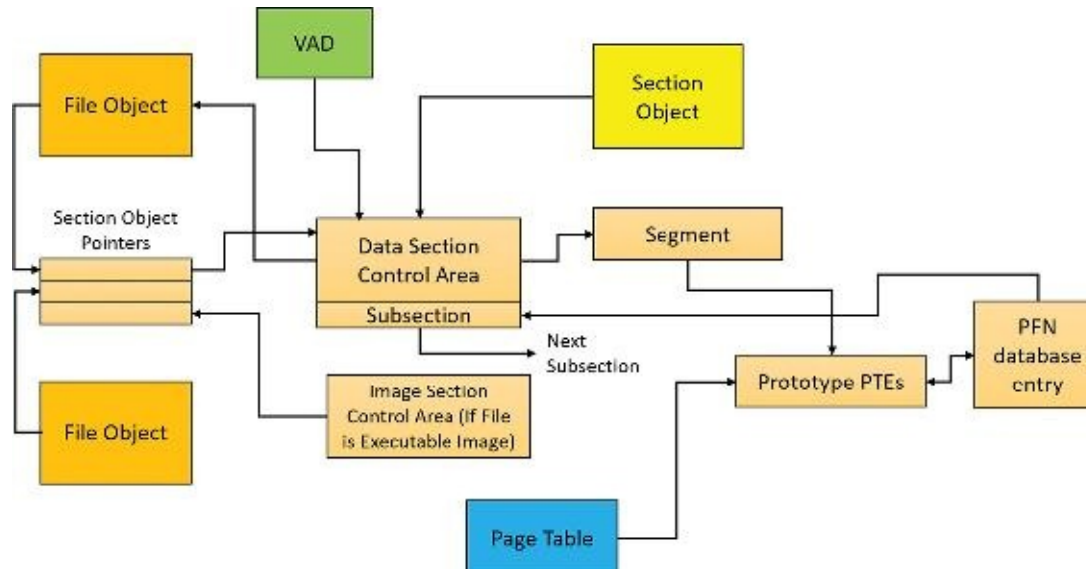


FIGURE 5-32 Internal section structures.

Although Windows ensures that any process that accesses (reads or writes) a file will always see the same consistent data, there is one case in which two copies of pages of a file can reside in physical memory. (Even in this case, all accessors get the latest copy and data consistency is maintained.) This duplication can happen when an image file has been accessed as a data file (having been read or written) and is then run as an executable image. (An example might be when an image is linked and then run; the linker had the file open for data access, and then when the image was run, the image loader mapped it as an executable.) Internally, the following actions occur:

1. If the executable file was created using the file-mapping APIs or the cache manager, a data control area is created to represent the data pages in the image file being read or written.
2. When the image is run and the section object is created to map the image as an executable, the memory manager finds that the section object pointers for the image file point to a data control area and flushes the section. This step is necessary to ensure that any modified pages have been written to disk before accessing the image through the image control area.
3. The memory manager creates a control area for the image file.
4. As the image begins execution, its (read-only) pages are faulted in from the image file or copied directly over from the data file if the corresponding data page is resident.

Because the pages mapped by the data control area might still be resident (on the standby list), this is the one case in which two copies of the same data are in two different pages in memory. However, this duplication doesn't result in a data consistency issue. This is because, as mentioned, the data control area has already been flushed to disk, so the pages read from the image are up to date (and these pages are never written back to disk).

EXPERIMENT: Viewing control areas

To find the address of the control area structures for a file, you must first get the address of the file object in question. You can obtain this address through the kernel debugger by dumping the process handle table with the `!handle` command and noting the object address of a file object. Although the kernel debugger `!file` command displays the basic information in a file object, it doesn't display the pointer to the section object pointers structure. Then, using the `dt` command, format the file object to get the address of the section object pointers structure. This structure consists of three pointers: a pointer to the data control area, a pointer to the shared cache map (explained in Chapter 14 in Part 2), and a pointer to the image control area. From the section object pointers structure, you can obtain the address of a control area for the file (if one exists) and feed that address into the `!ca` command.

For example, if you open a PowerPoint file and use `!handle` to display the handle table for that process, you will find an open handle to the PowerPoint file (you can do a text search). (For more information on using `!handle`, see the “Object manager” section in Chapter 8 in Part 2 or the debugger documentation.)

[Click here to view code image](#)

```
lkd> !process 0 0 powerpnt.exe
PROCESS fffffc8068913e080
    SessionId: 1  Cid: 2b64    Peb: 01249000  ParentCid: 1d38
    DirBase: 252e25000  ObjectTable: fffffda8f49269c40  HandleCount:
1915.
    Image: POWERPNT.EXE
lkd> .process /p fffffc8068913e080
Implicit process is now fffffc806'8913e080
lkd> !handle
...
0c08: Object: fffffc8068f56a630  GrantedAccess: 00120089  Entry:
ffffda8f491d0020
Object: fffffc8068f56a630  Type: (fffffc8068256cb00) File
    ObjectHeader: fffffc8068f56a600 (new version)
        HandleCount: 1  PointerCount: 30839
        Directory Object: 00000000  Name:
\WindowsInternals\7thEdition\Chapter05\
diagrams.pptx {HarddiskVolume2}
...
```

Taking the file object address (FFFFFC8068F56A630) and formatting it with `dt` results in this:

[Click here to view code image](#)

```
lkd> dt nt!_file_object fffffc8068f56a630
+0x000 Type           : 0n5
+0x002 Size           : 0n216
+0x008 DeviceObject   : 0xfffffc806'8408cb40 _DEVICE_OBJECT
+0x010 Vpb            : 0xfffffc806'82feba00 _VPB
+0x018 FsContext      : 0xfffffda8f'5137cbd0 Void
+0x020 FsContext2     : 0xfffffda8f'4366d590 Void
```

```
+0x028 SectionObjectPointer : 0xfffffc806'8ec0c558
_SECTION_OBJECT_POINTERS
...
```

Taking the address of the section object pointers structure and formatting it with dt results in this:

[Click here to view code image](#)

```
lkd> dt nt!_section_object_pointers 0xfffffc806'8ec0c558
+0x000 DataSectionObject : 0xfffffc806'8e838c10 Void
+0x008 SharedCacheMap    : 0xfffffc806'8d967bd0 Void
+0x010 ImageSectionObject : (null)
```

Finally, you can use !ca to display the control area using the address:

[Click here to view code image](#)

```
lkd> !ca 0xfffffc806'8e838c10
ControlArea @ fffffc8068e838c10
  Segment      fffffda8f4d97fdc0  Flink      fffffc8068ecf97b8  Blink
fffffc8068ecf97b8
  Section Ref      1  Pfn Ref      58  Mapped
Views      2
  User Ref      0  WaitForDel      0  Flush
Count      0
  File Object  fffffc8068e5d3d50  ModWriteCount      0  System
Views      2
  WritableRefs      0
  Flags (8080) File
WasPurged      \WindowsInternalsBook\7thEdition\Chapter05\diagrams.pptx

Segment @ fffffda8f4d97fdc0
  ControlArea      fffffc8068e838c10  ExtendInfo      0000000000000000
  Total Ptes      80
  Segment Size      80000  Committed      0
  Flags (c0000) ProtectionMask

Subsection 1 @ fffffc8068e838c90
  ControlArea      fffffc8068e838c10  Starting Sector      0  Number Of
Sectors      58
  Base Pte      fffffda8f48eb6d40  Ptes In Subsect      58  Unused
Ptes      0
  Flags      d  Sector
Offset      0  Protection      6
  Accessed
  Flink      fffffc8068bb7fcf0  Blink      fffffc8068bb7fcf0  MappedView

Subsection 2 @ fffffc8068c2e05b0
  ControlArea      fffffc8068e838c10  Starting Sector      58  Number Of
Sectors      28
```

```

Base Pte      fffffda8f3cc45000  Ptes In Subsect      28  Unused
Ptes          1d8
Flags                                d  Sector
Offset         0  Protection                                6
Accessed
Flink          fffffc8068c2e0600  Blink    fffffc8068c2e0600  MappedView

```

Another technique is to display the list of all control areas with the `!memusage` command. The following excerpt is from the output of this command. (The command might take a long time to complete on a system with a large amount of memory.)

[Click here to view code image](#)

```

lkd> !memusage
loading PFN database
loading (100% complete)

Compiling memory usage data (99% Complete).

```

```

          Zeroed:      98533 (   394132 kb)
          Free:        1405 (     5620 kb)
        Standby:     331221 (  1324884 kb)
        Modified:      83806 (   335224 kb)
ModifiedNoWrite:       116 (     464 kb)
    Active/Valid:  1556154 (  6224616 kb)
      Transition:         5 (        20 kb)
      SLIST/Bad:     1614 (     6456 kb)
        Unknown:         0 (          0 kb)
        TOTAL:     2072854 (  8291416 kb)

```

```

Dangling Yes Commit:      130 (     520 kb)
Dangling No Commit:   514812 (  2059248 kb)
Building kernel map
Finished building kernel map

```

```
(Master1 0 for 1c0)
```

```
(Master1 0 for e80)
```

```
(Master1 0 for ec0)
```

```
(Master1 0 for f00)
```

```
Scanning PFN database - (02% complete)
```

```
(Master1 0 for de80)
```

```
Scanning PFN database - (100% complete)
```

```
Usage Summary (in Kb):
```

```
Control      Valid Standby Dirty Shared Locked PageTables  name

```

```

fffffffffffd 1684540      0      0      0 1684540      0      0 AWE
ffff8c0b7e4797d0      64      0      0      0      0      0 mapped_file(
Microsoft-
Windows-Kernel-PnP%4Configuration.evtx )
ffff8c0b7e481650      0      4      0      0      0      0 mapped_file(
No name for file )
ffff8c0b7e493c00      0      40      0      0      0      0 mapped_file(
FSD-{ED5680AF-
0543-4367-A331-850F30190B44}.FSD )
ffff8c0b7e4a1b30      8      12      0      0      0      0 mapped_file(
msidle.dll )
ffff8c0b7e4a7c40     128      0      0      0      0      0 mapped_file(
Microsoft-
Windows-Diagnosis-PCW%4Operational.evtx )
ffff8c0b7e4a9010     16      8      0     16      0      0 mapped_file(
netjoin.dll )8a04db00
ffff8c0b7f8cc360 8212      0      0      0      0      0 mapped_file(
OUTLOOK.EXE )
ffff8c0b7f8cd1a0     52     28      0      0      0      0 mapped_file(
verdanab.ttf )
ffff8c0b7f8ce910      0      4      0      0      0      0 mapped_file(
No name for file )
ffff8c0b7f8d3590      0      4      0      0      0      0 mapped_file(
No name for file )
...

```

The `Control` column points to the control area structure that describes the mapped file. You can display control areas, segments, and subsections with the kernel debugger `!ca` command. For example, to dump the control area for the mapped file Outlook.exe in this example, type the `!ca` command followed by the `Control` column number, as shown here:

[Click here to view code image](#)

```

lkd> !ca fffff8c0b7f8cc360

```

```

ControlArea @ fffff8c0b7f8cc360
Segment fffffdf08d8a55670 Flink fffff8c0b834f1fd0 Blink
fffff8c0b834f1fd0
Section Ref 1 Pfn Ref 806 Mapped
Views 1
User Ref 2 WaitForDel 0 Flush
Count c5a0
File Object fffff8c0b7f0e94e0 ModWriteCount 0 System
Views ffff
WritableRefs 80000161
Flags (a0) Image File

\Program Files (x86)\Microsoft

```

Office\root\Office16\OUTLOOK.EXE

Segment @ fffffdf08d8a55670

ControlArea	fffff8c0b7f8cc360	BasedAddress	0000000000be0000
Total Ptes	1609		
Segment Size	1609000	Committed	0
Image Commit	f4	Image Info	fffffdf08d8a556b8
ProtoPtes	fffffdf08dab6b000		
Flags (c20000)	ProtectionMask		

Subsection 1 @ fffff8c0b7f8cc3e0

ControlArea	fffff8c0b7f8cc360	Starting Sector	0	Number Of
Sectors	2			
Base Pte	fffffdf08dab6b000	Ptes In Subsect	1	Unused
Ptes	0			
Flags	2	Sector		
Offset	0	Protection	1	

Subsection 2 @ fffff8c0b7f8cc418

ControlArea	fffff8c0b7f8cc360	Starting Sector	2	Number Of
Sectors	7b17			
Base Pte	fffffdf08dab6b008	Ptes In Subsect	f63	Unused
Ptes	0			
Flags	6	Sector		
Offset	0	Protection	3	

Subsection 3 @ fffff8c0b7f8cc450

ControlArea	fffff8c0b7f8cc360	Starting Sector	7b19	Number Of
Sectors	19a4			
Base Pte	fffffdf08dab72b20	Ptes In Subsect	335	Unused
Ptes	0			
Flags	2	Sector		
Offset	0	Protection	1	

Subsection 4 @ fffff8c0b7f8cc488

ControlArea	fffff8c0b7f8cc360	Starting Sector	94bd	Number Of
Sectors	764			
Base Pte	fffffdf08dab744c8	Ptes In Subsect	f2	Unused
Ptes	0			
Flags	a	Sector		
Offset	0	Protection	5	

Subsection 5 @ fffff8c0b7f8cc4c0

ControlArea	fffff8c0b7f8cc360	Starting Sector	9c21	Number Of
Sectors	1			
Base Pte	fffffdf08dab74c58	Ptes In Subsect	1	Unused
Ptes	0			

Flags	a	Sector		
Offset	0	Protection	5	
Subsection 6 @ fffff8c0b7f8cc4f8				
ControlArea	fffff8c0b7f8cc360	Starting Sector	9c22	Number Of
Sectors 1				
Base Pte	fffffdf08dab74c60	Ptes In Subsect	1	Unused
Ptes 0				
Flags	a	Sector		
Offset	0	Protection	5	
Subsection 7 @ fffff8c0b7f8cc530				
ControlArea	fffff8c0b7f8cc360	Starting Sector	9c23	Number Of
Sectors c62				
Base Pte	fffffdf08dab74c68	Ptes In Subsect	18d	Unused
Ptes 0				
Flags	2	Sector		
Offset	0	Protection	1	
Subsection 8 @ fffff8c0b7f8cc568				
ControlArea	fffff8c0b7f8cc360	Starting Sector	a885	Number Of
Sectors 771				
Base Pte	fffffdf08dab758d0	Ptes In Subsect	ef	Unused
Ptes 0				
Flags	2	Sector		
Offset	0	Protection	1	

Working sets

Now that you’ve looked at how Windows keeps track of physical memory and how much memory it can support, we’ll explain how Windows keeps a subset of virtual addresses in physical memory.

As you’ll recall, a subset of virtual pages resident in physical memory is called a *working set*. There are three kinds of working sets:

- **Process working sets** These contain the pages referenced by threads within a single process.
- **System working sets** These contain the resident subset of the pageable system code (for example, Ntoskrnl.exe and drivers), paged pool, and the system cache.
- **Session’s working set** Each session has a working set that contains the resident subset of the kernel-mode session-specific data structures allocated by the kernel-mode part of the Windows subsystem (Win32k.sys), session paged pool, session mapped views, and other session-space device drivers.

Before examining the details of each type of working set, let’s look at the overall policy for deciding which pages are brought into physical memory and how long they remain.

Demand paging

The Windows memory manager uses a demand-paging algorithm with clustering to load pages into memory. When a thread receives a page fault, the memory manager loads into memory the faulted page plus a small number of pages preceding and/or following it. This strategy attempts to minimize the number of paging I/Os a thread will incur. Because programs—especially large ones—tend to execute in small regions of their address space at any given time, loading clusters of virtual pages reduces the number of disk reads. For page faults that reference data pages in images, the cluster size is three pages. For all other page faults, the cluster size is seven pages.

However, a demand-paging policy can result in a process incurring many page faults when its threads first begin executing or when they resume execution at a later point. To optimize the startup of a process (and the system), Windows has an intelligent prefetch engine called the *logical prefetcher*, described in the next section. Further optimization and prefetching is performed by another component, called *SuperFetch*, described later in the chapter.

Logical prefetcher and ReadyBoot

During a typical system boot or application startup, the order of faults is such that some pages are brought in from one part of a file, then perhaps from a distant part of the same file, then from a different file, then perhaps from a directory, and then again from the first file. This jumping around slows down each access considerably. Indeed, analysis shows that disk seek times are a dominant factor in slowing boot and application startup times. By prefetching batches of pages all at once, you can achieve a more sensible ordering of access without excessive backtracking, thus improving the overall time for system and application startup. The pages that are needed can be known in advance because of the high correlation in accesses across boots or application starts.

The prefetcher tries to speed the boot process and application startup by monitoring the data and code accessed by boot and application startups and using that information at the beginning of a subsequent boot or application startup to read in the code and data. When the prefetcher is active, the memory manager notifies the prefetcher code in the kernel of page faults—those that require that data be read from disk (hard faults) and those that simply require data already in memory to be added to a process's working set (soft faults). The prefetcher monitors the first 10 seconds of application startup. For boot, the prefetcher by default traces from system start through the 30 seconds following the start of the user's shell (typically Explorer) or, failing that, through 60 seconds following Windows service initialization or through 120 seconds, whichever comes first.

The trace assembled in the kernel notes faults taken on the NTFS master file table (MFT) metadata file (if the application accesses files or directories on NTFS volumes), referenced files, and referenced directories. With the trace assembled, the kernel prefetcher code waits for requests from the prefetcher component of the *Superfetch* service (%SystemRoot%\System32\Sysmain.dll), running in an instance of Svchost. The *Superfetch* service is responsible for both the logical prefetching component in the kernel and the *SuperFetch* component that we'll talk about later. The prefetcher signals the \KernelObjects\PrefetchTracesReady event to inform the *Superfetch* service that it can now query trace data.



The Superfetch service (which hosts the logical prefetcher, although it is a completely separate component from the actual SuperFetch functionality) performs a call to the internal `NtQuerySystemInformation` system call requesting the trace data. The logical prefetcher post-processes the trace data, combining it with previously collected data, and writes it to a file in the `%SystemRoot%\Prefetch` folder. (See [Figure 5-33](#).) The file's name is the name of the application to which the trace applies followed by a dash and the hexadecimal representation of a hash of the file's path. The file has a .pf extension. An example would be `NOTEPAD.EXE-9FB27C0E.PF`.



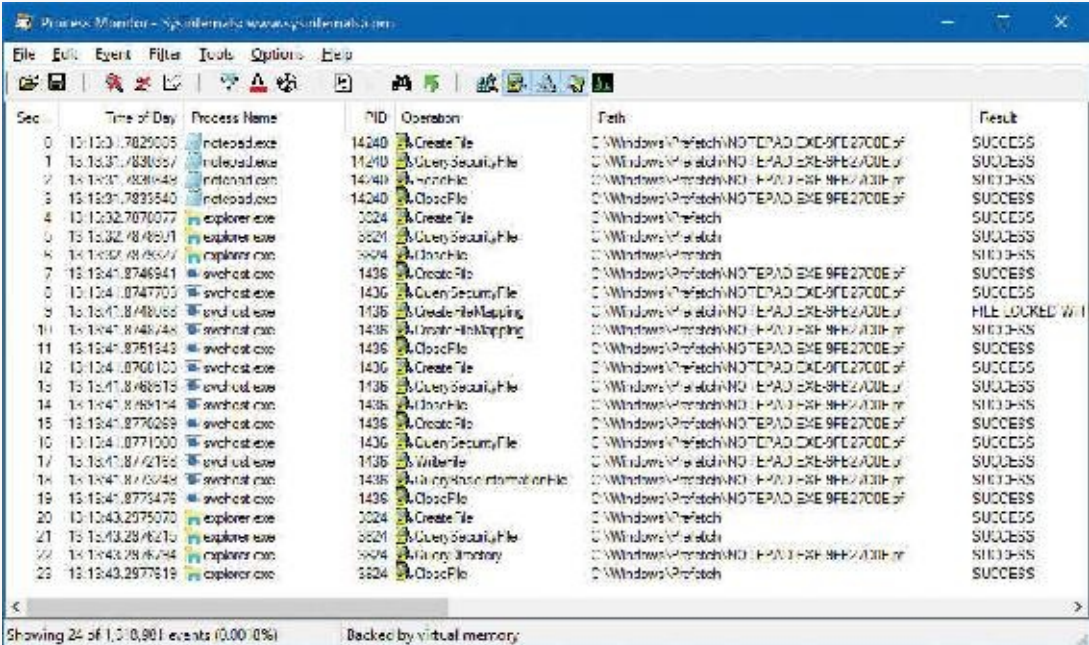
There is an exception to the file name rule for images that host other components, including the Microsoft Management Console (%SystemRoot%\System32\Mmc.exe), the service hosting process (%SystemRoot%\System32\Svchost.exe), the RunDLL component (%SystemRoot%\System32\Rundll32.exe), and Dllhost (%SystemRoot%\System32\Dllhost.exe). Because add-on components are specified on the command line for these applications, the prefetcher includes the command line in the generated hash. Thus, invocations of these applications with different components on the command line will result in different traces.

For a system boot, a different mechanism is used, called *ReadyBoot*. ReadyBoot tries to optimize I/O operations by creating large and efficient I/O reads and storing the data in RAM. When system components require the data, it's serviced through the stored RAM. This especially benefits mechanical disks, but can be also useful for SSDs. Information on the files to prefetch is stored after boot in the ReadyBoot subdirectory of the Prefetch directory shown in [Figure 5-33](#). Once boot is complete, the cached data in RAM is deleted. For very fast SSDs, ReadyBoot is off by default because its gains are marginal, if any.

When the system boots or an application starts, the prefetcher is called to give it an opportunity to prefetch. The prefetcher looks in the prefetch directory to see if a trace file exists for the prefetch scenario in question. If it does, the prefetcher calls NTFS to prefetch any MFT metadata file references, reads in the contents of each of the directories referenced, and finally opens each file referenced. It then calls the memory manager function `MmPrefetchPages` to read in any data and code specified in the trace that's not already in memory. The memory manager initiates all the reads asynchronously and then waits for them to complete before letting an application's startup continue.

EXPERIMENT: Watching prefetch file reads and writes

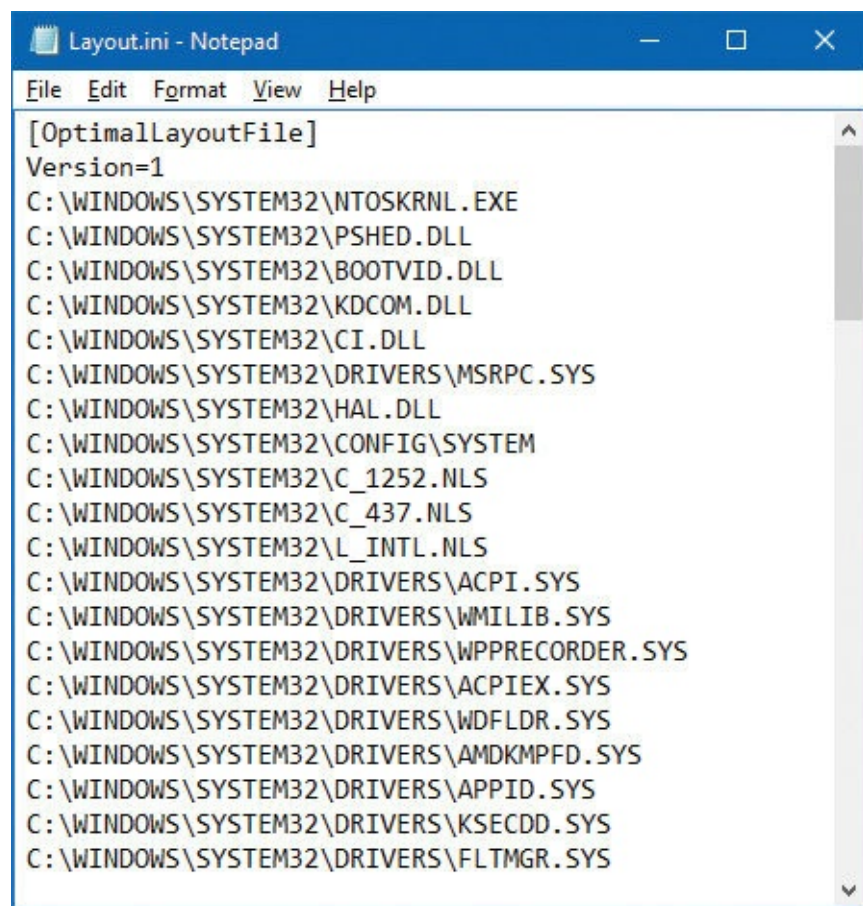
If you capture a trace of application startup with Process Monitor from Sysinternals on a client edition of Windows (Windows Server editions disable prefetching by default), you can see the prefetcher check for and read the application's prefetch file (if it exists). In addition, you can see the prefetcher write out a new copy of the file roughly 10 seconds after the application starts. Here is a capture of Notepad startup with an `Include` filter set to `prefetch` so that Process Monitor shows only accesses to the `%SystemRoot%\Prefetch` directory:

The screenshot shows the Process Monitor application window with a table of events. The 'Filter' menu is open, and 'Include' is selected, with 'prefetch' entered in the filter text. The table displays file operations for 'notepad.exe' and 'svchost.exe'. The 'Path' column shows the full path to the prefetch file in the Windows directory. The 'Result' column indicates the outcome of each operation, such as 'SUCCESS' or 'FILE LOCKED WITH OTHER APPLICATIONS'.

Sec	Time of Day	Process Name	PID	Operation	Path	Result
0	12:13:11.7029005	notepad.exe	14240	Create File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
1	12:13:31.7830657	notepad.exe	14240	Query Security File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
2	12:13:31.7830657	notepad.exe	14240	Open File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
3	12:13:31.7830657	notepad.exe	14240	Close File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
4	12:13:32.7070077	explorer.exe	3024	Create File	C:\Windows\Prefetch	SUCCESS
5	12:13:32.7876571	explorer.exe	3024	Query Security File	C:\Windows\Prefetch	SUCCESS
6	12:13:32.7876571	explorer.exe	3024	Open File	C:\Windows\Prefetch	SUCCESS
7	12:13:41.8746541	svchost.exe	1436	Create File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
8	12:13:41.8746541	svchost.exe	1436	Query Security File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
9	12:13:41.8746541	svchost.exe	1436	Create File Mapping	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	FILE LOCKED WITH OTHER APPLICATIONS
10	12:13:41.8746541	svchost.exe	1436	Unmap File Mapping	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
11	12:13:41.8746541	svchost.exe	1436	Close File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
12	12:13:41.8746541	svchost.exe	1436	Create File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
13	12:13:41.8746541	svchost.exe	1436	Query Security File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
14	12:13:41.8746541	svchost.exe	1436	Open File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
15	12:13:41.8746541	svchost.exe	1436	Create File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
16	12:13:41.8746541	svchost.exe	1436	Query Security File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
17	12:13:41.8746541	svchost.exe	1436	Write File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
18	12:13:41.8746541	svchost.exe	1436	Unmap File Mapping	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
19	12:13:41.8746541	svchost.exe	1436	Close File	C:\Windows\Prefetch\NOTEPAD.CXC-9FE2700C.pf	SUCCESS
20	12:13:43.2975070	explorer.exe	3024	Create File	C:\Windows\Prefetch	SUCCESS
21	12:13:43.2975070	explorer.exe	3024	Query Security File	C:\Windows\Prefetch	SUCCESS
22	12:13:43.2975070	explorer.exe	3024	Open File	C:\Windows\Prefetch	SUCCESS
23	12:13:43.2975070	explorer.exe	3024	Close File	C:\Windows\Prefetch	SUCCESS

Lines 0–3 show the Notepad prefetch file being read in the context of the Notepad process during its startup. Lines 7–19 (which have time stamps 10 seconds later than the first four lines) show the Superfetch service—running in the context of a Svchost process—writing out the updated prefetch file.

To minimize seeking even further, every three days or so, during system idle periods, the Superfetch service organizes a list of files and directories in the order that they are referenced during a boot or application start and stores it in a file named `%SystemRoot%\Prefetch\Layout.ini` (see [Figure 5-34](#)). This list also includes frequently accessed files tracked by Superfetch.



```
[OptimalLayoutFile]
Version=1
C:\WINDOWS\SYSTEM32\NTOSKRNL.EXE
C:\WINDOWS\SYSTEM32\PSHED.DLL
C:\WINDOWS\SYSTEM32\BOOTVID.DLL
C:\WINDOWS\SYSTEM32\KDCOM.DLL
C:\WINDOWS\SYSTEM32\CI.DLL
C:\WINDOWS\SYSTEM32\DRIVERS\MSRPC.SYS
C:\WINDOWS\SYSTEM32\HAL.DLL
C:\WINDOWS\SYSTEM32\CONFIG\SYSTEM
C:\WINDOWS\SYSTEM32\C_1252.NLS
C:\WINDOWS\SYSTEM32\C_437.NLS
C:\WINDOWS\SYSTEM32\L_INTL.NLS
C:\WINDOWS\SYSTEM32\DRIVERS\ACPI.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\WMILIB.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\WPPREORDER.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\ACPIEX.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\WDFLDR.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\AMDKMPFD.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\APPID.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\KSECDD.SYS
C:\WINDOWS\SYSTEM32\DRIVERS\FLTMGR.SYS
```

FIGURE 5-34 Prefetch defragmentation layout file.

It then launches the system defragmenter with a command-line option that tells the defragmenter to defragment based on the contents of the file instead of performing a full defrag. The defragmenter finds a contiguous area on each volume large enough to hold all the listed files and directories that reside on that volume and then moves them in their entirety into that area so that they are stored one after the other. Thus, future prefetch operations will even be more efficient because all the data read in is now stored physically on the disk in the order it will be read. Because the files defragmented for prefetching usually number only in the hundreds, this defragmentation is much faster than full-volume defragmentations.

Placement policy

When a thread receives a page fault, the memory manager must determine where in physical memory to put the virtual page. The set of rules it uses to determine the best position is called a *placement policy*. Windows considers the size of CPU memory caches when choosing page frames to minimize unnecessary thrashing of the cache.

If physical memory is full when a page fault occurs, Windows uses a replacement policy to determine which virtual page must be removed from memory to make room for the new page. Common replacement policies include least recently used (LRU) and first in, first out (FIFO). The LRU algorithm (also known as the *clock algorithm*, as implemented in most versions of UNIX) requires the virtual memory system to track when a page in memory is used. When a new page frame is required, the page that hasn't been used for the greatest amount of time is removed from the working set. The FIFO algorithm is somewhat simpler: It removes the page that has been in physical memory for the greatest amount of time, regardless of how often it's been used.

Replacement policies can be further characterized as either global or local. A global replacement policy allows a page fault to be satisfied by any page frame, regardless of whether that frame is owned by

another process. For example, a global replacement policy using the FIFO algorithm would locate the page that has been in memory the longest and would free it to satisfy a page fault. A local replacement policy would limit its search for the oldest page to the set of pages already owned by the process that incurred the page fault. Be aware that global replacement policies make processes vulnerable to the behavior of other processes. For example, an ill-behaved application can undermine the entire operating system by inducing excessive paging activity in all processes.

Windows implements a combination of local and global replacement policies. When a working set reaches its limit and/or needs to be trimmed because of demands for physical memory, the memory manager removes pages from working sets until it has determined there are enough free pages.

Working set management

Every process starts with a default working set minimum of 50 pages and a working set maximum of 345 pages. Although it has little effect, you can change these working set limits with the `SetProcessWorkingSetSize` function, although you must have the increase scheduling priority (`SeIncreaseBasePriorityPrivilege`) privilege to do this. However, unless you have configured the process to use hard working set limits, these limits are ignored. That is, the memory manager will permit a process to grow beyond its maximum if it is paging heavily and there is ample memory. (Conversely the memory manager will shrink a process below its working set minimum if it is not paging and there is a high demand for physical memory on the system.) You can set hard working set limits using the `SetProcessWorkingSetSizeEx` function along with the `QUOTA_LIMITS_HARDWS_MAX_ENABLE` flag, but it is almost always better to let the system manage your working set.

On 32 bit systems, the maximum working set size can't exceed the system-wide maximum calculated at system initialization time, stored in the `MiMaximumWorkingSet` kernel variable. On x64 systems, physical memory would be the practical upper limit, as the virtual address space is so vast. The working set maximums are listed in [Table 5-15](#).

Windows Version	Working Set Maximum
x86, ARM	2 GB—64 KB (0x7FFF0000)
x86 versions of Windows booted with <code>increaseuserva</code>	2 GB—64 KB + user virtual address increase
x64 (Windows 8, Server 2012)	8.192 GB (8 TB)
X64 (Windows 8.1, 10, Server 2012 R2, 2016)	128 TB

TABLE 5-15 Upper limit for working set maximums

When a page fault occurs, the process's working set limits and the amount of free memory on the system are examined. If conditions permit, the memory manager allows a process to grow to its working set maximum (or beyond if the process does not have a hard working set limit and there are enough free pages available). However, if memory is tight, Windows replaces rather than adds pages in a working set when a fault occurs.

Windows attempts to keep memory available by writing modified pages to disk. Still, when modified pages are being generated at a very high rate, more memory is required to meet memory demands. Therefore, when physical memory runs low, the working set manager, a routine that runs in the context of the balance set manager system thread (described in the next section), initiates automatic working set trimming to increase the amount of free memory available in the system. You can also initiate working set trimming of your own process—for example, after process initialization—with the aforementioned `Windows SetProcessWorkingSetSizeEx` function.

The working set manager examines available memory and decides which, if any, working sets need to be trimmed. If there is ample memory, the working set manager calculates how many pages could be removed from working sets if needed. If trimming is needed, it looks at working sets that are above their minimum setting. It also dynamically adjusts the rate at which it examines working sets and arranges the list of processes that are candidates to be trimmed into an optimal order. For example, processes with many pages that have not been accessed recently are examined first; larger processes that have been idle longer are considered before smaller processes that are running more often; the process running the foreground application is considered last; and so on.

When the working set manager finds processes that are using more than their minimums, it looks for

pages to remove from the working sets, making the pages available for other uses. If the amount of free memory is still too low, the working set manager continues removing pages from processes' working sets until it achieves a minimum number of free pages on the system.

The working set manager tries to remove pages that haven't been accessed recently by checking the accessed bit in the hardware PTE to see whether a page has been accessed. If the bit is clear, the page is said to be *aged*. That is, a count is incremented indicating that the page hasn't been referenced since the last working set trim scan. Later, the age of pages is used to locate candidate pages to remove from the working set.

If the hardware PTE accessed bit is set, the working set manager clears it and goes on to examine the next page in the working set. In this way, if the accessed bit is clear the next time the working set manager examines the page, it knows that the page hasn't been accessed since the last time it was examined. This scan for pages to remove continues through the working set list until either the number of desired pages has been removed or the scan has returned to the starting point. The next time the working set is trimmed, the scan picks up where it left off last.

EXPERIMENT: Viewing process working set sizes

You can use Performance Monitor to examine process working set sizes by looking at the performance counters shown in the following table. Several other process viewer utilities (such as Task Manager and Process Explorer) also display the process working set size.

Counter	Description
Process: Working Set	This notes the current size of the selected process's working set in bytes.
Process: Working Set Peak	This tracks the peak size of the selected process's working set in bytes.
Process: Page Faults/Sec	This indicates the number of page faults for the process that occur each second.

You can also get the total of all the process working sets by selecting the `_Total` process in the instance box in Performance Monitor. This process isn't real; it's simply a total of the process-specific counters for all processes currently running on the system. The total you see is larger than the actual RAM being used, however, because the size of each process working set includes pages being shared by other processes. Thus, if two or more processes share a page, the page is counted in each process's working set.

EXPERIMENT: Working set versus virtual size

Earlier in this chapter, you used the TestLimit utility to create two processes: one with a large amount of memory that was merely reserved, and one in which the memory was private committed. You then examined the difference between them with Process Explorer. Now we will create a third TestLimit process—one that not only commits the memory but also accesses it, thus bringing it into its working set. Follow these steps:

1. Create a new TestLimit process.

[Click here to view code image](#)

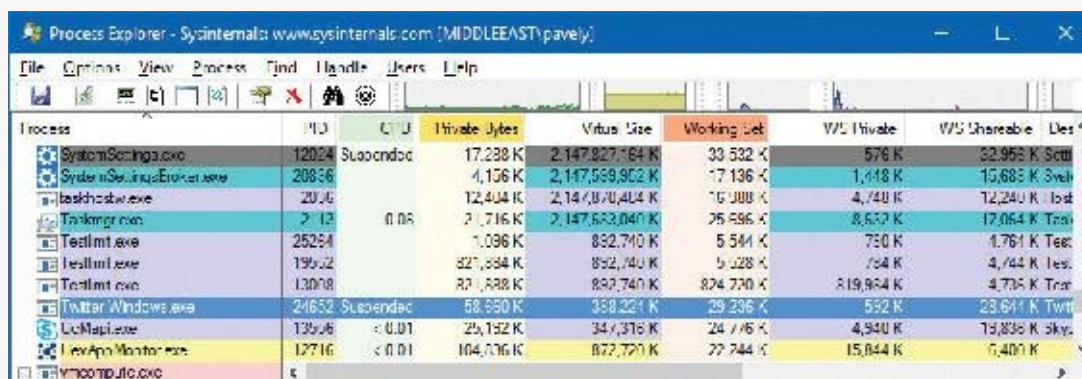
```
C:\Users\pavely>testlimit -d 1 -c 800
```

```
Testlimit v5.24 - test Windows limits  
Copyright (C) 2012-2015 Mark Russinovich  
Sysinternals - www.sysinternals.com
```

```
Process ID: 13008
```

```
Leaking private bytes with touch 1 MB at a time...  
Leaked 800 MB of private memory (800 MB total leaked).  
Lasterror: 0  
The operation completed successfully.
```

2. Open Process Explorer.
3. Open the **View** menu, choose **Select Columns**, and click the **Process Memory** tab.
4. Enable the **Private Bytes**, **Virtual Size**, **Working Set Size**, **WS Shareable Bytes**, and **WS Private Bytes** counters.
5. Find the three instances of TestLimit, as shown in the display:



Process	PID	CPU	Private Bytes	Virtual Size	Working Set	WS Private	WS Shareable	Des
SystemSettings.exe	12024	Suspended	17,238 K	2,147,827,164 K	33,532 K	576 K	32,956 K	Scrt
SystemSettings.exe	20836		4,156 K	2,147,599,902 K	17,136 K	1,418 K	15,688 K	Svch
taskhost.exe	2036		12,404 K	2,147,827,164 K	16,000 K	4,740 K	11,260 K	Task
Taskmgr.exe	2112	0.0%	21,716 K	2,147,599,902 K	25,596 K	8,552 K	17,044 K	Task
TestLimit.exe	25254		1,096 K	892,740 K	5,544 K	750 K	1,764 K	Test
TestLimit.exe	19502		821,888 K	892,740 K	5,528 K	754 K	4,744 K	Test
TestLimit.exe	13008		821,888 K	892,740 K	824,720 K	819,964 K	4,736 K	Test
Twitter Windows.exe	24652	Suspended	58,590 K	358,224 K	29,236 K	552 K	28,644 K	Twitter
UxMapi32.exe	13506	< 0.01	25,152 K	34,376 K	24,116 K	4,940 K	19,836 K	Sky
UxAppMntr.exe	12716	< 0.01	104,036 K	872,720 K	22,744 K	15,844 K	6,400 K	UxAppMntr
vmtoolsd.exe	8							vmtoolsd

The new TestLimit process is the third one shown, PID 13008. It is the only one of the three that actually referenced the memory allocated, so it is the only one with a working set that reflects the size of the test allocation.

Note that this result is possible only on a system with enough RAM to allow the process to grow to such a size. Even on this system, not quite all of the private bytes (821,888 K) are in the WS Private portion of the working set. A small number of the private pages have been pushed out of the process working set due to replacement or have not been paged in yet.

EXPERIMENT: Viewing the working set list in the debugger

You can view the individual entries in the working set by using the kernel debugger !wsle command. The following example shows a partial output of the working set list of WinDbg (32-bit system):

[Click here to view code image](#)

```
lkd> !wsle 7

Working Set Instance @ c0802d50
Working Set Shared @ c0802e30

      FirstFree      f7d FirstDynamic      6
      LastEntry      203d NextSlot      6 LastInitialized      20
      NonDirect      0 HashTable      0 HashTableSize
```

Reading the WSLE data

.....
.....

Virtual Address	Age	Locked	ReferenceCount
c0603009	0	0	1
c0602009	0	0	1
c0601009	0	0	1
c0600009	0	0	1
c0802d59	6	0	1
c0604019	0	0	1
c0800409	2	0	1
c0006209	1	0	1
77290a05	5	0	1
7739aa05	5	0	1
c0014209	1	0	1
c0004209	1	0	1
72a37805	4	0	1
b50409	2	0	1
b52809	4	0	1
7731dc05	6	0	1
bbec09	6	0	1
bbfc09	6	0	1
6c801805	4	0	1
772a1405	2	0	1
944209	1	0	1
77316a05	5	0	1
773a4209	1	0	1
77317405	2	0	1
772d6605	3	0	1
a71409	2	0	1
c1d409	2	0	1
772d4a05	5	0	1

77342c05	6	0	1
6c80f605	3	0	1
77320405	2	0	1
77323205	1	0	1
77321405	2	0	1
7ffe0215	1	0	2
a5fc09	6	0	1
7735cc05	6	0	1

...

Notice that some entries in the working set list are page table pages (the ones with addresses greater than 0xC0000000), some are from system DLLs (the ones in the 0x7nnnnnnn range), and some are from the code of Windbg.exe itself.

Balance set manager and swapper

Working set expansion and trimming take place in the context of a system thread called the *balance set manager* (KeBalanceSetManager function). The balance set manager is created during system initialization. Although the balance set manager is technically part of the kernel, it calls the memory manager's working set manager (MmWorkingSetManager) to perform working set analysis and adjustment.

The balance set manager waits for two different event objects: an event that is signaled when a periodic timer set to fire once per second expires and an internal working set manager event that the memory manager signals at various points when it determines that working sets need to be adjusted. For example, if the system is experiencing a high page fault rate or the free list is too small, the memory manager wakes up the balance set manager so that it will call the working set manager to begin trimming working sets. When memory is more plentiful, the working set manager permits faulting processes to gradually increase the size of their working sets by faulting pages back into memory. However, the working sets will grow only as needed.

When the balance set manager wakes up because its 1-second timer has expired, it takes the following steps:

1. If the system supports Virtual Secure Mode (VSM, Windows 10 and Server 2016), then the secure kernel is called to do its periodic housekeeping (VslSecureKernelPeriodicTick).
2. Calls a routine to adjust IRP credits to optimize the usage of the per-processor look-aside lists used in IRP completion (IoAdjustIrpCredits). This allows better scalability when certain processors are under heavy I/O load. (See [Chapter 6](#) for more on IRPs.)
3. Checks the look-aside lists and adjusts their depths (if necessary) to improve access time and reduce pool usage and pool fragmentation (ExAdjustLookasideDepth).
4. Calls to adjust the Event Tracing for Windows (ETW) buffer pool size to use ETW memory buffers more efficiently (EtwAdjustTraceBuffers). (For more on ETW, see Chapter 8 in Part 2.)
5. Calls the memory manager's working set manager. The working set manager has its own internal counters that regulate when to perform working set trimming and how aggressively to trim.
6. Enforces execution time for jobs (PsEnforceExecutionLimits).
7. Every eighth time the balance set manager wakes up because its 1-second timer has expired, it signals an event that wakes up another system thread called the *swapper* (KeSwapProcess -

OrStack). It attempts to outswap kernel stacks for threads that have not executed for a long time. The swapper thread (which runs at priority 23) looks for threads that have been in a user mode wait state for 15 seconds. If it finds one, it puts the thread's kernel stack in transition (moving the pages to the modified or standby lists) to reclaim its physical memory, operating on the principle that if a thread has been waiting that long, it's going to be waiting even longer. When the last thread in a process has its kernel stack removed from memory, the process is marked to be entirely outswapped. That's why, for example, processes that have been idle for a long time (such as Winit or Winlogon) can have a working set size of zero.

System working sets

Just as processes have working sets that manage pageable portions of the process address space, the pageable code and data in the system address space is managed using three global working sets, collectively known as the *system working sets*. These global working sets are as follows:

- **System cache working set** This contains pages that are resident in the system cache.
- **Paged pool working set** This contains pages that are resident in the paged pool.
- **System PTEs working set** This contains pageable code and data from loaded drivers and the kernel image and pages from sections that have been mapped into the system space.

[Table 5-16](#) shows where these system working set types are stored.

System Working Set Type	Stored in (Windows 8.x, Server 2012/R2)	Stored in (Windows 10, Server 2016)
System cache	MmSystemCacheWs	MmStale.SystemVa.SysLenWs[0]
Paged pool	MmPagedPoolWs	MmStale.SystemVa.SysLenWs[2]
System PTEs	MmSystemPtesWs	MmStale.SystemVa.SysLenWs[1]

TABLE 5-16 System working sets

You can examine the sizes of these working sets or the sizes of the components that contribute to them with the performance counters or system variables shown in [Table 5-17](#). (Note that the performance counter values are in bytes, whereas the system variables are measured in pages.)

Performance Counter (In Bytes)	System Variable (In Pages)	Description
Memory: Cache Bytes Memory: System Cache Resident Bytes	WorkingSetSize member	This is the physical memory consumed by the file system cache.
Memory: Cache Bytes Peak	PeakWorkingSetSize member (Windows 10 and 2016) Peak member (Windows 8.x and 2012/R2)	This is the peak system working set size.
Memory: System Driver Resident Bytes	SysLenPageCounts.SysLenDriverPage (global, Windows 10 and Server 2016) MmSystemDriverPage (global, Windows 8.x and Server 2012/R2)	This is the physical memory consumed by pageable device driver code.
Memory: Pool Paged Resident Bytes	WorkingSetSize member	This is the physical memory consumed by paged pool.

TABLE 5-17 System working set performance counters

You can also examine the paging activity in the system cache working set by examining the Memory: Cache Faults/Sec performance counter. This counter describes page faults that occur in the system cache working set (both hard and soft). The PageFaultCount member in the system cache working set structure contains the value for this counter.

Memory notification events

Windows provides a way for user-mode processes and kernel-mode drivers to be notified when physical memory, paged pool, non-paged pool, and commit charge are low and/or plentiful. This information can be used to determine memory usage as appropriate. For example, if available memory is low, the application can reduce memory consumption. If available paged pool is high, the driver can allocate more memory. Finally, the memory manager also provides an event that permits notification when corrupted pages have been detected.

User-mode processes can be notified only of low or high memory conditions. An application can call the `CreateMemoryResourceNotification` function, specifying whether low or high memory notification is desired. The returned handle can be provided to any of the wait functions. When memory is low (or high), the wait completes, thus notifying the thread of the condition. Alternatively, the `QueryMemory-ResourceNotification` can be used to query the system memory condition at any time without blocking the calling thread.

Drivers, on the other hand, use the specific event name that the memory manager has set up in the `\KernelObjects` object manager directory. This is because notification is implemented by the memory manager signaling one of the globally named event objects it defines, shown in [Table 5-18](#). When a given memory condition is detected, the appropriate event is signaled, thus waking up any waiting threads.

Event Name	Description
HighCommitCondition	This event is set when the commit charge is near the maximum commit limit—in other words, memory usage is very high, very little space is available in physical memory or paging files, and the operating system cannot increase the size of its paging files.
HighMemoryCondition	This event is set whenever the amount of free physical memory exceeds the defined amount.
HighNonPagedPoolCondition	This event is set whenever the amount of non-paged pool exceeds the defined amount.
HighPagedPoolCondition	This event is set whenever the amount of paged pool exceeds the defined amount.
LowCommitCondition	This event is set when the commit charge is low relative to the current commit limit—in other words, memory usage is low and a lot of space is available in physical memory or paging files.
LowMemoryCondition	This event is set whenever the amount of free physical memory falls below the defined amount.
LowNonPagedPoolCondition	This event is set whenever the amount of free non-paged pool falls below the defined amount.
LowPagedPoolCondition	This event is set whenever the amount of free paged pool falls below the defined amount.
MaximumCommitCondition	This event is set when the commit charge is near the maximum commit limit—in other words, memory usage is very high, very little space is available in physical memory or paging files, and the operating system cannot increase the size or number of paging files.
MemoryErrors	This indicates that a bad page (non-zeroed zero page) has been detected.

TABLE 5-18 Memory manager notification events

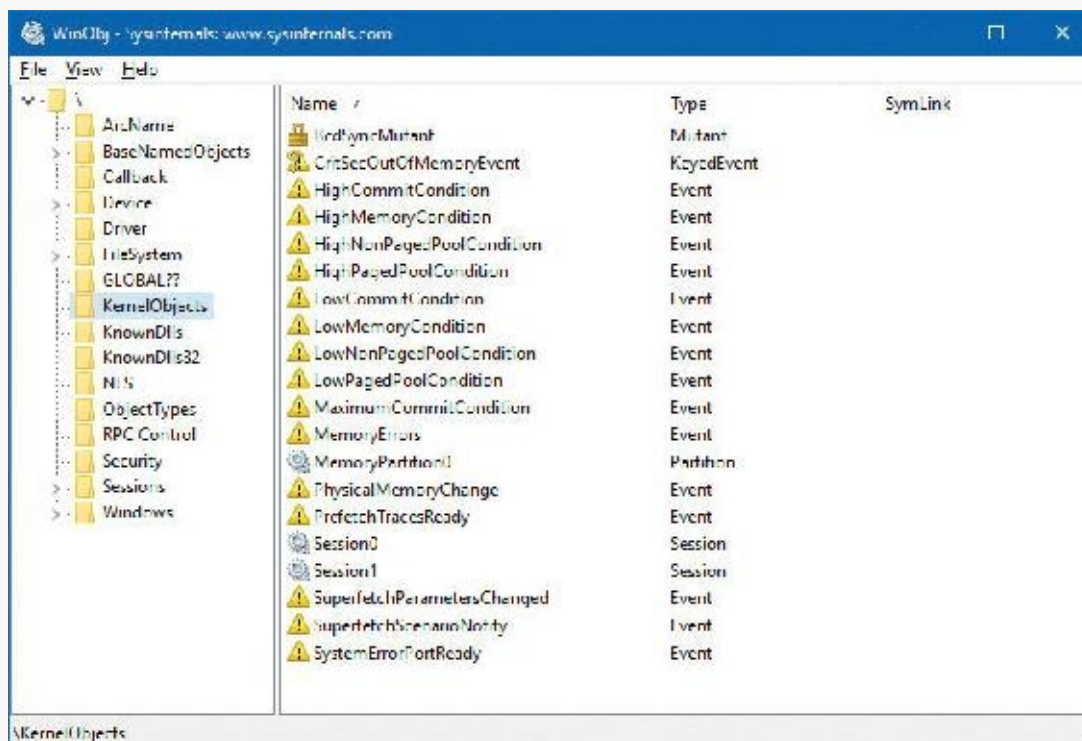


Note

You can override the high and low memory values by adding the `LowMemoryThreshold` or `HighMemoryThreshold` DWORD registry value under `HKLM\SYSTEM\CurrentControlSet\Session Manager\Memory Management`. This specifies the number of megabytes to use as the low or high threshold. You can also configure the system to crash when a bad page is detected instead of signaling a memory error event by setting the `PageValidationAction` DWORD registry value in the same key to 1.

EXPERIMENT: Viewing the memory resource notification events

To see the memory resource notification events, run WinObj from Sysinternals and click the **KernelObjects** folder. You will see both the low and high memory condition events shown in the pane on the right:



If you double-click either event, you can see how many handles and/or references have been made to the objects. To see whether any processes in the system have requested memory resource notification, search the handle table for references to `LowMemoryCondition` or `HighMemoryCondition`. To do so, use Process Explorer’s **Find** menu (choose **Find Handle or DLL**) or use WinDbg. (For a description of the handle table, see the section “Object manager” in Chapter 8 in Part 2.)

Page frame number database

Several previous sections concentrated on the virtual view of a Windows process—page tables, PTEs, and VADs. The remainder of this chapter will explain how Windows manages physical memory, starting with how Windows keeps track of physical memory. Whereas working sets describe the resident pages owned by a process or the system, the PFN database describes the state of each page in physical memory. The page states are listed in [Table 5-19](#).

Status	Description
Active (also called valid)	The page is part of a working set (either a process working set, a session working set, or a system working set), or it's not in any working set (for example, a non-paged kernel page) and a valid PTE usually points to it.
Transition	This is a temporary state for a page that isn't owned by a working set and isn't on any paging list. A page is in this state when an I/O to the page is in progress. The PTE is encoded so that collided page faults can be recognized and handled properly. (This use of the term transition differs from the use of the word in the section on invalid PTEs. An invalid transition PTE refers to a page on the standby or modified list.)
Standby	The page previously belonged to a working set but was removed or was prefetched/clustered directly into the standby list. The page wasn't modified since it was last written to disk. The PTE still refers to the physical page but it is marked invalid and in transition.
Modified	The page previously belonged to a working set but was removed. However, the page was modified while it was in use and its current contents haven't yet been written to disk or remote storage. The PTE still refers to the physical page but is marked invalid and in transition. It must be written to the backing store before the physical page can be reused.
Modified no-write	This is the same as a modified page except that the page has been marked so that the memory manager's modified page writer won't write it to disk. The cache manager marks pages as modified no-write at the request of file system drivers. For example, NTFS uses this state for pages containing file system metadata so that it can first ensure that transaction log entries are flushed to disk before the pages they are protecting are written to disk. (NTFS transaction logging is explained in Chapter 13, "File systems," in Part 2.)
Free	The page is free but has unspecified dirty data in it. For security reasons, these pages can't be given as a user page to a user process without being initialized with zeroes, but they can be overwritten with new data (for example, from a file) before being given to a user process.
Zeroed	The page is free and has been initialized with zeroes by the zero page thread or was determined to already contain zeroes.
Rom	The page represents read-only memory.
Bad	The page has generated parity or other hardware errors and can't be used (or used as part of an enclave).

TABLE 5-19 Physical page states

The PFN database consists of an array of structures that represent each physical page of memory on the system. The PFN database and its relationship to page tables are shown in [Figure 5-35](#). As this figure shows, valid PTEs usually point to entries in the PFN database (and the PFN index points to the page in physical memory), and the PFN database entries (for non-prototype PFNs) point back to the page table that is using them (if it is being used by a page table). For prototype PFNs, they point back to the prototype PTE.

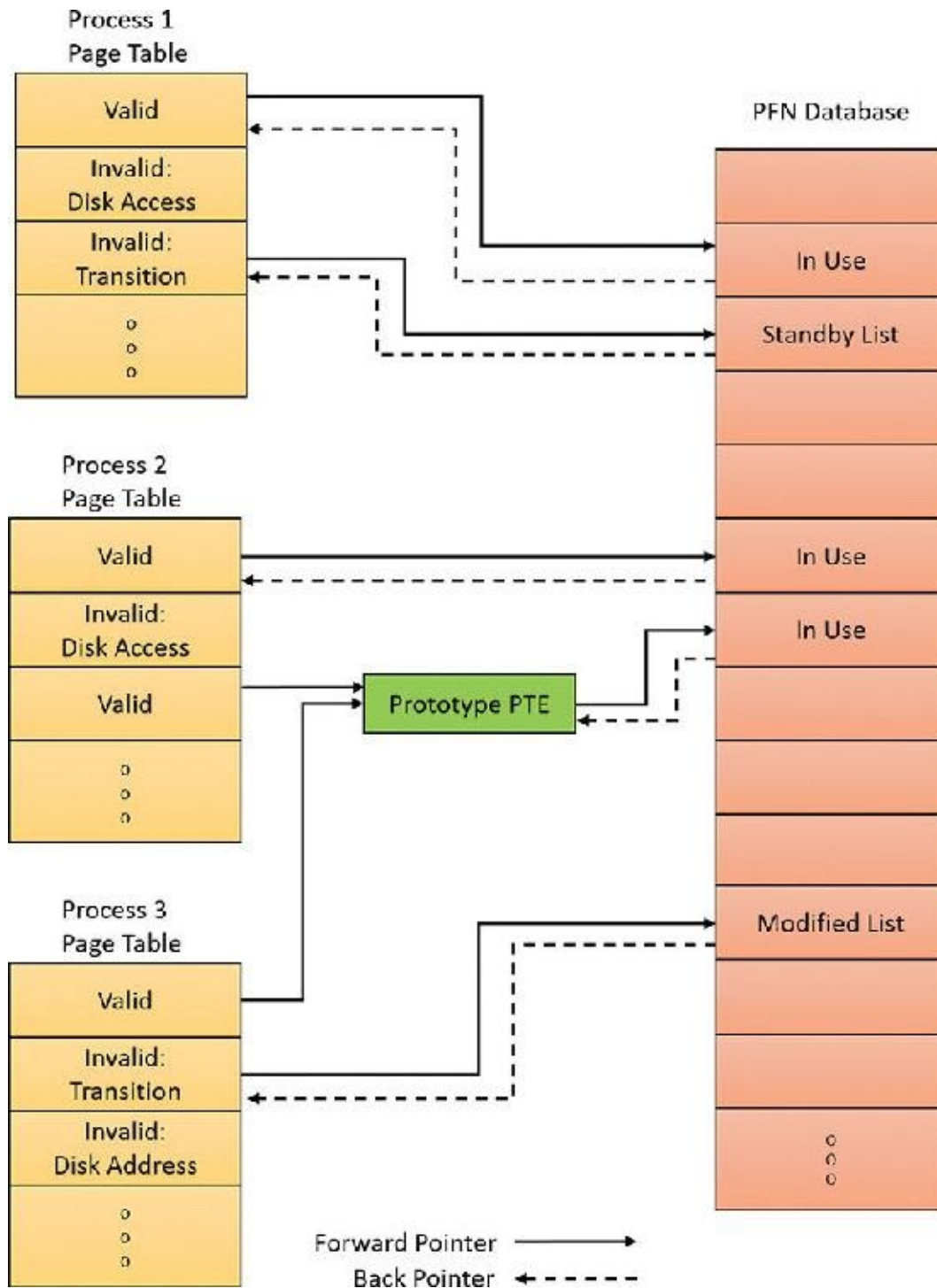


FIGURE 5-35 Page tables and the PFN database.

Of the page states listed in [Table 5-19](#), six are organized into linked lists so that the memory manager can quickly locate pages of a specific type. (Active/valid pages, transition pages, and overloaded “bad” pages aren’t in any system-wide page list.) Additionally, the standby state is associated with eight different lists ordered by priority. (We’ll talk about page priority later in this section.) [Figure 5-36](#) shows an example of how these entries are linked together.

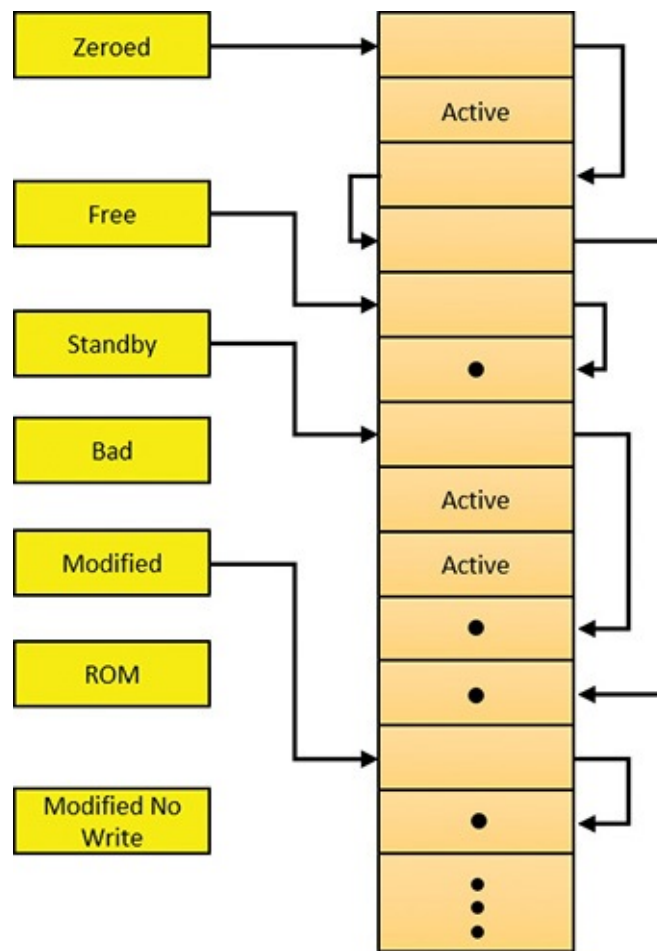


FIGURE 5-36 Page lists in the PFN database.

In the next section, you'll find out how these linked lists are used to satisfy page faults and how pages move to and from the various lists.

EXPERIMENT: Viewing the PFN database

You can use the MemInfo tool from the *Windows Internals* book's website to dump the size of the various paging lists by using the -S flag. The following is the output from this command:

[Click here to view code image](#)

```
C:\Tools>MemInfo.exe -s
MemInfo v3.00 - Show PFN database information
Copyright (C) 2007-2016 Alex Ionescu
www.alex-ionescu.com
```

```
Initializing PFN Database... Done
```

PFN Database List Statistics

Zeroed:	4867	(	19468	kb)
Free:	3076	(	12304	kb)
Standby:	4669104	(	18676416	kb)
Modified:	7845	(	31380	kb)
ModifiedNoWrite:	117	(	468	kb)
Active/Valid:	3677990	(	14711960	kb)
Transition:	5	(	20	kb)
Bad:	0	(	0	kb)
Unknown:	1277	(	5108	kb)
TOTAL:	8364281	(	33457124	kb)

Using the kernel debugger !memusage command, you can obtain similar information, although this will take considerably longer to execute.

Page list dynamics

[Figure 5-37](#) shows a state diagram for page frame transitions. For simplicity, the modified-no-write, bad and ROM lists aren't shown.

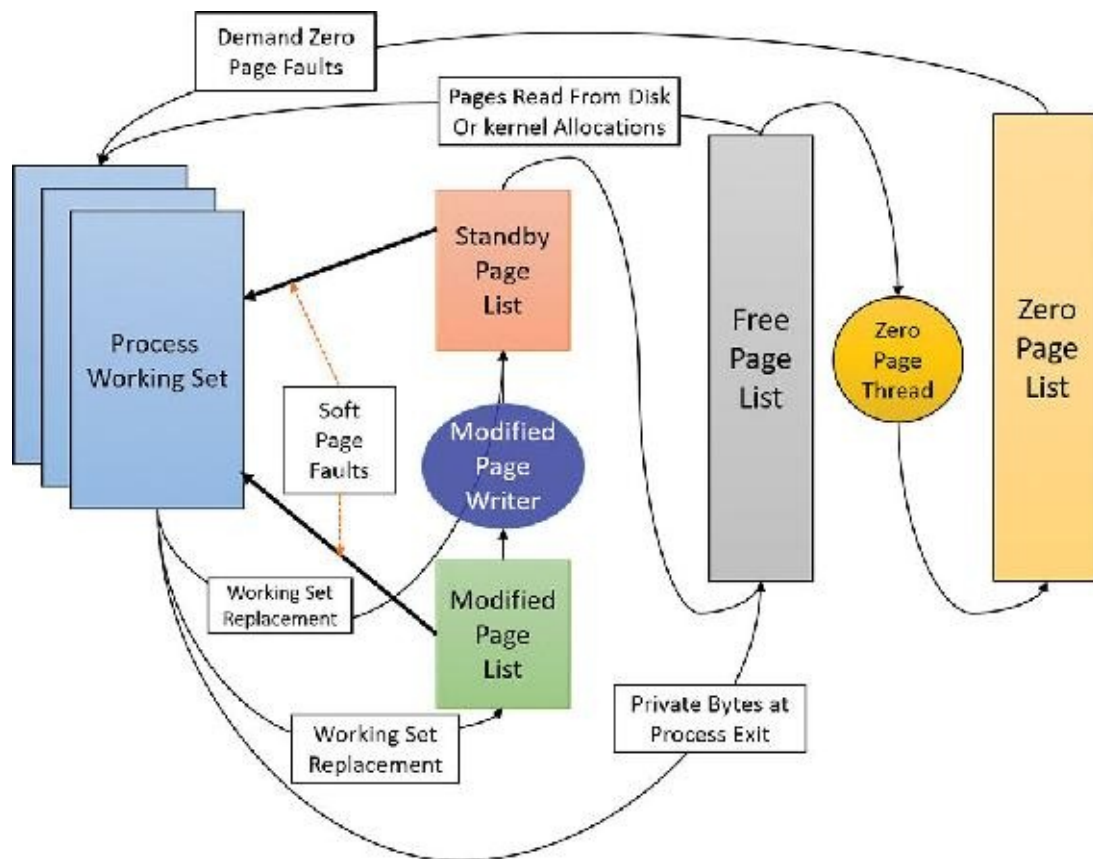


FIGURE 5-37 State diagram for physical pages.

Page frames move between the paging lists in the following ways:

- When the memory manager needs a zero-initialized page to service a demand-zero page fault (a reference to a page that is defined to be all zeroes or to a user-mode committed private page that has never been accessed), it first attempts to get one from the zero page list. If the list is empty, it gets one from the free page list and zeroes the page. If the free list is empty, it goes to the standby list and zeroes that page.

One reason zero-initialized pages are needed is to meet security requirements such as the Common Criteria (CC). Most CC profiles specify that user-mode processes be given initialized page frames to prevent them from reading a previous process's memory contents. Thus, the memory manager gives user-mode processes zeroed page frames unless the page is being read in from a backing store. In that case, the memory manager prefers to use non-zeroed page frames, initializing them with the data off the disk or remote storage. The zero page list is populated from the free list by the zero page thread system thread (thread 0 in the `System` process). The zero page thread waits on a gate object to signal it to go to work. When the free list has eight or more pages, this gate is signaled. However, the zero page thread will run only if at least one processor has no other threads running, because the zero page thread runs at priority 0 and the lowest priority that a user thread can be set to is 1.



Note

When memory needs to be zeroed as a result of a physical page allocation by a driver that calls `MmAllocatePagesForMdl(Ex)`, by a Windows application that calls `AllocateUserPhysicalPages` or `AllocateUserPhysicalPagesNuma`, or when an application allocates large pages, the memory manager zeroes the memory by using a higher-performing function called `MiZeroInParallel` that maps larger regions than the zero page thread, which only zeroes a page at a time. In addition, on multiprocessor systems, the memory manager creates additional system threads to perform the zeroing in parallel (and in a NUMA-optimized fashion on NUMA platforms).

- When the memory manager doesn't require a zero-initialized page, it goes first to the free list. If that's empty, it goes to the zeroed list. If the zeroed list is empty, it goes to the standby lists. Before the memory manager can use a page frame from the standby lists, it must first backtrack and remove the reference from the invalid PTE (or prototype PTE) that still points to the page frame. Because entries in the PFN database contain pointers back to the previous user's page table page (or to a page of prototype PTE pool for shared pages), the memory manager can quickly find the PTE and make the appropriate change.
- When a process must give up a page out of its working set either because it referenced a new page and its working set was full or the memory manager trimmed its working set, the page goes to the standby lists if the page was clean (not modified) or to the modified list if the page was modified while it was resident.
- When a process exits, all the private pages go to the free list. Also, when the last reference to a page-file-backed section is closed, and the section has no remaining mapped views, these pages also go to the free list.

EXPERIMENT: The free and zero page lists

You can observe the release of private pages at process exit with Process Explorer's System Information display. Begin by creating a process with numerous private pages in its working set. We did this in an earlier experiment with the TestLimit utility:

[Click here to view code image](#)

```
C:\Tools\Sysinternals>Testlimit.exe -d 1 -c 1500
```

```
Testlimit v5.24 - test Windows limits  
Copyright (C) 2012-2015 Mark Russinovich  
Sysinternals - www.sysinternals.com
```

```
Process ID: 13928
```

```
Leaking private bytes with touch 1 MB at a time...  
Leaked 1500 MB of private memory (1500 MB total leaked). Lasterror:  
0  
The operation completed successfully.
```

The `-d` option causes TestLimit to not only allocate the memory as private committed, but to touch it—that is, to access it. This causes physical memory to be allocated and assigned to the process to realize the area of private committed virtual memory. If there is sufficient available RAM on the system, the entire 1,500 MB should be in RAM for the process. The process will now wait until you cause it to exit or terminate (perhaps by pressing **Ctrl+C** in its command window). After you do, follow these steps:

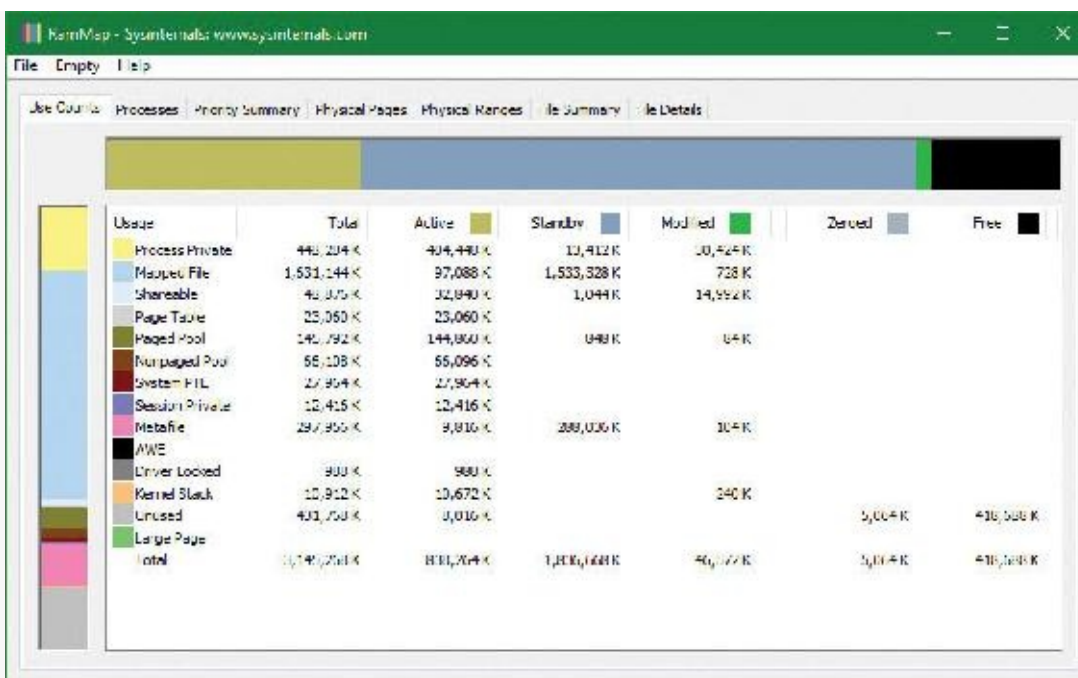
1. Open Process Explorer.
2. Open the **View** menu, choose **System Information**, and click the **Memory** tab.
3. Observe the size of the Free and Zeroed lists.
4. Terminate or exit the TestLimit process.

You *may* see the free page list briefly increase in size. We say “may” because the zero page thread is awakened as soon as there are only eight pages on the free list, and it acts very quickly. Process Explorer updates this display only once per second, and it is likely that most of the pages were already zeroed and moved to the zeroed page list before it happened to “catch” this state. If you can see the temporary increase in the free list, you will then see it drop to zero, and a corresponding increase will occur in the zeroed page list. If not, you will simply see the increase in the zeroed list.

EXPERIMENT: The modified and standby page lists

You can observe the movement of pages from process working set to the modified page list and then to the standby page list with the VMMap and RAMMap Sysinternals tools and the live kernel debugger. Follow these steps:

1. Open RAMMap and observe the state of the quiet system. This is an x86 system with 3 GB of RAM. The columns in this display represent the various page states shown in [Figure 5-37](#) (a few of the columns not important to this discussion have been narrowed for ease of reference).



- The system has about 420 MB of RAM free (sum of the free and zeroed page lists). About 580 MB is on the standby list (hence part of “available,” but likely containing data recently lost from processes or being used by SuperFetch). About 830 MB is “active,” being mapped directly to virtual addresses via valid page table entries.
- Each row further breaks down into page state by usage or origin (process private, mapped file, and so on). For example, at the moment, of the active 830 MB, about 400 MB is due to process private allocations.
- Now, as in the previous experiment, use the TestLimit utility to create a process with a large number of pages in its working set. Again, we will use the `-d` option to cause TestLimit to write to each page, but this time we will use it without a limit, so as to create as many private modified pages as possible:

[Click here to view code image](#)

```
C:\Tools\Sysinternals>Testlimit.exe -d
```

```
Testlimit v5.24 - test Windows limits
Copyright (C) 2012-2015 Mark Russinovich
Sysinternals - www.sysinternals.com
```

```
Process ID: 7548
```

```
Leaking private bytes with touch (MB)...
Leaked 1975 MB of private memory (1975 MB total leaked).
Lasterror: 8
```

- TestLimit has now created 1975 allocations of 1 MB each. In RAMMap, use the **File | Refresh** command to update the display (because of the cost of gathering its information, RAMMap does not update continuously).

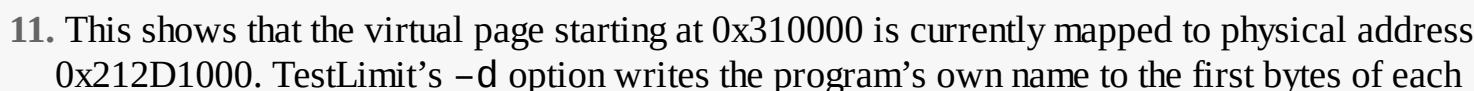
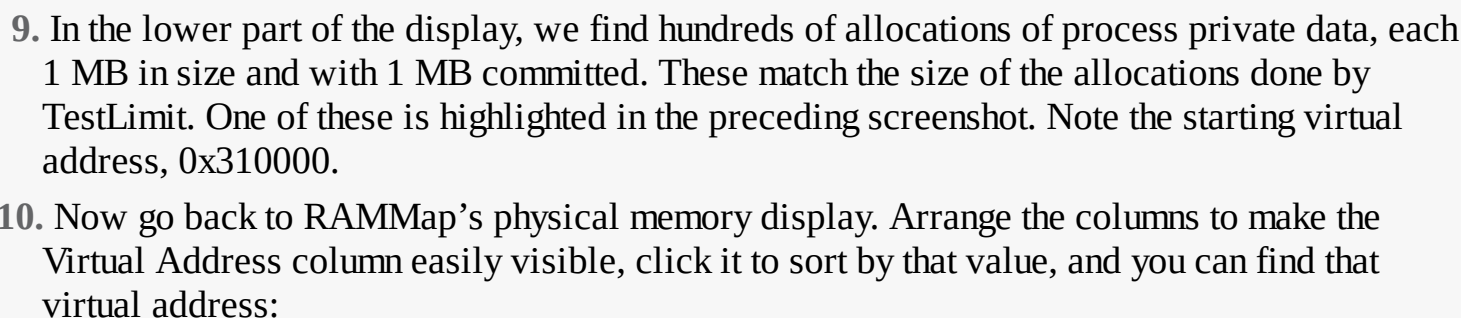
Image	Total	Active	Standby	Modified	Zeroed
Process Private	2,505,628 K	2,457,236 K	11,296 K	27,296 K	
Mapped File	243,416 K	116,836 K	135,820 K	760 K	
Shareable	45,248 K	41,256 K	48 K	5,044 K	
Page Table	27,656 K	27,956 K			
Paged Pool	71,500 K	71,300 K	60 K	96 K	
Nonpaged Pool	65,120 K	66,100 K			
System File	25,164 K	25,154 K			
Session Private	18,828 K	18,828 K			
Memfile	25,668 K	8,408 K	18,160 K	100 K	
AWF					
Driver Locked	500 K	900 K			
Kernel Stack	10,024 K	10,024 K			
Unused	100,392 K	17,244 K			38,856 K
Large Page					
Total	3,145,268 K	2,863,428 K	163,584 K	33,296 K	38,856 K

- You will see that over 2.8 GB are now active, of which 2.4 GB are in the Process Private row. This is due to the memory allocated and accessed by the TestLimit process. Note also that the standby, zeroed, and free lists are now much smaller. Most of the RAM allocated to TestLimit came from these lists.
- Next, in RAMMap, check the process's physical page allocations. Change to the **Physical Pages** tab, and set the filter at the bottom to the **Process** column and the value **Testlimit.exe**. This display shows all the physical pages that are part of the process working set.

Physical Address	List	Use	Priority	Virtual Address
0x2D000	Active	Process Private	5	0x144E2000
0x9D000	Active	Process Private	5	0x57F40000
0x9E000	Active	Process Private	5	0x72D00000
0x10F000	Active	Process Private	5	0x555B0000
0x130000	Active	Process Private	5	0x560A0000
0x1A5000	Active	Process Private	5	0x555F0000
0x191000	Active	Process Private	5	0x30570000
0x1C0000	Active	Process Private	5	0x2F5A0000
0x195000	Active	Process Private	5	0x44L25000
0x1C7000	Active	Process Private	5	0x735F0000
0x1L1000	Active	Process Private	5	0x57000000
0x1B0000	Active	Process Private	5	0x57F70000
0x1L3000	Active	Process Private	5	0x44000000
0x1B6000	Active	Process Private	5	0x507F0000
0x1L4000	Active	Process Private	5	0x30L00000
0x1B8000	Active	Process Private	5	0x5B410000
0x1L9000	Active	Process Private	5	0x305A0000
0x1BB000	Active	Process Private	5	0x7B1B0000
0x10K000	Active	Process Private	5	0x10000000

Filter: Process > Testlimit.exe (7518)

- We would like to identify a physical page involved in the allocation of virtual address space done by TestLimit's -d option. RAMMap does not give an indication about which virtual allocations are associated with RAMMap's VirtualAlloc calls. However, we can get a good hint of this through the VMMap tool. Using VMMap on the same process, we find the following:

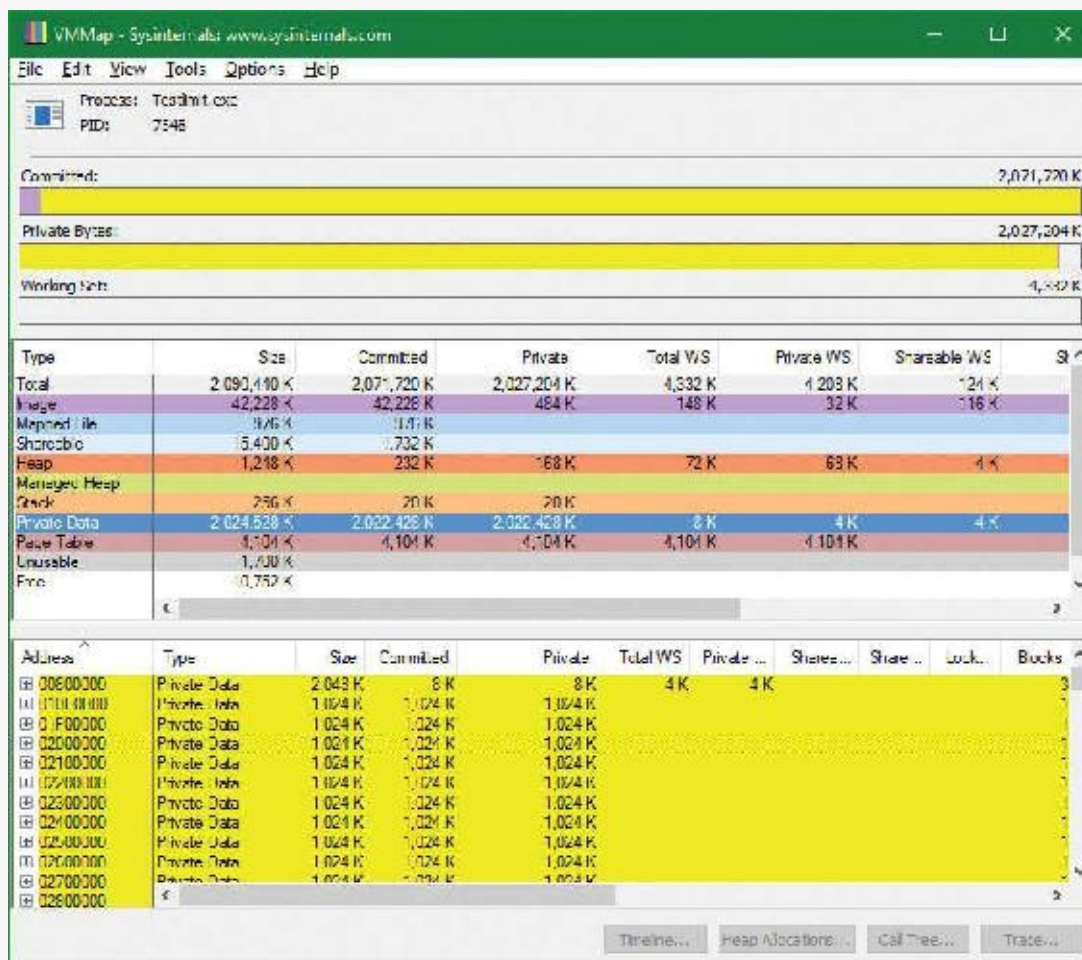


allocation. We can demonstrate this with the !dc (display characters using physical address) command in the local kernel debugger:

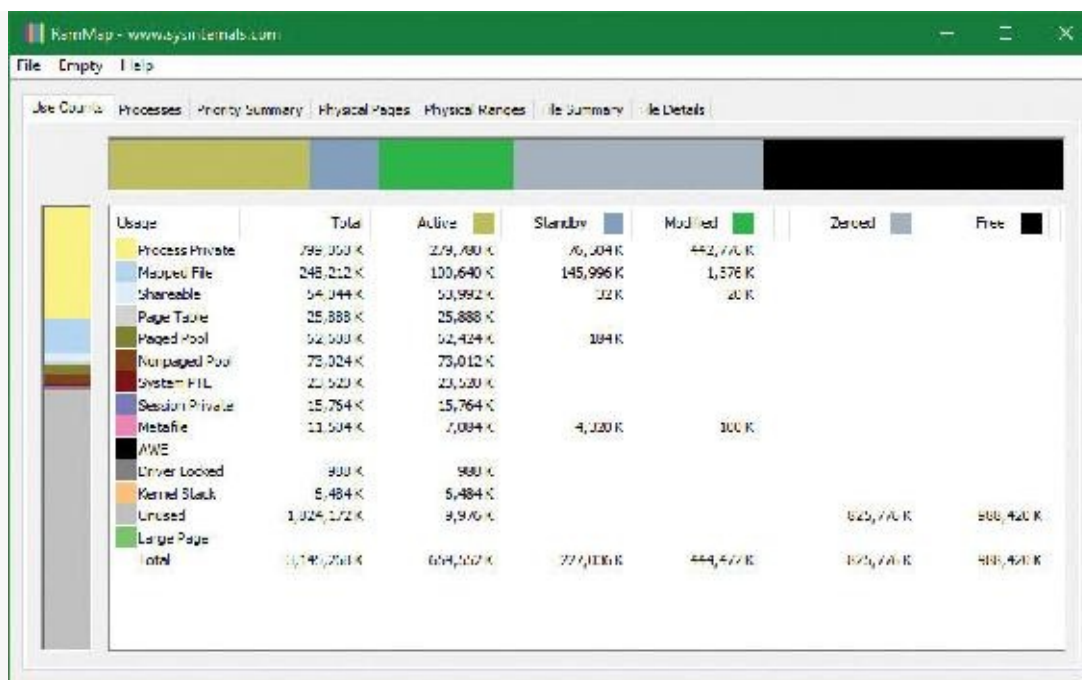
[Click here to view code image](#)

```
lkd> !dc 0x212d1000
#212d1000 74736554 696d694c 00000074 00000000 TestLimit.....
#212d1010 00000000 00000000 00000000 00000000 .....
...
```

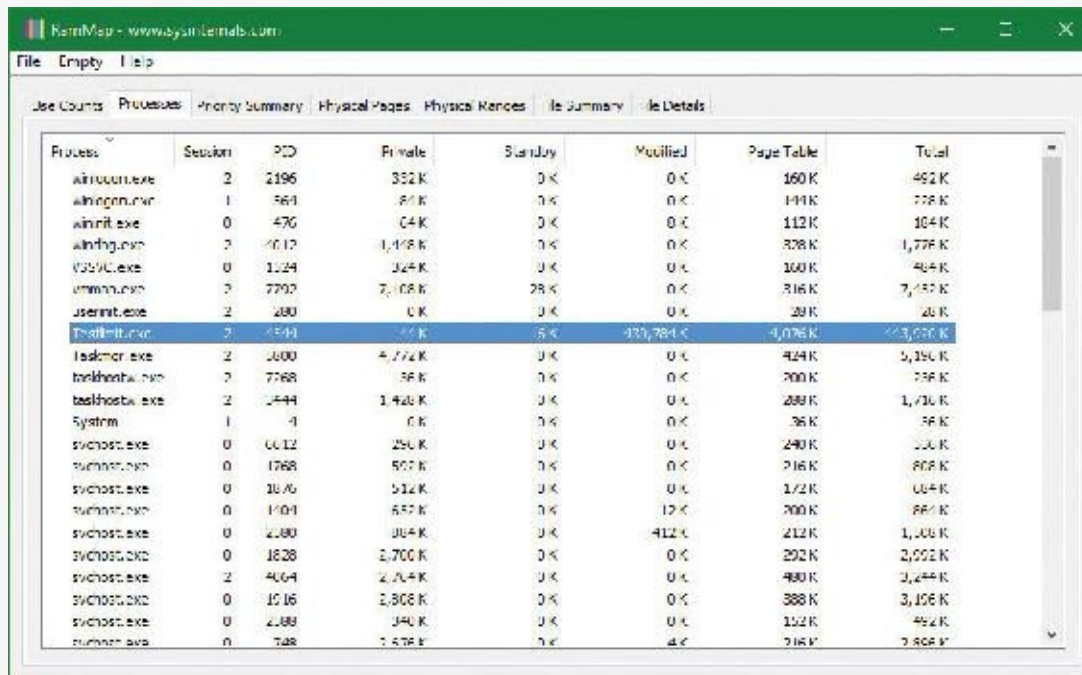
12. If you're not quick enough, this may fail—the page may be removed from the working set. For the final leg of the experiment, we will demonstrate that this data remains intact (for a while, anyway) after the process working set is reduced and this page is moved to the modified and then the standby page list.
13. In VMMap, having selected the TestLimit process, open the **View** menu and choose **Empty Working Set** to reduce the process's working set to the bare minimum. VMMap's display should now look like this:



14. Notice that the Working Set bar graph is practically empty. In the middle section, the process shows a total working set of only 4 KB, and almost all of it is in page tables. Now return to RAMMap and refresh it. On the Use Counts tab, you will find that active pages have been reduced tremendously, with a large number of pages on the modified list and some on the standby list:



15. RAMMap's Processes tab confirms that the TestLimit process contributed most of those pages to those lists:



Page priority

Every physical page in the system has a page priority value assigned to it by the memory manager. The page priority is a number in the range 0 to 7. Its main purpose is to determine the order in which pages are consumed from the standby list. The memory manager divides the standby list into eight sublists that each stores pages of a particular priority. When the memory manager wants to take a page from the standby list, it takes pages from low-priority lists first.

Each thread and process in the system is also assigned a page priority. A page's priority usually reflects the page priority of the thread that first causes its allocation. (If the page is shared, it reflects the highest page priority among the sharing threads.) A thread inherits its page-priority value from the process to which it belongs. The memory manager uses low priorities for pages it reads from disk speculatively when anticipating a process's memory accesses.

By default, processes have a page-priority value of 5, but the `SetProcessInformation` and `SetThreadInformation` user-mode functions allow applications to change process and thread page-priority values. These functions call the native `NtSetInformationProcess` and `NtSetInformationThread` functions. You can look at the memory priority of a thread with Process Explorer (per-page priority can be displayed by looking at the PFN entries, as you'll see in an experiment later in the chapter). [Figure 5-38](#) shows Process Explorer's Threads tab displaying information about Winlogon's main thread. Although the thread priority itself is high, the memory priority is still the standard 5.

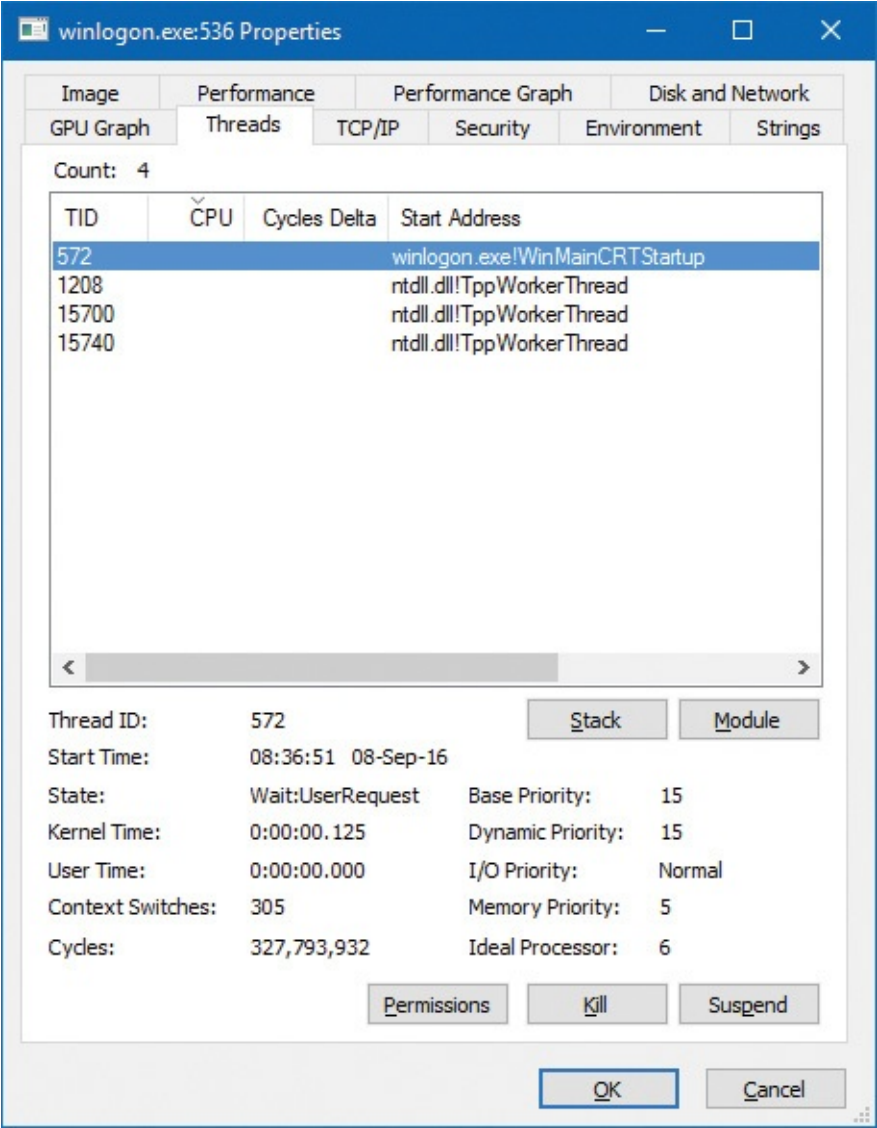
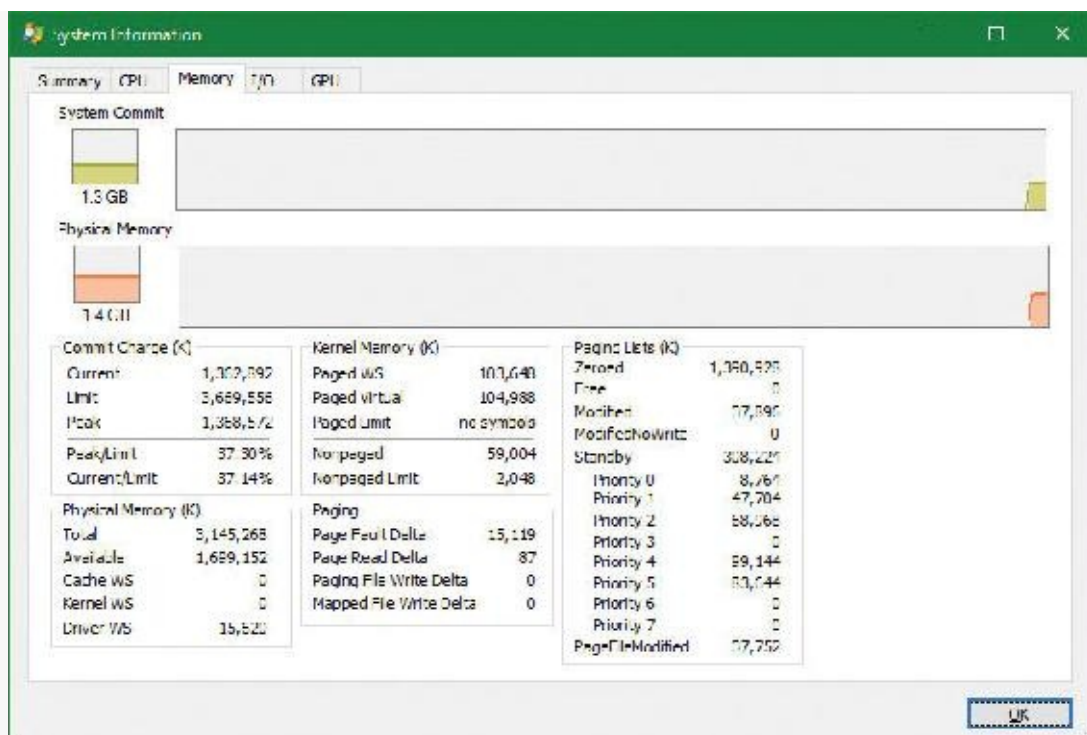


FIGURE 5-38 Process Explorer's Threads tab.

The real power of memory priorities is realized only when the relative priorities of pages are understood at a high level, which is the role of SuperFetch, covered at the end of this chapter.

EXPERIMENT: Viewing the prioritized standby lists

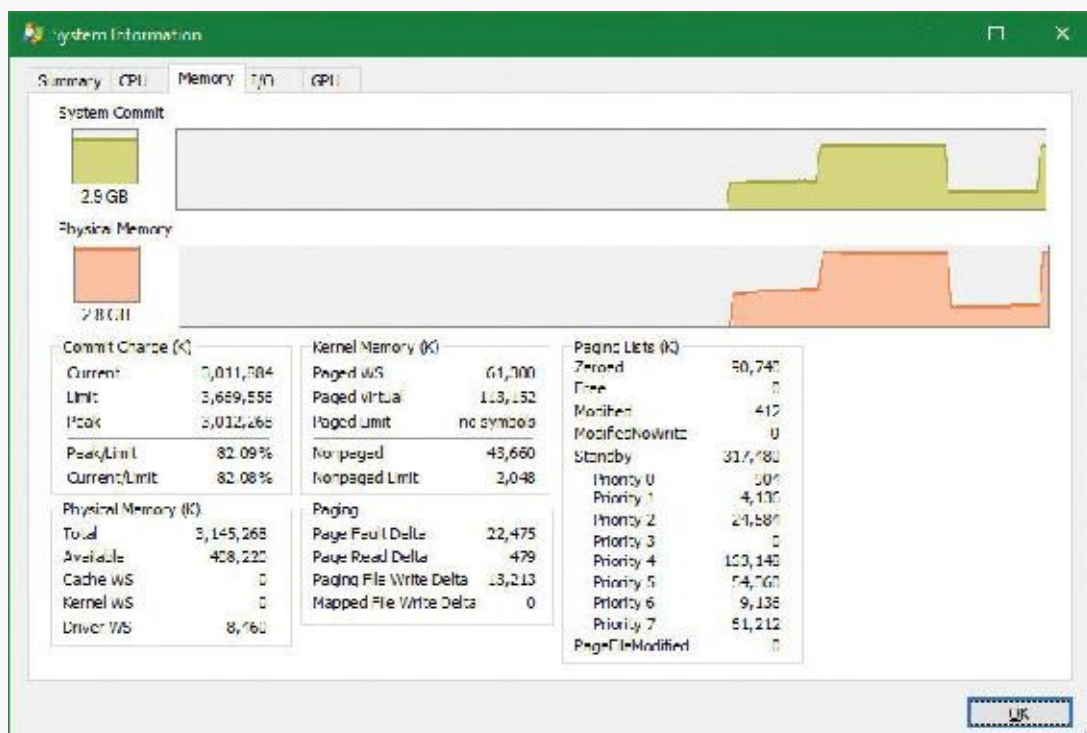
You can use Process Explorer to look at the size of each standby paging list by opening the System Information dialog box and selecting the **Memory** tab:



On the recently started x86 system used in this experiment, there is about 9 MB of data cached at priority 0, about 47 MB at priority 1, about 68 MB at priority 2, etc. The following shows what happens when we use the TestLimit tool from Sysinternals to commit and touch as much memory as possible:

[Click here to view code image](#)

C:\Tools\Sysinternals>Testlimit.exe -d



Note how the lower-priority standby page lists were used first (shown by the repurposed count) and are now much smaller, while the higher lists still contain valuable cached data.

Modified page writer and mapped page writer

The memory manager employs two system threads to write pages back to disk and move those pages back to the standby lists (based on their priority). One system thread writes out modified pages (`MiModifiedPageWriter`) to the paging file, and a second one writes modified pages to mapped files (`MiMappedPageWriter`). Two threads are required to avoid creating a deadlock. This would occur if the writing of mapped file pages caused a page fault that in turn required a free page when no free pages were available, thus requiring the modified page writer to create more free pages. By having the modified page writer perform mapped file paging I/Os from a second system thread, that thread can wait without blocking regular page file I/O.

Both threads run at priority 18, and after initialization they wait for separate event objects to trigger their operation. The mapped page writer waits on 18 event objects:

- An exit event, signaling the thread to exit (not relevant to this discussion).
- The mapped writer event, stored in the global variable `MiSystemPartition.Modwriter.MappedPageWriterEvent` (`MmMappedPageWriterEvent` on Windows 8.x and Server 2012/R2). This event can be signaled in the following instances:
 - During a page list operation (`MiInsertPageInList`); this routine inserts a page into one of the lists (standby, modified, etc.) based on its input arguments. The routine signals this event if the number of file-system-destined pages on the modified page list has reached more than 16 and the number of available pages has fallen below 1024.
 - In an attempt to obtain free pages (`MiObtainFreePages`).
 - By the memory manager's working set manager (`MmWorkingSetManager`), which runs as part of the kernel's balance set manager (once every second). The working set manager signals this event if the number of file-system-destined pages on the modified page list has reached more than 800.
 - Upon a request to flush all modified pages (`MmFlushAllPages`).
 - Upon a request to flush all file-system-destined modified pages (`MmFlushAllFilesystemPages`). Note that in most cases, writing modified mapped pages to their backing store files does not occur if the number of mapped pages on the modified page list is less than the maximum write cluster size, which is 16 pages. This check is not made in `MmFlushAllFilesystemPages` or `MmFlushAllPages`.
- An array of 16 events associated with 16 mapped page lists, stored in `MiSystemPartition.PageLists.MappedPageListHeadEvent` (`MiMappedPageListHeadEvent` on Windows 8.x and Server 2012/R2). Each time a mapped page is dirtied, it is inserted into one of these 16 mapped page lists based on a bucket number, stored in `MiSystemPartition.WorkingSetControl->CurrentMappedPageBucket` (`MiCurrentMappedPageBucket` on Windows 8.x and Server 2012/R2). This bucket number is updated by the working set manager whenever the system considers that mapped pages have gotten old enough, which is currently 100 seconds (stored in the `WriteGapCounter` variable in the same structure [`MiWriteGapCounter` on Windows 8.x and Server 2012/R2] and incremented whenever the working set manager runs). The reason for these additional events is to reduce data loss in the case of a system crash or power failure by eventually writing out modified mapped pages even if the modified list hasn't reached its threshold

of 800 pages.

The modified page writer waits on two events: the first is an Exit event, and the second stored in `MiSystemPartition.Modwriter.ModifiedPageWriterEvent` (on Windows 8.x and Server 2012/R2 waits on a kernel gate stored in `MmModifiedPageWriterGate`), which can be signaled in the following scenarios:

- A request to flush all pages has been received.
- The number of available pages—stored in `MiSystemPartition.Vp.AvailablePages` (`MmAvailablePages` on Windows 8.x and Server 2012/R2)—drops below 128 pages.
- The total size of the zeroed and free page lists drops below 20,000 pages, and the number of modified pages destined for the paging file is greater than the smaller of one-sixteenth of the available pages or 64 MB (16,384 pages).
- When a working set is being trimmed to accommodate additional pages, if the number of pages available is less than 15,000.
- During a page list operation (`MiInsertPageInList`). This routine signals this event if the number of page-file-destined pages on the modified page list has reached more than 16 pages and the number of available pages has fallen below 1,024.

Additionally, the modified page writer waits on two other events after the preceding event is signaled. One is used to indicate rescanning of the paging file is required (for example, a new page file may have been created), stored in `MiSystemPartition.Modwriter.RescanPageFilesEvent` (`MiRescanPageFilesEvent` on Windows 8.x and Server 2012/R2). The second event is internal to the paging file header (`MiSystemPartition.Modwriter.PagingFileHeader` [`MmPagingFileHeader` on Windows 8.x and Server 2012/R2]), which allows the system to manually request flushing out data to the paging file when needed.

When invoked, the mapped page writer attempts to write as many pages as possible to disk with a single I/O request. It accomplishes this by examining the original PTE field of the PFN database elements for pages on the modified page list to locate pages in contiguous locations on the disk. Once a list is created, the pages are removed from the modified list, an I/O request is issued, and, at successful completion of the I/O request, the pages are placed at the tail of the standby list corresponding to their priority.

Pages that are in the process of being written can be referenced by another thread. When this happens, the reference count and the share count in the PFN entry that represents the physical page are incremented to indicate that another process is using the page. When the I/O operation completes, the modified page writer notices that the reference count is no longer 0 and doesn't place the page on any standby list.

PFN data structures

Although PFN database entries are of fixed length, they can be in several different states, depending on the state of the page. Thus, individual fields have different meanings depending on the state. [Figure 5-39](#) shows the formats of PFN entries for different states.

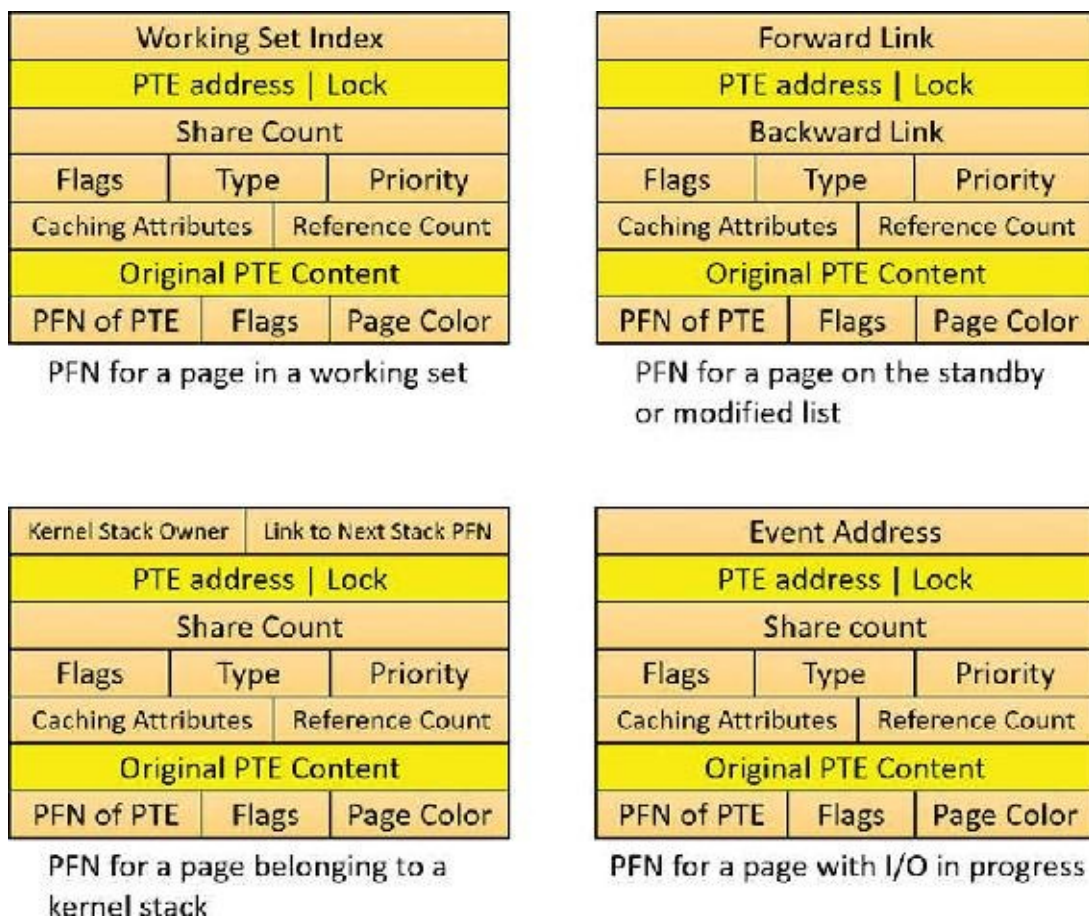


FIGURE 5-39 States of PFN database entries (specific layouts are conceptual).

Several fields are the same for several PFN types, but others are specific to a given type of PFN. The following fields appear in more than one PFN type:

- **PTE address** This is the virtual address of the PTE that points to this page. Also, since PTE addresses will always be aligned on a 4-byte boundary (8 bytes on 64-bit systems), the two low-order bits are used as a locking mechanism to serialize access to the PFN entry.
- **Reference count** This is the number of references to this page. The reference count is incremented when a page is first added to a working set and/or when the page is locked in memory for I/O (for example, by a device driver). The reference count is decremented when the share count becomes 0 or when pages are unlocked from memory. When the share count becomes 0, the page is no longer owned by a working set. Then, if the reference count is also zero, the PFN database entry that describes the page is updated to add the page to the free, standby, or modified list.
- **Type** This is the type of page represented by this PFN. (Types include active/valid, standby, modified, modified-no-write, free, zeroed, bad, and transition.)
- **Flags** The information contained in the flags field is shown in [Table 5-20](#).
- **Priority** This is the priority associated with this PFN, which will determine on which standby list it will be placed.
- **Original PTE contents** All PFN database entries contain the original contents of the PTE that pointed to the page (which could be a prototype PTE). Saving the contents of the PTE allows it to be restored when the physical page is no longer resident. PFN entries for AWE allocations are exceptions; they store the AWE reference count in this field instead.
- **PFN of PTE** This is the physical page number of the page table page containing the PTE that points to this page.

- **Color** Besides being linked together on a list, PFN database entries use an additional field to link physical pages by “color,” which is the page’s NUMA node number.
- **Flags** A second flags field is used to encode additional information on the PTE. These flags are described in [Table 5-21](#).

Flag	Meaning
Write in progress	This indicates that a page write operation is in progress. The first DWORD contains the address of the event object that will be signaled when the I/O is complete.
Modified state	This indicates whether the page was modified. (If the page was modified, its contents must be saved to disk before removing it from memory.)
Read in progress	This indicates that an in-page operation is in progress for the page. The first DWORD contains the address of the event object that will be signaled when the I/O is complete.
ROM	This indicates that this page comes from the computer's firmware or another piece of read-only memory such as a device register.
In-page error	This indicates that an I/O error occurred during the in-page operation on this page. (In this case, the first field in the PFN contains the error code.)
Kernel stack	This indicates that this page is being used to contain a kernel stack. In this case, the PFN entry contains the owner of the stack and the next stack PFN for this thread.
Removal requested	This indicates that the page is the target of a remove (due to ECC/scrubbing or hot memory removal).
Parity error	This indicates that the physical page contains parity or error correction control errors.

TABLE 5-20 Flags within PFN database entries

Flag	Meaning
PFN image verified	This indicates that the code signature for this PFN (contained in the cryptographic signature catalog for the image being backed by this PFN) has been verified.
AWE allocation	This indicates that this PFN backs an AWE allocation.
Prototype PTE	This indicates that the PTE referenced by the PFN entry is a prototype PTE. For example, this page is shareable.

TABLE 5-21 Secondary flags within PFN database entries

The remaining fields are specific to the type of PFN. For example, the first PFN in [Figure 5-39](#) represents a page that is active and part of a working set. The share count field represents the number of PTEs that refer to this page. (Pages marked read-only, copy-on-write, or shared read/write can be shared by multiple processes.) For page table pages, this field is the number of valid and transition PTEs in the page table. As long as the share count is greater than 0, the page isn’t eligible for removal from memory.

The working set index field is an index into the process working set list (or the system or session working set list, or zero if not in any working set) where the virtual address that maps this physical page resides. If the page is a private page, the working set index field refers directly to the entry in the working set list because the page is mapped only at a single virtual address. In the case of a shared page, the working set index is a hint that is guaranteed to be correct only for the first process that made the page valid. (Other processes will try to use the same index where possible.) The process that initially sets this field is guaranteed to refer to the proper index and doesn’t need to add a working set list hash entry referenced by the virtual address into its working set hash tree. This guarantee reduces the size of the working set hash tree and makes searches faster for these entries.

The second PFN in [Figure 5-39](#) is for a page on either the standby or the modified list. In this case, the forward and backward link fields link the elements of the list together within the list. This linking allows pages to be easily manipulated to satisfy page faults. When a page is on one of the lists, the share count is by definition 0 (because no working set is using the page) and therefore can be overlaid with the backward link. The reference count is also 0 if the page is on one of the lists. If it is non-zero (because an I/O could be in progress for this page—for example, when the page is being written to disk), it is first

removed from the list.

The third PFN in [Figure 5-39](#) is for a page that belongs to a kernel stack. As mentioned earlier, kernel stacks in Windows are dynamically allocated, expanded, and freed whenever a callback to user mode is performed and/or returns, or when a driver performs a callback and requests stack expansion. For these PFNs, the memory manager must keep track of the thread actually associated with the kernel stack, or if it is free it keeps a link to the next free look-aside stack.

The fourth PFN in [Figure 5-39](#) is for a page that has an I/O in progress (for example, a page read). While the I/O is in progress, the first field points to an event object that will be signaled when the I/O completes. If an in-page error occurs, this field contains the Windows error status code representing the I/O error. This PFN type is used to resolve collided page faults.

In addition to the PFN database, the system variables in [Table 5-22](#) describe the overall state of physical memory.

Variable (Windows 10 and Server 2016)	Variable (Windows 8.x and Server 2012/R2)	Description
MmSystemPartition.Vp.NumberOfPhysicalPages	MmNumberOfPhysicalPages	This is the total number of physical pages available on the system.
MmSystemPartition.Vp.AvailablePages	MmAvailablePages	This is the total number of available pages on the system—the sum of the pages on the zeroed, free, and standby lists.
MmSystemPartition.Vp.ResidentAvailablePages	MmResidentAvailablePages	This is the total number of physical pages that would be available if every process was trimmed to its minimum working set size and all modified pages were flushed to disk.

TABLE 5-22 System variables that describe physical memory

EXPERIMENT: Viewing PFN entries

You can examine individual PFN entries with the kernel debugger `!pfn` command. You need to supply the PFN as an argument. (For example, `!pfn 0` shows the first entry, `!pfn 1` shows the second, and so on.) In the following example, the PTE for virtual address `0xD20000` is displayed, followed by the PFN that contains the page directory, and then the actual page:

[Click here to view code image](#)

```
lkd> !pte d20000
                                VA 00d20000
PDE at C0600030                PTE at C0006900
contains 000000003E989867      contains 8000000093257847
pfn 3e989    ---DA--UWEV      pfn 93257    ---D---UW-V

lkd> !pfn 3e989
    PFN 0003E989 at address 868D8AFC
    flink          00000071  blink / share count 00000144  pteaddress
C0600030
    reference count 0001    Cached        color 0    Priority 5
    restore pte 00000080    containing page 0696B3    Active      M
    Modified

lkd> !pfn 93257
    PFN 00093257 at address 87218184
    flink          000003F9  blink / share count 00000001  pteaddress
C0006900
    reference count 0001    Cached        color 0    Priority 5
    restore pte 00000080    containing page 03E989    Active      M
    Modified
```

You can also use the MemInfo tool to obtain information about a PFN. MemInfo can sometimes give you more information than the debugger's output, and it does not require being booted into debugging mode. Here's MemInfo's output for those same two PFNs:

[Click here to view code image](#)

```
C:\Tools>MemInfo.exe -p 3e989

0x3E989000 Active      Page
Table          5      N/A          0xC0006000 0x8E499480

C:\Tools>MemInfo.exe -p 93257

0x93257000 Active      Process
Private 5    windbg.exe    0x00D20000 N/A
```

From left to right, the information shown includes the physical address, type, page priority, process name, virtual address, and potential extra information. MemInfo correctly recognized that the first PFN was a page table and that the second PFN belongs to WinDbg, which was the active process when the `!pte d20000` command was used in the debugger.

Page file reservation

We have already seen some mechanisms used by the memory manager to attempt to reduce physical memory consumption and thus reduce accessing page files. Using the standby and modified list is one such mechanism, and so is memory compression (see the “[Memory compression](#)” section later in this chapter). Another optimization the memory uses is directly related to accessing page files themselves.

Rotational hard disks have a moving head that travels to a target sector before the disk can actually read or write. This seek time is relatively expensive (in the order of milliseconds) and so the total disk activity is the seek time added to the actual read/write time. If the amount of data accessed contiguously from the seek position is large, then the seek time may be negligible. But if the head must seek a lot while accessing scattered data on the disk, the aggregated seek time becomes the main issue.

When the Session Manager (Smss.exe) creates a page file, it queries the disk of the file’s partition to find whether it’s a rotational disk or a solid-state drive (SSD). If it’s rotational, it activates a mechanism called *page file reservations* that tries to keep contiguous pages in physical memory contiguous in the page file as well. If the disk is an SSD (or a hybrid, which for the sake of page file reservation is treated as SSD), then page file reservation adds no real value (since there is no moving head), and the feature is not utilized for this particular page file.

Page file reservation is handled in three locations within the memory manager: working set manager, modified page writer, and page fault handler. The working set manager performs working set trimming by calling the `MiFreeWsleList` routine. The routine takes a list of pages from a working set and for each page it decrements its share count. If it reaches zero, the page can be placed on the modified list, changing the relevant PTE into a transition PTE. The old valid PTE is saved in the PFN.

The invalid PTE has two bits related to page file reservation: page file reserved and page file allocated (refer to [Figure 5-24](#)). When a physical page is needed and is taken from one of the “free” page lists (free, zero or standby) to become an active (valid) page, an invalid PTE is saved into the Original PTE field of the PFN. This field is the key for tracking page file reservation.

The `MiCheckReservePageFileSpace` routine tries to create page file reservation cluster starting from a specified page. It checks if page file reservation is disabled for the target page file and if there is already page file reservation for this page (based on the original PTE), and if any of these conditions is true, the function aborts further processing for this page. The routine also checks if the page type is of user pages, and if not, it bails out. Page file reservation is not attempted for other page types (such as paged pool), because it was not found to be particularly beneficial (because of unpredictable usage patterns, for example), which led to small clusters. Finally, `MiCheckReservePageFileSpace` calls `MiReservePageFileSpace` to do the actual work.

The search for page file reservation starts backward from the initial PTE. The goal is to locate eligible consecutive pages where reservation is possible. If the PTE that maps the neighboring page represents a decommitted page, a non-paged pool page, or if it’s already reserved, then the page cannot be used; the current page will become the lower limit of the reservation cluster. Otherwise, the search continues backward. Then the search starts from the initial page forward, trying to gather as many eligible pages as possible. The cluster size must be at least 16 pages for the reservation to take place (the maximum cluster size is 512 pages). [Figure 5-40](#) shows an example of a cluster bound by invalid page on one hand and an existing cluster on the other (note that it can span page tables within the same page directory).

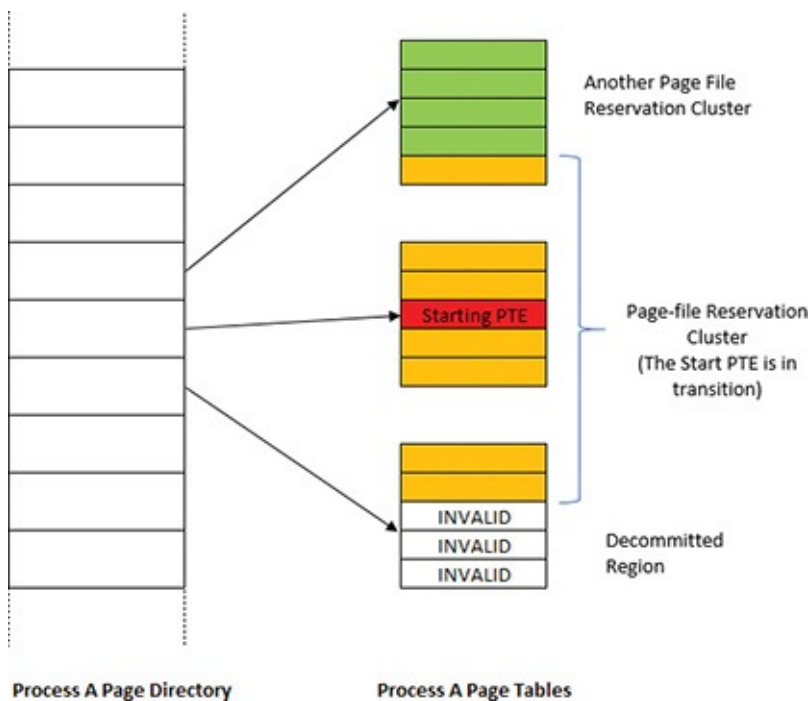


FIGURE 5-40 Page file reservation cluster.

Once the page cluster is computed, free page file space must be located to be reserved for this cluster of pages. Page file allocations are managed by a bitmap (where each set bit indicates a used page in the file). For page file reservation, a second bitmap is used that indicates pages that have been reserved (but not necessarily written to yet—this is the job of the page file allocation bitmap). Once there is page file space that is not reserved and not allocated (based on these bitmaps), the relevant bits are set in the reservation bitmap only. It is the job of the modified page writer to set these bits in the allocation bitmap when it writes the contents of the pages to disk. If not enough page file space could be found for the required cluster size, page file expansion is attempted, and if that already happened (or the maximum page file size is the expanded size), then the cluster size is reduced to fit in the reservation size that was located.



Note

The clustered pages (except the original starting PTE) are not linked to any of the physical page lists. The reservation information is placed in the Original PTE of the PFN.

The modified page writer needs to handle writing pages that have reservations as a special case. It uses all the gathered information described previously to build an MDL that contains the correct PFNs for the cluster that is used as part of writing to the page file. Building the cluster includes finding contiguous pages that can span reservation clusters. If there are “holes” between clusters, a dummy page is added in between (a page that contains all bytes with 0xFF value). If the dummy page count is above 32, the cluster is broken. This “walk” is done going forward and then backward to build the final cluster to write. [Figure 5-41](#) shows an example of the state of pages after such a cluster has been built by the modified page writer.

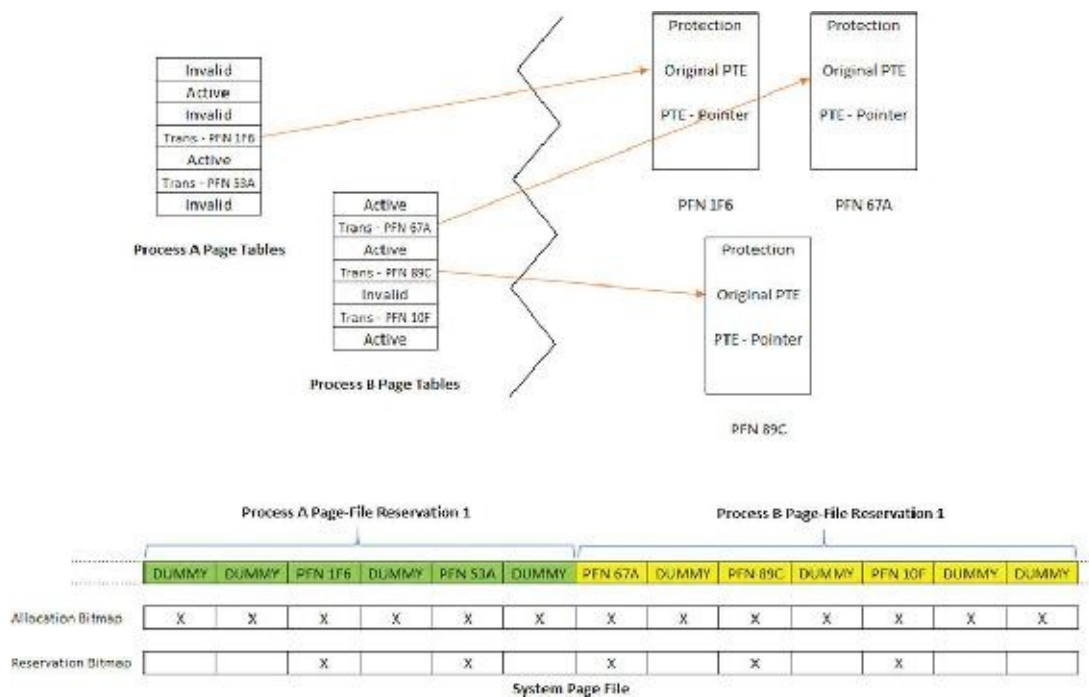


FIGURE 5-41 Cluster built before writing.

Finally, the page fault handler uses the built information from the reservation bitmap and the PTEs to determine the start and end points of clusters, to efficiently load back needed pages with minimal seeking of the mechanical disk head.

Physical memory limits

Now that you've learned how Windows keeps track of physical memory, we'll describe how much of it Windows can actually support. Because most systems access more code and data than can fit in physical memory as they run, physical memory is essentially a window into the code and data used over time. The amount of memory can therefore affect performance because when data or code that a process or the operating system needs is not present, the memory manager must bring it in from disk or remote storage.

Besides affecting performance, the amount of physical memory affects other resource limits. For example, the amount of non-paged pool is backed by physical memory, thus obviously constrained by physical memory. Physical memory also contributes to the system virtual memory limit, which is the sum of roughly the size of physical memory plus the current configured size of all paging files. Physical memory also can indirectly limit the maximum number of processes.

Windows support for physical memory is dictated by hardware limitations, licensing, operating system data structures, and driver compatibility. The following URL shows the memory limits of the various Windows editions: <https://msdn.microsoft.com/en-us/library/windows/desktop/aa366778.aspx>. Table 5-23 summarizes the limits in Windows 8 and higher versions.

Operating System Version/Edition	32-Bit Maximum	64-Bit Maximum
Windows 8.x Professional and Enterprise	4 GB	512
Windows 8.x (all other editions)	4 GB	128 GB
Windows Server 2012/R2 Standard and Datacenter	N/A	4 TB
Windows Server 2012/R2 Essentials	N/A	64 GB
Windows Server 2012/R2 Foundation	N/A	32 GB
Windows Storage Server 2012 Workgroup	N/A	32 GB
Windows Storage Server 2012 Standard Hyper-V Server 2012	N/A	4 TB
Windows 10 Home	4 GB	128 GB
Windows 10 Pro, Education and Enterprise	4 GB	2 TB
Windows Server 2016 Standard and Datacenter	N/A	24 TB

TABLE 5-23 Limitations on physical memory support in Windows

At the time of this writing, the maximum supported physical memory is 4 TB on some Server 2012/R2 editions and 24 TB on Server 2016 editions. The limitations don't come from any implementation or hardware limitation, but because Microsoft will support only configurations it can test. As of this writing, these were the largest tested and supported memory configurations.

Windows client memory limits

64-bit Windows client editions support different amounts of memory as a differentiating feature, with the low end being 4 GB increasing to 2 TB for Enterprise and Professional editions. All 32-bit Windows client editions, however, support a maximum of 4 GB of physical memory, which is the highest physical address accessible with the standard x86 memory management mode.

Although client SKUs support PAE addressing modes on x86 systems in order to provide hardware no-execute protection (which would also enable access to more than 4 GB of physical memory), testing revealed that systems would crash, hang, or become unbootable because some device drivers, commonly those for video and audio devices found typically on clients but not servers, were not programmed to expect physical addresses larger than 4 GB. As a result, the drivers truncated such addresses, resulting in memory corruptions and corruption side effects. Server systems commonly have more generic devices, with simpler and more stable drivers, and therefore had not generally revealed these problems. The problematic client driver ecosystem led to the decision for client editions to ignore physical memory that resides above 4 GB, even though they can theoretically address it. Driver developers are encouraged to test their systems with the `noLowmem` BCD option, which will force the kernel to use physical addresses above 4 GB only if sufficient memory exists on the system to allow it. This will immediately lead to the detection of such issues in faulty drivers.

Although 4 GB is the licensed limit for 32-bit client editions, the effective limit is actually lower and depends on the system's chipset and connected devices. This is because the physical address map includes not only RAM but device memory, and x86 and x64 systems typically map all device memory below the 4 GB address boundary to remain compatible with 32-bit operating systems that don't know how to handle addresses larger than 4 GB. Newer chipsets do support PAE-based device remapping, but client editions of Windows do not support this feature for the driver compatibility problems explained earlier. (Otherwise, drivers would receive 64-bit pointers to their device memory.)

If a system has 4 GB of RAM and devices such as video, audio, and network adapters that implement windows into their device memory that sum to 500 MB, then 500 MB of the 4 GB of RAM will reside

above the 4 GB address boundary, as shown in [Figure 5-42](#).

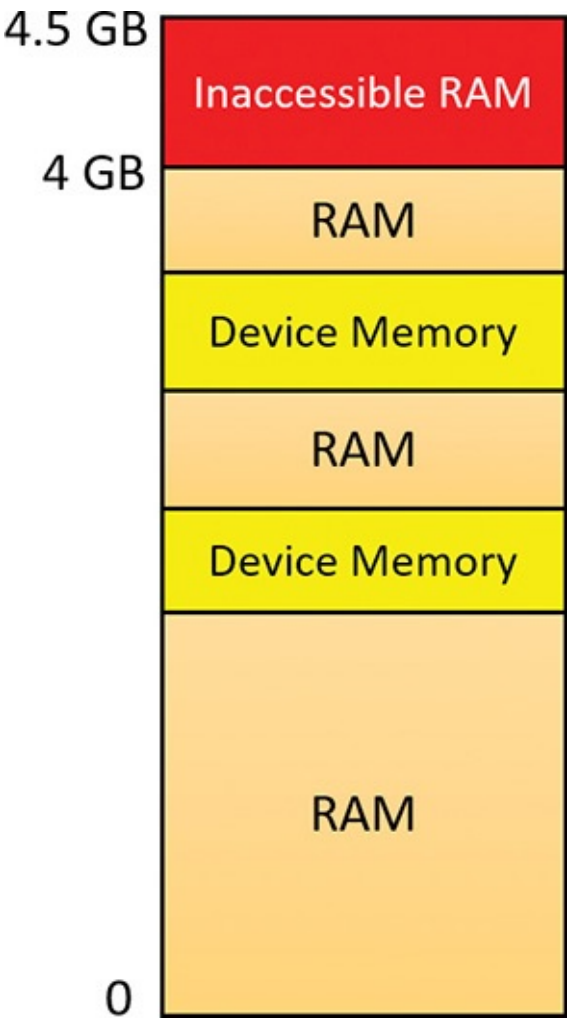


FIGURE 5-42 Physical memory layout on a 4 GB system.

The result is that if you have a system with 3 GB or more of memory and you are running a 32-bit Windows client, you may not get the benefit of all the RAM. You can see how much RAM Windows has detected as being installed in the System Properties dialog box, but to see how much memory is *really* available to Windows, you need to look at Task Manager’s Performance page or the Msinfo32 utility. For example, on a Hyper-V virtual machine configured with 4 GB of RAM, with 32-bit Windows 10 installed, the amount of physical memory available is 3.87 GB, as shown in the Msinfo32 utility:

[Click here to view code image](#)

```
Installed Physical Memory (RAM)  4.00 GB
Total Physical Memory            3.87 GB
```

You can see the physical memory layout with the MemInfo tool. The following output is from MemInfo when run on a 32-bit system, using the -r switch to dump physical memory ranges:

[Click here to view code image](#)

```
C:\Tools>MemInfo.exe -r
MemInfo v3.00 - Show PFN database information
Copyright (C) 2007-2016 Alex Ionescu
www.alex-ionescu.com
```

```
Physical Memory Range: 00001000 to 0009F000 (158 pages, 632 KB)
```

Physical Memory Range: 00100000 to 00102000 (2 pages, 8 KB)
Physical Memory Range: 00103000 to F7FF0000 (1015533 pages, 4062132 KB)
MmHighestPhysicalPage: 1015792

Note the gap in the memory address range from A0000 to 100000 (384 KB), and another gap from F8000000 to FFFFFFFF (128 MB).

You can use Device Manager on your machine to see what is occupying the various reserved memory regions that can't be used by Windows (and that will show up as holes in MemInfo's output). To check Device Manager, follow these steps:

1. Run `Devmgmt.msc`.
2. Open the **View** menu and select **Resources by Connection**.
3. Expand the **Memory** node. On the laptop computer used for the output shown in [Figure 5-43](#), the primary consumer of mapped device memory is, unsurprisingly, the video card (Hyper-V S3 Cap), which consumes 128 MB in the range F8000000–FBFFFFFFF.

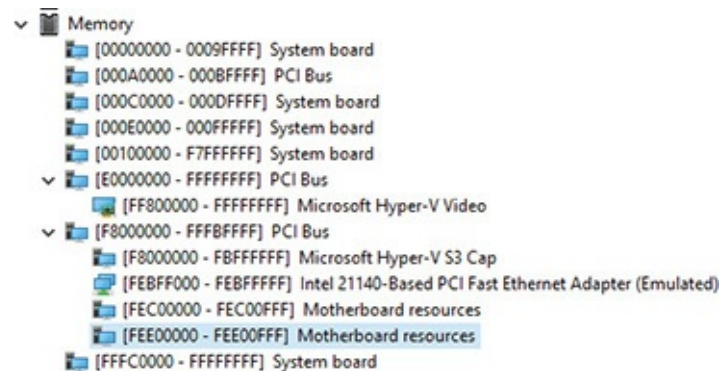


FIGURE 5-43 Hardware-reserved memory ranges on a 32-bit Windows system.

Other miscellaneous devices account for most of the rest, and the PCI bus reserves additional ranges for devices as part of the conservative estimation the firmware uses during boot.

Memory compression

The Windows 10 memory manager implements a mechanism that compresses private and page-file-backed section pages that are on the modified page list. The primary candidates for compression are private pages belonging to UWP apps because compression works very well with the working set swapping and emptying that already occurs for such applications if memory is tight. After an application is suspended and its working set is outswapped, the working set can be emptied at any time and dirty pages can be compressed. This will create additional available memory that may be enough to hold another application in memory without making the first application's pages leave memory.



Note

Experiments have shown that pages compress to around 30–50 percent of their original size using Microsoft's Xpress algorithm, which balances speed with size, thus resulting in considerable memory savings.

The memory compression architecture must adhere to the following requirements:

- A page cannot be in memory in a compressed and an uncompressed form because this would waste physical memory due to duplication. This means that whenever a page is compressed, it must become a free page after successful compression.
- The compression store must maintain its data structures and store the compressed data such that it is always saving memory for the system overall. This means that if a page doesn't compress well enough, it will not be added to the store.
- Compressed pages must appear as available memory (because they can really be repurposed if needed) to avoid creating a perception issue that compressing memory somehow increases memory consumption.

Memory compression is enabled by default on client SKUs (phone, PC, Xbox, and so on). Server SKUs do not currently use memory compression, but that is likely to change in future server versions.



Note

In Windows 2016, Task Manager still shows a number in parentheses for compressed memory, but that number is always zero. Also, the memory compression process does not exist.

During system startup, the `Superfetch` service (`sysmain.dll`, hosted in a `svchost.exe` instance, described in the upcoming “[Proactive memory management \(SuperFetch\)](#)” section) instructs the Store Manager in the executive through a call to `NtSetSystemInformation` to create a single system store (always the first store to be created), to be used by non-UWP applications. Upon app startup, each UWP application communicates with the `Superfetch` service and requests the creation of a store for itself.

Compression illustration

To get a sense of how memory compression works, let's look at an illustrative example. Assume that at some point in time, the following physical pages exist:

Zero / Free



Active

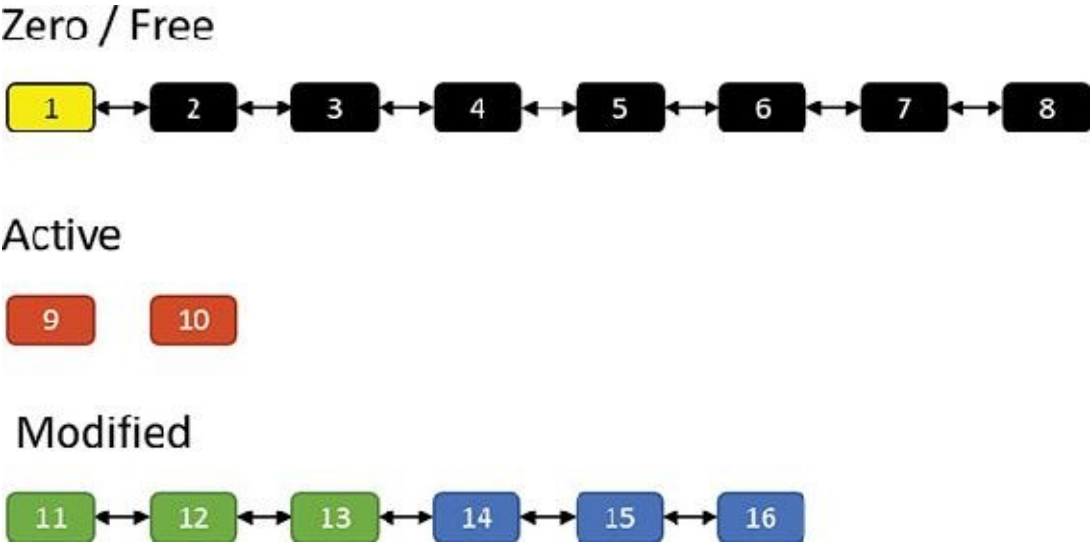


Modified

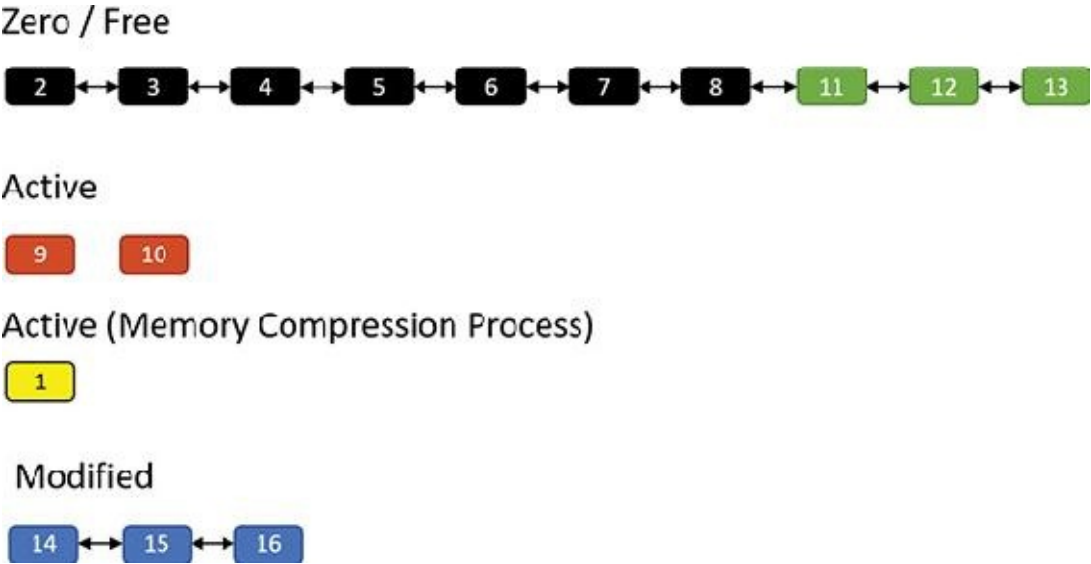


The zero and free page lists contain pages that have garbage and zeroes, respectively, and can be used to satisfy memory commits; for the sake of this discussion, we'll treat them as one list. The active pages belong to various processes, while the modified pages have dirty data that has not yet been written to a page file, but can be soft-faulted without an I/O operation to a process working set if that process references a modified page.

Now assume the memory manager decides to trim the modified page list—for example, because it has become too large or the zero/free pages have become too small. Assume three pages are to be removed from the modified list. The memory manager compresses their contents into a single page (taken from the zero/free list):



Pages 11, 12 and 13 are compressed into page 1. After that's done, page 1 is no longer free and is in fact active, part of the working set of the memory compression process (described in the next section). Pages 11, 12, and 13 are no longer needed and move to the free list; the compression saved two pages:



Suppose the same process repeats. This time, pages 14, 15, and 16 are compressed into (say) two pages (2 and 3) as shown here:

Zero / Free



Active



Active (Memory Compression Process)



Modified



The result is that pages 2 and 3 join the working set of the memory compression process, while pages 14, 15, and 16 become free:

Zero / Free



Active



Active (Memory Compression Process)



Modified

Suppose the memory manager later decides to trim the working set of the memory compression process. In that case, such pages are moved to the modified list because they contain data not yet written to a page file. Of course, they can at any time be soft-faulted back into their original process (decompressing in the process by using free pages). The following shows pages 1 and 2 being removed from the active pages of the memory compression process and moved to the modified list:

Zero / Free



Active



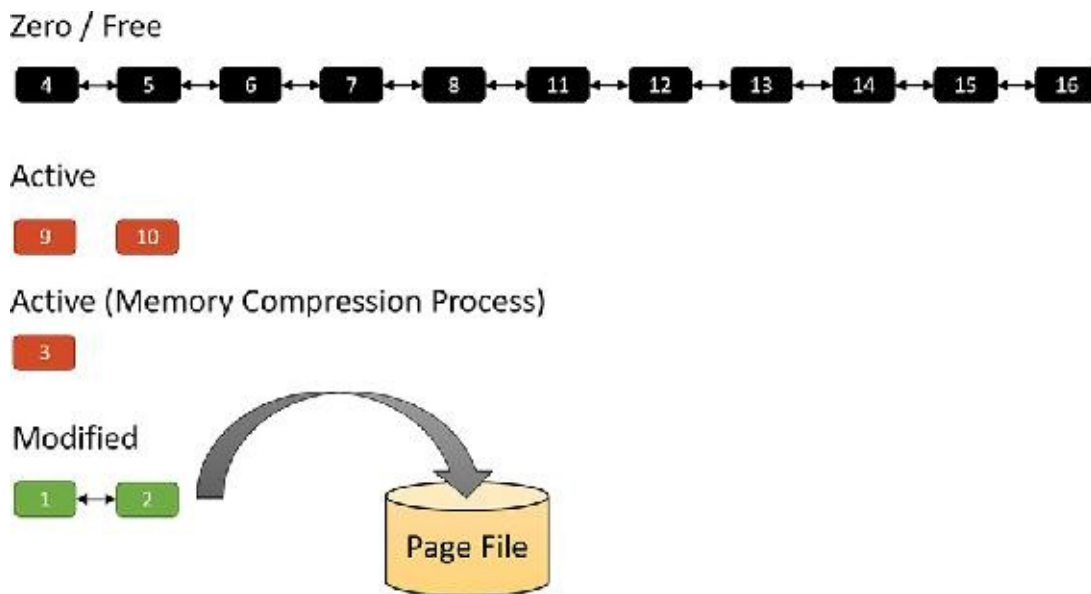
Active (Memory Compression Process)



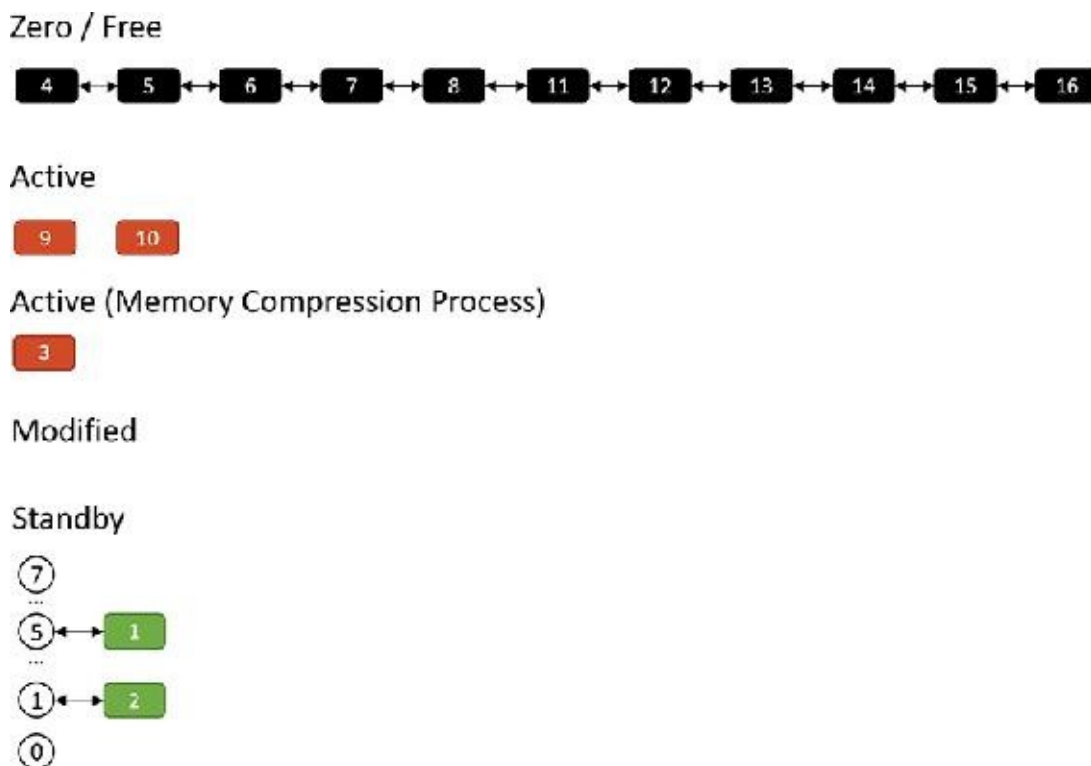
Modified



If memory becomes tight, the memory manager may decide to write the compressed modified pages to a page file:



Finally, after such pages have been written to a page file, they move to the standby list because their content is saved, so they can be repurposed if necessary. They can also be soft-faulted (as when they are part of the modified list) by decompressing them and moving the resulting pages to the active state under the relevant process working set. When in the standby list, they are attached to the appropriate sub-list, depending on their priority (as described in the [“Page priority and rebalancing”](#) section later in this chapter):



Compression architecture

The compression engine needs a “working area” memory to store compressed pages and the data structures that manage them. In Windows 10 versions prior to 1607, the user address space of the System process was used. Starting with Windows 10 Version 1607, a new dedicated process called *Memory Compression* is used instead. One reason for creating this new process was that the System process memory consumption looked high to a casual observer, which implied the system was consuming a lot of memory. That was not the case, however, because compressed memory does not count against the commit limit. Nevertheless, sometimes perception is everything.

The Memory Compression process is a minimal process, which means it does not load any DLLs. Rather, it just provides an address space to work with. It's not running any executable image either—the kernel is just using its user mode address space. (See [Chapter 3](#), “[Processes and jobs](#),” for more information on minimal processes.)



Note

By design, Task Manager does not show the Memory Compression process in its details view, but Process Explorer does. Using a kernel debugger, the compression process image name is MemCompression.

For each store, the Store Manager allocates memory in regions with a configurable region size. Currently, the size used is 128 KB. The allocations are done by normal VirtualAlloc calls as needed. The actual compressed pages are stored in 16 byte chunks within a region. Naturally, a compressed page (4 KB) can span many chunks. [Figure 5-44](#) shows a store with an array of regions and some of the data structures associated with managing such a store.

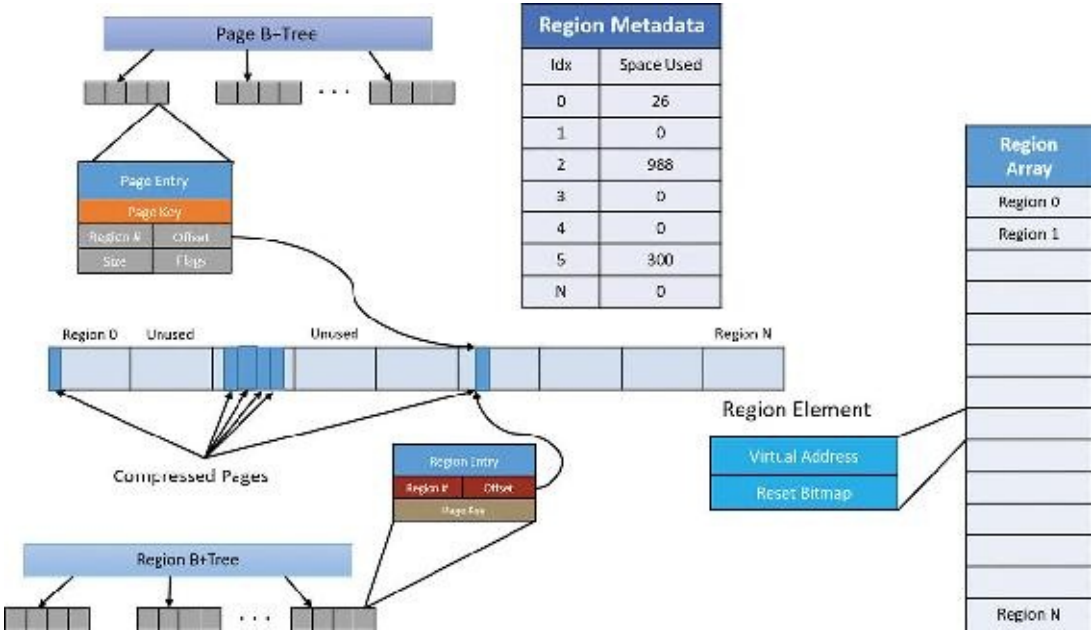


FIGURE 5-44 Store data structures.

As shown in [Figure 5-44](#), pages are managed with a B+Tree—essentially a tree where a node can have any number of children—where each page entry points to its compressed content within one of the regions. A store starts with zero regions, and regions are allocated and deallocated as needed. Regions are also associated with priorities, as described in the “[Page priority and rebalancing](#)” section later in this chapter.

Adding a page involves the following major steps:

1. If there is no current region with the page’s priority, allocate a new region, lock it in physical memory, and assign it the priority of the page to be added. Set the current region for that priority to the allocated region.
2. Compress the page and store it in the region, rounding up to the granularity unit (16 bytes). For example, if a page compresses to 687 bytes, it consumes 43 16-byte units (always rounding up). Compression is done on the current thread, with low CPU priority (7) to minimize interference. When decompression is needed, it’s performed in parallel using all available processors.

3. Update the page and region information in the Page and Region B+Trees.
4. If the remaining space in the current region is not large enough to store the compressed page, a new region is allocated (with the same page priority) and set as the current region for that priority.

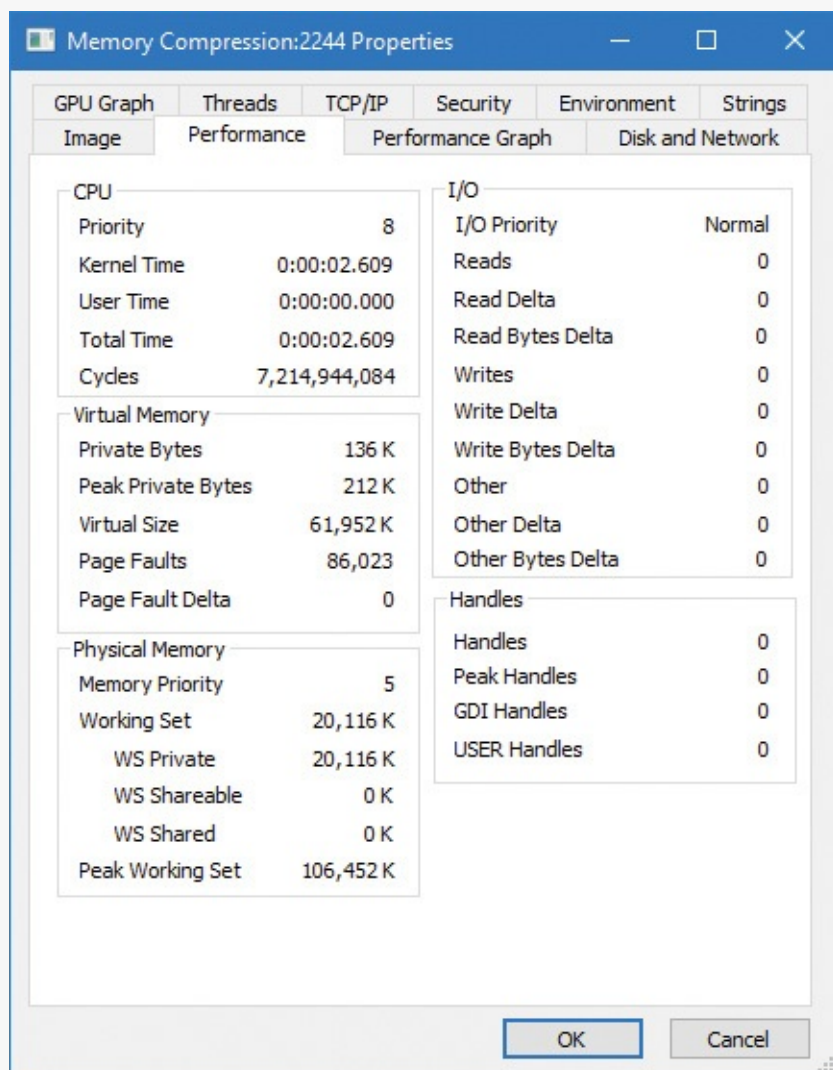
Removing a page from the store involves the following steps:

1. Find the page entry in the Page B+Tree and the region entry in the Region B+Tree.
2. Remove the entries and update the space used in the region.
3. If the region becomes empty, deallocate the region.

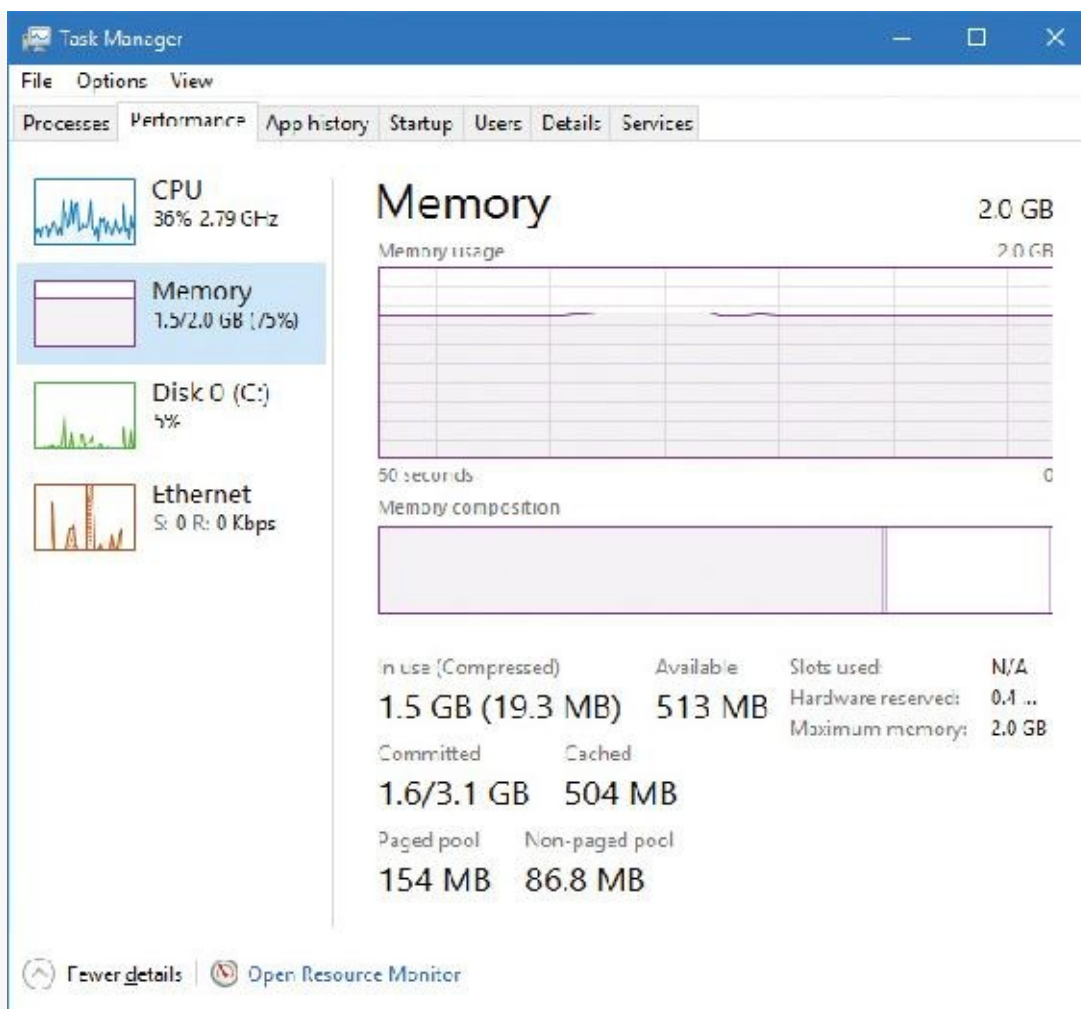
Regions become fragmented over time as compressed pages are added and removed. The memory for a region is not freed until the region is completely empty. This means some kind of compaction is necessary to reduce memory waste. A compaction operation is lazily scheduled with aggressiveness depending on the amount of fragmentation. Region priorities are taken into account when consolidating regions.

EXPERIMENT: Memory compression

There is very little visibility to the memory compression going on in the system. You can view the memory compression process with Process Explorer or a kernel debugger. The following figure shows the Performance tab in the properties of the Memory Compression process in Process Explorer (which must run with admin privileges):



Notice that the process has no user time (since only kernel threads “work” within this process), and its working set is private only (not shared). This is because the compressed memory is not sharable in any sense. Compare that to the Memory view in Task Manager:



Here, the compressed memory in parentheses should correlate with the working set of the Memory Compression process—this screenshot was taken about a minute after the previous one—because this is the amount of space that the compressed memory is consuming.

Memory partitions

Traditionally, virtual machines (VMs) are used to isolate applications so that separate VMs can run completely isolated applications (or groups of applications) at least from a security standpoint. VMs cannot interact with each other, providing strong security and resource boundaries. Although this works, VMs have a high resource cost in terms of hardware that hosts the VMs and management costs. This gave a rise to container-based technologies, such as Docker. These technologies attempt to lower the barrier for isolation and resource management by creating sandbox containers that host applications, all on the same physical or virtual machine.

Creating such containers is difficult, as it would require kernel drivers that perform some form of virtualization on top of the regular Windows. Some of these drivers are the following (a single driver can encompass all these functionalities):

- File system (mini) filter that would create an illusion of an isolated file system
- Registry virtualization driver, creating an illusion of a separate registry (CmRegisterCallbacksEx)
- Private object manager namespace, by utilizing silos (see [Chapter 3](#) for more details)
- Process management for associating processes with the correct container by using process create/notifications (PsSetCreateNotifyRoutineEx)

Even with these in place, some things are difficult to virtualize, specifically memory management. Each container may want to use its own PFN database, its own page file, and so on. Windows 10 (64-bit versions only) and Windows Server 2016 provide such possible memory control through Memory Partitions.

A memory partition consists of its own memory-related management structures, such as page lists (standby, modified, zero, free, etc.), commit charge, working set, page trimmer, modified page writer, zero-page thread, and so on, but isolated from other partitions. Memory partitions are represented in the system by Partition objects, which are securable, nameable objects (just like other executive objects). One partition always exists, called the *System Partition*, and it represents the system as a whole and is the ultimate parent of any explicitly created partition. The system partition's address is stored in a global variable (`MiSystemPartition`) and its name is `KernelObjects\MemoryPartition0`, visible with tools such as WinObj from Sysinternals as shown in [Figure 5-45](#).

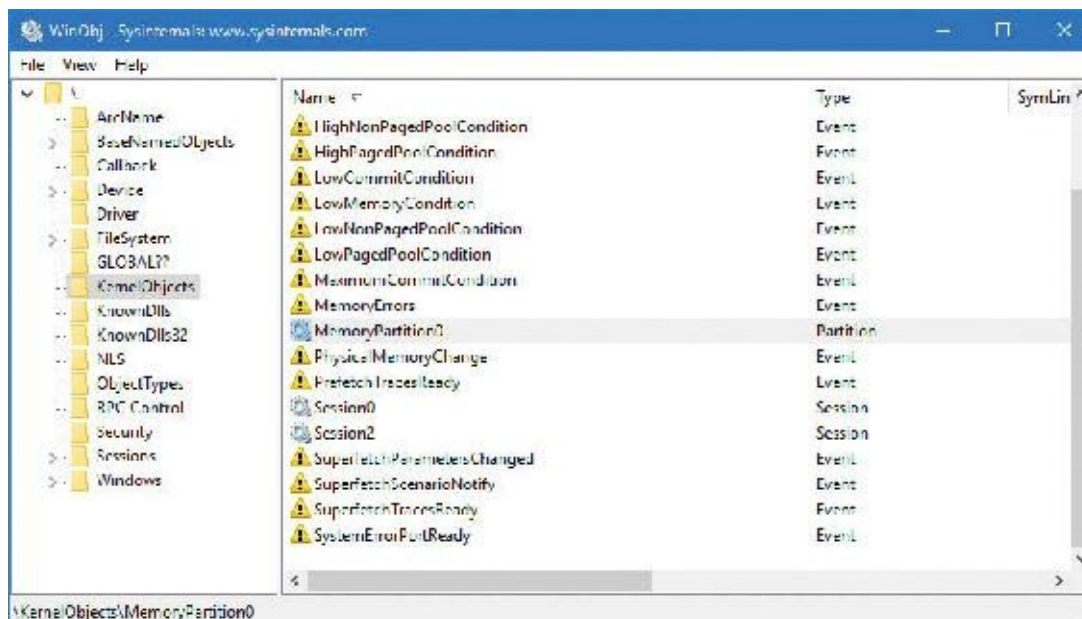


FIGURE 5-45 The System Partition in WinObj.

All partition objects are stored in global list, where the current maximum partition count is 1024 (10 bits), because the partition index must be encoded in PTEs for quick access to partition information where applicable. One of these indices is the system partition and two other values are used as special sentinels, leaving 1021 partitions available.

Memory partitions can be created from user mode or kernel mode by using the `NtCreatePartition` internal (and undocumented) function; user mode callers must have the `SeLockMemory` privilege for the call to succeed. The function can accept a parent partition, which initial pages will come from and eventually return to, when the partition is destroyed; the system partition is the default parent if none is specified. `NtCreatePartition` delegates the actual work to the internal memory manager `MiCreatePartition` function.

An existing partition can be opened by name using `NtOpenPartition` (no special privilege required for this as the object can be protected by ACLs as usual). The actual manipulation of a partition is reserved for the `NtManagePartition` function. This is the function that can be used to add memory to the partition, add a paging file, copy memory from one partition to another, and generally obtain information about a partition.

EXPERIMENT: Viewing memory partitions

In this experiment, you'll use the kernel debugger to look at partition objects.

1. Start local kernel debugging and issue the `!partition` command. It lists all the partition objects in the system.

[Click here to view code image](#)

```
lkd> !partition
Partition0 fffff803eb5b2480 MemoryPartition0
```

2. By default, the always existing system partition is shown. The `!partition` command can accept the address of a partition object and show more details:

[Click here to view code image](#)

```
lkd> !partition fffff803eb5b2480
PartitionObject @ ffffc808f5355920 (MemoryPartition0)
_MI_PARTITION 0 @ fffff803eb5b2480
MemoryRuns: 0000000000000000
MemoryNodeRuns: ffffc808f521ade0
AvailablePages: 0n4198472 ( 16 Gb 16 Mb 288 Kb)
ResidentAvailablePages: 0n6677702 ( 25 Gb 484 Mb 792 Kb)
0 _MI_NODE_INFORMATION @ fffff10000003800
TotalPagesEntireNode: 0x7f8885
Zeroed Free
1GB 0 (
0) 0 (
0)
2MB 41 ( 82
Mb) 0 (
0)
64KB 3933 ( 245 Mb 832
Kb) 0 (
0)
4KB 82745 ( 323 Mb 228
Kb) 0 (
0)
Node Free Memory: ( 651 Mb 36 Kb )
InUse Memory: ( 31 Gb 253 Mb 496 Kb )
TotalNodeMemory: ( 31 Gb 904 Mb 532 Kb )
```

The output shows some of the information stored in the underlying `MI_PARTITION` structure (whose address is given as well). Notice that the command shows memory information on a NUMA node basis (just one in this case). Since this is the system partition, the numbers related to used, free, and total memory should correspond to the values reported by tools such as Task Manager and Process Explorer. You can also examine the `MI_PARTITION` structure with the usual `dt` command.

Future scenarios may leverage the memory partitioning capability for specific processes (through job

objects) to be associated with a partition, such as when exclusive control over physical memory may be beneficial. One such scenario slated for the Creators Update release is game mode (more information on game mode is included in Chapter 8 in Part 2).

Memory combining

The memory manager uses several mechanisms in an attempt to save as much RAM as possible, such as sharing pages for images, copy-on-write for data pages, and compression. In this section, we'll take a look at yet another such mechanism called *memory combining*.

The general idea is simple: Find duplicates of pages in RAM and combine them into one, thus removing the rest of the duplicates. Clearly there are a few issues to resolve:

- What are the “best” pages to use as candidates for combining?
- When is it appropriate to initiate memory combining?
- Should combining be targeted at a particular process, a memory partition, or the entire system?
- How can the combining process be made quick so it does not adversely impact normally executing code?
- If a writeable combined page is later modified by one of its clients, how would it get a private copy?

We'll answer these questions throughout this section, starting with the last. The copy-on-write mechanism is used here: as long as a combined page is not written to, do nothing. If a process tries to write to the page, make a private copy for the writing process, and remove the copy-on-write flag for that newly allocated private page.



Note

Page combining can be disabled by setting a DWORD value named `DisablePageCombining` to 1 in the `HKLM\System\CurrentControlSet\Control\Session Manager\Memory Management` registry key.



Note

In this section, the terms *CRC* and *hash* are used interchangeably. They indicate a statistically unique (with high probability) 64-bit number referencing a page's contents.

The memory manager's initialization routine, `MmInitSystem`, creates the system partition (see the previous section on memory partitions). Within the `MI_PARTITION` structure that describes a partition lies an array of 16 AVL trees that identify the duplicated pages. The array is sorted by the last 4 bits of a combined page CRC value. We'll see in a moment how this fits into the algorithm.

Two special page types are called *common pages*. One includes an all zero bytes, and the other includes an all one bits (filled with the byte `0xFF`); their CRC is calculated just once and stored. Such pages can easily be identified when scanning pages' contents.

To initiate memory combining, the `NtSetSystemInformation` native API is called with the `System-CombinePhysicalMemoryInformation` system information class. The caller must have the `SeProfile-SingleProcessPrivilege` in its token, normally granted to the local

administrators group. The argument to the API provides the following options through a combination of flags:

- Perform memory combining on the entire (system) partition or just the current process.
- Search for common pages (all zeros or all ones) to combine only, or any duplicate pages regardless of content.

The input structure also provides an optional event handle that can be passed in, and if signaled (by another thread), will abort the page combining. Currently, the `SuperFetch` service (see the section “SuperFetch” at the end of this chapter for more information) has a special thread, running in low priority (4) that initiates memory combining for the entire system partition when the user is away, or if the user is busy, every 15 minutes.

In the Creators Update, if the amount of physical memory is higher than 3.5 GB (3584 MB), most built-in Svchost-ed services host a single service in each Svchost process. This creates dozens of processes out of the box but removes the likelihood of one service affecting another (either because of some instability or security issues). In this scenario, the Service Control Manager (SCM) uses a new option of the memory combining API, and initiates page combining in each of the Svchost processes every three minutes by utilizing a thread pool timer running with base priority of 6 (`ScPerformPageCombineOnServiceImages` routine). The rationale is to try to reduce RAM consumption that may be higher than with fewer Svchost instances. Note that non-Svchost services are not page combined, nor are services running with per-user or private user accounts.

The `MiCombineIdenticalPages` routine is the actual entry point to the page combining process. For each NUMA node of the memory partition, it allocates and stores a list of pages with their CRC inside the page combining support (PCS) structure, which is the one managing all the needed information for the page combining operation. (That’s the one holding the AVL trees array mentioned earlier.) The requesting thread is the one doing the work; it should run on CPUs belonging to the current NUMA node, and its affinity is modified accordingly if needed. We’ll divide the memory combining algorithm into three stages to simplify the explanation: search, classification, and page sharing. The following sections assume that a complete page combining is requested (rather than for the current process) and for all pages (not just the common pages); the other cases are similar in principle, and somewhat simpler.

The search phase

The goal of this initial stage is to calculate the CRC of all the physical pages. The algorithm analyses each physical page that belongs to the active, modified, or standby list, skipping the zeroed and free pages (since they are effectively unused).

A good page candidate for memory combining should be an active non-shared page that belongs to a working set, and that should not map a paging structure. The candidate could be even in the standby or modified state, but needs to have a reference counter of 0. Basically, the system identifies three types of pages for combining: user process, paged pool, and session space. Other types of pages are skipped.

To correctly calculate the CRC of the page, the system should map the physical page to a system address (because the process context is mostly different from the calling thread, making the page inaccessible in low user-mode addresses) using a new system PTE. The CRC of the page is then calculated with a customized algorithm (`MiComputeHash64` routine), and the system PTE freed (the page is now unmapped from system address space).

The page hashing algorithm

The algorithm that the system uses to calculate an 8-bytes page hash (CRC) is the following: two 64-bit large prime numbers are multiplied together, and the result is used as the starting hash. The page is scanned from the end to the beginning; the algorithm hashes 64 bytes per cycle. Each read 8-byte value from the page in question is added to the starting hash, then the hash is rotated right a prime-number of bits (starting with 2, then 3, 5, 7, 11, 13, 17 and 19). 512 memory access operations ($4096/8$) are needed to completely hash a page.

The classification phase

When all the hashes of the pages that belong to a NUMA node have been successfully calculated, the second part of the algorithm commences. The goal of this phase is to process each CRC/PFN entry in the list and organize them in a strategic way. The page sharing algorithm must minimize the process contexts switches, and be as fast as possible.

The `MiProcessCrCList` routine starts by sorting the CRC/PFN list by hash (using a quick sort algorithm). Another key data structure, *combine block*, is used to keep track of all the pages that share the same hash, and, more importantly, to store the new prototype PTE that will map the new combined page. Each CRC/PFN of the new sorted list is processed in order. The system needs to verify if the current hash is common (belongs to a zeroed or a complete-filled page) and if it's equal to the previous or next hash (remember that the list is sorted). If this is not the case, the system checks if a combine block already exists in the PCS structure. If so, it means that a combined page has been already identified in a previous execution of the algorithm or in another node of the system. Otherwise it means that the CRC is unique and the page couldn't be combined, and the algorithm continues to the next page in the list.

If the found common hash has never been seen before, the algorithm allocates a new empty combine block (used for the master PFN) and inserts it in a list used by the actual page-sharing code (next stage). Otherwise if the hash already existed (the page is not the master copy), a reference to the combine block is added in the current CRC/PFN entry.

At this point, the algorithm has prepared all the data that the page-sharing algorithm needs: a list of combine blocks used to store the master physical pages and their prototype PTEs, a list of CRC/PFN entries organized by the owning working set, and some physical memory needed to store the content of the new shared pages.

The algorithm then obtains the address of the physical page (that should exist, due to the initial check performed previously by the `MiCombineIdenticalPages` routine) and searches a data structure used to store all the pages that belongs to the specific working set (from now on we will call this structure *WS CRC node*). If this doesn't exist, it allocates a new one and inserts it in another AVL tree. The CRC/PFN and virtual address of the page are linked together inside the WS CRC node.

After all the identified pages have been processed, the system allocates the physical memory for the new master shared pages (using an MDL), and processes each WS CRC node; for performance reasons, the candidate pages located inside the node are sorted by their original virtual address. The system is now ready to perform the actual page combining.

The page combining phase

The page combining phase starts with a WS CRC node structure that contains all the pages that belong to a specific working set and are all candidates for combining, and with a list of free combine blocks, used to store the prototype PTE and the actual shared page. The algorithm attaches to the target process and locks its working set (raising IRQL to dispatch level). In this way, it will be able to directly read and write each page without the need to remap it.

The algorithm processes every CRC/PFN entry in the list, but since it's running at dispatch level IRQL and execution could take some time, it checks if the processor has some DPCs or scheduled items in its queue (by calling `KeShouldYieldProcessor`) before analyzing the next entry. If the answer is yes, the algorithm does the right thing and takes appropriate precautions to maintain state.

The actual page sharing strategy expects three possible scenarios:

- The page is active and valid, but it contains all zeroes, so rather than combining, it replaces its PTE with a demand-zero PTE. Recall that this is the initial state of normal `VirtualAlloc`-like memory allocation.
- The page is active and valid, but it is not zeroed out, meaning it has to be shared. The algorithm checks if the page has to be promoted as the master: if the CRC/PFN entry has a pointer to a valid combine block, it means that it's not the master page; otherwise, the page is the master copy. The master page hash is rechecked and a new physical page assigned for the sharing. Otherwise, the already existing combine block is used (and its reference count incremented). The system is now ready to convert the private page into a shared one, and calls the `MiConvertPrivateToProto` routine to perform the actual job.
- The page is in the modified or standby list. In this case, it's mapped to a system address as a valid page and its hash recalculated. The algorithm performs the same step as the previous scenario, with the only difference being that the PTE is converted from shared to prototype using the `MiConvertStandbyToProto` routine.

When the sharing of the current page ends, the system inserts the combine block of the master copy into the PCS structure. This is important because the combine block becomes the link between each private PTE and the combined page.

From private to shared PTE

The goal of `MiConvertPrivateToProto` is to convert a PTE of an active and valid page. If the routine detects that the prototype PTE inside the combine block is zero, it means that the master page must be created (together with the master shared prototype PTE). It then maps the free physical page into a system address and copies the content of the private page into the new shared one. Before actually creating the shared PTE, the system should free any page file reservation (see the section "[Page file reservation](#)" earlier in this chapter) and fill the PFN descriptor of the shared page. The shared PFN has the prototype bit set, and the PTE frame pointer set to the PFN of the physical page that contains the Prototype PTE. Most importantly, it has the PTE pointer set to the PTE located inside the combine block, but with the 63rd bit set to zero. This signifies to the system that the PFN belongs to a page that has been combined.

Next, the system needs to modify the PTE of the private page so that its target PFN is set to the shared physical page, its protection mask changed to copy-on-write, and the private page PTE is marked as valid. The prototype PTE inside the combine block is marked as a valid hardware PTE too: the content is identical to the new PTE of the private page. Finally, the page file space allocated for the private page is

freed and the original PFN of the private page is marked as deleted. The TLB cache is flushed and the private process working set size is decremented by one page.

Otherwise (the prototype PTE inside the combine block is non-zero), it means that the private page should be a copy of the master one. Only the active PTE of the private page must be converted. The PFN of the shared page is mapped to a system address and the content of the two pages is compared. This is important because the CRC algorithm does not produce unique values in the general case. If the two pages don't match, the function stops processing and returns. Otherwise, it unmaps the shared page and sets the page priority of the shared PFN to the higher of the two. [Figure 5-46](#) shows the state where only a master page exists.

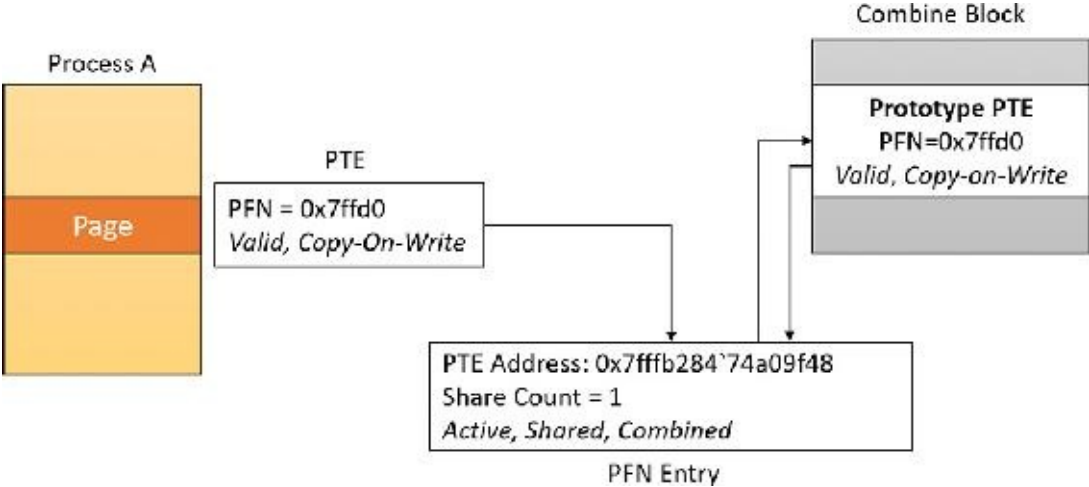


FIGURE 5-46 Combined master page.

The algorithm now calculates the new invalid software prototype PTE that should be inserted into the process private page table. To do that, it reads the address of the hardware PTE that maps the shared page (located in the combine block), shifts it, and sets the Prototype and Combined bits. A check is made that the share count of the private PFN is 1. If it is not, processing is aborted. The algorithm writes the new software PTE in the private page table of the process and decrements the share count of the page table of the old private PFN (keep in mind that an active PFN always has a pointer to its page table). The target process working set size is decremented by one page and the TLB is flushed. The old private page is moved into the transition state, and its PFN is marked for deletion. [Figure 5-47](#) shows two pages, where the new page points to the prototype PTE but is not yet valid.

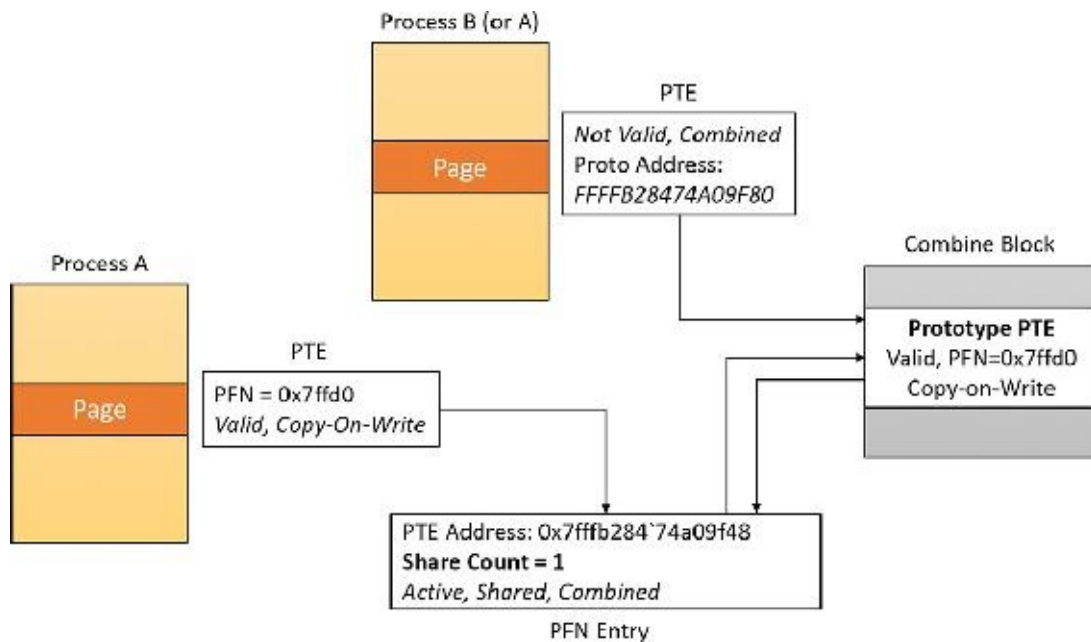


FIGURE 5-47 Combined pages before simulated page fault.

Finally, the system uses another trick to prevent working set trimming on the shared pages by simulating a page fault. That way the share count of the shared PFN is again incremented, and no fault would occur at a time the process will try to read the shared page. The end result is that the private PTE is again a valid hardware PTE. [Figure 5-48](#) shows the effect of a soft page fault on the second page, making it valid and incrementing the share count.

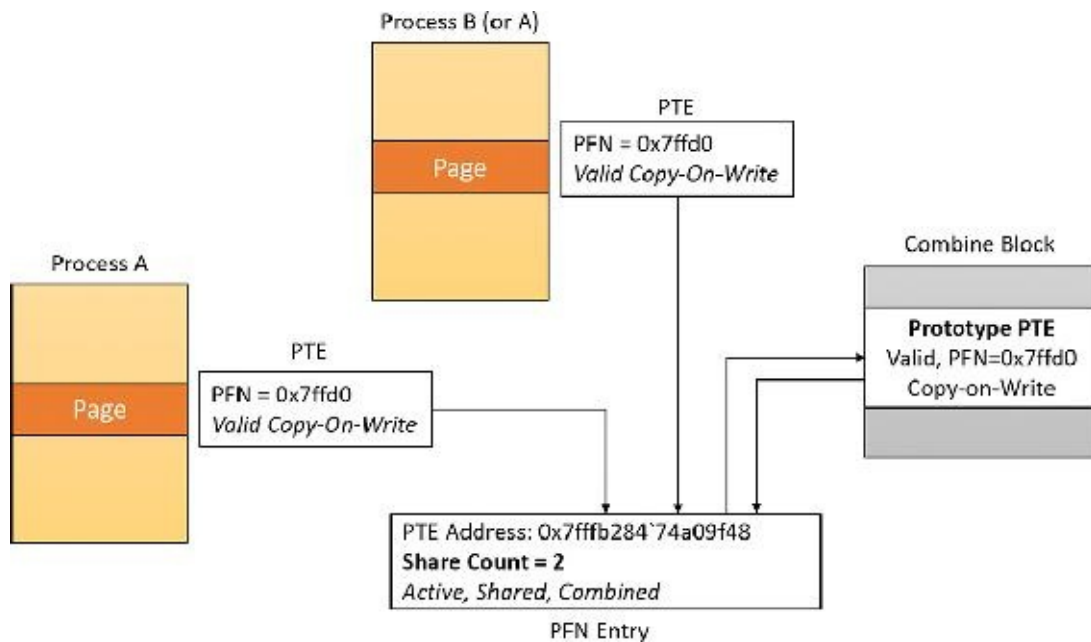


FIGURE 5-48 Combined pages after simulated page fault.

Combined pages release

When the system needs to free a particular virtual address, it first locates the address of the PTE that maps it. The pointed PFN of a combined page has the prototype and combined bits set. The free request for a combined PFN is managed exactly like the one for a prototype PFN. The only difference is that the system (if the combined bit is set) calls `MiDecrementCombinedPte` after processing the prototype PFN.

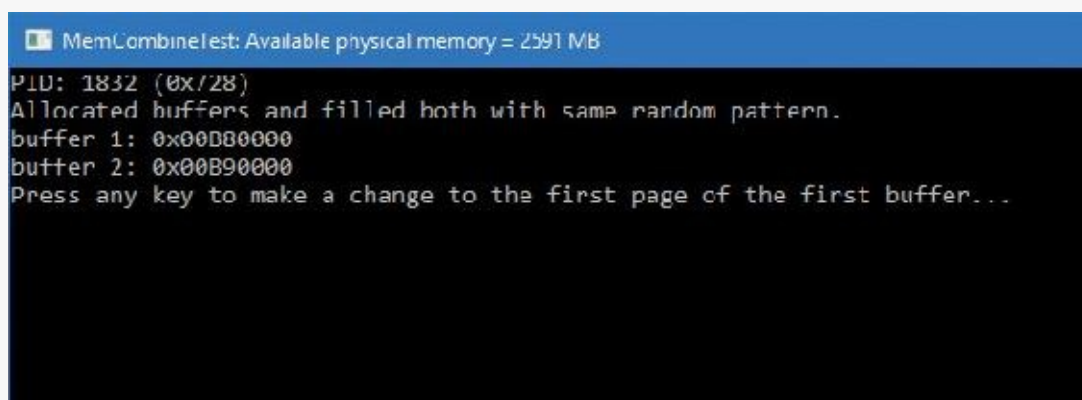
`MiDecrementCombinedPte` is a simple function that decrements the reference count of the

combine block of the Prototype PTE. (Keep in mind that at this stage the PTE is in transition because the memory manager has already dereferenced the physical page that it maps. The share count of the physical page has already dropped to zero, and so the system put the PTE in transition.) If the reference count drops to zero, the prototype PTE will be freed, the physical page put in the free list, and the combine block returned to the combine free list of the PCS structure of the memory partition.

EXPERIMENT: Memory combining

In this experiment, you will see the effects of memory combining. Follow these steps:

1. Start a kernel debugging session with a VM target (as described in [Chapter 4](#), “[Threads](#)”).
2. Copy the MemCombine32.exe (for 32 bit targets) or MemCombine64.exe (for 64 bit targets), and MemCombineTest.exe executables from this book’s downloadable resources to the target machine.
3. Run MemCombineTest.exe on the target machine. You should see something like the following:



4. Note the two addresses shown. These are two buffers filled with a random generated pattern of bytes, repeated so that each page has the same content.
5. Break into the debugger. Locate the MemCombineTest process:

[Click here to view code image](#)

```
0: kd> !process 0 0 memcombinetest.exe
PROCESS fffff70a3cb29080
    SessionId: 2  Cid: 0728    Peb: 00d08000  ParentCid: 11c4
    DirBase: c7c95000  ObjectTable:
ffff918ede582640  HandleCount: <Data Not
Accessible>
    Image: MemCombineTest.exe
```

6. Switch to the located process:

[Click here to view code image](#)

```
0: kd> .process /i /r fffff70a3cb29080
You need to continue execution (press 'g' <enter>) for the
context
to be switched. When the debugger breaks in again, you will be
in
the new process context.
0: kd> g
```

```
Break instruction exception - code 80000003 (first chance)
nt!DbgBreakPointWithStatus:
fffff801'94b691c0 cc int 3
```

7. Use the !pte command to locate the PFN where the two buffers are stored:

[Click here to view code image](#)

```
0: kd> !pte b80000
VA 00000000000b80000
PXE at FFFFA25128944000 PPE at FFFFA25128800000 PDE at
FFFA251000000028 PTE at FFFFA200000005C00
contains 00C0000025BAC867 contains 0FF00000CAA2D867 contains
00F0000003B22F867 contains B9200000DEDFB867
pfn 25bac ---DA--UWEV pfn caa2d ---DA--UWEV pfn
3b22f ---DA--
UWEV pfn dedfb ---DA--UW-V
```

```
0: kd> !pte b90000
VA 00000000000b90000
PXE at FFFFA25128944000 PPE at FFFFA25128800000 PDE at
FFFA251000000028 PTE at FFFFA200000005C80
contains 00C0000025BAC867 contains 0FF00000CAA2D867 contains
00F0000003B22F867 contains B9300000F59FD867
pfn 25bac ---DA--UWEV pfn caa2d ---DA--UWEV pfn
3b22f ---DA--
UWEV pfn f59fd ---DA--UW-V
```

8. Notice the PFN values are different, indicating that these pages are mapped to different physical addresses. Resume the target.

9. On the target, open an elevated command window, navigate to the directory where you copied MemCombine(32/64), and run it. The tool forces a full memory combining, which may take a few seconds.

10. When it's finished, break into the debugger again. Repeat steps 6 and 7. You should see the PFNs change:

[Click here to view code image](#)

```
1: kd> !pte b80000
VA 00000000000b80000
PXE at FFFFA25128944000 PPE at FFFFA25128800000 PDE at
FFFA251000000028 PTE at FFFFA200000005C00
contains 00C0000025BAC867 contains 0FF00000CAA2D867 contains
00F0000003B22F867 contains B9300000EA886225
pfn 25bac ---DA--UWEV pfn caa2d ---DA--UWEV pfn
3b22f ---DA--
UWEV pfn ea886 C---A--UR-V
```

```
1: kd> !pte b90000
VA 00000000000b90000
```

```

PXE at FFFFA25128944000    PPE at FFFFA25128800000    PDE at
FFFA251000000028    PTE at FFFFA20000005C80
contains 00C0000025BAC867    contains 0FF00000CAA2D867    contains
00F000003B22F867    contains BA600000EA886225
pfn 25bac    ---DA--UWEV    pfn caa2d    ---DA--UWEV    pfn
3b22f    ---DA--
UWEV    pfn ea886    C---A--UR-V

```

11. Notice the PFN values are the same, meaning the pages are mapped to the exact same address in RAM. Also note the C flag in the PFN, indicating copy-on-write.
12. Resume the target and press any key in the MemCombineTest window. This changes a single byte in the first buffer.
13. Break into the target again and repeat steps 6 and 7 yet again:

[Click here to view code image](#)

```

1: kd> !pte b80000
                                                    VA 00000000000b80000
PXE at FFFFA25128944000    PPE at FFFFA25128800000    PDE at
FFFA251000000028    PTE at FFFFA20000005C00
contains 00C0000025BAC867    contains 0FF00000CAA2D867    contains
00F000003B22F867    contains B9300000813C4867
pfn 25bac    ---DA--UWEV    pfn caa2d    ---DA--UWEV    pfn
3b22f    ---DA--
UWEV    pfn 813c4    ---DA--UW-V

```

```

1: kd> !pte b90000
                                                    VA 00000000000b90000
PXE at FFFFA25128944000    PPE at FFFFA25128800000    PDE at
FFFA251000000028    PTE at FFFFA20000005C80
contains 00C0000025BAC867    contains 0FF00000CAA2D867    contains
00F000003B22F867    contains BA600000EA886225
pfn 25bac    ---DA--UWEV    pfn caa2d    ---DA--UWEV    pfn
3b22f    ---DA--
UWEV    pfn ea886    C---A--UR-V

```

14. The PFN for the first buffer has changed, and the copy-on-write flag was removed. The page has changed and was relocated to a different address in RAM.

Memory enclaves

Threads executing in a process have access to the entire process address space (as determined by page protection, which can be changed). This is desirable most of the time; however, in case malicious code manages to get itself injected into a process, it has the exact same power. It can freely read data that may contain sensitive information and even change it.

Intel has created a technology called Intel Software Guard Extensions (SGX) that allows the creation of protected memory enclaves—secure zones in a process address space where code and data are protected by the CPU from code running outside the enclave. Conversely, code running inside an enclave has full

(normal) access to process address space outside the enclave. Naturally, the protection extends to access from other processes and even code running in kernel mode. A simplified diagram of memory enclaves is shown in [Figure 5-49](#).

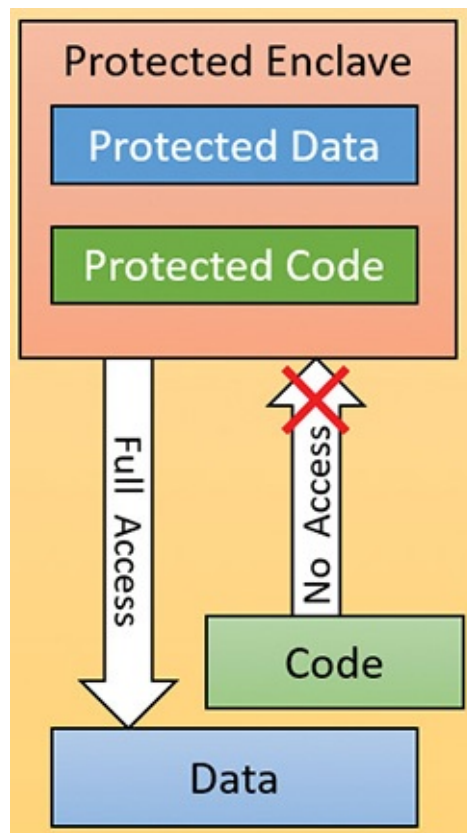


FIGURE 5-49 Memory enclaves.

Intel SGX is supported by sixth generation Core processors (“Skylake”) and later generations. Intel has its own SDK for application developers that can be used on Windows 7 and later systems (64 bit only). Starting with Windows 10 version 1511 and Server 2016, Windows provide an abstraction using Windows API functions that removes the need to use Intel’s SDK. Other CPU vendors may create similar solutions in the future, and these will also be wrapped by the same API, providing a relatively portable layer for application developers to use for creating and populating enclaves.



Note

Not all sixth generation Core processors support SGX. Also, an appropriate BIOS update must be installed on the system for SGX to work. Consult the Intel SGX documentation for more information. The Intel SGX website can be found at <https://software.intel.com/en-us/sgx>.



Note

At the time of this writing, Intel produced two versions of SGX (versions 1.0 and 2.0). Windows currently supports version 1.0 only. The differences are outside the scope of this book; consult the SGX documentation for more information.



Note

Current SGX versions do not support enclaves in ring 0 (kernel mode). Only ring 3 (user mode) enclaves are supported.

Programmatic interface

From an application developer's perspective, creating and working with an enclave consists of the following steps. (The internal details are described in the following sections.)

1. Initially the program should determine whether memory enclaves are supported by calling `IsEnclaveTypeSupported`, passing a value representing the enclave technology, which currently can only be `ENCLAVE_TYPE_SGX`.
2. A new enclave is created by calling the `CreateEnclave` function, which is similar in its arguments to `VirtualAllocEx`. For example, it's possible to create an enclave in a different process than the caller process. The complication in this function is the need to provide a vendor-specific configuration structure, which for Intel means a 4 KB data structure called *SGX Enclave Control Structure (SECS)*, which Microsoft does not define explicitly. Instead, developers are expected to create their own structure based on the particular technology used and defined in its documentation.
3. Once an empty enclave is created, the next step is to populate the enclave with code and data from outside of the enclave. This is accomplished by calling `LoadEnclaveData` where an external memory to the enclave is used to copy data to the enclave. Multiple calls to `LoadEnclaveData` may be used to populate the enclave.
4. The last step that is required to "activate" the enclave is achieved with the `InitializeEnclave` function. At that point, code that was configured to execute within the enclave can start executing.
5. Unfortunately, executing the code in the enclave is not wrapped by the API. Direct use of assembly language is required. The `EENTER` instruction transfers execution to the enclave and the `EEXIT` instruction causes return to the calling function. Abnormal termination of the enclave execution is also possible with Asynchronous Enclave Exit (AEX), such as due to a fault. The exact details are beyond the scope of this book, as they are not Windows-specific. Consult the SGX documentation to get the fine details.
6. Finally, to destroy the enclave, a normal `VirtualFree(Ex)` function can be used on the pointer to the enclave obtained in `CreateEnclave`.

Memory enclave initializations

During boot, Winload (the Windows Boot Loader) calls `Os1EnumerateEnclavePageRegions`, which first checks if SGX is supported using the CPUID instruction. If it is, it issues instructions to enumerate Enclave Page Cache (EPC) descriptors. EPC is the protected memory provided by the processor for creating and using enclaves. For each enumerated EPC, `Os1EnumerateEnclavePageRegions` calls `BlMm- AddEnclavePageRange` to add the page range information to a sorted list of memory descriptors with a type value of `LoaderEnclaveMemory`. This list is eventually stored in the `MemoryDescriptorListHead` member of the `LOADER_PARAMETER_BLOCK` structure used to pass information from the boot loader to the kernel.

During phase 1 initialization, the memory manager routine `MiCreateEnclaveRegions` is called to create an AVL tree for the discovered enclave regions (allowing quick lookups when needed); the tree is stored in the `MiState.Hardware.EnclaveRegions` data member. The kernel adds a new enclave page list, and a special flag passed to `MiInsertPageInFreeOrZeroedList` enables functionality to utilize this new list. However, since the memory manager has actually run out of list identifiers (3 bits are used for a maximum of 8 values, all taken), the kernel actually identifies these pages as being “bad” pages currently suffering an in-page error. The memory manager knows never to use bad pages, so calling enclave pages “bad” keeps them from being used by normal memory management operations, and so such pages end up in the bad page list.

Enclave construction

The `CreateEnclave` API ends up calling `NtCreateEnclave` in the kernel. As noted, a SECS structure must be passed in, documented by Intel SGX as shown in [Table 5-24](#).

Field	Offset (bytes)	Size (bytes)	Description
SIZE	0	8	Size of enclave in bytes; must be power of 2
BASEADDR	8	8	Enclave base linear address must be naturally aligned to size
SSAFRAMEST/FF	16	4	Size of one SSA frame in pages (including XSAVE, pad, GPR, and conditionally MISC)
MRSIGNER	128	32	Measurement register extended with the public key that verified the enclave. See <code>STGSTRUCT</code> for proper format
RESERVED	160	96	
ISVPRODID	256	2	Product ID of enclave
ISVSVN	258	2	Security version number (SVN) of the enclave
EID	Implementation dependent	8	Enclave identifier
PADDING	Implementation dependent	352	Padding pattern from the signature (used for key derivation strings)
RESERVED	260	3836	Includes EID, other non-zero reserved field and must-be-zero fields

TABLE 5-24 SECS structure layout

`NtCreateEnclave` first checks if memory enclaves are supported by looking at the root of the AVL tree (not by using the slower CPUID instruction). It then creates a copy of the passed-in structure (as is usual for kernel functions obtaining data from user mode) and attaches to the target process (`KeStackAttachProcess`) if the enclave is to be created in a different process than the caller’s. Then it transfers control to `MiCreateEnclave` to begin the actual work.

The first thing `MiCreateEnclave` does is allocate the Address Windowing Extension (AWE)

information structure that AWE APIs also utilize. This is because similar to the AWE functionality, an enclave allows a user-mode application to directly have access over physical pages (that is to say, the physical pages that are EPC pages based on the detection described earlier). Anytime a user-mode application has such direct control over physical pages, AWE data structures and locks must be used. This data structure is stored in the `AweInfo` field of the `EPROCESS` structure.

Next, `MiCreateEnclave` calls `MiAllocateEnclaveVad` to allocate an enclave-type VAD describing the enclave virtual memory range. This VAD has the `VadAwd` flag (as all AWE VADs) but also an additional `Enclave` flag, to differentiate it from a true AWE VAD. Finally, as part of VAD allocation, this is where the user-mode address for the enclave memory will be chosen (if not explicitly specified in the original `CreateEnclave` call).

The next step in `MiCreateEnclave` is to acquire an enclave page regardless of the enclave size or initial commitment. This is because, as per Intel's SGX documentation, all enclaves require at least a one-page control structure to be associated with them. `MiGetEnclavePage` is used to obtain the required allocation. This function simply scans the enclave page list described earlier, and extracts one page as needed. The returned page is mapped using a system PTE stored as part of the enclave VAD; the `MiInitializeEnclavePfn` function sets up the related PFN data structure and marks it `Modified` and `ActiveAndValid`.

There are no actual bits that would help you differentiate this enclave PFN from any other active region of memory (such as non-paged pool). This is where the enclave regions AVL tree comes into play, and `MI_PFN_IS_ENCLAVE` is a function that the kernel uses whenever it needs to check if a PFN is indeed describing an EPC region.

With the PFN initialized, the system PTE is now converted to a final global kernel PTE, and its resulting virtual address is computed. The final step in `MiCreateEnclave` is to call `KeCreateEnclave`, which will now do the low-level kernel enclave creation steps, including communication with the actual SGX hardware implementation. One job that `KeCreateEnclave` is responsible for is filling in the base address required by the SECS structure if the caller did not specify one, as it must be set in the SECS structure before communicating with the SGX hardware to create an enclave.

Loading data into an enclave

Once an enclave has been created, it's time to load information into it. The `LoadEnclaveData` function is exposed for that purpose. The function merely forwards the request to the underlying executive function, `NtLoadEnclaveData`. The function resembles a combination of a memory copy operation with some `VirtualAlloc` attributes (such as page protection).

If the enclave created with `CreateEnclave` doesn't yet have any committed enclave pages, they must first be obtained, which will result in zeroed out memory being added to the enclave, which can then be filled with non-zero memory from outside the enclave. Otherwise, if an initial pre-committed initialization size was passed in, then the enclave's pages can directly be filled in with non-zero memory from outside of the enclave.

Because enclave memory is described by a VAD, many of the traditional memory management APIs will function, at least partly, on this memory as well. For example, calling `VirtualAlloc` (ending up in `NtAllocateVirtualMemory`) on such an address with the `MEM_COMMIT` flag will result in `MiCommitEnclavePages` being called, which will validate that the protection mask for the new pages is compatible (i.e., a combination of read, write, and/or execute, without any special caching or

write combining flags), and then call `MiAddPagesToEnclave`, passing a pointer to the enclave VAD associated with the address range, the protection mask that was specified to `VirtualAlloc`, and the PTE addresses that correspond to the virtual address range being committed.

`MiAddPagesToEnclave` first checks if the enclave VAD has any existing EPC pages associated with it, and if there's enough of them to satisfy the commit. If not, `MiReserveEnclavePages` is called to obtain a sufficient amount. `MiReserveEnclavePages` looks at the current enclave page list and counts the total. If there aren't enough physical EPC pages provided by the processor (based on the information obtained at boot), the function fails. Otherwise, it calls `MiGetEnclavePage` in a loop for the required amount of pages.

For each PFN entry that is retrieved, it is linked into the PFN array in the enclave VAD. Essentially, this means that once an enclave PFN is removed from the enclave page list and put into an active state, the enclave VAD acts as the list of active enclave PFNs.

Once the required committed pages have been obtained, `MiAddPagesToEnclave` translates the page protection passed to `LoadEnclaveData` into their SGX equivalents. Next, it reserves the appropriate number of system PTEs to hold paging information for each EPC pages that will be required. With this information in hand, it will eventually call `KeAddEnclavePage` that calls the SGX hardware to do the actual page adding.

One special page protection attribute is `PAGE_ENCLAVE_THREAD_CONTROL`, which indicates the memory is for a Thread Control Structure (TCS) defined by SGX. Each TCS represents a different thread that can execute independently within the enclave.

`NtLoadEnclaveData` validates parameters and then calls `MiCopyPagesIntoEnclave` to do the actual work, which may require getting committed pages as described earlier.

Initializing an enclave

Now that the enclave has been created and data has been transferred into it, there is another last step to perform before actual code in the enclave can execute. `InitializeEnclave` must be called to notify SGX that the enclave is in its final state before execution can begin. `InitializeEnclave` requires two SGX-specific structures to be passed in (`SIGSTRUCT` and `EINITTOKEN`; see the SGX documentation for the details).

The executive function `NtInitializeEnclave` called by `InitializeEnclave` does some parameter validation and makes sure the enclave VAD obtained has the correct attributes and then passed the structures along to the SGX hardware. Note that an enclave can only be initialized once.

The final step would be to use the Intel assembly instruction `EENTER` to start code execution (again, see the SGX documentation for the details).

Proactive memory management (SuperFetch)

Traditional memory management in operating systems has focused on the demand-paging model discussed thus far, with some advances in clustering and prefetching so that disk I/O can be optimized at the time of the demand-page fault. Client versions of Windows, however, include a significant improvement in the management of physical memory with the implementation of SuperFetch, a memory management scheme that enhances the least-recently accessed approach with historical file access information and proactive memory management.

The standby list management in older versions of Windows had two limitations. First, the prioritization

of pages relies only on the recent past behavior of processes and does not anticipate their future memory requirements. Second, the data used for prioritization is limited to the list of pages owned by a process at any given point in time. These shortcomings can result in scenarios in which the computer is left unattended for a brief period of time, during which a memory-intensive system application runs (doing work such as an antivirus scan or a disk defragmentation) and then causes subsequent interactive application use (or launch) to be sluggish. The same thing can happen when a user purposely runs a data- and/or memory-intensive application and then returns to use other programs, which appear to be significantly less responsive.

This decline in performance occurs because the memory-intensive application forces the code and data that active applications had cached in memory to be overwritten by the memory-intensive activities—applications perform sluggishly as they have to request their data and code from disk. Client versions of Windows take a big step toward resolving these limitations with SuperFetch.

Components

SuperFetch has several components in the system that work hand in hand to proactively manage memory and limit the impact on user activity when SuperFetch is performing its work. These components include the following:

- **Tracer** The tracer mechanisms are part of a kernel component (Pf) that allows SuperFetch to query detailed page-usage, session, and process information at any time. SuperFetch also uses the `FileInfo` mini-filter driver (`%SystemRoot%\System32\Drivers\Fileinfo.sys`) to track file usage.
- **Trace collector and processor** This collector works with the tracing components to provide a raw log based on the tracing data that has been acquired. This tracing data is kept in memory and handed off to the processor. The processor then hands the log entries in the trace to the agents, which maintain history files (described next) in memory and persist them to disk when the service stops, such as during a reboot.
- **Agents** SuperFetch keeps file page access information in history files, which keep track of virtual offsets. Agents group pages by attributes, such as the following:
 - Page access while the user was active
 - Page access by a foreground process
 - Hard fault while the user was active
 - Page access during an application launch
 - Page access upon the user returning after a long idle period
- **Scenario manager** This component, also called the *context agent*, manages the three SuperFetch scenario plans: hibernation, standby, and fast-user switching. The kernel-mode part of the scenario manager provides APIs for initiating and terminating scenarios, managing the current scenario state, and associating tracing information with these scenarios.
- **Rebalancer** Based on the information provided by the SuperFetch agents, as well as the current state of the system (such as the state of the prioritized page lists), the rebalancer—a specialized agent in the `Superfetch` user-mode service—queries the PFN database and reprioritizes it based on the associated score of each page, thus building the prioritized standby lists. The rebalancer can also issue commands to the memory manager that modify the working sets of processes on the system, and it is the only agent that actually takes action on the system. Other agents merely filter information for the rebalancer to use in its decisions. In addition to reprioritization, the rebalancer initiates prefetching through the prefetcher thread, which uses `FileInfo` and kernel services to

At any time during tracing, the rebalancer might be invoked to repopulate pages differently. These decisions are made by analyzing information such as the distribution of memory within working sets, the zero page list, the modified page list and the standby page lists, the number of faults, the state of PTE access bits, the per-page usage traces, current virtual address consumption, and working set size.

A given trace can be a page-access trace, in which the tracer uses the access bit to keep track of which pages were accessed by the process (both file page and private memory). Or, it can be a name-logging trace, which monitors the file name-to-file key mapping updates to the actual file on disk. These allow SuperFetch to map a page associated with a file object.

Although a SuperFetch trace only keeps track of page accesses, the `Superfetch` service processes this trace in user mode and goes much deeper, adding its own richer information such as where the page was loaded from (for example, in resident memory or a hard page fault), whether this was the initial access to that page, and what the rate of page access actually is. Additional information, such as the system state, is also kept, as well as information about recent scenarios in which each traced page was last referenced. The generated trace information is kept in memory through a logger into data structures, which—in the case of page-access traces—identify traces, a virtual address-to-working set pair, or, in the case of a name-logging trace, a file-to-offset pair. SuperFetch can thus keep track of which range of virtual addresses for a given process have page-related events and which range of offsets for a given file have similar events.

Scenarios

One aspect of SuperFetch that is distinct from its primary page reprioritization and prefetching mechanisms (covered in more detail in the next section) is its support for scenarios, which are specific actions on the machine for which SuperFetch strives to improve the user experience. These scenarios are as follows:

- **Hibernation** The goal of hibernation is to intelligently decide which pages are saved in the hibernation file other than the existing working-set pages. The idea is to minimize the amount of time it takes for the system to become responsive after a resume.
- **Standby** The goal of standby is to completely remove hard faults after resume. Because a typical system can resume in less than 2 seconds, but can take 5 seconds to spin up the hard drive after a long sleep, a single hard fault could cause such a delay in the resume cycle. SuperFetch prioritizes pages needed after a standby to remove this chance.
- **Fast user switching** The goal of fast user switching is to keep an accurate priority and understanding of each user's memory. That way, switching to another user will cause the user's session to be immediately usable, and won't require a large amount of lag time to allow pages to be faulted in.

Each of these scenarios has different goals, but all are centered around the main purpose of minimizing or removing hard faults.

Scenarios are hardcoded, and SuperFetch manages them through the `NtSetSystemInformation` and `NtQuerySystemInformation` APIs that control system state. For SuperFetch purposes, a special information class, `SystemSuperfetchInformation`, is used to control the kernel-mode components and to generate requests such as starting, ending, and querying a scenario or associating one or more traces with a scenario.

Each scenario is defined by a plan file, which contains, at minimum, a list of pages associated with the scenario. Page priority values are also assigned according to certain rules (described next). When a

scenario starts, the scenario manager is responsible for responding to the event by generating the list of pages that should be brought into memory and at which priority.

Page priority and rebalancing

You’ve already seen that the memory manager implements a system of page priorities to define which standby list pages will be repurposed for a given operation and in which list a given page will be inserted. This mechanism provides benefits when processes and threads have associated priorities—for example, ensuring that a defragmenter process doesn’t pollute the standby page list and/or steal pages from an interactive foreground process. But its real power is unleashed through SuperFetch’s page-prioritization schemes and rebalancing, which don’t require manual application input or hardcoded knowledge of process importance.

SuperFetch assigns page priority based on an internal score it keeps for each page, part of which is based on frequency-based usage. This usage counts how many times a page was used in given relative time intervals, such as by hour, day, or week. The system also keeps track of time of use, recording how long it’s been since a given page was accessed. Finally, data such as where this page comes from (which list) and other access patterns is used to compute this score.

The score is translated into a priority number, which can be anywhere from 1 to 6. (A priority of 7 is used for another purpose, described later.) Going down each level, the lower standby page list priorities are repurposed first, as shown in the “[Viewing the prioritized standby lists](#)” experiment. Priority 5 is typically used for normal applications, while priority 1 is meant for background applications that third-party developers can mark as such. Finally, priority 6 is used to keep a certain number of high-importance pages as far away as possible from repurposing. The other priorities are a result of the score associated with each page.

Because SuperFetch “learns” a user’s system, it can start from scratch with no existing historical data and slowly build an understanding of the different page accesses associated with the user. However, this would result in a significant learning curve whenever a new application, user, or service pack was installed. Instead, by using an internal tool, Windows can pre-train SuperFetch to capture SuperFetch data and then turn it into prebuilt traces. These prebuilt traces were generated by the SuperFetch team, who traced common usages and patterns that all users will probably encounter, such as clicking the Start menu, opening Control Panel, or using the File Open/Save dialog box. This trace data was then saved to history files (which ship as resources in Sysmain.dll) and is used to prepopulate the special priority 7 list. This list is where the most critical data is placed and is rarely repurposed. Pages at priority 7 are file pages kept in memory even after the process has exited and even across reboots (by being repopulated at the next boot). Finally, pages with priority 7 are static, in that they are never reprioritized, and SuperFetch will never dynamically load pages at priority 7 other than the static pretrained set.

The prioritized list is loaded into memory (or prepopulated) by the rebalancer, but the actual act of rebalancing is handled by both SuperFetch and the memory manager. As shown, the prioritized standby page list mechanism is internal to the memory manager, and decisions as to which pages to throw out first and which to protect are innate, based on the priority number. The rebalancer does its job not by manually rebalancing memory but by reprioritizing it, which causes the memory manager to perform the needed tasks. The rebalancer is also responsible for reading the actual pages from disk, if needed, so that they are present in memory (prefetching). It then assigns the priority that is mapped by each agent to the score for each page, and the memory manager ensures that the page is treated according to its importance.

The rebalancer can take action without relying on other agents—for example, if it notices that the distribution of pages across paging lists is suboptimal or that the number of repurposed pages across

different priority levels is detrimental. The rebalancer can also trigger working-set trimming, which might be required for creating an appropriate budget of pages that will be used for SuperFetch prepopulated cache data. The rebalancer will typically take low-utility pages—such as those that are already marked as low priority, that are zeroed, or that have valid content but not in any working set and have been unused—and build a more useful set of pages in memory, given the budget it has allocated itself. After the rebalancer has decided which pages to bring into memory and at which priority level they need to be loaded (as well as which pages can be thrown out), it performs the required disk reads to prefetch them. It also works in conjunction with the I/O manager’s prioritization schemes so that I/Os are performed with very low priority and do not interfere with the user.

The memory consumption used by prefetching is backed by standby pages. As described in the discussion of page dynamics, standby memory is available memory because it can be repurposed as free memory for another allocator at any time. In other words, if SuperFetch is prefetching the wrong data, there is no real impact on the user because that memory can be reused when needed and doesn’t actually consume resources.

Finally, the rebalancer also runs periodically to ensure that pages it has marked as high priority have actually been recently used. Because these pages will rarely (sometimes never) be repurposed, it is important not to waste them on data that is rarely accessed but may have appeared to be frequently accessed during a certain period. If such a situation is detected, the rebalancer runs again to push those pages down in the priority lists.

A special agent called the *application launch agent* is involved in a different kind of prefetching mechanism, which attempts to predict application launches and builds a Markov chain model that describes the probability of certain application launches given the existence of other application launches within a time segment. These time segments are divided across four different periods of roughly 6 hours each—morning, noon, evening, and night—and by weekday or weekend. For example, if on Saturday and Sunday evening a user typically launches Outlook after having launched Word, the application launch agent will likely prefetch Outlook based on the high probability of it running after Word during weekend evenings.

Because systems today have sufficiently large amounts of memory—on average more than 2 GB (although SuperFetch works well on low-memory systems, too)—the actual real amount of memory that frequently used processes on a machine need resident for optimal performance ends up being a manageable subset of their entire memory footprint. Often, SuperFetch can fit all the pages required into RAM. When it can’t, technologies such as ReadyBoost and ReadyDrive can further prevent disk usage.

Robust performance

A final performance-enhancing functionality of SuperFetch is called *robustness*, or *robust performance*. This component—managed by the user-mode `Superfetch` service but ultimately implemented in the kernel (`Pf` routines)—watches for specific file I/O access that might harm system performance by populating the standby lists with unneeded data. For example, if a process were to copy a large file across the file system, the standby list would be populated with the file’s contents, even though that file might never be accessed again (or not for a long period of time). This would throw out any other data within that priority—and if this was an interactive and useful program, chances are its priority would be at least 5.

SuperFetch responds to two specific kinds of I/O access patterns:

- **Sequential file access** With this type of I/O access pattern, the system goes through all the data in a file.

- **Sequential directory access** With this type of I/O access, the system goes through every file in a directory.

When SuperFetch detects that a certain amount of data past an internal threshold has been populated in the standby list as a result of this kind of access, it applies aggressive deprioritization (called *robustion*) to the pages being used to map this file. This occurs within the targeted process only so as not to penalize other applications. These pages, which are said to be *robusted*, essentially become reprioritized to priority 2.

Because this component of SuperFetch is reactive and not predictive, it does take some time for the robustion to kick in. SuperFetch will therefore keep track of this process for the next time it runs. Once SuperFetch has determined that it appears that this process always performs this kind of sequential access, it remembers this and robusts the file pages as soon as they're mapped instead of waiting for the reactive behavior. At this point, the entire process is now considered robusted for future file access.

Just by applying this logic, however, SuperFetch could potentially hurt many legitimate applications or user scenarios that perform sequential access in the future. For example, by using the Sysinternals Strings.exe utility, you can look for a string in all executables that are part of a directory. If there are many files, SuperFetch would likely perform robustion. Now, next time you run Strings.exe with a different search parameter, it would run just as slowly as it did the first time even though you'd expect it to run much faster. To prevent this, SuperFetch keeps a list of processes that it watches into the future, as well as an internal hard-coded list of exceptions. If a process is detected to later re-access robusted files, robustion is disabled on the process to restore the expected behavior.

The main point to remember when thinking about robustion—and SuperFetch optimizations in general—is that SuperFetch constantly monitors usage patterns and updates its understanding of the system to avoid fetching useless data. Although changes in a user's daily activities or application startup behavior might cause SuperFetch to pollute the cache with irrelevant data or to throw out data that it might think is useless, it will quickly adapt to any pattern changes. If the user's actions are erratic and random, the worst that can happen is that the system will behave in a similar state as if SuperFetch were not present at all. If SuperFetch is ever in doubt or cannot track data reliably, it quiets itself and doesn't make changes to a given process or page.

ReadyBoost

These days, RAM is easily available and relatively cheap compared to a decade ago. Still, it doesn't beat the cost of secondary storage such as hard disk drives. Unfortunately, mechanical hard disks contain many moving parts, are fragile, and, more importantly, are relatively slow compared to RAM, especially during seeking. As a result, storing active SuperFetch data on the drive would be as bad as paging out a page and hard-faulting it inside memory.

Solid state disks and hybrid drives offset some of these disadvantages but they are still pricier and slower compared to RAM. Portable solid state media such as USB flash disk (UFD), CompactFlash cards, and Secure Digital cards, however, provide a useful compromise. They are cheaper than RAM and available in larger sizes, but also have shorter seek times than mechanical hard drives because of the lack of moving parts.



Note

In practice, CompactFlash cards and Secure Digital cards are almost always interfaced through a USB adapter, so they all appear to the system as USB flash disks.

Random disk I/O is especially expensive because disk head seek time plus rotational latency for typical desktop hard drives total about 10 milliseconds—an eternity for today’s 3 or 4 GHz processors. Flash memory, however, can service random reads up to 10 times faster than a typical hard disk. Windows therefore includes a feature called *ReadyBoost* to take advantage of flash memory storage devices by creating an intermediate caching layer on them that logically sits between memory and disks.

ReadyBoost (not to be confused with ReadyBoot) is implemented with the aid of a driver (%SystemRoot%\System32\Drivers\Rdyboost.sys) that is responsible for writing the cached data to the non-volatile RAM (NVRAM) device. When you insert a USB flash disk into a system, ReadyBoost looks at the device to determine its performance characteristics and stores the results of its test in HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Emdmgmt. (*Emd* is short for *external memory device*, the working name for ReadyBoost during its development.)

If the new device is between 256 MB and 32 GB in size, has a transfer rate of 2.5 MB per second or higher for random 4 KB reads, and has a transfer rate of 1.75 MB per second or higher for random 512 KB writes, then ReadyBoost will ask if you’d like to dedicate some of the space for disk caching. If you agree, ReadyBoost creates a file named *ReadyBoost.sfcache* in the root of the device, which it uses to store cached pages.

After initializing caching, ReadyBoost intercepts all reads and writes to local hard disk volumes (C:\, for example) and copies any data being read or written into the caching file that the service created. There are exceptions—for example, data that hasn’t been read in a long while or data that belongs to Volume Snapshot requests. Data stored on the cached drive is compressed and typically achieves a 2:1 compression ratio, so a 4 GB cache file will usually contain 8 GB of data. Each block is encrypted as it is written using Advanced Encryption Standard (AES) encryption with a randomly generated per-boot session key to guarantee the privacy of the data in the cache if the device is removed from the system.

When ReadyBoost sees random reads that can be satisfied from the cache, it services them from there. However, because hard disks have better sequential read access than flash memory, it lets reads that are part of sequential access patterns go directly to the disk even if the data is in the cache. Likewise, when reading the cache, if large I/Os must be done, the on-disk cache will be read instead.

One disadvantage of depending on flash media is that the user can remove it at any time, which means the system can never solely store critical data on the media. (As you’ve seen, writes always go to the secondary storage first.) A related technology, ReadyDrive, covered in the next section, offers additional benefits and solves this problem.

ReadyDrive

ReadyDrive is a Windows feature that takes advantage of hybrid hard disk drives (H-HDDs). An *H-HDD* is a disk with embedded NVRAM. Typical H-HDDs include between 50 MB and 512 MB of cache.

Under ReadyDrive, the drive’s flash memory does not simply act as an automatic, transparent cache, as does the RAM cache common on most hard drives. Instead, Windows uses ATA-8 commands to define the disk data to be held in the flash memory. For example, Windows saves boot data to the cache when the system shuts down, allowing for faster restarting. It also stores portions of hibernation file data in the cache when the system hibernates so that the subsequent resume is faster. Because the cache is enabled even when the disk is spun down, Windows can use the flash memory as a disk-write cache, which avoids spinning up the disk when the system is running on battery power. Keeping the disk spindle turned off can save much of the power consumed by the disk drive under normal usage.

Another consumer of ReadyDrive is SuperFetch. It offers the same advantages as ReadyBoost with some enhanced functionality, such as not requiring an external flash device and having the ability to work

persistently. Because the cache is on the actual physical hard drive, which a user typically cannot remove while the computer is running, the hard drive controller typically doesn't have to worry about the data disappearing and can avoid making writes to the actual disk using solely the cache.

Process reflection

There are often cases where a process exhibits problematic behavior, but because it's still providing service, suspending it to generate a full memory dump or interactively debug it is undesirable. The length of time a process is suspended to generate a dump can be minimized by taking a minidump, which captures thread registers and stacks along with pages of memory referenced by registers, but that dump type has a very limited amount of information, which many times is sufficient for diagnosing crashes but not for troubleshooting general problems. With process reflection, the target process is suspended only long enough to generate a minidump and create a suspended cloned copy of the target, and then the larger dump that captures all of a process's valid user-mode memory can be generated from the clone while the target is allowed to continue executing.

Several Windows Diagnostic Infrastructure (WDI) components make use of process reflection to capture minimally intrusive memory dumps of processes their heuristics identify as exhibiting suspicious behavior. For example, the Memory Leak Diagnoser component of Windows Resource Exhaustion Detection and Resolution (also known as RADAR), generates a reflected memory dump of a process that appears to be leaking private virtual memory so that it can be sent to Microsoft via Windows Error Reporting (WER) for analysis. WDI's hung process detection heuristic does the same for processes that appear to be deadlocked with one another. Because these components use heuristics, they can't be certain the processes are faulty and therefore can't suspend them for long periods of time or terminate them.

The `RtlCreateProcessReflection` function in `Ntdll.dll` drives the implementation of process reflection. It works as follows:

1. It creates a shared memory section.
2. It populates the shared memory section with parameters.
3. It maps the shared memory section into the current and target processes.
4. It creates two event objects and duplicates them into the target process so that the current process and target process can synchronize their operations.
5. It injects a thread into the target process via a call to `RtlpCreateUserThreadEx`. The thread is directed to begin execution in `Ntdll`'s `RtlpProcessReflectionStartup` function. (Because `Ntdll.dll` is mapped at the same address (randomly generated at boot) into every process's address space, the current process can simply pass the address of the function it obtains from its own `Ntdll.dll` mapping.
6. If the caller of `RtlCreateProcessReflection` specified that it wants a handle to the cloned process, `RtlCreateProcessReflection` waits for the remote thread to terminate, otherwise it returns to the caller.
7. The injected thread in the target process allocates an additional event object that it will use to synchronize with the cloned process once it's created.
8. The injected thread calls `RtlCloneUserProcess`, passing parameters it obtains from the memory mapping it shares with the initiating process.
9. If the `RtlCreateProcessReflection` option that specifies the creation of the clone when the process is not executing in the loader, performing heap operations, modifying the process environment block (PEB), or modifying fiber-local storage is present, then

`RtlCreateProcessReflection` acquires the associated locks before continuing. This can be useful for debugging because the memory dump's copy of the data structures will be in a consistent state.

10. `RtlCloneUserProcess` finishes by calling `RtlpCreateUserProcess`, the user-mode function responsible for general process creation, passing flags that indicate the new process should be a clone of the current one. `RtlpCreateUserProcess` in turn calls `ZwCreateUserProcess` to request the kernel to create the process.

When creating a cloned process, `ZwCreateUserProcess` executes most of the same code paths as when it creates a new process, with the exception that `PspAllocateProcess`, which it calls to create the process object and initial thread, calls `MmInitializeProcessAddressSpace` with a flag specifying that the address should be a copy-on-write copy of the target process instead of an initial process address space. The memory manager uses the same support it provides for the Services for Unix Applications `fork` API to efficiently clone the address space. Once the target process continues execution, any changes it makes to its address space are seen only by it, not the clone. This enables the clone's address space to represent a consistent point-in-time view of the target process.

The clone's execution begins at the point just after the return from `RtlpCreateUserProcess`. If the clone's creation is successful, its thread receives the `STATUS_PROCESS_CLONED` return code, whereas the cloning thread receives `STATUS_SUCCESS`. The cloned process then synchronizes with the target and, as its final act, calls a function optionally passed to `RtlCreateProcessReflection`, which must be implemented in `Ntdll.dll`. RADAR, for instance, specifies `RtlDetectHeapLeaks`, which performs heuristic analysis of the process heaps and reports the results back to the thread that called `RtlCreateProcessReflection`. If no function was specified, the thread suspends itself or terminates, depending on the flags passed to `RtlCreateProcessReflection`.

When RADAR and WDI use process reflection, they call `RtlCreateProcessReflection`, asking for the function to return a handle to the cloned process and for the clone to suspend itself after it has initialized. Then they generate a minidump of the target process, which suspends the target for the duration of the dump generation. Next, they generate a more comprehensive dump of the cloned process. After they finish generating the dump of the clone, they terminate the clone. The target process can execute during the time window between the minidump's completion and the creation of the clone, but for most scenarios any inconsistencies do not interfere with troubleshooting. The `Procdump` utility from Sysinternals also follows these steps when you specify the `-r` switch to have it create a reflected dump of a target process.

Conclusion

This chapter examined how the Windows memory manager implements virtual memory management. As with most modern operating systems, each process is given access to a private address space, protecting one process's memory from another's but allowing processes to share memory efficiently and securely. Advanced capabilities, such as the inclusion of mapped files and the ability to sparsely allocate memory, are also available. The Windows environment subsystem makes most of the memory manager's capabilities available to applications through the Windows API.

The next chapter covers another critical part of any operating system—the I/O system.